

A 64-bit ARX-box NeoAlzette

Executable Specification, Exact Local Models, and Resource-Limited MILP Analysis

Twilight-Dream¹

Affiliation, City, Country, yujiang1187791459@gmail.com

Abstract. We present NeoAlzette, a 64-bit reversible ARX box over two 32-bit words, together with an executable specification and a model-to-code analysis of its differential and linear behavior. The paper treats `neoalzette_core.hpp` and `neoalzette_core.cpp` as the normative implementation and gives a stage-by-stage refinement argument from the mathematical round map to the C++ statements. We prove reversibility by factoring the round into triangular word updates whose inverses are executed in the reverse order; consequently, the two branch-dependent injection maps need not be permutations by themselves.

The cryptanalytic model distinguishes four non-interchangeable arithmetic objects: two-variable modular addition/subtraction in the differential setting, fixed-public-addend addition/subtraction in the differential setting, two-variable modular addition/subtraction in the linear setting, and fixed-public-addend addition/subtraction in the linear setting. For each object, the local oracle, its MILP realization, and the exported trace are connected by explicit verification obligations. The joint injection is modeled as a coupled two-output object: the XOR-routed component and the injection-derived addend component share the same source and may not be split into independent local transitions. We additionally formalize the semantic characteristic signature, the exact no-good cut used for HULL enumeration, the SCIP reoptimization invariant, and the multi-layer Forest search in which an output state becomes the input to the next analyzed box.

The numerical conclusions are unchanged from the designated 43200-second experiments. The final differential incumbent has weight 80.431292345473963, corresponding to $2^{-80.431292345}$. The final linear incumbent has objective weight 16.61538595228642, with exact trace-evaluated weight 16.615385959600772 and absolute correlation approximately $2^{-16.6153859522}$. Both runs terminated by the resource limit; hence the reported values are verified incumbents for the implemented model, not solver-proven global optima and not complete HULL bounds.

Keywords: ARX-box, symmetric cryptography, modular addition, linear cryptanalysis, differential cryptanalysis, MILP, SCIP

NeoAlzette

Contents

Introduction	4
Related Work	7
Version 6.5 Revision Scope	10
Methodology, Refinement Layers, and Verification Obligations	11

I	Engineering Implementation	12
1	Implemented Object	12
1.1	Implementation Source Boundary	13
2	Primitive Operations	13
2.1	Word Space and Bit Numbering	13
2.2	Boolean Word Operations	13
2.3	Modular Addition and Subtraction	14
2.4	Rotations	14
2.5	Zero-Filling Shifts	15
2.6	State and Joint Injection Values	16
3	Round Constants	16
4	Linear Diffusion-Mask Generators	16
4.1	Mask mask0	16
4.2	Mask mask1	17
5	Cross-Branch Bridges	17
6	Injection Constants	17
7	Current <i>B</i>-to-<i>A</i> Injection	18
8	Current <i>A</i>-to-<i>B</i> Injection	18
9	Anti-Bypass Hybrid Schedule	19
9.1	Design Background	19
9.2	Inheritance from Alzette	19
9.3	Relation to the Prior Fallback Variant	19
9.4	Schedule Overview	20
9.5	Implemented Forward Transformation	20
9.6	Checkpointed Arrangement	20
9.7	Implemented Backward Transformation	22
9.8	Executable Refinement and Permutation Proof	22
9.9	Round Structure and Inversion	24
9.10	First Half-Round Engineering Structure	24
9.11	Second Half-Round Engineering Structure	24
9.12	Fixed Public Subtraction	25
9.13	Modular Addition and Subtraction in the Model	25
9.14	Why the Constant Does Not Belong Only Inside the Injection	25
9.15	Rotation Roles	25
9.16	Bridge Selection Derivation	26
9.17	First Bridge: 22 and 13	27
9.18	Second Bridge: 5 and 25	27
9.19	Why Not Reuse the Same Rotation Family	27
9.20	Joint Injection Semantics	28
9.21	Threat Model	28
9.22	MILP, SAT, and Trace Modeling Requirements	28
9.23	Code Synchronization Requirements	29
9.24	Design Summary	29

II	Resource-Limited MILP Analysis	29
10	Modular Addition and Subtraction Oracles for MILP Analysis	30
10.1	Terminology Before the Local Boxes	30
10.2	From Local Oracles to Distinguishers	32
10.3	Why Modular Addition and Subtraction Are Isolated Here	33
10.4	Two-Variable Addition and Subtraction Oracles	34
10.5	Two-Variable Differential Q_1 : Lipmaa–Moriai 2002 Point Oracle	35
10.6	Two-Variable Differential Q_2 : Lipmaa–Moriai 2002 Optimal- γ Completion	39
10.7	Two-Variable Linear Q_1 : Wallén 2003 to Schulte–Geers 2013 Point Correlation	42
10.8	Two-Variable Linear Q_1 and Schulte–Geers Matrix Directions	52
10.9	Two-Variable Linear Row Completion: Our Fixed-Row Direction	61
10.10	Two-Variable Linear Subtraction: Addition-Oracle Permutation	65
11	Fixed-Addend Addition and Subtraction Oracles	66
11.1	Why the Fixed Addend Is a Separate Local Object	66
11.2	Fixed-Addend Differential Q_1 : Exact Probability, Exact Count, and Code Paths	66
11.3	Fixed-Addend Linear Q_1 : Signed-Numerator Oracle	75
11.4	Fixed-Addend Subtraction Wrappers	81
12	MILP Constraints for Two-Variable Addition and Subtraction	81
12.1	From the Differential Oracle to a One-Hot MILP Box	81
12.2	Code-Level Derivation of the Two-Variable MILP Boxes	82
13	MILP Constraints for Fixed-Addend Addition and Subtraction	87
13.1	Code-Level Derivation of the Fixed-Addend MILP Boxes	87
13.2	Fixed-Addend Differential MILP: The Exact Code-Level Constraints	88
13.3	Fixed-Addend Linear MILP: Exact Numerator and Exact-Log Weight	98
14	Mathematical Analysis of the Joint Injection Layer	107
14.1	Design Rationale Under the Four-Addition Budget	107
14.2	Construction Validity of the Dynamic Diffusion Masks	109
14.3	Quadratic Boolean Front Half	116
14.4	Quadratic Boolean Injection Function Differential Behavior	119
14.5	Quadratic Boolean Injection Function Linear Behavior	122
14.6	Oracle-to-MILP Bridge for the Joint Injection Layer	124
15	Top-Level MILP Question Model Assembly	134
15.1	Differential	134
15.2	Linear	137
16	Executable Model Correspondence, Characteristic Certification, and Forest/HULL Analysis	143
16.1	Round Graph and Four-Case Arithmetic Dispatch	143
16.2	Global Refinement of the Round MILP	143
16.3	Semantic Characteristic Certificates	144
16.4	Semantic No-Good Cuts and Reoptimization	145
16.5	Forest Layer as Iterated Weakness Growth	146
16.6	Multi-Process Campaigns Without Model Mutation	147
16.7	Research-Artifact Acceptance Checklist	148

17 Resource-Limited MILP Experiment	148
17.1 Differential Result	148
17.2 Linear Result	151
A Detailed Code-to-Mathematics Correspondence	154
B Verification Protocol for Future Revisions	155
C Citations and Source Lineage	156

Introduction

In symmetric-key design, a linear diffusion layer is not a decorative component. It is the mechanism that connects local nonlinearity to global security by spreading differences, linear masks, and active bits across a larger part of the state. In word-oriented software designs, ARX structures, bitsliced implementations, and lightweight primitives, one especially natural form of such a layer is an XOR of rotated copies of a 32-bit word. The design problem is therefore not merely whether one can write a rotation-XOR formula, but how the rotation set is chosen, how it interacts with the nonlinear operations around it, and how it affects differential, linear, integral, and automated-search models.

Standardized and widely studied primitives already illustrate this pattern. The SM4 round function uses the 32-bit linear transform

$$L(B) = B \oplus B \lll^2 \oplus B \lll^{10} \oplus B \lll^{18} \oplus B \lll^{24},$$

and the SM4 key schedule uses

$$L'(B) = B \oplus B \lll^{13} \oplus B \lll^{23}.$$

These layers do not rely on explicit multiplication by a large dense matrix; they obtain diffusion from a small number of word rotations and XORs [TW18]. The ZUC stream cipher uses the same design idiom inside its nonlinear function F , with

$$L_1(X) = X \oplus X \lll^2 \oplus X \lll^{10} \oplus X \lll^{18} \oplus X \lll^{24}$$

and

$$L_2(X) = X \oplus X \lll^8 \oplus X \lll^{14} \oplus X \lll^{22} \oplus X \lll^{30}.$$

Thus, even outside a conventional block-cipher round structure, 32-bit XOR-rotation maps remain a compact way to move local activity through a word [Dat]. These examples are not cited as direct ancestors of NeoAlzette; rather, they show that rotation-XOR diffusion is a mature engineering idiom in deployed word-oriented cryptography.

The common abstraction is

$$M_R(X) = \bigoplus_{r \in R} X \lll^r,$$

where R is a set of rotation offsets. Different choices of R can produce very different branch numbers, implementation costs, and propagation patterns for differential and linear trails. Recent work by Leurent and Pernot makes this point explicit by treating high-branch-number XOR/rotation linear layers for bitsliced 32-bit implementations as a search problem, using information-set-decoding techniques to find efficient candidates in this restricted implementation-friendly space [LP24]. This paper adopts the same general engineering stance: a rotation-XOR diffusion map should be treated as a searched and audited component, not as an arbitrary collection of constants.

The name NeoAlzette is deliberately genealogical, but it is not merely a lighter or renamed Alzette. It refers to Alzette, the 64-bit ARX-box introduced by Beierle et al. and used in the SPARKLE family, as well as to the broader ARX-box design line represented by SPARX/LAX [DPU⁺16, BBdS⁺20, BBdS⁺21]. Alzette’s contribution was important for exactly the reason relevant here: it showed that a wide S-box-like object over two 32-bit words can be built from a compact ADD/ROT/XOR/constant schedule, with constant-time software implementation and dedicated differential and linear analysis. The Alzette paper reports a 64-bit ARX-based S-box using only twelve instructions on modern processors and uses it as a building block for CRAX and TRAX [BBdS⁺20]. In other words, Alzette is not merely a set of rotation constants; it is a design principle: arithmetic nonlinearity, rotational transport, XOR feedback, and public constants are arranged so that local word operations behave like a compact nonlinear box.

The prefix “Neo” is used here in the stricter sense of a security-specialized Alzette descendant. The design goal is not to preserve Alzette’s instruction count at all costs, but to explore a nearby 64-bit reversible ARX/SRX box in which extra work is spent on suppressing low-round bypasses visible to differential and linear trail search. Thus NeoAlzette is allowed to trade some of Alzette’s minimality for coupled Boolean–arithmetic injection, fixed-public constant arithmetic, and non-duplicated bridge structure. This is why the object is called NeoAlzette rather than Alzette: it inherits the wide ARX-box motivation, but changes the security objective and the internal mechanism.

Large-state S-box work following Alzette reinforces this viewpoint. Fan, Feng, and Wei explicitly formulate the problem of designing a 64-bit ARX-based lightweight S-box with performance comparable to Alzette but stronger differential and linear bounds. Their hierarchy filtering method sets target bounds for best differential and linear characteristics in advance and then searches rotation parameters satisfying these constraints [FFW23]. This is close in spirit to the engineering stance of NeoAlzette: the round schedule, rotation choices, and injected nonlinear paths are not ornamental constants, but searched and audited components.

NeoAlzette therefore keeps the Alzette-like scheduling spirit but changes the design objective. It is not a drop-in replacement for Alzette and it is not an attempt to claim that Alzette lacks a necessary feature. Instead, NeoAlzette explores a security-biased engineering point: the main production path is still modular addition/subtraction plus bit rotation, while a side injection path feeds branch-derived Boolean material back into the arithmetic path. In this paper, “SRX” is used conservatively to mean an ARX-like schedule augmented by a substitution-like Boolean injection path; it is not meant as a standard cipher-design taxonomy. The side path is implemented by the two asymmetric procedures `CdInjectionFromB` and `CdInjectionFromA`. Their local diffusion masks are 32-bit XOR-rotation maps, and their Boolean outputs are not used as isolated XOR gadgets; the same injected material is also transformed into operands for subsequent modular addition. The injection layer is therefore intended as a security hardening path for activity spreading, even when this costs more than the original Alzette schedule.

This distinction is central to the present paper. Earlier automated experiments showed that a solver can sometimes drive an internal injection window to zero difference or zero linear mask, avoid paying for a local Boolean layer, and then reactivate the state later through bridge or modular addition constraints. NeoAlzette addresses this internal zero-window bypass phenomenon by combining four ingredients: fixed public subtractions as carry/borrow coordinate anchors, searched XOR-rotation diffusion masks inside the injection pre-mix, coupled Boolean injection outputs, and injection-derived modular-addition operands. The bridge rotations are also not duplicated: the first bridge uses the ordered pair (22, 13), while the second uses (5, 25), so that the solver does not see two copies of the same zero-weight rotation template.

Accordingly, the technical line followed in this paper is

- mature XOR-rotation diffusion
- search-based 32-bit bitsliced linear layers
- large-state Alzette-like ARX boxes
- NeoAlzette joint Boolean–arithmetic injection
- MILP/SAT-style differential and linear evaluation.

The purpose is not to present the injection layer as mysterious complexity. The purpose is to explain why the layer is shaped as a joint object: its front half provides branch-derived Boolean diffusion and confusion, while its back half couples the same derived material to modular addition/subtraction. A differential or linear model that separates those two halves would analyze a different object from the implemented one.

This paper therefore has two scopes. Part I specifies the implemented NeoAlzette V6.5 core: primitive operations, diffusion-mask generators, injection procedures, bridge choices, forward schedule, and inverse schedule. Part II reports a resource-limited C++/SCIP MILP analysis of exactly that implementation. All numerical conclusions are drawn only from the final 43200-second differential and linear runs designated for this version. Consequently, “best” means best incumbent found under the stated time limit; it does not mean solver-proven global optimality unless the solver status explicitly proves optimality.

Contributions and claim discipline. The technical contribution of this version is not a new numerical record but a complete chain of correspondence between specification, local mathematics, solver model, and exported artifact. More precisely:

1. We define the implemented 64-bit map as an ordered composition of triangular word updates and prove that the C++ backward routine is its inverse. This proof identifies the controlling branch that must be restored before each injection value is recomputed.
2. We separate the four arithmetic cases that occur in the code. The two-variable cases are averaged over two variable operands and use the Lipmaa–Moriai and Wallén/Schulte–Geers theories; the fixed-public cases average over one variable operand and use constant-dependent carry-state recurrences. No case is obtained by silently substituting a zero mask or zero difference into a different probability space.
3. We state oracle-to-MILP equivalence claims for the arithmetic and joint injection components and identify the shared variables that enforce the Boolean–arithmetic coupling of the implemented round.
4. We define a semantic characteristic certificate and give independent differential and linear audit procedures. These procedures recompute feasibility, local weight, and, in the linear case, sign and public constant phase from the exported masks or differences.
5. We formalize the HULL reoptimization and Forest layers implemented in the repository. HULL enumeration adds one semantic no-good inequality after each accepted characteristic; Forest search composes boxes by feeding the output of one analyzed box into the next layer. These mechanisms extend the analysis workflow but do not alter the numerical one-box results reported in this paper.
6. We preserve a strict claim boundary: a time-limited incumbent is a verified witness, whereas optimality, endpoint completeness, and HULL completeness require the corresponding solver termination certificate.

Related Work

Large-state ARX S-boxes and Alzette-like designs

The closest conceptual ancestor of NeoAlzette is the line of large word-oriented ARX boxes represented by SPARX and Alzette. SPARX introduced an ARX-box design methodology with provable single-trail bounds and long-trail style arguments for ARX-based lightweight block ciphers [DPU⁺16]. Alzette then showed that a 64-bit S-box-like object can be built directly from two 32-bit branches using only additions, rotations, XORs, and constants, while still admitting dedicated automated differential and linear analysis [BBdS⁺20]. The SPARKLE specification places Alzette inside a full permutation family and records its role as the nonlinear ARX-box used by the Schwaemm and Esch families [BBdS⁺21].

A particularly close reference point is the recent Chinese line of large-state lightweight cryptographic S-box design following Alzette. Fan, Feng, and Wei present a new 64-bit ARX-based lightweight S-box, introduce a hierarchy filtering method for choosing rotation parameters by pre-setting differential and linear bounds, and compare the resulting bounds against Alzette across several rounds [FFW23]. This work matters for NeoAlzette because it frames the same high-level research question: whether a 64-bit lightweight S-box can remain close to Alzette in implementation cost while improving the automated-search profile. NeoAlzette answers that question from a different direction: instead of only selecting new rotation constants inside a pure ARX schedule, it couples an Alzette-like ARX skeleton with a joint Boolean–arithmetic injection path.

This is also the point of the prefix “Neo”. In the related-work landscape, NeoAlzette should not be read as a reimplementing of Alzette, nor as a claim that Alzette needs this particular modification. It is a security-specialized descendant: the baseline rhythm of wide ADD/ROT/XOR/constant operations is kept, but extra operations are deliberately spent to make the branch-derived Boolean injection and the following arithmetic operand depend on the same source. This design point is allowed to be less instruction-minimal than Alzette, because its purpose is to harden a visible low-round bypass pattern rather than to reproduce the original Alzette performance point.

The security analysis of Alzette has also developed beyond ordinary differential and linear trails. Subsequent work studied impossible differentials, rotational-XOR propagation, rotational differential-linear cryptanalysis, and arbitrary-output-mask rotational differential-linear distinguishers for Alzette and related ARX boxes [XLW22, HXW22, LSL21, NSL⁺22, LNS⁺23]. Lv, Jin, and Cui later formulated automatic differential-linear trail search for ARX ciphers as a MIQCP problem and applied the method to Alzette and SPECK [LJC23]. These studies are relevant for NeoAlzette because they show that a 64-bit ARX-box must be evaluated not only as a sequence of additions and rotations, but as a wide nonlinear component whose low-round behavior may admit specialised distinguishers. The present paper does not claim to close all of those attack surfaces; it focuses on the implemented V6.5 schedule and on resource-limited one-round differential and linear single-characteristic MILP experiments.

Rotation-XOR diffusion layers in standardized designs

The SM4 and ZUC examples in the introduction are retained deliberately. They are not direct ancestors of NeoAlzette, but they show that 32-bit rotation-XOR linear layers are a mature implementation pattern. SM4 uses rotation-XOR linear transforms in both its round function and key schedule, while ZUC uses closely related transforms inside the nonlinear function F [TW18, Dat]. This supports the design decision to keep the local diffusion masks inside `CdInjectionFromB` and `CdInjectionFromA` within the same software-friendly rotation-XOR family. The difference is that NeoAlzette places these maps inside a joint injection layer and then feeds the derived material into modular arithmetic,

so the diffusion layer is not an independent post-S-box linear layer.

Differential and linear theory of modular addition

The mathematical treatment of modular addition is the second required background. Lipmaa and Moriai gave efficient algorithms for computing differential properties of addition modulo 2^n , replacing infeasible table-based reasoning by structural conditions on carries [LM02]. Wallen developed the corresponding algorithmic theory for linear approximations of addition and subtraction modulo 2^n , including efficient computation of correlations and generation of masks with prescribed nonzero correlation [Wal03a]. Wallen’s longer report also places XOR-difference probabilities, additive differential probabilities of XOR, and linear correlations of addition into a common rational-series framework [Wal03b]. Schulte-Geers showed that addition modulo 2^n is CCZ-equivalent to a quadratic vectorial Boolean function and used this viewpoint to obtain explicit Walsh-transform formulas and linear-equation reductions for differential equations of addition [SG13]. Dehnavi, Mahmoodi Rishakani, and Mirzaee Shamsabad further refined explicit formulas for linear probabilities of modular addition [DRS15]. Related Boolean-function analyses study the component functions, Walsh spectra, carry-bit distributions, algebraic degree, and ANF of modular addition and constant modular addition [LHS⁺11, DRS⁺16]. Mouha et al. revisit the additive differential probability of XOR and prove maximal-probability facts that are useful when ARX trails mix XOR and modular addition as competing difference groups [MKA⁺21]. These works justify why NeoAlzette does not model modular addition as a generic black box: carry and borrow structure is precisely where the arithmetic nonlinearity lives.

Fixed-addend modular addition and public constants

Constant modular addition is particularly relevant because NeoAlzette V6.5 uses fixed public subtractions before the two injection windows. Miyano already emphasized that, when an addend is fixed as a key or constant, the differential and linear probabilities are functions of that addend rather than universal constants [Miy98]. Machado studied the differential probability of the one-variable map $X \mapsto A + X \bmod 2^n$ with a constant operand [Mac01]. Azimi et al. gave a bit-vector differential model for modular addition by a constant, enabling SMT-based automated search for ARX primitives that contain such operations [ARS⁺20, ARS⁺22]. Bagherzadeh and Ahmadian observed that the differential model for modular addition with one constant input can be represented with fewer linear inequalities than the fully variable case, and used this to speed up MILP differential searches for LEA and HIGHT [BA20]. Guan and Ding studied the probability structure connecting $(X + K) \bmod 2^n$ and $X \oplus K$ for fixed constants and used it in a guess-and-determine context [GD10].

These results support the modeling decision used here: the steps $B \leftarrow B - \text{RC}[1]$ and $A \leftarrow A - \text{RC}[6]$ are fixed-public arithmetic operators with carry/borrow constraints, not XOR whitening and not zero-weight constants. On the differential side, the fixed-addend view can be formulated through a carry-pair or 4-state recurrence; on the linear side, the same distinction leads to a carry-conditioned signed transfer or 2-state recurrence. The implementation accompanying this paper separates two-variable addition/subtraction from fixed-public-constant addition/subtraction at the oracle level. This is not merely a coding preference: it prevents the analysis from silently replacing a concrete public constant by an averaged two-variable operation.

The current implementation also contains an exact fixed-addend log-weight engineering layer for the linear model. In conceptual terms, the signed numerator N of the fixed-addend Walsh correlation is computed from an exact carry-transfer recurrence, the absolute numerator $A = |N|$ is bound to the MILP state, and the local linear weight is charged

as $W = n - \log_2 A$. Earlier engineering experiments used outer numerator sweeps or ladder-style charging; those should not be described as the final V6.5 objective model unless the corresponding command-line mode is explicitly selected. The V6.5 paper should therefore distinguish the historical sweep mode from the fixed-addend exact-log-weight model used for the final one-round linear run.

Two companion technical drafts are useful for documenting this fixed-addend layer. The first gives explicit 4-state differential and 2-state linear recurrences for modular addition by a constant, together with linear-time evaluators and exhaustive small-width checks [Twi26b]. The second develops a run-aware bit-vector linear correlation model for modular addition by a constant, including flattened carry-state evaluators and windowed estimators [Twi26a]. These are cited as technical manuscripts, not as peer-reviewed standards; their role is to document the engineering theory used in the implementation.

Automatic search for ARX trails and hulls

A third body of work concerns automatic search for ARX differential and linear trails. Biryukov and Velichkov introduced a tool for automatic differential-trail search in ARX ciphers by extending Matsui’s branch-and-bound strategy through partial difference distribution tables for modular addition [BV]. Biryukov, Velichkov, and Le Corre then adapted Matsui’s algorithm to find best differential and linear trails for ARX ciphers without heuristic pruning, demonstrating the method on reduced-round SPECK [BVC16]. Sun et al. showed how automated enumeration can find differential and linear characteristics with predefined properties for bit-oriented block ciphers [SHW⁺14]. Fu et al. formulated differential and linear trail search for SPECK as MILP problems by encoding modular-addition propagation with linear inequalities [FWG⁺16]. The later CDDT/CLAT approach of Liu, Li, Jiao, and Wang viewed a wide addition as a sequence of smaller additions linked by carry bits, thereby recovering an S-box-like table interface for ARX search without building an infeasible full DDT or LAT [LLJW].

Linear search and linear-hull effects require separate attention. Liu, Wang, and Rijmen modeled optimal linear-trail search for ARX designs as a SAT problem and applied the method to SPECK and Chaskey, including enumeration of trails inside a linear hull [LWR16]. Huang and Wang revisited the Walsh transform of modular addition, constructed combinational linear approximation tables, and improved automatic search for linear characteristics and hulls of SPECK, SPARX, Chaskey, and CHAM-64 [HW19]. We cite the hull literature to position the scope of this paper, not to claim a hull bound. The experiments reported here are single-characteristic MILP searches under a time limit. They are not linear-hull enumerations and should not be read as bounds on the full linear hull.

Partitioning and differential-linear methods provide another warning about scope. Leurent refined partitioning techniques for ARX differential and linear attacks, reducing data complexity at the cost of more processing time and applying the method to FEAL and Chaskey [Leu15]. The MIQCP differential-linear work mentioned above shows that even for Alzette one can ask solver-driven questions beyond separate differential or linear single trails [LJC23]. These are important adjacent attack models; they are not covered by the one-round V6.5 MILP claims.

Several works also warn that simplified independence assumptions in ARX models can be dangerous. ElSheikh, Abdelkhalek, and Youssef showed that a MILP model for differential propagation through modular additions may become invalid when it assumes independence between consecutive additions, and they proposed a more accurate model that accounts for dependencies induced by carry bits [EAY]. ElSheikh’s later thesis places this issue in a broader MILP-aided cryptanalysis program and explicitly discusses automation for differential, impossible-differential, and integral analysis of block ciphers [ELS21]. This is directly aligned with the modeling discipline used in this paper: the Boolean front half

of the NeoAlzette injection and the arithmetic back half are not independent variables. They form one joint source whose XOR output and modular-addition operand must remain coupled.

Classical RC5/RC6 analyses and multiple approximations

The classical RC5 and RC6 literature explains why word-oriented ARX operations cannot be treated as simple symbolic decoration. Kaliski and Yin studied differential and linear cryptanalysis of RC5, whose security relies on the mixed use of addition, XOR, and data-dependent rotations [JY95]. Knudsen and Meier improved the differential attack on RC5 by exploiting data-dependent rotations and weak-key phenomena, and Biryukov and Kushilevitz later introduced a partial-differential “good pair” approach that further reduced the chosen-plaintext cost for RC5-32/12/16 [KM96, BK98]. On the linear side, Kaliski and Robshaw developed the general multiple-approximation technique, Selçuk gave further linear cryptanalysis results for RC5, and Borst, Preneel, and Vandewalle showed that linear-hull effects and multiple linear approximations can materially affect RC5 and the AES candidate RC6 [JR94, Sel98, BPV99]. Reduced-round and simplified-variant analyses of RC6 further isolate the roles of fixed rotations, quadratic mixing, and multiple linear paths [CRRY99, MN02, STK02]. These older ARX analyses are not structurally the same as NeoAlzette, but they reinforce the same lesson: the interaction between arithmetic carries, rotations, Boolean masks, and key or constant material must be modeled at the bit level if the resulting security claims are to be meaningful.

Scope of this paper and position of NeoAlzette

Alzette was also evaluated against attacks beyond ordinary differential and linear characteristics, including division-property based integral searches, algebraic-degree experiments, invariant-subspace searches, and nonlinear invariant tests [BBdS⁺20]. NeoAlzette V6.5 does not claim to complete the same full analysis in this version. Instead, this paper focuses on the engineering specification of the V6.5 core and on a resource-limited MILP study of one-round differential and linear single characteristics. Extending the analysis to integral, invariant-subspace, algebraic-degree, rotational, rotational differential-linear, and nonlinear invariant properties is left as future work.

In summary, previous work provides four foundations used by this paper: large ARX boxes can be meaningful S-box-like components; rotation-XOR maps are mature software-friendly diffusion layers; modular addition has exact differential and linear theories, including constant-addend variants; and automated ARX search is powerful but sensitive to modeling assumptions. NeoAlzette contributes a different internal coupling pattern within that landscape: the injection layer is designed and analyzed as a joint Boolean–arithmetic object, so that bypassing the XOR injection and bypassing the injection-derived modular-addition operand are no longer independent choices. The final 43200-second runs are therefore interpreted as verified resource-limited one-round incumbents, not as solver-proven global optimums, not as hull bounds, and not as full-cipher security proofs.

Version 6.5 Revision Scope

Version 6.5 is a paper-level consolidation of the implemented Core and the final one-round resource-limited MILP artifacts. It is not a research diary and it is not a compressed engineering note. The revision has three explicit boundaries:

1. the engineering part specifies the current C++ implementation, including the exact primitive operations, injection maps, bridge order, forward schedule, and inverse schedule;

2. the linear diffusion-mask generators inside the injection layer are retained as implemented and are not redesigned or relocated in this revision;
3. the analysis part reports the final 43200-second C++/SCIP single-characteristic incumbents, together with their solver resource limits, logged node counts, logged memory use, and trace validation.

The word “best” in Part II always means “best incumbent exported by the resource-limited run described in this paper”. It does not mean a solver-certified global optimum unless the corresponding SCIP status says so; the two final runs reported here both ended by time limit.

Methodology, Refinement Layers, and Verification Obligations

A long implementation-oriented cryptanalytic paper is credible only if it states which layer each claim belongs to. We therefore use the following five-layer hierarchy throughout the remainder of the paper.

Definition 1 (Five-layer correspondence hierarchy). Let $F : \mathbb{W}_{32}^2 \rightarrow \mathbb{W}_{32}^2$ denote the implemented one-box map. The analysis is partitioned into the following layers.

1. *Value semantics*: the mathematical operations over $\mathbb{Z}/2^{32}\mathbb{Z}$ and $\mathbb{F}_2^{32}$, including the ordered round map and its inverse.
2. *Local oracle semantics*: exact feasibility and local probability/correlation for one arithmetic or joint-injection component.
3. *MILP semantics*: binary, integer, and continuous variables whose feasible integral points project to the local oracle relation and whose objective equals the local weight.
4. *Characteristic semantics*: a globally consistent assignment of intermediate differences or masks to every stage of the round graph.
5. *Experimental semantics*: the incumbent, bound, status, resource usage, and exported trace returned by a concrete SCIP run.

A statement at layer i does not automatically imply a stronger statement at layer $i + 1$; the connecting refinement claim must be stated and checked.

Verification Obligation 1 (Functional correspondence). The mathematical forward map, Algorithm 5, and `NeoAlzetteCore::forward` shall execute the same ordered word updates. The analogous statement shall hold for the backward map.

Verification Obligation 2 (Local-model correspondence). For each of the four arithmetic cases, the oracle and MILP box shall agree on feasibility and local weight for every integral mask/difference assignment. For linear transitions, the independent oracle shall additionally determine the sign used by the trace and HULL accumulator.

Verification Obligation 3 (Joint-injection coupling). At each injection site, the XOR-routed output and the addend-routed output shall be generated from one source word and one joint oracle relation. The addend component used by the injection relation shall be the same variable used by the following modular-addition box.

Verification Obligation 4 (Objective and trace identity). For every accepted SCIP solution, the sum of independently recomputed local weights shall agree with the solver objective within the declared numerical tolerance. An oracle-impossible stage or a local-weight mismatch invalidates the characteristic rather than merely annotating it.

Verification Obligation 5 (Enumeration identity). A semantic no-good cut shall exclude exactly the characteristic signature that has already been counted. Reoptimization may reuse solver state, but it shall not alter endpoints, the objective window, local models, or the feasible set except for the accumulated no-good cuts.

Table 1 records the source boundary used for these obligations. It is deliberately a responsibility map rather than a line number map, because line numbers change during maintenance while interfaces and semantic roles should remain stable.

Table 1: Normative source responsibility map.

Source	Normative responsibility
<code>neoalzette_core.hpp/.cpp</code>	Value-domain constants, diffusion maps, joint injections, forward map, and inverse map.
<code>differential/model/ *oracle.hpp</code>	Differential point oracles for two-variable arithmetic and the joint-injection affine-image/rank relation.
<code>differential/model/ *constraint.hpp</code>	Differential MILP realization of two-variable and fixed-public arithmetic.
<code>differential/model/ *round_function.hpp</code>	Round composition, semantic characteristic capture, trace audit, and differential HULL reoptimization session.
<code>linear/model/*oracle.hpp</code>	Signed two-variable and fixed-public Walsh oracles and joint-injection Walsh support/rank relation.
<code>linear/model/*constraint. hpp</code>	Linear MILP realization, including the exact fixed-addend numerator state and exact-log epigraph constraint.
<code>linear/model/*round_ function.hpp</code>	Transpose-mask round composition, sign/phase trace, semantic no-good capture, and linear HULL reoptimization session.
<code>*/neoalzette_scip_round_ hull_search.cpp</code>	Multi-layer Forest growth, endpoint-local HULL enumeration, stopping conditions, and JSON artifacts.
<code>*/hull_multiple_thread. runner.cpp</code>	External multi-process campaign scheduling; it does not modify the local cryptanalytic models.

Part I

Engineering Implementation

1 Implemented Object

NeoAlzette is a 64-bit ARX box over a pair of 32-bit words

$$(A, B) \in \mathbb{W}_{32}^2, \quad \mathbb{W}_{32} = \mathbb{Z}/2^{32}\mathbb{Z}.$$

Modular addition and subtraction are taken modulo 2^{32} ; bitwise operators act on 32-bit words; and rotations use the C++ convention in the Core helpers, with the rotation amount interpreted modulo 32.

The construction combines an ARX/SRX skeleton with two asymmetric branch-derived Boolean injection functions. Each injection returns a pair of words. One component is XOR-injected into the opposite branch, and the other component contributes to a modular-addition operand. The surrounding round order restores the controlling branch before inversion, so the box remains reversible even though the local injection functions are not standalone permutations.

1.1 Implementation Source Boundary

The source-of-truth implementation used by this specification is:

`neozette_core.hpp`, `neozette_core.cpp`.

The header fixes the public constants, bridge rotation constants, primitive rotation helpers, diffusion-mask generators, injection function interface, and forward/backward entry points. The source file fixes the concrete injection expressions, the exact forward schedule, and the inverse schedule. The MILP backends in `differential/` and `linear/` are analysis artifacts for this implemented object; they do not define a different cipher.

This distinction is important for V6.5. The paper names the object that is implemented and analyzed. It does not infer a new design by simplifying the MILP trace, and it does not silently replace the injection layer by a smaller symbolic surrogate. When the Core says that an injection consumes the already-subtracted or already-bridged branch state, the paper follows that ordering.

2 Primitive Operations

This section fixes the mathematical meaning of every primitive operation used later in the paper. The definitions are deliberately explicit because the implementation mixes arithmetic over $\mathbb{Z}/2^{32}\mathbb{Z}$, Boolean operations over $\mathbb{F}_2^{32}$, rotations, and zero-filling shifts. These are not interchangeable operations in ARX analysis.

2.1 Word Space and Bit Numbering

Definition 2 (32-bit word model and little-endian coordinates). Let

$$\mathbb{W}_{32} = \mathbb{Z}/2^{32}\mathbb{Z}.$$

Every word $x \in \mathbb{W}_{32}$ is identified with its canonical integer representative $\text{val}(x) \in \{0, \dots, 2^{32} - 1\}$ and with its little-endian bit vector

$$x = (x_0, x_1, \dots, x_{31}) \in \mathbb{F}_2^{32}, \quad \text{val}(x) = \sum_{i=0}^{31} x_i 2^i.$$

Bit index 0 is therefore the least significant bit. All algorithms in this paper are read over 32-bit words unless a packed 64-bit joint value is explicitly named.

2.2 Boolean Word Operations

Definition 3 (Boolean word operations over $\mathbb{F}_2$). For $x, y \in \mathbb{W}_{32}$, bitwise XOR, AND, and OR are the word maps induced by the corresponding operations in $\mathbb{F}_2$:

$$(x \oplus y)_i = x_i + y_i \pmod{2}, \quad (x \wedge y)_i = x_i y_i, \quad (x \vee y)_i = x_i + y_i + x_i y_i \pmod{2}.$$

The word complement is width-limited:

$$(\neg x)_i = 1 + x_i \pmod{2}, \quad \neg x = x \oplus (2^{32} - 1).$$

Thus $\neg$ always complements exactly 32 bits. This convention matches the C++ type `std::uint32_t` used by the Core.

Equivalently, at the scalar bit level all Boolean operations are polynomial functions over the finite field $\mathbb{F}_2 = \{0, 1\}$. For $a, b \in \mathbb{F}_2$,

$$a \oplus b = a + b, \quad a \wedge b = ab, \quad a \vee b = a + b + ab, \quad \neg a = 1 + a,$$

where both addition and multiplication are the field operations of $\mathbb{F}_2$. The OR identity follows from the Boolean-ring relation

$$a \vee b = 1 + (1 + a)(1 + b) = a + b + ab.$$

Hence XOR is linear over $\mathbb{F}_2$, NOT is affine over $\mathbb{F}_2$, and AND/OR are quadratic Boolean maps over $\mathbb{F}_2$. In particular, OR is not a new algebraic primitive beyond field addition and multiplication; it is the quadratic polynomial $a + b + ab$. For word operations, these scalar polynomials are applied independently to every coordinate i , so bitwise AND and OR have algebraic degree two as maps $\mathbb{F}_2^{64} \rightarrow \mathbb{F}_2^{32}$, while bitwise NOT has algebraic degree one as an affine map $\mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$.

2.3 Modular Addition and Subtraction

Definition 4 (Modular addition and subtraction recurrences). For $x, y \in \mathbb{W}_{32}$, modular addition and subtraction are

$$x + y = (\text{val}(x) + \text{val}(y)) \bmod 2^{32}, \quad x - y = (\text{val}(x) - \text{val}(y)) \bmod 2^{32}.$$

The bit-level meaning is fixed by the carry and borrow chains. For $z = x + y \bmod 2^{32}$, set $c_0 = 0$ and, for $0 \leq i < 32$,

$$\begin{aligned} z_i &= x_i \oplus y_i \oplus c_i, \\ c_{i+1} &= (x_i \wedge y_i) \vee (x_i \wedge c_i) \vee (y_i \wedge c_i). \end{aligned}$$

The outgoing carry c_{32} is discarded by the reduction modulo 2^{32} . Thus the i -th output bit is not a function only of (x_i, y_i) ; it also depends on the complete lower-bit carry history generated by positions $0, \dots, i-1$.

For $z = x - y \bmod 2^{32}$, set $b_0 = 0$ and, for $0 \leq i < 32$,

$$\begin{aligned} z_i &= x_i \oplus y_i \oplus b_i, \\ b_{i+1} &= ((\neg x_i) \wedge y_i) \vee ((\neg x_i) \wedge b_i) \vee (y_i \wedge b_i). \end{aligned}$$

Equivalently, subtraction may be expressed as

$$x - y = x + (\neg y) + 1 \pmod{2^{32}},$$

but the paper keeps the borrow-chain notation when the value-domain operation is written as a subtraction in the Core. The outgoing borrow b_{32} is discarded by the same word-size convention as the outgoing carry.

When one operand is a public constant K , the same recurrences are used with $y_i = K_i$ fixed. This specialization is still not a linear layer: constant bits select different carry or borrow transitions, and therefore change exact differential probabilities and exact linear correlations. The MILP backends in Part II model fixed-public add/sub operations with dedicated constant-addend operators for precisely this reason. Modular addition and subtraction are therefore the main arithmetic source of differential and linear weight in the round, together with the nonlinear Boolean injection maps.

2.4 Rotations

Definition 5 (32-bit rotations). For any integer r , the implementation reduces r modulo 32. The left and right rotations are permutations of bit positions:

$$(x \lll^r)_i = x_{(i-r) \bmod 32}, \quad (x \ggg^r)_i = x_{(i+r) \bmod 32}.$$

Equivalently, in C++ word syntax,

$$x \lll^r = ((x \ll r) \vee (x \gg (32 - r))) \bmod 2^{32}$$

after reducing r modulo 32, with the special case $r = 0$ returning x . The `rotr` definition is analogous with the directions exchanged. The shifts in this expression are the zero-filling unsigned word shifts defined below, not arithmetic signed shifts. For $0 < r < 32$, the same rotation can be written directly in the integer representative as

$$\begin{aligned} \text{val}(x \lll^r) &= (2^r \text{val}(x) \bmod 2^{32}) + \left\lfloor \frac{\text{val}(x)}{2^{32-r}} \right\rfloor, \\ \text{val}(x \ggg_r) &= \left\lfloor \frac{\text{val}(x)}{2^r} \right\rfloor + 2^{32-r} (\text{val}(x) \bmod 2^r). \end{aligned}$$

The first term keeps the shifted-in-place bits, and the second term wraps the discarded end bits to the other side. Rotations are linear over $\mathbb{F}_2^{32}$ and preserve Hamming weight.

2.5 Zero-Filling Shifts

Definition 6 (Zero-filling unsigned shifts). The implementation also uses zero-filling shifts in injection pre-mix expressions. These are logical unsigned shifts on 32-bit words. They are defined for $0 \leq s < 32$; shifts by the word size or more are outside the primitive operation used in this paper and are not needed by the implemented Core.

In the integer representative, the left shift is multiplication by 2^s followed by the 32-bit word reduction:

$$\text{val}(x \ll s) = 2^s \text{val}(x) \bmod 2^{32} = \sum_{i=0}^{31-s} x_i 2^{i+s}.$$

When $s > 0$, the bits $x_{32-s}, \dots, x_{31}$ are the terms that would land at powers $2^{32}, \dots, 2^{31+s}$; they are discarded by the reduction modulo 2^{32} . For $s = 0$ no bit is discarded. Equivalently,

$$(x \ll s)_i = \begin{cases} x_{i-s}, & i \geq s, \\ 0, & i < s, \end{cases}$$

so the low s output bits are zero-filled.

The right shift is integer division by 2^s with the fractional part discarded:

$$\text{val}(x \gg s) = \left\lfloor \frac{\text{val}(x)}{2^s} \right\rfloor = \sum_{i=s}^{31} x_i 2^{i-s}.$$

The floor is required because $\text{val}(x)/2^s$ need not be an integer. The fractional part is exactly the contribution of the discarded low bits $x_0, \dots, x_{s-1}$:

$$\frac{\text{val}(x)}{2^s} = \sum_{i=s}^{31} x_i 2^{i-s} + \sum_{i=0}^{s-1} x_i 2^{i-s}.$$

The second sum lies in $[0, 1)$, and taking the floor removes it. This is the mathematical form of C++ unsigned right shift: it drops the low s bits and does not propagate a sign bit. Equivalently,

$$(x \gg s)_i = \begin{cases} x_{i+s}, & i + s < 32, \\ 0, & i + s \geq 32. \end{cases}$$

Thus the high s output bits are zero-filled.

These shifts are not rotations. In particular, $B \ll 9$ discards the top nine bits and injects zeros into the low nine positions, while $A \gg 9$ discards the low nine bits and injects zeros into the high nine positions. The distinction matters in the B -to- A and A -to- B injection bridge-state terms.

2.6 State and Joint Injection Values

Definition 7 (State and local joint-injection value). The full box state is a pair $(A, B) \in \mathbb{W}_{32}^2$. Each local injection function returns a pair $(C, D) \in \mathbb{W}_{32}^2$. In the implementation and in the MILP analysis, the two components are used differently: one component is part of a zero-weight XOR/rotation injection into the opposite branch, while a derived word from the pair contributes to a modular-addition operand. When the analysis describes a joint injection output, it packs the two 32-bit components into a 64-bit bookkeeping value only for reporting; the Core itself operates on two 32-bit words.

3 Round Constants

The Core exposes a 16-word constant array:

Index	Hex word	Source comment in implementation
0	0x16B2C40B	Fibonacci-derived prefix
1	0xC117176A	Fibonacci-derived prefix
2	0x0F9A2598	Fibonacci-derived prefix
3	0xA1563ACA	Fibonacci-derived prefix
4	0x243F6A88	hexadecimal expansion of π
5	0x85A308D3	hexadecimal expansion of π
6	0x13198A2E	hexadecimal expansion of π
7	0x03707344	hexadecimal expansion of π
8	0x9E3779B9	hexadecimal expansion of the golden ratio
9	0x7F4A7C15	hexadecimal expansion of the golden ratio
10	0xF39CC060	hexadecimal expansion of the golden ratio
11	0x5CEDC834	hexadecimal expansion of the golden ratio
12	0xB7E15162	hexadecimal expansion of e
13	0x8AED2A6A	hexadecimal expansion of e
14	0xBF715880	hexadecimal expansion of e
15	0x9CF4F3C7	hexadecimal expansion of e

The current Core transformation uses RC[0], RC[1], RC[4], RC[5], RC[6], RC[9], RC[10], and RC[11] directly in the value-domain round schedule. The injection functions also use pre-rotated constants derived from RC[2], RC[3], RC[7], RC[8], RC[12], and RC[13]. The constants RC[14] and RC[15] are present in the exported array but are not used by the current forward or backward functions.

4 Linear Diffusion-Mask Generators

This section states only the executable straight-line programs used by the implementation. The search procedure, ISD screening, circulant-matrix interpretation, inverse certificates, and differential/linear branch-number proofs are given later in Section 14.2.

4.1 Mask mask0

Algorithm 1 Engineering SLP for `generate_dynamic_diffusion_mask0`

Require: a 32-bit word x

Ensure: the left-rotation dynamic diffusion mask `mask0(x)`

- 1: $v_0 \leftarrow x$
- 2: $v_1 \leftarrow v_0 \oplus v_0 \lll 2$

```

3:  $v_2 \leftarrow v_0 \oplus v_1 \lll^{17}$ 
4:  $v_3 \leftarrow v_0 \oplus v_2 \lll^4$ 
5:  $v_4 \leftarrow v_3 \oplus v_3 \lll^{24}$ 
6: return  $v_2 \oplus v_4 \lll^7$ 

```

4.2 Mask mask1

Algorithm 2 Engineering SLP for `generate_dynamic_diffusion_mask1`

Require: a 32-bit word x

Ensure: the right-rotation dynamic diffusion mask `mask1(x)`

```

1:  $v_0 \leftarrow x$ 
2:  $v_1 \leftarrow v_0 \oplus v_0 \ggg_2$ 
3:  $v_2 \leftarrow v_0 \oplus v_1 \ggg_{17}$ 
4:  $v_3 \leftarrow v_0 \oplus v_2 \ggg_4$ 
5:  $v_4 \leftarrow v_3 \oplus v_3 \ggg_{24}$ 
6: return  $v_2 \oplus v_4 \ggg_7$ 

```

5 Cross-Branch Bridges

The implementation contains two ordered bridge locations. The first bridge is the pair

$$B \leftarrow B \oplus A \lll^{22} \oplus \text{RC}[4], \quad A \leftarrow A \oplus B \lll^{13}.$$

The second bridge is

$$A \leftarrow A \oplus B \lll^5 \oplus \text{RC}[9], \quad B \leftarrow B \oplus A \lll^{25}.$$

The first pair is the named Core pair

$$R_0 = 22, \quad R_1 = 13.$$

The implementation records the following structural screening rule:

$$\gcd((R_0 + R_1) \bmod 32, 32) = 1.$$

For the chosen pair,

$$(22 + 13) \bmod 32 = 3, \quad \gcd(3, 32) = 1.$$

The structural filter is intended to avoid a visible class of periodic cancellation subspaces in expressions of the form

$$\Delta B \oplus \Delta B \lll^{R_0+R_1}.$$

This is an empirical design filter rather than a theorem. Its role is to exclude rotation choices that repeatedly behaved poorly in automated search.

6 Injection Constants

The injection functions precompute a small set of rotated public constants:

$$\begin{aligned}
\text{RC7_R24} &= \text{RC}[7] \ggg_{24}, \\
\text{RC8_R24} &= \text{RC}[8] \ggg_{24}, \\
\text{RC13_R24} &= \text{RC}[13] \ggg_{24}, \\
\text{RC2_L8} &= \text{RC}[2] \lll^8, \\
\text{RC3_L8} &= \text{RC}[3] \lll^8, \\
\text{RC12_L8} &= \text{RC}[12] \lll^8.
\end{aligned}$$

They also precompute two constant-derived masks:

$$\text{MASK0_RC7} = \text{mask0}(\text{RC}[7]), \quad \text{MASK1_RC2} = \text{mask1}(\text{RC}[2]).$$

These precomputations do not alter the mapping; they simply cache repeated constant work.

7 Current B -to- A Injection

The B -controlled injection returns a pair (C, D) . The Core uses one expression derived from C, D as the XOR-injection term and another as the modular-add operand term. The local function itself is not required to be invertible.

Algorithm 3 Implemented B -to- A injection core

```

1: function CDINJECTIONFROMB( $B$ )
2:    $companion0 \leftarrow B \ggg_{24}$ 
3:    $mask \leftarrow \text{MASK0}(B)$ 
4:    $companion\_mask \leftarrow mask \ggg_{24} \oplus \text{MASK0\_RC7}$ 
5:    $mask\_r1 \leftarrow mask \ggg_5$ 
6:    $x_0 \leftarrow companion0 \oplus mask$ 
7:    $x_1 \leftarrow B \oplus mask$ 
8:    $view \leftarrow companion0 \oplus companion\_mask$ 
9:    $bridge\_state \leftarrow B \ggg_{19} \oplus (B \ll 9)$ 
10:   $q_{state} \leftarrow \text{RC7\_R24} \oplus \neg(B \wedge mask)$ 
11:   $q_{comp} \leftarrow companion0 \oplus B \oplus \text{RC8\_R24} \oplus \neg(companion0 \vee mask\_r1)$ 
12:   $q_{bridge} \leftarrow bridge\_state \oplus B \oplus \text{RC13\_R24} \oplus \neg(bridge\_state \wedge companion\_mask)$ 
13:   $q_{shared} \leftarrow q_{state} \oplus q_{comp}$ 
14:   $cross\_q \leftarrow (B \oplus mask\_r1) \wedge mask \oplus companion\_mask \ggg_7$ 
15:   $anti\_q \leftarrow ((x_1 \gg 3) \oplus (view \gg 5) \oplus mask\_r1) \wedge (B \oplus x_0 \ggg_{11})$ 
16:   $C \leftarrow q_{shared} \oplus q_{comp} \ggg_5 \oplus q_{comp} \ggg_{11} \oplus anti\_q$ 
17:   $D \leftarrow q_{shared} \oplus q_{state} \ggg_5 \oplus q_{bridge} \ggg_{13} \oplus cross\_q \oplus anti\_q$ 
18:  return  $(C, D)$ 
19: end function

```

8 Current A -to- B Injection

The A -controlled injection is not a literal mirror of the B -controlled one. It uses mask1 , left-rotated companion material, and a different mixture of Boolean operators.

Algorithm 4 Implemented A -to- B injection core

```

1: function CDINJECTIONFROMA( $A$ )
2:    $companion0 \leftarrow A \lll^8$ 
3:    $mask \leftarrow \text{MASK1}(A)$ 
4:    $companion\_mask \leftarrow mask \lll^8 \oplus \text{MASK1\_RC2}$ 
5:    $mask\_r1 \leftarrow mask \ggg_5$ 
6:    $x_0 \leftarrow companion0 \oplus mask$ 
7:    $x_1 \leftarrow A \oplus mask$ 
8:    $view \leftarrow companion0 \oplus companion\_mask$ 
9:    $bridge\_state \leftarrow A \lll^{19} \oplus (A \gg 9)$ 
10:   $q_{state} \leftarrow \text{RC2\_L8} \oplus \neg(A \vee mask)$ 

```

```

11:   $q_{comp} \leftarrow companion0 \oplus A \oplus RC3\_L8 \oplus \neg(companion0 \wedge mask\_r1)$ 
12:   $q_{bridge} \leftarrow bridge\_state \oplus A \oplus RC12\_L8 \oplus \neg(bridge\_state \vee companion\_mask)$ 
13:   $q_{shared} \leftarrow q_{state} \oplus q_{comp}$ 
14:   $cross\_q \leftarrow (A \oplus mask\_r1) \wedge mask \oplus companion\_mask \lll^{13}$ 
15:   $anti\_q \leftarrow ((x_1 \ll 3) \oplus (view \ll 5) \oplus mask\_r1) \vee (A \oplus x_0 \lll^{11})$ 
16:   $C \leftarrow q_{shared} \oplus q_{comp} \lll^5 \oplus q_{comp} \lll^{11} \oplus anti\_q$ 
17:   $D \leftarrow q_{shared} \oplus q_{state} \lll^5 \oplus q_{bridge} \lll^{13} \oplus cross\_q \oplus anti\_q$ 
18:  return  $(C, D)$ 
19: end function

```

9 Anti-Bypass Hybrid Schedule

The primitives and injection-layer diffusion masks above remain unchanged. This section rewrites the implemented round as the official Part I engineering description for the backup anti-bypass schedule. The goal is not to make zero trails impossible, which no deterministic 64-bit box can do, but to stop a solver from cheaply blanking the injection window and then reviving the branch after the local nonlinearity has already been skipped.

9.1 Design Background

The relevant failure mode is an internal zero-window bypass. This is not the trivial statement that an external all-zero difference maps to an all-zero output difference. It is the more specific phenomenon in which the controlling branch is forced to zero exactly at the injection point, allowing the local injection to be skipped, and is then restored only after the local work is already over. The schedule below is arranged so that any successful bypass attempt must also remain compatible with the derived modular-addition operand and with the later bridge feedback.

The intended design principle is therefore:

source branch $\rightarrow$ joint injection $\rightarrow$ XOR routing
 $\rightarrow$ derived modular addition $\rightarrow$ bridge feedback.

The injection layer is not an isolated XOR gadget. It is a joint source for both the XOR injection material and the subsequent arithmetic operand.

9.2 Inheritance from Alzette

The schedule inherits the ARX rhythm of Alzette, not its implementation economy. The important part of the Alzette spirit is the alternation of addition, rotation, XOR, and constant disturbance. NeoAlzette keeps that rhythm, but adds a fixed public subtraction at the beginning of each half-round, branch-derived injection into the opposite branch, and an injection-derived modular-addition operand. In short:

fixed subtraction $\rightarrow$ joint injection $\rightarrow$ derived addition
 $\rightarrow$ bridge feedback $\rightarrow$ mirror half-round.

9.3 Relation to the Prior Fallback Variant

This backup schedule is not a cosmetic restatement of the previous fallback version. The earlier variant let the post-addition intermediate state feed the bridge and the injection. That was still vulnerable to a low-cost source collapse: the solver could cleanly zero the relevant intermediate branch and let the local injection pass for free.

The present version reverses that dependency. The source branch is injected first, and the injection output then generates the modular-addition operand. The schedule therefore becomes:

$$\text{injection source} \rightarrow \text{XOR injection} + \text{injection-derived addend.}$$

This is the essential correction. The injection layer is no longer a local XOR-only patch that can be skipped before the arithmetic path is touched.

9.4 Schedule Overview

The first half-round follows

$$B \rightarrow J_B(B) \rightarrow A \rightarrow B \rightarrow A,$$

and the second half-round follows

$$A \rightarrow J_A(A) \rightarrow B \rightarrow A \rightarrow B.$$

Each half-round therefore consists of four coupled pieces: fixed public subtraction, joint injection, injection-derived modular addition, and bridge feedback.

9.5 Implemented Forward Transformation

The following algorithm is the forward schedule used by the Core and by the Part II MILP builders.

Algorithm 5 Implemented forward transformation

```

1: function FORWARD( $A, B$ )
2:    $B \leftarrow B - \text{RC}[1]$ 
3:    $(C_0, D_0) \leftarrow \text{CDINJECTIONFROMB}(B)$ 
4:    $CD0 \leftarrow (C_0 \ll 2) \oplus (D_0 \gg 2)$ 
5:    $CD1 \leftarrow (C_0 \gg 5) \oplus (D_0 \ll 5)$ 
6:    $A \leftarrow A \oplus B \lll^{24} \oplus C_0 \lll^{16} \oplus B \lll^8$ 
7:    $A \leftarrow A + (CD0 \lll^{31} \oplus CD1 \lll^{17} \oplus \text{RC}[0])$ 
8:    $B \leftarrow B \oplus A \lll^{22} \oplus \text{RC}[4]$ 
9:    $A \leftarrow A \oplus B \lll^{13}$ 
10:   $A \leftarrow A - \text{RC}[6]$ 
11:   $(C_1, D_1) \leftarrow \text{CDINJECTIONFROMA}(A)$ 
12:   $CD2 \leftarrow (C_1 \gg 3) \oplus (D_1 \ll 3)$ 
13:   $CD3 \leftarrow (C_1 \ll 1) \oplus (D_1 \gg 1)$ 
14:   $B \leftarrow B \oplus A \ggg_{24} \oplus D_1 \ggg_{16} \oplus A \ggg_8$ 
15:   $B \leftarrow B + (CD2 \oplus CD3 \oplus \text{RC}[5])$ 
16:   $A \leftarrow A \oplus B \lll^5 \oplus \text{RC}[9]$ 
17:   $B \leftarrow B \oplus A \lll^{25}$ 
18:   $A \leftarrow A \oplus \text{RC}[10]$ 
19:   $B \leftarrow B \oplus \text{RC}[11]$ 
20:  return ( $A, B$ )
21: end function
```

9.6 Checkpointed Arrangement

For analysis and trace validation, the same forward schedule is often written with explicit checkpoints. No linear diffusion-mask formula changes in this presentation; the masks remain inside $\text{CDINJECTIONFROMB}(\cdot)$ and $\text{CDINJECTIONFROMA}(\cdot)$.

Algorithm 6 V6.5 forward arrangement with explicit checkpoints

```

1: function FORWARDV65CHECKPOINTS( $A_0, B_0$ )
2:    $B_1 \leftarrow B_0 - \text{RC}[1]$ 
3:    $(C_0, D_0) \leftarrow \text{CDINJECTIONFROMB}(B_1)$ 
4:    $U_0 \leftarrow (C_0 \ll 2) \oplus (D_0 \gg 2)$ 
5:    $V_0 \leftarrow (C_0 \gg 5) \oplus (D_0 \ll 5)$ 
6:    $X_0 \leftarrow B_1 \lll^{24} \oplus C_0 \lll^{16} \oplus B_1 \lll^8$ 
7:    $A_1 \leftarrow A_0 \oplus X_0$ 
8:    $A_2 \leftarrow A_1 + (U_0 \lll^{31} \oplus V_0 \lll^{17} \oplus \text{RC}[0])$ 
9:    $B_2 \leftarrow B_1 \oplus A_2 \lll^{22} \oplus \text{RC}[4]$ 
10:   $A_3 \leftarrow A_2 \oplus B_2 \lll^{13}$ 
11:   $A_4 \leftarrow A_3 - \text{RC}[6]$ 
12:   $(C_1, D_1) \leftarrow \text{CDINJECTIONFROMA}(A_4)$ 
13:   $U_1 \leftarrow (C_1 \gg 3) \oplus (D_1 \ll 3)$ 
14:   $V_1 \leftarrow (C_1 \ll 1) \oplus (D_1 \gg 1)$ 
15:   $X_1 \leftarrow A_4 \ggg_{24} \oplus D_1 \ggg_{16} \oplus A_4 \ggg_8$ 
16:   $B_3 \leftarrow B_2 \oplus X_1$ 
17:   $B_4 \leftarrow B_3 + (U_1 \oplus V_1 \oplus \text{RC}[5])$ 
18:   $A_5 \leftarrow A_4 \oplus B_4 \lll^5 \oplus \text{RC}[9]$ 
19:   $B_5 \leftarrow B_4 \oplus A_5 \lll^{25}$ 
20:   $A_6 \leftarrow A_5 \oplus \text{RC}[10]$ 
21:   $B_6 \leftarrow B_5 \oplus \text{RC}[11]$ 
22:  return  $(A_6, B_6)$ 
23: end function

```

Table 2: Checkpointed V6.5 arrangement as consumed by the MILP builders.

Stage	Value-domain operation	Analysis role
0	$B_1 = B_0 - \text{RC}[1]$	fixed-public subtraction on B
1	(C_0, D_0) $\text{CDINJECTIONFROMB}(B_1)$	= joint injection oracle/support relation
2	$A_1 = A_0 \oplus X_0$	zero-weight XOR/rotation routing of the first injection output
3	$A_2 = A_1 + (U_0 \lll^{31} \oplus V_0 \lll^{17} \oplus \text{RC}[0])$	+ first source-dependent modular-addition box
4	$B_2 = B_1 \oplus A_2 \lll^{22} \oplus \text{RC}[4]$	first cross-branch feedback line
5	$A_3 = A_2 \oplus B_2 \lll^{13}$	first bridge feedback line
6	$A_4 = A_3 - \text{RC}[6]$	second fixed-public subtraction, now on A
7	(C_1, D_1) $\text{CDINJECTIONFROMA}(A_4)$	= second joint injection oracle/support relation
8	$B_3 = B_2 \oplus X_1$	zero-weight XOR/rotation routing of the second injection output
9	$B_4 = B_3 + (U_1 \oplus V_1 \oplus \text{RC}[5])$	second source-dependent modular-addition box
10	$A_5 = A_4 \oplus B_4 \lll^5 \oplus \text{RC}[9]$	final B -to- A bridge line
11	$B_5 = B_4 \oplus A_5 \lll^{25}$	final A -to- B bridge line

Stage	Value-domain operation	Analysis role
12	$A_6 = A_5 \oplus \text{RC}[10], B_6 = B_5 \oplus \text{RC}[11]$	public whitening

9.7 Implemented Backward Transformation

The inverse uses the same branch-derived injection functions. The local maps do not need to be invertible on their own, because the controlling branch has already been restored when the inverse recomputes $\text{CdInjectionFromB}(\cdot)$ or $\text{CdInjectionFromA}(\cdot)$.

Algorithm 7 Implemented backward transformation

```

1: function BACKWARD( $A, B$ )
2:    $B \leftarrow B \oplus \text{RC}[11]$ 
3:    $A \leftarrow A \oplus \text{RC}[10]$ 
4:    $B \leftarrow B \oplus A \lll^{25}$ 
5:    $A \leftarrow A \oplus B \lll^5 \oplus \text{RC}[9]$ 
6:    $(C_1, D_1) \leftarrow \text{CdInjectionFromA}(A)$ 
7:    $CD2 \leftarrow (C_1 \gg 3) \oplus (D_1 \ll 3)$ 
8:    $CD3 \leftarrow (C_1 \ll 1) \oplus (D_1 \gg 1)$ 
9:    $B \leftarrow B - (CD2 \oplus CD3 \oplus \text{RC}[5])$ 
10:   $B \leftarrow B \oplus A \ggg_{24} \oplus D_1 \ggg_{16} \oplus A \ggg_8$ 
11:   $A \leftarrow A + \text{RC}[6]$ 
12:   $A \leftarrow A \oplus B \lll^{13}$ 
13:   $B \leftarrow B \oplus A \lll^{22} \oplus \text{RC}[4]$ 
14:   $(C_0, D_0) \leftarrow \text{CdInjectionFromB}(B)$ 
15:   $CD0 \leftarrow (C_0 \ll 2) \oplus (D_0 \gg 2)$ 
16:   $CD1 \leftarrow (C_0 \gg 5) \oplus (D_0 \ll 5)$ 
17:   $A \leftarrow A - (CD0 \lll^{31} \oplus CD1 \lll^{17} \oplus \text{RC}[0])$ 
18:   $A \leftarrow A \oplus B \lll^{24} \oplus C_0 \lll^{16} \oplus B \lll^8$ 
19:   $B \leftarrow B + \text{RC}[1]$ 
20:  return ( $A, B$ )
21: end function

```

9.8 Executable Refinement and Permutation Proof

The executable specification can be proved reversible without assuming that the maps CdInjectionFromA or CdInjectionFromB are themselves invertible. The relevant structural property is triangularity.

Definition 8 (Triangular word update). A map on $\mathbb{W}_{32}^2$ is called triangular if it updates one branch by a bijection whose parameter is a function of the unchanged branch. The four forms used here are

$$\begin{aligned}
(A, B) &\mapsto (A \oplus f(B), B), & (A, B) &\mapsto (A \boxplus f(B), B), \\
(A, B) &\mapsto (A, B \oplus g(A)), & (A, B) &\mapsto (A, B \boxplus g(A)).
\end{aligned}$$

Their inverses are obtained by replacing $\oplus$ by itself and $\boxplus$ by $\boxminus$, while evaluating the same controlling-branch function.

Lemma 1 (Invertibility of the elementary updates). *Every elementary update in Algorithm 5 is a bijection on $\mathbb{W}_{32}^2$.*

Proof. Translations by public constants are permutations of $\mathbb{Z}/2^{32}\mathbb{Z}$. XOR by a fixed 32-bit word is self-inverse. For a triangular update such as $(A, B) \mapsto (A \boxplus f(B), B)$, the second coordinate is unchanged, so $f(B)$ can be recomputed from the output and subtracted from the first coordinate. The remaining three forms are identical after exchanging the branches or replacing modular addition by XOR. Rotations and zero-filling shifts occur only inside the parameter functions; they need not be invertible because those parameter functions are recomputed from the unchanged controlling branch. $\square$

Theorem 1 (Value-domain refinement of the C++ forward routine). *For every $(A, B) \in \mathbb{W}_{32}^2$, the value returned by `NeoAlzetteCore::forward` is exactly the value returned by Algorithm 5.*

Proof. The proof is a statement-by-statement simulation. Both descriptions first replace B by $B - \text{RC}[1]$. On this same post-subtraction value, both call the B -controlled injection and form $CD0 = (C_0 \ll 2) \oplus (D_0 \gg 2)$ and $CD1 = (C_0 \gg 5) \oplus (D_0 \ll 5)$. They next apply the identical XOR-routed injection to A , followed by the identical modular-addition operand derived from $CD0$, $CD1$, and $\text{RC}[0]$. The ordered bridge updates use rotations 22 and 13 in both descriptions. The second half repeats the same argument after subtracting $\text{RC}[6]$: the same A -controlled injection values generate both the XOR-routed update of B and the addend-routed update of B . The final bridge rotations 5 and 25 and the whitening constants $\text{RC}[10]$, $\text{RC}[11]$ also coincide. Because every intermediate variable is obtained by the same operation from identical preceding variables, induction over the listed statements proves identity of the final pair. $\square$

Theorem 2 (Permutation theorem and correctness of the backward routine). *The implemented forward map is a permutation of $\mathbb{W}_{32}^2$, and `NeoAlzetteCore::backward` is its two-sided inverse.*

Proof. Factor the forward map into the ordered elementary bijections

$$F = T_{11} \circ T_{10} \circ \cdots \circ T_1,$$

where T_1 is the subtraction of $\text{RC}[1]$, T_2, T_3 are the first XOR and modular-addition triangular updates, T_4, T_5 are the first bridge, T_6 is the subtraction of $\text{RC}[6]$, T_7, T_8 are the second XOR and modular-addition triangular updates, T_9, T_{10} are the second bridge, and T_{11} is the final whitening. By Lemma 1, every T_i is bijective.

Algorithm 7 executes $T_1^{-1} \circ \cdots \circ T_{11}^{-1}$, which is the inverse composition in execution order. The only subtle point is recomputation of the injection values. Before the inverse recomputes $\text{CDINJECTIONFROMA}(A)$, it has already undone the two final bridge lines, so A equals the value that controlled the forward $A \rightarrow B$ injection. Likewise, before recomputing $\text{CDINJECTIONFROMB}(B)$, it has undone the first bridge in reverse order, so B equals the value that controlled the forward $B \rightarrow A$ injection. Consequently the same XOR words and modular-addition operands are reconstructed and cancelled exactly. Hence $\text{Backward}(\text{Forward}(A, B)) = (A, B)$; the reverse identity follows because both maps are bijections on the finite state space. $\square$

Algorithm 8 Checkpoint refinement audit for the value-domain implementation

- 1: **function** AUDITCOREREFINEMENT(A_0, B_0)
- 2: execute Algorithm 6 and retain every labeled checkpoint
- 3: execute `NeoAlzetteCore::forward` with instrumentation at the same labels
- 4: **for** each checkpoint label ℓ **do**
- 5: require $(A_\ell^{\text{math}}, B_\ell^{\text{math}}) = (A_\ell^{\text{C++}}, B_\ell^{\text{C++}})$
- 6: **end for**
- 7: execute `NeoAlzetteCore::backward` on the final C++ state

```

8:   require the recovered pair to equal  $(A_0, B_0)$ 
9:   return ACCEPT
10: end function

```

9.9 Round Structure and Inversion

The round is asymmetric in two respects. First, the two injection functions are intentionally different. The B -side injection uses $B \ggg_{24}$, `mask0`, AND-heavy nonlinear terms, and the bridge state $B \ggg_{19} \oplus (B \ll 9)$. The A -side injection uses $A \lll^8$, `mask1`, a different Boolean mix, and $A \lll^{19} \oplus (A \gg 9)$. Second, the schedule is asymmetric: the first half-round consumes the subtracted B before the first bridge, while the second half-round consumes the subtracted and bridged A before the second modular addition.

This asymmetry is deliberate. It prevents a trivial zero-injection bypass in which an automated search nullifies the controlling branch around the injection window and then restores activity only after the local work is finished.

9.10 First Half-Round Engineering Structure

Writing $J_B(B) = (C_0, D_0)$, the first half-round is

$$\begin{aligned}
B_0 &= B - \text{RC}[1], \\
CD0 &= (C_0 \ll 2) \oplus (D_0 \gg 2), \\
CD1 &= (C_0 \gg 5) \oplus (D_0 \ll 5), \\
X_B &= B_0 \lll^{24} \oplus C_0 \lll^{16} \oplus B_0 \lll^8, \\
H_B &= CD0 \lll^{31} \oplus CD1 \lll^{17} \oplus \text{RC}[0], \\
A_1 &= A \oplus X_B, \\
A_2 &= A_1 + H_B, \\
B_1 &= B_0 \oplus A_2 \lll^{22} \oplus \text{RC}[4], \\
A_3 &= A_2 \oplus B_1 \lll^{13}.
\end{aligned}$$

This is not a plain five-step ARX block. It is a source-anchored joint injection followed by a derived modular-addition operand, then by a feedback bridge into the opposite branch.

9.11 Second Half-Round Engineering Structure

Writing $J_A(A) = (C_1, D_1)$, the second half-round is

$$\begin{aligned}
A_0 &= A_3 - \text{RC}[6], \\
CD2 &= (C_1 \gg 3) \oplus (D_1 \ll 3), \\
CD3 &= (C_1 \ll 1) \oplus (D_1 \gg 1), \\
X_A &= A_0 \ggg_{24} \oplus D_1 \ggg_{16} \oplus A_0 \ggg_8, \\
H_A &= CD2 \oplus CD3 \oplus \text{RC}[5], \\
B_2 &= B_1 \oplus X_A, \\
B_3 &= B_2 + H_A, \\
A_4 &= A_0 \oplus B_3 \lll^5 \oplus \text{RC}[9], \\
B_4 &= B_3 \oplus A_4 \lll^{25}.
\end{aligned}$$

The second half-round mirrors the first in direction but not in template. The bridge rotations differ, and the injection output is routed through a distinct Boolean expansion before being used as an addend.

9.12 Fixed Public Subtraction

The steps $B \leftarrow B - \text{RC}[1]$ and $A \leftarrow A - \text{RC}[6]$ are not decorative whitening. They introduce fixed borrow chains, so the bit order becomes a real coordinate anchor rather than a rotation-covariant convenience. A solver that tries to erase activity in the injection window must still remain compatible with the borrow propagation induced by the public subtraction.

This is the practical role of the fixed subtraction: it does not make zero impossible, but it makes zero non-free.

9.13 Modular Addition and Subtraction in the Model

The construction contains three arithmetic roles that must not be conflated. First, the public subtractions are fixed-addend arithmetic operations and must be modeled as such. Second, the B -to- A modular addition is source-dependent: the addend is derived from the output of `CdInjectionFromB`, not from a fixed rotation of the branch itself. Third, the A -to- B modular addition is source-dependent in the same way, but with `CdInjectionFromA` as its source.

For the first side, the model must respect

$$A \leftarrow A + (CD0 \lll^{31} \oplus CD1 \lll^{17} \oplus \text{RC}[0]),$$

with $CD0$ and $CD1$ both derived from the same injection output. For the second side, it must respect

$$B \leftarrow B + (CD2 \oplus CD3 \oplus \text{RC}[5]).$$

Neither side can be split into unrelated free variables without changing the object being analyzed.

9.14 Why the Constant Does Not Belong Only Inside the Injection

Earlier variants tried to hide more complexity inside the local injection function alone. That is not enough. If a solver can avoid entering the injection window, then a complicated local function never gets touched. The anti-bypass schedule therefore makes the injection output feed the later modular addition as well. This turns the injection layer from a local patch into a global path constraint.

9.15 Rotation Roles

The rotation budget is partitioned by role rather than by symmetry. The four bridge numbers are not hidden in the sleeves of the schedule; they are the output of an explicit engineering filter against zero-window bypass and bridge template reuse:

$$\{31, 17, 22, 13, 24, 16, 8, 5, 25\}.$$

The roles are:

Rotation set	Role
31, 17	derived modular-addition rotations on the B -to- A side
22, 13	first cross-branch feedback bridge
24, 16, 8	local injection routing and diffusion on the B -side
5, 25	second cross-branch feedback bridge

The zero-filling shifts inside `CdInjectionFromB` and `CdInjectionFromA` are local Boolean expansion operators; they are not bridge rotations.

The bridge selection logic is therefore part of the specification, not an undocumented implementation habit. The numbers are written out because the paper must show where they come from.

9.16 Bridge Selection Derivation

For the first bridge, the ordered pair is (22, 13). In code-facing form, this is

$$\text{CROSS_ROT_A_TO_B} = 22, \quad \text{CROSS_ROT_B_TO_A} = 13.$$

The key structural quantity is the bridge sum:

$$(22 + 13) \bmod 32 = 35 \bmod 32 = 3.$$

Because $32 = 2^5$, the condition

$$\gcd((R_0 + R_1) \bmod 32, 32) = 1$$

is equivalent here to asking that $(R_0 + R_1) \bmod 32$ be odd. That matters because large common divisors would leave periodic rotation subspaces in which expressions such as

$$\Delta B \oplus \Delta B \lll^{R_0 + R_1}$$

can cancel too cheaply. So $\gcd = 1$ is only the first gate, not the whole story.

The actual first-bridge screening rule is stricter:

- exclude the quarter-turn positions 8, 16, 24;
- prefer a moderate circular distance

$$\text{CircularDistance}(r_1, r_2) = \min((r_2 - r_1) \bmod 32, (r_1 - r_2) \bmod 32),$$

$$\text{with } 5 \leq \text{CircularDistance}(r_1, r_2) \leq 11;$$

- prefer one rotation in [1, 15] and the other in [17, 31];
- verify the candidate through differential and linear MILP screening;
- reject any pair that induces an obvious zero-window injection bypass.

For the chosen pair (13, 22),

$$\text{CircularDistance}(13, 22) = 9, \quad (13 + 22) \bmod 32 = 3.$$

That is why the first bridge is kept. It is not kept because the numbers look pretty; it is kept because they pass the full engineering filter.

For the second bridge, the ordered pair is (5, 25). Its basic quantities are

$$5 + 25 = 30, \quad \gcd(30, 32) = 2, \quad 25 - 5 = 20.$$

So the second bridge is a controlled secondary periodic structure, not a clone of the first bridge. Its circular distance is

$$\text{CircularDistance}(5, 25) = \min(20, 12) = 12.$$

That places it outside the short-distance entry window used for the first bridge and keeps it from repeating the first bridge's template family.

The two bridges are intentionally different:

- first bridge: strong entrance lock, $\gcd = 1$, short circular distance;
- second bridge: controlled secondary feedback, $\gcd = 2$, different template family.

The order matters. If the first bridge were also given $\gcd = 2$, the solver could start from a period-2 entrance subspace, and the later $\gcd = 1$ choice would only be repairing an already weakened state. The chosen order is therefore:

strong entrance lock $\rightarrow$ controlled secondary feedback.

The mirrored distance windows are also deliberate. The first bridge uses the short circular window 5–11; the second bridge uses the mirrored raw-distance window 20–26. These are not unrelated rules. They are the two sides of the same engineering distance policy. The pair (5, 25) sits at the conservative edge of that mirrored window, is still coprime at the individual-word level, and does not repeat the (22, 13) bridge family.

9.17 First Bridge: 22 and 13

The first bridge therefore follows directly from the derivation above. In implementation order it is

$$B \oplus = A \lll^{22} \oplus \text{RC}[4], \quad A \oplus = B \lll^{13}.$$

This is the strong entrance lock.

9.18 Second Bridge: 5 and 25

The second bridge is selected in the same visible style. In implementation order it is

$$A \oplus = B \lll^5 \oplus \text{RC}[9], \quad B \oplus = A \lll^{25}.$$

This is not a copy of the first bridge. It is a different feedback pattern for the tail half of the schedule.

The reason it is (5, 25) rather than another (22, 13)-style copy is that the second bridge is meant to be a controlled secondary periodic structure:

$$5 + 25 = 30, \quad \gcd(30, 32) = 2, \quad \text{CircularDistance}(5, 25) = 12.$$

That puts it outside the short entry window and prevents bridge-template isomorphism. In plain terms, the first bridge locks the entrance; the second bridge changes the template family instead of repeating it.

9.19 Why Not Reuse the Same Rotation Family

If the same bridge template is used twice, the solver can learn one low-cost route and reuse it. The construction therefore keeps the first bridge and the second bridge structurally distinct. The first one locks the entry more aggressively; the second one is a different feedback pattern that prevents the two halves of the schedule from collapsing into the same trail family.

The design rule is therefore:

- first bridge: strong entry lock, $\gcd = 1$, short circular distance, and full screening against obvious periodic cancellation;
- second bridge: controlled secondary feedback, $\gcd = 2$, different template family, and no reuse of the first bridge's low-cost trail shape.

9.20 Joint Injection Semantics

The local injection maps are coupled objects. Each call returns a pair in $\mathbb{W}_{32}^2$, and the two components are consumed by two different downstream operations. One component contributes to the zero-weight XOR routing, while a derived component contributes to the modular-addition operand. The injection should therefore be modeled as a joint oracle, not as two independent free variables.

In the pseudocode, the names `CdInjectionFromB` and `CdInjectionFromA` are intentionally preserved. In trace and MILP tooling, the recommended checkpoints are:

- `JOINT_INJECTION_B_TO_A;`
- `ADD0_A_plus_JB_derived_addend;`
- `BRIDGE0_A_TO_B;`
- `BRIDGE1_B_TO_A;`
- `JOINT_INJECTION_A_TO_B;`
- `ADD1_B_plus_JA_derived_addend;`
- `LINEAR_B_TO_A_ROT5;`
- `FINAL_BRIDGE_A_TO_B_ROT25.`

9.21 Threat Model

The schedule is designed against the following solver behaviors:

1. injection-window zeroing, where the controlling branch is forced to zero at the injection point;
2. post-injection reactivation, where the branch is restored only after the local nonlinearity has been skipped;
3. addend cancellation, where a source-dependent modular addition is made cheap by manipulating the source branch;
4. zero-mask corridors, where a stretch of state is made to look free to the linear or differential model;
5. bridge-template reuse, where both feedback bridges collapse into the same trail template;
6. XOR-constant irrelevance, where public constants are modeled as if they carried no structural cost;
7. model mismatch, where implementation, MILP, and trace use different state orders or different derived-addend definitions.

9.22 MILP, SAT, and Trace Modeling Requirements

The differential and linear models must both treat the injection as a coupled source. They must not split `CdInjectionFromB` and the derived addend, or `CdInjectionFromA` and the derived addend, into unrelated variables. The trace layer should report the consistency flags

- `joint_oracle_valid;`

- `joint_oracle_weight_consistent;`
- `derived_addend_matches_injection_output;`
- `paper_usable_characteristic.`

These flags should be true whenever the schedule is being used as intended.

9.23 Code Synchronization Requirements

The implementation, MILP builders, trace emitters, and paper text must all use the same schedule:

Item	Required value
Cross-bridge rotations	22 and 13
Second feedback bridge	5 and 25
B-side derived addend	$CD0 \lll^{31} \oplus CD1 \lll^{17} \oplus RC[0]$
A-side derived addend	$CD2 \oplus CD3 \oplus RC[5]$

Any mismatch among `neoalzette_core.cpp`, `neoalzette_core.hpp`, the MILP construction files, and the trace tool would mean that different algorithms are being measured under one name.

9.24 Design Summary

The essential point is simple. The injection layer no longer behaves as an isolated XOR patch. It feeds both the XOR bridge and the modular-addition operand. The fixed subtraction anchors the bit order through borrow propagation, and the two bridges prevent the two halves of the state from remaining cleanly separable. This is the anti-bypass reading of the implemented round, and it is the reading that should be used in the paper text and in the corresponding MILP/trace tooling.

Part II

Resource-Limited MILP Analysis

MILP is short for Mixed-Integer Linear Programming, or mixed-integer linear programming. A MILP problem optimizes a linear objective function under a set of linear equality and inequality constraints, while some variables are required to take integer or binary values and the remaining variables may be continuous.

In this work, MILP is used as a constraint language for cryptanalytic search. Bit variables, carry states, mask choices, and selector variables are encoded as binary variables. Algebraic or bit-level rules are then translated into linear constraints. A feasible MILP solution corresponds to a valid differential or linear propagation pattern, and the objective value represents the accumulated weight of that pattern.

Thus, when we say that a differential or linear propagation rule is compiled into MILP constraints, we mean that every local Boolean or arithmetic condition is replaced by an equivalent set of linear equalities and inequalities over binary, integer, or continuous variables.

10 Modular Addition and Subtraction Oracles for MILP Analysis

This part has a deliberately local scope. It defines the arithmetic evaluators and MILP encodings used by the resource-limited search model for the maps

$$C = F_+(A, B) = A + B \pmod{2^{Bit}}, \quad C = F_-(A, B) = A - B \pmod{2^{Bit}},$$

and for their fixed-addend variants

$$Y = F_{+K}(X) = X + K \pmod{2^{Bit}}, \quad Y = F_{-K}(X) = X - K \pmod{2^{Bit}},$$

all modulo 2^N . These objects are not distinguishers by themselves. They are local ingredients from which trails, characteristics, hulls, and finally statistical tests may be built. Throughout this part, the word size is N , word differences are XOR differences, and linear masks are Boolean masks with the usual dot product over $\mathbb{F}_2$.

The mathematical sources are kept in separate compartments. For sampled two-input addition, the differential oracle uses the Lipmaa–Moriai characterization, while the linear oracle uses Wallen’s correlation formula [LM02, Wal03a]. For fixed public addends, the relevant issue is addend dependence: the constant bits are not random inputs. That line follows Miyano, Machado, and the fixed-constant bit-vector models of Azimi et al. [Miy98, Mac01, ARS⁺20, ARS⁺22]. The MILP layer is a further encoding layer: it makes these local rules visible to a global solver in the same spirit as automatic trail-search work for ARX designs [FWG⁺16, LWR16].

10.1 Terminology Before the Local Boxes

Definition 9 (Local oracle hierarchy and search objects). The local boxes below sit inside a larger analysis stack. We keep the stack separated into mathematical levels, because each level answers a different question.

Level 1: one local transition. Let M be one local word operation, and let X denote all of its random local operands. A differential point value has the form

$$\text{DP}_M(\Delta_{\text{in}} \rightarrow \Delta_{\text{out}}) = \Pr_X[M(X) \oplus M(X \oplus \Delta_{\text{in}}) = \Delta_{\text{out}}].$$

A linear point value has the form

$$\text{Cor}_M(a; b) = 2^{-\dim X} \sum_X (-1)^{a \cdot M(X) \oplus b \cdot X}.$$

For multi-input maps, X , Δ_{in} , and b are tuples of local operands, differences, and masks. This is the level of a Q_1 oracle. Given all local parameters, Q_1 returns whether the point is possible or nonzero and, if so, returns the local probability, signed correlation, sign, or logarithmic weight. It does not choose a missing difference or mask.

Level 2: one route through several local transitions. A differential trail is a fully specified route

$$\Delta X_0 \rightarrow \Delta X_1 \rightarrow \cdots \rightarrow \Delta X_r.$$

It specifies every intermediate difference, not only the endpoints. If the local transition probabilities are $p_0, \dots, p_{r-1}$, then the usual single-trail estimate is

$$p_{\text{trail}} = p_0 p_1 \cdots p_{r-1}.$$

With $w_i = -\log_2 p_i$,

$$W_{\text{trail}} = \sum_{i=0}^{r-1} w_i, \quad p_{\text{trail}} = 2^{-W_{\text{trail}}}.$$

The linear analogue is a linear characteristic

$$\Gamma_0 \rightarrow \Gamma_1 \rightarrow \cdots \rightarrow \Gamma_r,$$

where every intermediate mask is fixed. If the local correlations are $C_0, \dots, C_{r-1}$, then one characteristic contributes

$$C_{\text{char}} = \text{sgn} \cdot |C_0| |C_1| \cdots |C_{r-1}|.$$

The local linear weight is $w_i = -\log_2 |C_i|$, so

$$|C_{\text{char}}| = 2^{-\sum_i w_i}.$$

The sign is part of the mathematical object. A characteristic of weight W may contribute $+2^{-W}$ or -2^{-W} .

Level 3: endpoint aggregation. A hull fixes endpoints and aggregates all compatible routes between them. A differential hull aggregates trail probabilities for fixed input and output differences. A linear hull aggregates signed characteristic contributions:

$$C_{\text{hull}} = \sum_j \text{sgn}_j 2^{-W_j}.$$

Thus the best single linear characteristic is not the same object as the full linear hull. Two low-weight characteristics may cancel if their signs are opposite.

The correlation of a Boolean linear approximation is

$$C = 2 \Pr[A \cdot X \oplus B \cdot Y = 0] - 1.$$

If

$$\Pr[A \cdot X \oplus B \cdot Y = 0] = \frac{1}{2} + \epsilon,$$

then $C = 2\epsilon$. The bias ϵ measures the excess over random guessing; the signed correlation C is the value whose magnitude gives the standard weight $w = -\log_2 |C|$.

Level 4: search objects and solver encodings. We write examples of Q_1 oracles as

$$Q_1^{\text{DP},+}(\alpha, \beta, \gamma), \quad Q_1^{\text{DP},+\text{K}}(K, u, v),$$

and

$$Q_1^{\text{LC},+}(u, v, w), \quad Q_1^{\text{LC},+\text{K}}(K, \alpha, \beta).$$

A Q_2 object is different: it is a local completion or candidate-generation problem. For example,

$$Q_2^{\text{DP}}(\alpha, \beta) = \arg \min_{\gamma} Q_1^{\text{DP},+}(\alpha, \beta, \gamma). \text{weight}.$$

This expression hides an actual search problem: candidate generation, pruning, state compression, equivalence handling, caching, and proof of optimality. This paper therefore opens the Q_1 arithmetic oracles and the MILP boxes, while Q_2 is used only as a named role in the larger search stack.

A MILP box is not an oracle call. It replaces one local rule by binary variables, integer auxiliaries, linear constraints, and an objective contribution. MILP is convenient when the

search objective is a trail or characteristic weight. SAT asks only satisfiability of Boolean constraints; SMT, especially bit-vector SMT, can write the arithmetic semantics closer to the program. Here the emphasis is MILP because the local arithmetic cost must become part of a global linear objective.

Branch and bound is a global wrapper around routes. For a prefix of weight W_{prefix} and a current incumbent W_{best} , a branch can be discarded when

$$W_{\text{prefix}} \geq W_{\text{best}}.$$

With a sound remaining-round lower bound $LB_{\text{remaining}}$, it can also be discarded when

$$W_{\text{prefix}} + LB_{\text{remaining}} \geq W_{\text{best}}.$$

The explored space must cover the claimed target space, and the lower bound must never cut off a true optimum. The carry-dependent DDT/CLAT branch-and-bound line of Liu–Li–Jiao–Wang [LLJW] fits this pattern: extend a route, query local arithmetic information, order candidates, and prune only by valid bounds.

Level 5: statements of zero. Three zero-like statements have different meanings:

$DP = 0$	the differential transition is impossible,
$C = 0$	the signed linear correlation vanishes,
MILP/SAT/SMT infeasible	the written constraint system has no assignment.

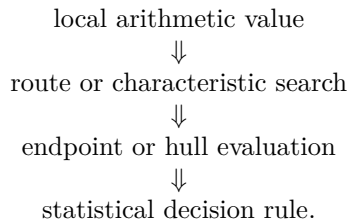
The last statement is a statement about the model, and inherits every correctness and completeness assumption of that model. In particular, an exact fixed-addend linear oracle for $Y = X + K$ returns the signed correlation

$$C_K(\alpha, \beta) = 2^{-N} \sum_X (-1)^{\alpha \cdot X \oplus \beta \cdot (X+K)}.$$

A single-characteristic MILP box may instead price one carry characteristic. Whenever a box below is a characteristic box rather than an exact signed-correlation oracle, the text states this explicitly.

10.2 From Local Oracles to Distinguishers

Definition 10 (Endpoint statistics and distinguisher boundary). The output of a local oracle is not yet a distinguisher. It becomes useful only after four increasingly global choices have been made:



The first line is where Q_1 lives. It answers a local question such as “is this addition transition possible?” or “what is this local signed correlation?” The second line chooses compatible local transitions across rounds. The third line decides whether one route is being reported or whether many endpoint-compatible routes are being aggregated. The last line is the actual distinguisher against a random permutation or ideal model.

For a permutation E on n -bit states, a differential endpoint statistic has the form

$$\text{DP}_E(\Delta \rightarrow \nabla) = \Pr_X[E(X) \oplus E(X \oplus \Delta) = \nabla].$$

A differential distinguisher uses this value only after comparing it with the random-permutation baseline and after accounting for data, time, and success probability. An impossible differential is the special endpoint statement

$$\text{DP}_E(\Delta \rightarrow \nabla) = 0.$$

The linear endpoint statistic is

$$\text{Cor}_E(a; b) = 2^{-n} \sum_X (-1)^{a \cdot X \oplus b \cdot E(X)}.$$

A linear distinguisher uses a nonrandom value of this signed correlation. A zero-correlation distinguisher uses the endpoint statement

$$\text{Cor}_E(a; b) = 0,$$

which is different from saying that no characteristic exists. It may result from cancellation among several nonzero signed contributions.

Mixed constructions, such as differential-linear distinguishers, add one more interface between the two sides. One may split the cipher into an early differential part and a later linear part,

$$\Delta X_0 \rightarrow \Delta X_a, \quad \Gamma_a \rightarrow \Gamma_r,$$

and then impose a compatibility condition at the middle state. The local addition and subtraction oracles supply only the local propagation values used inside this larger computation. They do not choose the split, prove the middle compatibility, generate data, or perform the final hypothesis test.

This is also why a good local candidate is not an answer. A Q_2 completion, a low-weight local transition, or a promising branch-and-bound prefix merely identifies something worth extending. It becomes a certified trail or characteristic only after the full path constraints and objective accounting have been checked. It becomes a distinguisher only after the corresponding endpoint statistic and decision rule have been specified.

10.3 Why Modular Addition and Subtraction Are Isolated Here

Theorem 3 (Separation of sampled and fixed-addend arithmetic). *The ARX round operations have unequal mathematical roles. XOR and rotation are linear over $\mathbb{F}_2^N$. They transport differences and masks without creating a nonlinear local probability table. Addition and subtraction modulo 2^N are different: their carry and borrow chains couple neighboring bits, and this coupling is where the local differential and linear costs arise.*

Two-input and fixed-addend arithmetic are also different objects. In

$$F_+(A, B) = A + B,$$

both operands are sampled. A two-variable differential oracle averages over (A, B) , and a two-variable linear oracle sums over (A, B) . In

$$F_{+K}(X) = X + K,$$

only X is sampled; the bits of K are public parameters that shape the carry chain. Thus F_{+K} is not obtained by silently setting the second input difference of F_+ to zero. The

probability space and the local state recurrence have changed. This is the addend-dependency issue treated by Miyano, Machado, and Azimi et al. [Miy98, Mac01, ARS⁺20, ARS⁺22].

The same separation is used for subtraction. Two-variable subtraction can be related to two-variable addition by rewriting

$$C = A - B \pmod{2^N} \iff A = C + B \pmod{2^N}.$$

Fixed subtraction by a public constant has its own borrow chain controlled by the constant bits. The following sections therefore keep four local families separate:

$$\begin{array}{ll} F_+, F_- & \text{sampled two-input arithmetic,} \\ F_{+K}, F_{-K} & \text{fixed-addend arithmetic.} \end{array}$$

This separation is a modeling choice, not a new design claim. It prevents the analysis from mixing sampled operands, public constants, exact endpoint correlations, and single-characteristic search boxes into one object.

10.4 Two-Variable Addition and Subtraction Oracles

Terminology and Search Role Before the Local Oracles

Definition 11 (Sampled two-variable arithmetic Q_1 contracts). This section has a narrower scope than the terminology above. It treats only the sampled two-input maps

$$F_+(A, B) = A + B, \quad F_-(A, B) = A - B \pmod{2^N},$$

where both operands are averaged or summed over. No fixed public addend is present in this section.

For the differential oracle, the local parameters are the XOR differences

$$\alpha \text{ on } A, \quad \beta \text{ on } B, \quad \gamma \text{ on } C.$$

The point oracle

$$Q_1^{\text{DP},+}(\alpha, \beta, \gamma)$$

evaluates exactly one local transition $(\alpha, \beta) \rightarrow \gamma$. It does not choose γ , and it does not search across several additions.

For the linear oracle, the local parameters are masks

$$u \text{ on } C, \quad v \text{ on } A, \quad w \text{ on } B.$$

The point oracle

$$Q_1^{\text{LC},+}(u, v, w)$$

evaluates the signed correlation of exactly this mask triple. Its output is not a linear hull and not a distinguisher; it is a local signed number or a zero-correlation local point.

The two subtraction oracles are not new mathematical laws. They are obtained from the identity

$$C = A - B \iff A = C + B \pmod{2^N},$$

so subtraction is represented by the addition oracle with permuted local variables. The MILP boxes introduced later encode these same local relations as constraints and objective terms for global trail or characteristic search.

10.5 Two-Variable Differential Q_1 : Lipmaa–Moriai 2002 Point Oracle

The local probability being evaluated.

Definition 12 (Two-variable addition differential point probability). For the two-variable addition

$$C = A + B \pmod{2^N},$$

let α, β, γ be the XOR differences on A, B, C , respectively. The local differential probability is

$$\text{DP}_+(\alpha, \beta \rightarrow \gamma) = \Pr_{A,B} [(A + B) \oplus ((A \oplus \alpha) + (B \oplus \beta)) = \gamma].$$

The probability space is the uniform choice of the two sampled operands (A, B) . No fixed addend and no trail aggregation is involved. The point-probability characterization used here is the 2002 result of Helger Lipmaa and Shiho Moriai for two-input modular addition [LM02].

Implementation-level input normalization.

Definition 13 (Truncated implementation domain for the Lipmaa–Moriai oracle). The implementation of this Q_1 oracle is the following pair in the differential-probability weight-evaluation header:

<code>xdp_add_lm2001</code>	$N = 32$ specialization,
<code>xdp_add_lm2001_n</code>	explicit- N mathematical contract used below.

The second function chooses

$$\mathcal{M}_N = \begin{cases} 2^{32} - 1, & N = 32, \\ 2^N - 1, & 1 \leq N < 32, \end{cases}$$

and truncate the three input words before every test:

$$\alpha \leftarrow \alpha \wedge \mathcal{M}_N, \quad \beta \leftarrow \beta \wedge \mathcal{M}_N, \quad \gamma \leftarrow \gamma \wedge \mathcal{M}_N.$$

The left-shift used by the code is therefore not an abstract infinite-word shift. It is

$$S_N(x) = ((x \ll 1) \wedge \mathcal{M}_N), \quad (S_N(x))_0 = 0, \quad (S_N(x))_i = x_{i-1} \ (i \geq 1).$$

The low charged mask is

$$\mathcal{L}_N = \begin{cases} 2^{31} - 1, & N = 32, \\ 2^{N-1} - 1, & 1 \leq N < 32. \end{cases}$$

This is the code's `mask_lower`. It deliberately removes bit $N - 1$, because the carry leaving the most significant bit is discarded modulo 2^N .

The equality mask used by the implementation.

Definition 14 (Lipmaa–Moriai equality and impossibility masks). All packed-word operations in this paragraph are truncated to N bits. The implementation in `weight_evaluation.hpp` writes the Lipmaa–Moriai equality test as the mask

$$\psi_N(\alpha, \beta, \gamma) = ((\mathcal{M}_N \oplus \alpha) \oplus \beta) \wedge ((\mathcal{M}_N \oplus \alpha) \oplus \gamma),$$

followed by truncation to $\mathcal{M}_N$. This is exactly `psi_with_mask`: complement first inside the active word, then XOR, then AND. Bitwise,

$$\psi_N(\alpha, \beta, \gamma)_i = 1 \iff \alpha_i = \beta_i = \gamma_i.$$

Thus $\overline{\psi_N(\alpha, \beta, \gamma)}$ marks the bit positions where the three local differences are not all equal.

Let

$$S(x) = \text{trunc}_N(x \ll 1), \quad t = \alpha \oplus \beta \oplus \gamma.$$

The bit-parallel impossible-differential mask used by the code is obtained by the following named temporary values:

$$\begin{aligned} \alpha_s &= S_N(\alpha), \\ \beta_s &= S_N(\beta), \\ \gamma_s &= S_N(\gamma), \\ \psi_s &= \psi_N(\alpha_s, \beta_s, \gamma_s), \\ t &= (\alpha \oplus \beta \oplus \gamma) \wedge \mathcal{M}_N, \\ \chi &= (t \oplus \beta_s) \wedge \mathcal{M}_N. \end{aligned}$$

Thus

$$B_{\text{bad}} = \psi_s \wedge \chi.$$

The transition is impossible exactly when

$$B_{\text{bad}} \neq 0.$$

Equivalently, the implementation returns `INFINITE_WEIGHT` as soon as one bit position satisfies this mask. Using the virtual boundary

$$\alpha_{-1} = \beta_{-1} = \gamma_{-1} = 0,$$

the i -th bit of the bad mask is

$$(B_{\text{bad}})_i = [\alpha_{i-1} = \beta_{i-1} = \gamma_{i-1}] \cdot [\alpha_i \oplus \beta_i \oplus \gamma_i \neq \beta_{i-1}].$$

Thus the implementation checks a family of N adjacent-bit implications, but it checks all of them at once by one shifted ψ -mask and one XOR condition mask.

Connection with the carry recurrence.

Lemma 2 (Packed bad-mask equivalence with the carry recurrence). *The previous display is the compact packed-word form of the local carry condition. To see the bit meaning, set*

$$\alpha_{-1} = \beta_{-1} = \gamma_{-1} = 0$$

and let r_i, r'_i be the carries produced at bit i by the two additions $A+B$ and $(A \oplus \alpha) + (B \oplus \beta)$, with $r_{-1} = r'_{-1} = 0$. At bit i ,

$$\begin{aligned} C_i &= A_i \oplus B_i \oplus r_{i-1}, \\ C'_i &= (A_i \oplus \alpha_i) \oplus (B_i \oplus \beta_i) \oplus r'_{i-1}. \end{aligned}$$

Thus $C_i \oplus C'_i = \gamma_i$ is equivalent to

$$r_{i-1} \oplus r'_{i-1} = \alpha_i \oplus \beta_i \oplus \gamma_i = t_i.$$

When $(\alpha_{i-1}, \beta_{i-1}, \gamma_{i-1})$ is 000, the entering carry-difference value is forced to 0; when it is 111, it is forced to 1. In the other six previous triples, both carry-difference values occur with equal multiplicity. Therefore the packed condition $B_{\text{bad}} = 0$ is equivalent to

$$\forall i \in \{0, \dots, N-1\}, \quad \alpha_{i-1} = \beta_{i-1} = \gamma_{i-1} \implies t_i = \beta_{i-1}.$$

Since the premise gives $\alpha_{i-1} = \beta_{i-1} = \gamma_{i-1}$, the right side may equivalently be written as α_{i-1} or γ_{i-1} . For $i = 0$, this forces $t_0 = 0$.

The same condition can be read as the following local table for the previous triple $(\alpha_{i-1}, \beta_{i-1}, \gamma_{i-1})$:

previous triple	required value of t_i	reason in the code
000	0	$\psi_{si} = 1, \beta_{i-1} = 0,$
111	1	$\psi_{si} = 1, \beta_{i-1} = 1,$
001, 010, 011, 100, 101, 110	no restriction	$\psi_{si} = 0.$

This is the exact content of the expression $\psi_s \wedge (t \oplus \beta_s)$. The mask ψ_s chooses only the two equal previous triples, and the XOR word $t \oplus \beta_s$ checks whether the current parity disagrees with the forced entering carry-difference value.

Probability and weight.

Theorem 4 (Lipmaa–Moriai probability and weight formula). *If the Lipmaa–Moriai condition fails, the local transition is impossible. If it holds, the code computes*

$$\psi_0 = \psi_N(\alpha, \beta, \gamma), \quad E = (\mathcal{M}_N \oplus \psi_0) \wedge \mathcal{L}_N,$$

and returns

$$W_+^{\text{diff}}(\alpha, \beta, \gamma) = \text{wt}(E).$$

The corresponding probability is

$$\text{DP}_+(\alpha, \beta \rightarrow \gamma) = 2^{-W_+^{\text{diff}}(\alpha, \beta, \gamma)}.$$

The temporary word E is the implementation's *masked_bad*; it is “bad” only for weight charging, not for impossibility. The actual impossible mask is B_{bad} above. Only bits $0, \dots, N-2$ are charged because the carry leaving bit $N-1$ is discarded modulo 2^N . Equivalently, the weight is the number of positions $i < N-1$ for which

$$(\alpha_i, \beta_i, \gamma_i) \notin \{(0, 0, 0), (1, 1, 1)\}.$$

Thus the weight bit at a charged position is

$(\alpha_i, \beta_i, \gamma_i)$	E_i
000, 111	0,
001, 010, 011, 100, 101, 110	1.

There is no dependence on (A_i, B_i) left at this stage: the averaging over the four input bit pairs has already collapsed into the Lipmaa–Moriai equal-triple test.

Proof. By Lemma 2, $B_{\text{bad}} \neq 0$ is exactly the failure of the Lipmaa–Moriai carry condition, so no carry-difference sequence realizes the transition. When the condition holds, the Lipmaa–Moriai formula charges precisely the bit positions $i < N-1$ at which the triple $(\alpha_i, \beta_i, \gamma_i)$ is not equal. Since $\psi_N(\alpha, \beta, \gamma)_i = 1$ exactly for the two equal triples 000 and 111, the word $(\mathcal{M}_N \oplus \psi_N(\alpha, \beta, \gamma)) \wedge \mathcal{L}_N$ is exactly the characteristic word of those charged positions. Its Hamming weight is the negative base-two logarithm of the local probability. $\square$

Q_1 Helger Lipmaa and Shiho Moriai 's Algorithm

Theorem 5 (Two-variable addition differential Q_1 oracle). *The two-variable addition differential oracle is therefore*

$$Q_1^{\text{DP},+}(\alpha, \beta, \gamma) = \begin{cases} (\text{false}, +\infty), & \text{if } B_{\text{bad}} \neq 0, \\ (\text{true}, W_+^{\text{diff}}(\alpha, \beta, \gamma)), & \text{otherwise.} \end{cases}$$

There is no hidden search in this definition. This is the point evaluator implemented by `xdp_add_lm2001` and `xdp_add_lm2001_n`. The API exposes the same value in three small wrappers:

<code>xdp_add_lm2001(alpha, beta, gamma)</code>	$N = 32$, return W or $INFINITE_WEIGHT$,
<code>xdp_add_lm2001_n(alpha, beta, gamma, N)</code>	$1 \leq N \leq 32$, return W or $INFINITE_WEIGHT$,
<code>xdp_add_probability(alpha, beta, gamma)</code>	return 0 if impossible, otherwise 2^{-W} .

The boolean wrapper `is_xdp_add_possible` is the same query with only the comparison between W and $INFINITE_WEIGHT$ retained.

All reference algorithms in this section use the following explicit helper functions. There is no implementation-dependent meaning hidden behind these names. For any integer word x ,

$$\text{mask}_N = 2^N - 1, \quad \text{trunc}_N(x) = x \bmod 2^N.$$

The bit reader is

$$\text{bit}_N(x, i) = \begin{cases} \lfloor \text{trunc}_N(x) / 2^i \rfloor \bmod 2, & 0 \leq i < N, \\ 0, & \text{otherwise.} \end{cases}$$

Thus out-of-range indices are always zero. This convention is used for virtual bits such as $\alpha_{-1}, \beta_{-1}, \gamma_{-1}$ in the two-variable formulas. The Hamming weight and the equality mask are

$$\text{wt}_N(x) = \sum_{i=0}^{N-1} \text{bit}_N(x, i),$$

and

$$\text{eq}_N(x, y) = \text{mask}_N \oplus \text{trunc}_N(x \oplus y),$$

so the i -th bit of $\text{eq}_N(x, y)$ is one exactly when $x_i = y_i$. In pseudocode, the names `bit(x, i)`, `mask(N)`, and `hamming_weight(x)` mean exactly $\text{bit}_N(x, i)$, mask_N , and $\text{wt}_N(x)$, with every word truncated to N bits before complements are used.

The reference implementation can be written without enumerating A, B :

Algorithm 9 Two-variable modular-addition differential oracle

```

1: function PSI( $a, b, c, N$ )
2:    $a \leftarrow a \wedge \text{MASK}(N)$ ,  $b \leftarrow b \wedge \text{MASK}(N)$ ,  $c \leftarrow c \wedge \text{MASK}(N)$ 
3:    $\bar{a} \leftarrow \text{MASK}(N) \oplus a$ 
4:   return  $((\bar{a} \oplus b) \wedge (\bar{a} \oplus c)) \wedge \text{MASK}(N)$ 
5: end function

6: function ADD2DIFFORACLE( $\alpha, \beta, \gamma, N$ )
7:    $\alpha \leftarrow \alpha \wedge \text{MASK}(N)$ ,  $\beta \leftarrow \beta \wedge \text{MASK}(N)$ ,  $\gamma \leftarrow \gamma \wedge \text{MASK}(N)$ 
8:    $\alpha_s \leftarrow (\alpha \ll 1) \wedge \text{MASK}(N)$ 
9:    $\beta_s \leftarrow (\beta \ll 1) \wedge \text{MASK}(N)$ 
10:   $\gamma_s \leftarrow (\gamma \ll 1) \wedge \text{MASK}(N)$ 
11:   $\psi_s \leftarrow \text{PSI}(\alpha_s, \beta_s, \gamma_s, N)$ 
12:   $t \leftarrow (\alpha \oplus \beta \oplus \gamma) \wedge \text{MASK}(N)$ 
13:   $bad \leftarrow \psi_s \wedge (t \oplus \beta_s)$ 
14:  if  $bad \neq 0$  then
15:    return (false,  $+\infty$ )
16:  end if
17:   $\psi \leftarrow \text{PSI}(\alpha, \beta, \gamma, N)$ 
18:   $low \leftarrow \text{MASK}(N - 1)$ 

```

```

19:    $neq \leftarrow \text{MASK}(N) \oplus \psi$ 
20:    $charged \leftarrow neq \wedge low$ 
21:    $W \leftarrow \text{HAMMINGWEIGHT}(charged)$ 
22:   return (true,  $W$ )
23: end function

```

10.6 Two-Variable Differential Q_2 : Lipmaa–Moriai 2002 Optimal- γ Completion

Definition 15 (Two-variable differential Q_2 completion contract). The file `differential_probability/optimal_gamma` implements Lipmaa–Moriai Algorithm 4 [LM02]. This is the second local differential service: it is a completion oracle, not a point probability oracle. Its input is only (α, β) , and its first output is one maximizer

$$\gamma^*(\alpha, \beta) \in \arg \max_{\gamma} \text{DP}_+(\alpha, \beta \rightarrow \gamma).$$

When the code needs the corresponding weight, the convenience wrapper

`find_optimal_gamma_with_weight`

immediately feeds that γ^* back into Q_1 :

$$W^*(\alpha, \beta) = W_+^{\text{diff}}(\alpha, \beta, \gamma^*).$$

Thus the local Q_2 contract is

$$Q_2^{\text{DP},+}(\alpha, \beta) = (\gamma^*(\alpha, \beta), W^*(\alpha, \beta)).$$

The implementation separates the two responsibilities exactly as this display does. The template function `find_optimal_gamma<WordT>` constructs the word γ^* for 32-, 64-, or 128-bit words. The wrapper `find_optimal_gamma_with_weight` is currently the 32-bit convenience path because the visible Q_1 backend in `weight_evaluation.hpp` exports 32-bit weight functions.

This is not the Q_1 point evaluator above. It is a local completion helper of the form

$$(\alpha, \beta) \mapsto \gamma^*,$$

after which the weight is still obtained by querying

$$Q_1^{\text{DP},+}(\alpha, \beta, \gamma^*).$$

Construction 1 (Lipmaa–Moriai optimal- γ implementation). All operations in the following construction are N -bit operations. The first helper layer in the file fixes the active word size. If the machine word has $D \in \{32, 64, 128\}$ bits, then

$$\text{lowmask}_D(N) = \begin{cases} 0, & N \leq 0, \\ 2^D - 1, & N \geq D, \\ 2^N - 1, & 0 < N < D. \end{cases}$$

This is `low_mask<WordT>(N)`; it avoids undefined shifts such as $1 \ll 64$ or $1 \ll 128$. Write $\mathcal{M}_N = \text{lowmask}_D(N)$. The inputs are first truncated:

$$\alpha \leftarrow \alpha \wedge \mathcal{M}_N, \quad \beta \leftarrow \beta \wedge \mathcal{M}_N.$$

The file also exposes two audit helpers that are separate from the public γ^* construction. First,

$$\text{carry_aop}(\alpha, \beta, \gamma) = \alpha \oplus \beta \oplus \gamma \oplus ((\alpha \wedge \beta) \oplus ((\alpha \oplus \beta) \wedge \gamma)) \ll 1,$$

again truncated to N bits. It marks positions where the carry state can affect the output XOR difference.

Second, $\text{aop}(x)$ marks positions whose one-run, starting at that same position and moving upward, has odd length:

$$\text{aop}(x)_i = 1 \iff x_i = x_{i+1} = \dots = x_{i+\ell-1} = 1, \ x_{i+\ell} = 0, \text{ and } \ell \text{ is odd.}$$

The generic implementation builds this in $\Theta(\log N)$ stages. With $x^{(0)} = x$ and

$$x^{(1)} = x \wedge (x \gg 1), \quad x^{(j)} = x^{(j-1)} \wedge (x^{(j-1)} \gg 2^{j-1}),$$

the auxiliary words $y^{(j)}$ are initialized by

$$y^{(0)} = 0, \quad y^{(1)} = x \wedge \overline{x^{(1)}},$$

and updated by

$$y^{(j)} = y^{(j-1)} \vee \left((y^{(j-1)} \gg 2^{j-1}) \wedge x^{(j-1)} \right).$$

The returned word is $y^{(\lceil \log_2 N \rceil)}$. This explains the `x_arr/y_arr` structure in the implementation. The bit-reversal helpers in the same file have the following role. The functions `bitreverse32`, `bitreverse64`, and `bitreverse128` reverse a full machine word by fixed SWAR-style permutations; the generic `bitreverse_n` first masks to the low N bits, reverses the full word, and shifts the image back down to the low N positions. These helpers are exposed for audit and reuse; the public `find_optimal_gamma` path below does not need them after it uses `aopr` directly.

The only run-parity helper used by the public construction below is `aopr`. For a maximal one-run $[l, r]$ in a word x ,

$$\text{aopr}(x)_i = 1 \iff l \leq i \leq r \text{ and } i - l + 1 \text{ is odd.}$$

Equivalently, the implementation scans a run from its low end, toggles a one-bit parity while the run continues, and resets the parity at the first zero. This is not the same helper as the exported `aop`, which describes odd run length from a starting position; the optimal- γ path below uses `aopr`.

Algorithm 4 can now be written in the same temporary variables as the implementation:

$$\begin{aligned} r &= \alpha \wedge 1, \\ e &= \overline{\alpha} \oplus \bar{\beta} \wedge \bar{r}, \\ a_0 &= e \wedge (e \ll 1) \wedge (\alpha \oplus (\alpha \ll 1)), \\ p &= \text{aopr}(a_0), \\ a_1 &= (a_0 \vee (a_0 \gg 1)) \wedge \bar{r}, \\ b &= (a_1 \vee e) \ll 1. \end{aligned}$$

Every complemented word in this display is immediately intersected with $\mathcal{M}_N$. The mask r is the one-bit word containing only the least significant bit of α ; therefore $\bar{r}$ clears bit 0 exactly when $\alpha_0 = 1$, and otherwise clears no active bit. The three masks a_1 , $\bar{a}_1 \wedge b$, and $\bar{a}_1 \wedge \bar{b}$ split the word positions used in the closed-form output:

$$\begin{aligned} \tilde{\gamma} &= ((\alpha \oplus p) \wedge a_1) \vee ((\alpha \oplus \beta \oplus (\alpha \ll 1)) \wedge \bar{a}_1 \wedge b) \\ &\quad \vee (\alpha \wedge \bar{a}_1 \wedge \bar{b}). \end{aligned}$$

Finally the lowest bit is fixed separately:

$$\gamma^* = (\tilde{\gamma} \wedge \bar{1}) \vee ((\alpha \oplus \beta) \wedge 1).$$

The implementation-level procedure is therefore:

Algorithm 10 Two-variable differential Q_2 : optimal output difference

```

1: function AOPR( $x, N$ )
2:    $x \leftarrow x \wedge \text{MASK}(N)$ 
3:    $p \leftarrow 0, \text{ run} \leftarrow 0$ 
4:   for  $i \leftarrow 0$  to  $N - 1$  do
5:     if  $\text{BIT}(x, i) = 1$  then
6:        $\text{run} \leftarrow \text{run} \oplus 1$ 
7:     else
8:        $\text{run} \leftarrow 0$ 
9:     end if
10:    if  $\text{run} = 1$  then
11:       $p \leftarrow p \vee (1 \ll i)$ 
12:    end if
13:  end for
14:  return  $p \wedge \text{MASK}(N)$ 
15: end function

16: function ADD2DIFFOPTIMALGAMMA( $\alpha, \beta, N$ )
17:    $\alpha \leftarrow \alpha \wedge \text{MASK}(N), \beta \leftarrow \beta \wedge \text{MASK}(N)$ 
18:    $r \leftarrow \alpha \wedge 1$ 
19:    $e \leftarrow (\text{MASK}(N) \oplus (\alpha \oplus \beta)) \wedge (\text{MASK}(N) \oplus r)$ 
20:    $a \leftarrow e \wedge ((e \ll 1) \wedge \text{MASK}(N)) \wedge (\alpha \oplus ((\alpha \ll 1) \wedge \text{MASK}(N)))$ 
21:    $p \leftarrow \text{AOPR}(a, N)$ 
22:    $a \leftarrow (a \vee (a \gg 1)) \wedge (\text{MASK}(N) \oplus r)$ 
23:    $b \leftarrow ((a \vee e) \ll 1) \wedge \text{MASK}(N)$ 
24:    $\gamma \leftarrow ((\alpha \oplus p) \wedge a)$ 
25:    $\gamma \leftarrow \gamma \vee ((\alpha \oplus \beta \oplus ((\alpha \ll 1) \wedge \text{MASK}(N))) \wedge (\text{MASK}(N) \oplus a) \wedge b)$ 
26:    $\gamma \leftarrow \gamma \vee (\alpha \wedge (\text{MASK}(N) \oplus a) \wedge (\text{MASK}(N) \oplus b))$ 
27:    $\gamma \leftarrow \gamma \wedge \text{MASK}(N)$ 
28:    $\gamma \leftarrow (\gamma \wedge (\text{MASK}(N) \oplus 1)) \vee ((\alpha \oplus \beta) \wedge 1)$ 
29:   return  $\gamma \wedge \text{MASK}(N)$ 
30: end function

31: function ADD2DIFFQ2( $\alpha, \beta, N$ )
32:    $\gamma^* \leftarrow \text{ADD2DIFFOPTIMALGAMMA}(\alpha, \beta, N)$ 
33:    $(ok, W^*) \leftarrow \text{ADD2DIFFORACLE}(\alpha, \beta, \gamma^*, N)$ 
34:   return  $(\gamma^*, W^*)$ 
35: end function

```

This paragraph is included to document the candidate-completion layer. It must not be read as changing the Q_1 contract: Q_1 evaluates a fully specified triple (α, β, γ) , while Algorithm 4 constructs a best γ for a partially specified pair (α, β) .

Corollary 1 (Two-variable subtraction differential permutation). *Subtraction does not need a separate differential oracle. If*

$$C = A - B \pmod{2^N},$$

then $A = C + B \pmod{2^N}$, and therefore

$$\text{DP}_-(\alpha, \beta \rightarrow \gamma) = \text{DP}_+(\gamma, \beta \rightarrow \alpha).$$

The same swap is used in the MILP model: a subtraction box is encoded as an addition box whose first input is the subtraction output and whose output is the subtraction minuend.

Algorithm 11 Two-variable modular-subtraction differential oracle

```

1: function SUB2DIFFORACLE( $\alpha, \beta, \gamma, N$ )
     $\triangleright C = A - B$  is equivalent to  $A = C + B$ .
     $\triangleright \alpha, \beta, \gamma$  are the differences on  $A, B, C$ .
2:   return ADD2DIFFORACLE( $\gamma, \beta, \alpha, N$ )
3: end function

```

10.7 Two-Variable Linear Q_1 : Wallén 2003 to Schulte–Geers 2013 Point Correlation

The local correlation being evaluated.

Definition 16 (Two-variable addition linear point correlation). The two-variable linear Q_1 oracle is an exact point evaluator for

$$C = A + B \pmod{2^N}.$$

The output mask on C is u , and the input masks on A and B are v and w . The correlation is

$$\text{Cor}_+(u; v, w) = 2^{-2N} \sum_{A, B \in \mathbb{F}_2^N} (-1)^{u \cdot (A+B) \oplus v \cdot A \oplus w \cdot B}.$$

This subsection evaluates this signed sum for one fixed triple (u, v, w) . It does not search for masks and it does not aggregate a hull. The two code files implement two mathematical descriptions of this same fixed-triple object. Johan Wallén’s 2003 carry/CPM theory gives a fixed-triple feasibility and magnitude route [Wal03a, Wal03b]. Ernst Schulte–Geers’ 2013 CCZ/Walsh formula gives the signed $\phi_2(u, v, w)$ point value [SG13]. In this document $Q_1^{\text{LC},+}$ means the signed point value; the Wallén route is therefore a weight/magnitude component of the implementation, not a replacement for the sign. Throughout the implementation-level formulas below, the three masks are first truncated to

$$\mathcal{M}_N = 2^N - 1,$$

and every complement is understood inside this N -bit word. The CCZ file uses the following primitive helpers:

<code>mask_n</code>	$\mathcal{M}_N$,
<code>leq_bitwise</code>	$x \preceq z$, implemented as $(x \wedge \bar{z}) = 0$,
<code>dot_parity</code>	$\text{parity}(x \wedge y)$,
<code>popcount_u64</code>	$\text{wt}(x)$,
<code>L_of</code>	paper $L(x) = (x_1, \dots, x_{N-1}, 0)$, code $x \gg 1$,
<code>R_of</code>	paper $R(x) = (0, x_0, \dots, x_{N-2})$, code $(x \ll 1) \wedge \mathcal{M}_N$.

The function `prefix_xor_inclusive` computes

$$P_i = x_0 \oplus x_1 \oplus \dots \oplus x_i,$$

and `M_of(x,N)` returns $R(P)$. The Q_1 formula below uses the transpose helper `MnT_of`, which is the suffix analogue.

Implementation-level routes and their outputs.

Theorem 6 (Signed point route separation). *The two implementation files expose three different ways to talk about this same fixed triple. They should not be collapsed into one object.*

Schulte–Geers 2013 route	signed point	<code>linear_correlation_add_ccz_value,</code>
carry-state matrix route, sequential product	signed point	<code>correlation_add_varvar_reference,</code>
carry-state matrix route, log-depth product	signed point	<code>correlation_add_varvar_logdepth,</code>
Wallén 2003 route	magnitude	<code>linear_correlation_add_value_logn.</code>

The first line is the Schulte–Geers CCZ formula [SG13]. The two carry-state matrix lines are Wallén-style exact signed evaluators [Wal03a, Wal03b]. The last line is Wallén’s CPM magnitude route [Wal03a, Wal03b]. The Q_1 contract in this document is the signed point value. Therefore the sign is part of Q_1 . A route that returns only $|\text{Cor}_+|$ is useful for feasibility and weight, but it is not by itself the full signed Q_1 output. There is also a width distinction in the code. The CCZ route takes `uint64_t` words and an explicit N . The matrix-chain route has 32- and 64-bit wrappers. The public Wallén log- N value route is the 32-bit `uint32_t` fixed-triple helper.

It is useful to fix the matrix orientation before any formulas are used. Let

$$\Phi_N = (\phi_2(u, v, w))_{u \in \mathbb{F}_2^N, (v, w) \in \mathbb{F}_2^N \times \mathbb{F}_2^N}.$$

Rows of Φ_N are indexed by the output mask u . Columns of Φ_N are indexed by the input-mask pair (v, w) . Hence the code uses three different local objects:

$$\begin{aligned} \text{point value} &: (u, v, w) \mapsto \phi_2(u, v, w), & Q_1^{\text{LC},+}, \\ \text{column completion} &: (v, w) \mapsto u^*(v, w), & \text{ColumnBestU}, \\ \text{row completion} &: u \mapsto (v^*(u), w^*(u)), & \text{RowBestMasks}. \end{aligned}$$

Only the first line is the Q_1 oracle. The Schulte–Geers column construction is the second line. The row line is the implementation direction used when the global search has already fixed the output mask u and needs a row-side best column entry. In the C++ file these two completion services are exposed separately:

<code>ColumnBestU</code>	<code>find_optimal_output_u_ccz,</code>
<code>RowBestMasks</code>	<code>linear_correlation_add_phi2_row_max.</code>

Q_1 Wallén’s $O(n)$ Time Algorithm

Theorem 7 (Two-variable addition linear Q_1 oracle). *The two-variable addition linear oracle is therefore*

$$Q_1^{\text{LC},+}(u, v, w) = \begin{cases} (\text{false}, 0, +\infty), & p \not\leq z \text{ or } q \not\leq z, \\ (\text{true}, (-1)^{p \cdot q}, \text{wt}(z)), & \text{otherwise.} \end{cases}$$

The three output fields are the nonzero flag, the sign of the correlation, and the correlation weight. The actual correlation is zero in the first case and $\text{sign} \cdot 2^{-W}$ in the second.

Algorithm 12 Two-variable modular-addition linear oracle

```

1: function MNTRANPOSE( $s, N$ )
2:    $s \leftarrow s \wedge \text{MASK}(N)$ 
3:    $\sigma \leftarrow s$ 

```

```

4:   for  $d \leftarrow 1, 2, 4, \dots$  while  $d < N$  do
5:        $\sigma \leftarrow \sigma \oplus (\sigma \gg d)$ 
6:   end for
7:    $\sigma \leftarrow \sigma \wedge \text{MASK}(N)$ 
8:   return  $(\sigma \gg 1) \wedge \text{MASK}(N)$ 
9: end function

10: function  $\text{ADD2LINEARORACLE}(u, v, w, N)$ 
                                 $\triangleright u$  is the output mask;  $v, w$  are input masks.
11:    $u \leftarrow u \wedge \text{MASK}(N), \quad v \leftarrow v \wedge \text{MASK}(N), \quad w \leftarrow w \wedge \text{MASK}(N)$ 
12:    $z \leftarrow \text{MNTRANSPOSE}(u \oplus v \oplus w, N)$ 
13:    $p \leftarrow u \oplus v$ 
14:    $q \leftarrow u \oplus w$ 
15:    $\text{outside} \leftarrow \text{MASK}(N) \oplus z$ 
16:   if  $(p \wedge \text{outside}) \neq 0$  or  $(q \wedge \text{outside}) \neq 0$  then
17:       return  $(\text{false}, 0, +\infty)$ 
18:   end if
19:   if  $\text{HAMMINGWEIGHT}(p \wedge q)$  is odd then
20:        $\text{sign} \leftarrow -1$ 
21:   else
22:        $\text{sign} \leftarrow +1$ 
23:   end if
24:    $W \leftarrow \text{HAMMINGWEIGHT}(z)$ 
25:   return  $(\text{true}, \text{sign}, W)$ 
26: end function

```

Wallén's local common-prefix segment.

Definition 17 (Wallén local common-prefix segment). This paragraph is only the Wallén CPM object [Wal03a, Wal03b]; it is not the Schulte-Geers signed point formula. We use the implementation bit order throughout: bit 0 is the least significant bit, an N -bit word is $y = \sum_{i=0}^{N-1} y_i e_i$, and e_i is the word with only bit i set. For a start position k and a width g , define the window

$$I(k, g) = \{k, k+1, \dots, k+g-1\}.$$

The C++ helper `cpm_k_i(vec, k, width)` returns 0 when $g \leq 0$ or the window is outside $\{0, \dots, N-1\}$. For a valid positive-width window, Wallén's local common-prefix mask is

$$\text{cpm}_k^g(y) = \sum_{t=0}^{g-1} \pi_t e_{k+t}, \quad \pi_t = \prod_{r=t+1}^{g-1} y_{k+r},$$

with the empty product equal to one.

Lemma 3 (Local CPM equivalence and loop invariant). *Equivalently, for a valid positive-width window $I(k, g)$,*

$$(\text{cpm}_k^g(y))_\ell = 1 \iff y_s = 1 \quad \text{for every } s \in \{\ell+1, \dots, k+g-1\}, \quad \ell \in I(k, g).$$

Thus the output bit at position ℓ depends on the bits strictly above ℓ in the same window, not on y_ℓ itself. In particular, the highest output bit $k+g-1$ is always one, because the product above it is empty. The recurrence form seen by the loop is

$$\pi_{g-1} = 1, \quad \pi_t = y_{k+t+1} \pi_{t+1} \quad (0 \leq t < g-1).$$

This is exactly the invariant of the Boolean variable `all_ones_above`: when the loop is about to process bit j , that variable equals

$$\prod_{s=j+1}^{k+g-1} y_s.$$

After the possible output bit is written, the code multiplies this invariant by y_j before moving to the next lower position.

Proof. The bit $\ell = k + t$ of $\text{cpm}_k^g(y)$ is the product $\prod_{r=t+1}^{g-1} y_{k+r}$. Over $\mathbb{F}_2$, this product is one exactly when every factor is one, giving the displayed equivalence; when $t = g - 1$, the product is empty and equals one. The recurrence is the same product written from the top of the window downward. The loop invariant follows by descending induction on j : it is true at the highest position by the empty-product convention, and multiplication by y_j prepares precisely the product needed at the next lower position. $\square$

Wallén's recursive CPM definition.

Definition 18 (Wallén recursive common-prefix mask). We now define the full $\text{cpm}(x, y)$ used by Wallén's magnitude route [Wal03a, Wal03b]. For a word x , write its support in decreasing order as

$$\text{supp}(x) = \{s_1 > s_2 > \dots > s_m\}, \quad x = \sum_{q=1}^m e_{s_q}.$$

The r -fold stripping operator removes the r highest one-bits:

$$\text{strip}^r(x) = \sum_{q=r+1}^m e_{s_q}, \quad \text{strip}^0(x) = x,$$

and the sum is 0 when $r \geq m$. Hence $\text{strip}(x) = \text{strip}^1(x)$ is the C++ operation `strip_msb_once_u64`.

The recursion starts from

$$\text{cpm}(0, y) = 0.$$

For $x \neq 0$, let $j = s_1$ be the highest active bit of x . The lower anchor k is the second highest active bit when it exists, and 0 otherwise:

$$k = \begin{cases} s_2, & m \geq 2, \\ 0, & m = 1. \end{cases}$$

The active gap is

$$g = j - k,$$

and the local mask selected by this gap is

$$z(x, y) = \text{cpm}_k^g(y).$$

If $m = 1$ and $j = 0$, then $g = 0$, so the guard in the local definition gives $z(x, y) = 0$. This is the single-bit bottom case implemented by the same code path, not a separate mathematical exception.

The branch is controlled by support inclusion. For two N -bit masks,

$$a \preceq b \iff \text{supp}(a) \subseteq \text{supp}(b) \iff a_i \leq b_i \text{ for all } i \iff (a \wedge b) = a.$$

Equivalently, $a \preceq b$ means $(a \wedge \bar{b}) = 0$ inside $\mathcal{M}_N$. Define the number of removed highest one-bits by

$$b(x, y) = 1 + \mathbf{1}[z(x, y) \preceq y].$$

Wallén's recursive common-prefix mask is then

$$\text{cpm}(x, y) = z(x, y) \oplus \text{cpm}(\text{strip}^{b(x, y)}(x), y).$$

The recursion terminates because $b(x, y) \in \{1, 2\}$, so each call removes at least the current highest one-bit of x . The extra removal occurs exactly when every one-bit produced by the local mask $z(x, y)$ is also a one-bit of y . In Wallén's notation this is the component-wise product test $z \odot y = z$; in the C++ implementation it is

$$(z \ \& \ y) = z.$$

The variables in the reference implementation line up as follows:

<code>msb_index_u64(xx)</code>	$j = s_1$, the current highest active bit,
<code>stripped_once</code>	$\text{strip}(x)$, the word after removing s_1 ,
<code>second_msb_index</code>	$k = s_2$ when s_2 exists, otherwise 0,
<code>gap</code>	$g = j - k$, the width passed to the local CPM segment,
<code>cpm_k_i(yy, k, g)</code>	$z(x, y) = \text{cpm}_k^g(y)$,
<code>z_is_subset_of_y</code>	the predicate $z(x, y) \preceq y$,
<code>next_x</code>	$\text{strip}(x)$, then $\text{strip}^2(x)$ if $z(x, y) \preceq y$.

The outer guards $N \leq 0$, $N > 64$, and the mask $\mathcal{M}_N$ are word-size guards of the implementation. The mathematical recursion above is the reference semantics of the function

`compute_cpm_recursive.`

Algorithm 13 Wallén 2003 CPM reference implementation

```

1: function MSB( $x$ )
2:   if  $x = 0$  then
3:     return  $-1$ 
4:   end if
5:   return highest index  $j$  with  $\text{BIT}(x, j) = 1$ 
6: end function

7: function STRIP( $x$ )
8:    $j \leftarrow \text{MSB}(x)$ 
9:   if  $j < 0$  then
10:    return  $0$ 
11:  end if
12:  return  $x \wedge \neg(1 \ll j)$ 
13: end function

14: function CPMSEGMENT( $y, k, g, N$ )
15:   if  $g \leq 0$  or  $k < 0$  or  $k + g - 1 \geq N$  then
16:    return  $0$ 
17:  end if
18:   $z \leftarrow 0$ 
19:   $all \leftarrow \text{true}$ 
20:  for  $j \leftarrow k + g - 1$  downto  $k$  do
21:    if  $all$  then
22:       $z \leftarrow z \vee (1 \ll j)$ 

```

```

23:     end if
24:      $all \leftarrow all \wedge (\text{BIT}(y, j) = 1)$ 
25:   end for
26:   return  $z \wedge \text{MASK}(N)$ 
27: end function

28: function CPMRECURSIVE( $x, y, N$ )
29:    $x \leftarrow x \wedge \text{MASK}(N)$ ,  $y \leftarrow y \wedge \text{MASK}(N)$ 
30:   if  $x = 0$  then
31:     return 0
32:   end if
33:    $j \leftarrow \text{MSB}(x)$ 
34:    $x_1 \leftarrow \text{STRIP}(x)$ 
35:   if  $x_1 = 0$  then
36:      $k \leftarrow 0$ 
37:   else
38:      $k \leftarrow \text{MSB}(x_1)$ 
39:   end if
40:    $g \leftarrow j - k$ 
41:    $z \leftarrow \text{CPMSEGMENT}(y, k, g, N)$ 
42:    $x_{\text{next}} \leftarrow x_1$ 
43:   if  $(z \wedge y) = z$  then
44:      $x_{\text{next}} \leftarrow \text{STRIP}(x_{\text{next}})$ 
45:   end if
46:   return  $(z \oplus \text{CPMRECURSIVE}(x_{\text{next}}, y, N)) \wedge \text{MASK}(N)$ 
47: end function

```

The $\Theta(\log N)$ CPM implementation.

Construction 2 (Wallén log-depth CPM implementation). The public Wallén path uses the bit-sliced CPM routine. Its code entry is

`CpmLogN  $\equiv$  compute_cpm_logn_bitsliced.`

If N is not a power of two, the code falls back to the recursive reference `CpmRecursive`. For power-of-two N , the implementation follows the stage structure of Wallén’s Theorem 2 [Wal03a, Wal03b]. It maintains two masks z_0, z_1 , initialized as

$$z_0 = 0, \quad z_1 = \mathcal{M}_N,$$

and a stage mask β . The alternating mask is not an external helper in the C++ file. It is the local lambda `alpha_local` inside `run_variant_general`. For $B = 2^i$, the lambda builds a period $2B$ mask by placing a block of B ones in each period:

$$\alpha_i^{(f_0)} = \bigvee_{p \in \{0, 2B, 4B, \dots\}} \begin{cases} ((2^B - 1) \ll p), & f_0 = 1, \\ ((2^B - 1) \ll (p + B)), & f_0 = 0, \end{cases} \wedge \mathcal{M}_N.$$

If $B \geq N$, the lambda returns $\mathcal{M}_N$. Thus f_0 is exactly the code’s `alpha_ones_first` flag: it decides whether the one-block of α_i starts at the LSB side of each period or after the first zero block.

For each branch $b \in \{0, 1\}$, the implementation forms

$$\Gamma_b = (((y \wedge z_b) \wedge x) \vee ((y \wedge \overline{z_b}) \wedge \overline{x})) \wedge \beta,$$

unless f_4 selects the conditional variant

$$\Gamma_b = ((y \wedge z_b) ? x : \bar{x}) \wedge \beta.$$

These are the two C++ branches inside `run_variant_general`; the exhaustive $N = 8$ check decides which branch is consistent with `compute_cpm_recursive`. It then expands Γ_b by one stage shift, updates

$$\begin{aligned} t_0 &= (z_0 \wedge \alpha_i) \vee (z_0 \wedge \Gamma_0 \wedge \bar{\alpha}_i) \vee (z_1 \wedge \bar{\Gamma}_0), \\ t_1 &= (z_1 \wedge \alpha_i) \vee (z_0 \wedge \Gamma_1 \wedge \bar{\alpha}_i) \vee (z_1 \wedge \bar{\Gamma}_1), \end{aligned}$$

sets $z_0 \leftarrow t_0$, $z_1 \leftarrow t_1$, and moves β to the next stage. The exact shift directions are not guessed silently: the file enumerates five Boolean flags and accepts the unique assignment that matches `compute_cpm_recursive` on all 8-bit inputs. The five flags are:

- f_0 α_i starts with ones at the LSB side,
- f_1 β is initialized as $\bar{\alpha}_0$ rather than α_0 ,
- f_2 Γ_b is shifted right rather than left,
- f_3 β is shifted right rather than left,
- f_4 Γ_b uses the conditional form rather than the expanded form.

Algorithm 14 Wallén CPM log-depth implementation skeleton

```

1: function ALPHALocal( $i, N, f_0$ )
2:   if  $i < 0$  then
3:     return 0
4:   end if
5:    $B \leftarrow 2^i$ 
6:   if  $B \geq N$  then
7:     return MASK( $N$ )
8:   end if
9:    $P \leftarrow 0$ 
10:  for  $p \leftarrow 0, 2B, 4B, \dots < N$  do
11:    if  $f_0$  then
12:       $q \leftarrow p$ 
13:    else
14:       $q \leftarrow p + B$ 
15:    end if
16:     $P \leftarrow P \vee (((1 \ll B) - 1) \ll q)$ 
17:  end for
18:  return  $P \wedge \text{MASK}(N)$ 
19: end function

20: function RUNCPMVariant( $x, y, N, f$ )
21:  unpack  $f$  into  $f_0, \dots, f_4$  as the five bits of CommonPrefixMaskConfig::flags
22:   $x \leftarrow x \wedge \text{MASK}(N)$ ,  $y \leftarrow y \wedge \text{MASK}(N)$ 
23:   $\alpha_0 \leftarrow \text{ALPHALocal}(0, N, f_0)$ 
24:  if  $f_1$  then
25:     $\beta \leftarrow \text{MASK}(N) \oplus \alpha_0$ 
26:  else
27:     $\beta \leftarrow \alpha_0$ 
28:  end if
29:   $z_0 \leftarrow 0$ ,  $z_1 \leftarrow \text{MASK}(N)$ 

```

```

30:   for  $i \leftarrow 0$  to  $\log_2 N - 1$  do
31:      $\alpha \leftarrow \text{ALPHALOCAL}(i, N, f_0)$ 
32:      $\bar{\alpha} \leftarrow \text{MASK}(N) \oplus \alpha$ 
33:     for  $b \in \{0, 1\}$  do
34:        $z_b \leftarrow z_0$  if  $b = 0$ , else  $z_1$ 
35:       if  $f_4$  then
36:          $c_b \leftarrow y \wedge z_b$ 
37:          $r_b \leftarrow (c_b \wedge x) \vee ((\text{MASK}(N) \oplus c_b) \wedge (\text{MASK}(N) \oplus x))$ 
38:          $\Gamma_b \leftarrow r_b \wedge \beta$ 
39:       else
40:          $g_0 \leftarrow y \wedge z_b \wedge x$ 
41:          $g_1 \leftarrow y \wedge (\text{MASK}(N) \oplus z_b) \wedge (\text{MASK}(N) \oplus x)$ 
42:          $\Gamma_b \leftarrow (g_0 \vee g_1) \wedge \beta$ 
43:       end if
44:       if  $f_2$  then
45:          $\Gamma_b \leftarrow \Gamma_b \vee (\Gamma_b \gg 2^i)$ 
46:       else
47:          $\Gamma_b \leftarrow \Gamma_b \vee (\Gamma_b \ll 2^i)$ 
48:       end if
49:        $\Gamma_b \leftarrow \Gamma_b \wedge \text{MASK}(N)$ 
50:     end for
51:      $t_0 \leftarrow (z_0 \wedge \alpha) \vee (z_0 \wedge \Gamma_0 \wedge \bar{\alpha})$ 
52:      $t_0 \leftarrow t_0 \vee (z_1 \wedge (\text{MASK}(N) \oplus \Gamma_0))$ 
53:      $t_1 \leftarrow (z_1 \wedge \alpha) \vee (z_0 \wedge \Gamma_1 \wedge \bar{\alpha})$ 
54:      $t_1 \leftarrow t_1 \vee (z_1 \wedge (\text{MASK}(N) \oplus \Gamma_1))$ 
55:      $z_0 \leftarrow t_0 \wedge \text{MASK}(N)$ ,  $z_1 \leftarrow t_1 \wedge \text{MASK}(N)$ 
56:      $\alpha_{\text{next}} \leftarrow \text{ALPHALOCAL}(i + 1, N, f_0)$ 
57:     if  $f_3$  then
58:        $\beta \leftarrow (\beta \gg 2^i) \wedge \alpha_{\text{next}}$ 
59:     else
60:        $\beta \leftarrow (\beta \ll 2^i) \wedge \alpha_{\text{next}}$ 
61:     end if
62:   end for
63:   return  $z_0 \wedge \text{MASK}(N)$ 
64: end function

65: function CPMLOGN( $x, y, N$ )
66:   if  $N \leq 0$  then
67:     return 0
68:   end if
69:   if  $N > 64$  then
70:      $N \leftarrow 64$ 
71:   end if
72:    $x \leftarrow x \wedge \text{MASK}(N)$ ,  $y \leftarrow y \wedge \text{MASK}(N)$ 
73:   if  $x = 0$  then
74:     return 0
75:   end if
76:   if  $N$  is not a power of two then
77:     return CPMRECURSIVE( $x, y, N$ )
78:   end if
79:    $\mathcal{F} \leftarrow \emptyset$ 
80:   for  $f \leftarrow 0$  to 31 do

```

```

81:      if RUNCPMVARIANT( $t_x, t_y, 8, f$ ) = CPMRECURSIVE( $t_x, t_y, 8$ ) for all  $t_x, t_y < 2^8$ 
      then
82:          insert  $f$  into  $\mathcal{F}$ 
83:      end if
84:  end for
85:  if  $|\mathcal{F}| \neq 1$  then
86:      return CPMRECURSIVE( $x, y, N$ )
87:  end if
88:  let  $f^*$  be the unique element of  $\mathcal{F}$ 
89:  return RUNCPMVARIANT( $x, y, N, f^*$ )
90: end function

```

The exhaustive $N = 8$ flag selection is not a new mathematical definition of CPM. It is an implementation guard: after the unique flag assignment has been chosen, the returned value is required to agree with the recursive $\text{cpm}(x, y)$ definition above. This is why the Wallén route uses the log-depth word code for speed while still treating `compute_cpm_recursive` as the reference semantics.

Transfer-matrix signed evaluator.

Theorem 8 (Transfer-matrix signed evaluator). *The same implementation file also keeps an exact transfer-matrix reference. This is the Wallén carry-state linear-correlation viewpoint [Wal03a, Wal03b]. It evaluates the same signed point value. For one bit i , with carry-in c and carry-out c' ,*

$$M_i(c, c') = \frac{1}{4} \sum_{x_i, y_i \in \mathbb{F}_2} \mathbf{1}[\text{carry}(x_i, y_i, c) = c'] (-1)^{v_i x_i \oplus w_i y_i \oplus u_i (x_i \oplus y_i \oplus c)}.$$

This is exactly the loop nest in `make_Mi_add_varvar_bit`: the code iterates over $c \in \{0, 1\}$, $c' \in \{0, 1\}$, and the four pairs (x_i, y_i) . The helper

$$\text{carry_out}(x_i, y_i, c) = (x_i \wedge y_i) \vee (x_i \wedge c) \vee (y_i \wedge c)$$

selects which terms enter the matrix entry. The helper `signed_bit` implements

$$(-1)^{mb} = \begin{cases} -1, & m = 1, b = 1, \\ +1, & \text{otherwise,} \end{cases}$$

so the product of three `signed_bit` calls is the sign in the displayed sum. The factor $1/4$ is present because both sampled input bits vary. The full correlation is

$$\text{Cor}_+(u; v, w) = (1, 0) M_0 M_1 \cdots M_{N-1} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

*The sequential chain and the log-depth tree reduction differ only in multiplication order; mathematically they evaluate the same signed local correlation. The **Matrix2D** storage convention is row-vector left multiplication:*

$$[r_0, r_1] \begin{pmatrix} m_{00} & m_{01} \\ m_{10} & m_{11} \end{pmatrix} = [r_0 m_{00} + r_1 m_{10}, r_0 m_{01} + r_1 m_{11}].$$

The matrix product helper uses the ordinary associative product

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}.$$

The reference function processes the matrices in the order

$$M_0, M_1, \dots, M_{N-1}.$$

The code names are:

<code>sequential reference</code>	<code>correlation_add_varvar_reference,</code>
<code>public alias</code>	<code>correlation_add_varvar,</code>
<code>log-depth product</code>	<code>correlation_add_varvar_logdepth.</code>

That routine first stores all M_i , then replaces adjacent pairs by products

$$(M_0M_1), (M_2M_3), \dots$$

until one product remains. The work is still $O(N)$, while the multiplication depth is $\Theta(\log N)$. The wrappers `linear_add_varvar32` and `linear_add_varvar64` then convert the signed correlation C into the pair

$$(C, -\log_2 |C|),$$

using $+\infty$ as the weight when $C = 0$.

Wallén 2003 CPM route: fixed-triple magnitude.

Theorem 9 (Wallén CPM fixed-triple magnitude route). *The same `weight_evaluation.hpp` file also contains the Wallén common-prefix-mask path [Wal03a, Wal03b]. This route is still a fixed- (u, v, w) computation. It does not choose a mask. Its public value is nonnegative because it returns the magnitude of the correlation.*

The first step is Wallén's reduction from addition to carry. Since

$$A + B = A \oplus B \oplus \text{carry}(A, B),$$

the phase in the correlation can be rewritten as

$$u \cdot (A + B) \oplus v \cdot A \oplus w \cdot B = u \cdot \text{carry}(A, B) \oplus (u \oplus v) \cdot A \oplus (u \oplus w) \cdot B.$$

The implementation therefore sets

$$p' = v \oplus u, \quad q' = w \oplus u, \quad m = \text{cpm}(u, \text{eq}_N(p', q')).$$

This route checks

$$p' \preceq m, \quad q' \preceq m,$$

and returns the weight $\text{wt}(m)$, hence the magnitude

$$|\text{Cor}_+(u; v, w)| = 2^{-\text{wt}(m)}$$

when the support condition holds. If either support inclusion fails, the route returns zero. In the implementation, `linear_correlation_add_value_logn` returns this absolute value. It does not by itself provide the sign. The sign used by the Q_1 contract is therefore taken from the Schulte–Geers signed formula above, or equivalently from the exact matrix-chain evaluator.

The file contains two related Wallén helpers, and they should be kept separate. The helper `wallen_weight(mu, nu, omega, n)` computes

$$r = \mu \oplus \nu \oplus \omega, \quad z^* = M_{32}^t(r),$$

then sets

$$a = \mu \oplus \omega, \quad b = \nu \oplus \omega.$$

It returns $\text{wt}(z^*)$ when $a \preceq z^*$ and $b \preceq z^*$, otherwise nullopt. In the current code the parameter n is explicitly ignored in this helper, so this is a 32-bit support/weight routine. The public Wallén value entry is

`linear_correlation_add_value_logn.`

Internally it calls `internal_addition_wallen_logn`. This is the CPM version displayed above: it forms $p' = v \oplus u$, $q' = w \oplus u$, computes $m = \text{cpm}(u, \text{eq}(p', q'))$, checks support inclusion, and returns $2^{-\text{wt}(m)}$ if feasible.

The CPM itself is implemented in two layers. The function `compute_cpm_recursive` is the definition-level reference for Wallén’s Definition 3 and Definition 6 [Wal03a, Wal03b]. The function `compute_cpm_logn_bitsliced` is the word-parallel implementation of Wallén’s Theorem 2, with an implementation audit step that chooses the unique bit-order interpretation matching the recursive definition.

10.8 Two-Variable Linear Q_1 and Schulte–Geers Matrix Directions

Layer separation.

Remark 1 (Point, column, and row directions of $\mathcal{C}_N$). This subsection keeps three mathematical objects separate. First, Theorem 4 of Schulte–Geers gives the signed point value $Q_1^{\text{LC},+}(u, v, w)$ for one fixed triple (u, v, w) . Second, Schulte–Geers’ column theorem fixes the input-mask pair (v, w) and constructs one output mask u^* with maximal absolute correlation. Third, the project also needs the fixed-row direction: u is fixed by the trail state, and the code constructs one pair (v, w) attaining the row maximum. These three objects share the same correlation matrix, but they are not the same oracle and they must not be substituted for one another [SG13].

object	fixed data	free data	main code path
point value $Q_1^{\text{LC},+}$	(u, v, w)	none	<code>linear_correlation_add_ccz_weight</code>
column completion	(v, w)	u	<code>find_optimal_output_u_ccz</code>
row completion	u	(v, w)	<code>linear_correlation_add_phi2_row_max</code>

Schulte–Geers 2013 signed point formula.

Theorem 10 (Schulte–Geers signed point formula). *The signed Q_1 value is the single-entry formula in the Schulte–Geers/CCZ notation implemented by*

`weight_evaluation_ccz.hpp.`

Schulte–Geers writes vectors as rows, with coordinate 0 on the left and also as the least significant bit. This matches the implementation convention: bit i of the integer word is coordinate i . Define the addition correlation matrix by

$$\mathcal{C}_N[u, (v, w)] = \phi_2(u, v, w) = \text{Cor}_+(u; v, w).$$

Rows are indexed by the output mask u ; columns are indexed by the ordered input-mask pair (v, w) . Thus a Schulte–Geers column is

$$\mathcal{C}_N[*, (v, w)] = \{\phi_2(u, v, w) : u \in \mathbb{F}_2^N\},$$

whereas the row service used by this project is

$$\mathcal{C}_N[u, *] = \{\phi_2(u, v, w) : (v, w) \in \mathbb{F}_2^N \times \mathbb{F}_2^N\}.$$

The column theorem and the row constructor therefore optimize in different directions of the same matrix [SG13].

The implementation first realizes the linear maps used in the paper:

$$\begin{aligned} L(x) &= (x_1, \dots, x_{N-1}, 0), & \text{code: } L_of(\mathbf{x}, N) &= x \gg 1, \\ R(x) &= (0, x_0, \dots, x_{N-2}), & \text{code: } R_of(\mathbf{x}, N) &= (x \ll 1) \wedge \mathcal{M}_N, \\ M(x) &= R(x_0, x_0 \oplus x_1, \dots, x_0 \oplus \dots \oplus x_{N-1}). \end{aligned}$$

The point formula uses the transpose M^t . Define

$$s = u \oplus v \oplus w$$

and let $z = M_N^t(s)$, where, with bit 0 as the least significant bit,

$$z_i = s_{i+1} \oplus s_{i+2} \oplus \dots \oplus s_{N-1}, \quad z_{N-1} = 0.$$

Equivalently, z is obtained by a suffix-XOR scan followed by one shift toward the low bit. In the implementation this is the function `MnT_of`: first compute

$$\sigma_i = s_i \oplus s_{i+1} \oplus \dots \oplus s_{N-1}$$

by the bit-sliced loop

$$s \leftarrow s \oplus (s \gg 1), \quad s \leftarrow s \oplus (s \gg 2), \quad s \leftarrow s \oplus (s \gg 4), \quad \dots,$$

truncated to N bits after the loop. Then

$$z = (\sigma \gg 1) \wedge \mathcal{M}_N.$$

This explains the apparent one-bit shift in the code: z_i is the suffix parity above position i , not including s_i . Put

$$p = u \oplus v, \quad q = u \oplus w.$$

The exact signed correlation is

$$\text{Cor}_+(u; v, w) = \mathbf{1}[p \preceq z] \mathbf{1}[q \preceq z] (-1)^{p \cdot q} 2^{-\text{wt}(z)},$$

where $x \preceq z$ means bitwise support inclusion,

$$x \preceq z \iff x \wedge \bar{z} = 0.$$

Thus z is the support mask that can absorb the two remaining direct input phases. If either p or q has a one outside z , summing over all (A, B) cancels the approximation to zero. Equivalently, this is the pencil-and-paper condition stated after Schulte–Geers’ Theorem 4:

$$z_i = 0 \implies u_i = v_i = w_i.$$

Indeed $z_i = 0$ forces $p_i = u_i \oplus v_i = 0$ and $q_i = u_i \oplus w_i = 0$, hence all three masks agree at that coordinate [SG13]. The CCZ implementation follows this formula literally:

$$\begin{aligned} \text{linear_correlation_add_ccz_value} & \quad \text{nullopt if zero, otherwise } \pm 2^{-\text{wt}(z)}, \\ \text{linear_correlation_add_ccz_weight} & \quad \text{nullopt if zero, otherwise } \text{wt}(z). \end{aligned}$$

The sign bit is computed as

$$\text{parity}((u \oplus w) \wedge (u \oplus v)),$$

which is the function `dot_parity(uw, uv, N)` in the file. The exact code path is therefore:

Algorithm 15 Schulte–Geers signed point formula as implemented

```

1: function SUFFIXXORDOWN( $s, N$ )
2:    $s \leftarrow s \wedge \text{MASK}(N)$ 
3:    $d \leftarrow 1$ 
4:   while  $d < N$  do
5:      $s \leftarrow s \oplus (s \gg d)$ 
6:      $d \leftarrow 2d$ 
7:   end while
8:   return  $s \wedge \text{MASK}(N)$ 
9: end function

10: function MNTRANSPOSE( $s, N$ )
11:    $s \leftarrow s \wedge \text{MASK}(N)$ 
12:   if  $N \leq 0$  then
13:     return 0
14:   end if
15:    $\sigma \leftarrow \text{SUFFIXXORDOWN}(s, N)$ 
16:   return  $(\sigma \gg 1) \wedge \text{MASK}(N)$ 
17: end function

18: function CCZPOINTVALUE( $u, v, w, N$ )
19:    $u \leftarrow u \wedge \text{MASK}(N), \quad v \leftarrow v \wedge \text{MASK}(N), \quad w \leftarrow w \wedge \text{MASK}(N)$ 
20:    $s \leftarrow u \oplus v \oplus w$ 
21:    $z \leftarrow \text{MNTRANSPOSE}(s, N)$ 
22:    $p \leftarrow u \oplus v$ 
23:    $q \leftarrow u \oplus w$ 
24:   if  $(p \wedge (\text{MASK}(N) \oplus z)) \neq 0$  then
25:     return nullopt
26:   end if
27:   if  $(q \wedge (\text{MASK}(N) \oplus z)) \neq 0$  then
28:     return nullopt
29:   end if
30:    $W \leftarrow \text{HAMMINGWEIGHT}(z)$ 
31:   if  $\text{HAMMINGWEIGHT}(p \wedge q)$  is odd then
32:     return  $-2^{-W}$ 
33:   else
34:     return  $+2^{-W}$ 
35:   end if
36: end function

37: function CCZPOINTWEIGHT( $u, v, w, N$ )
38:    $u \leftarrow u \wedge \text{MASK}(N), \quad v \leftarrow v \wedge \text{MASK}(N), \quad w \leftarrow w \wedge \text{MASK}(N)$ 
39:    $s \leftarrow u \oplus v \oplus w$ 
40:    $z \leftarrow \text{MNTRANSPOSE}(s, N)$ 
41:    $p \leftarrow u \oplus v$ 
42:    $q \leftarrow u \oplus w$ 
43:   if  $(p \wedge (\text{MASK}(N) \oplus z)) \neq 0$  then
44:     return nullopt
45:   end if
46:   if  $(q \wedge (\text{MASK}(N) \oplus z)) \neq 0$  then
47:     return nullopt
48:   end if
49:   return  $\text{HAMMINGWEIGHT}(z)$ 

```

50: end function

The first two functions are not auxiliary guesses. After the stage with stride $d = 2^r$, the variable s_i equals the XOR of the block $s_i, \dots, s_{i+2^{r+1}-1}$, truncated at the word boundary. When d has passed N , this invariant gives

$$\sigma_i = s_i \oplus s_{i+1} \oplus \dots \oplus s_{N-1}.$$

The final shift computes

$$z_i = \sigma_{i+1} = s_{i+1} \oplus \dots \oplus s_{N-1},$$

which is exactly $M^t(s)_i$ in the row-vector, little-endian convention of Schulte–Geers [SG13]. The remaining lines implement the two indicator functions and the sign term of Theorem 4 without changing the mathematics: failed support inclusion returns `nullopt`, and a passed point returns either the signed value $\pm 2^{-\text{wt}(z)}$ or only the weight $\text{wt}(z)$, depending on which public routine is called [SG13]. The second function is not a new theorem. It is `linear_correlation_add_ccz_weight`: the same point formula with the sign and the factor 2^{-W} erased because the search layer only needs the exponent.

The same formula can also be read in the paper’s second notation. If the input masks are written as $u \oplus v$ and $u \oplus w$, then

$$\phi_2(u, u \oplus v, u \oplus w) = \mathbf{1}[v \preceq z] \mathbf{1}[w \preceq z] (-1)^{v \cdot w} 2^{-\text{wt}(z)}, \quad z = M^t(u \oplus (u \oplus v) \oplus (u \oplus w)).$$

This is not a second oracle. It is the same Q_1 point value after the change of variables used in Schulte–Geers’ proof.

Schulte–Geers matrix columns.

Theorem 11 (Schulte–Geers column completion theorem). *The point oracle above is only Theorem-4 evaluation of a fixed (u, v, w) . The column problem in Schulte–Geers’ Theorem 7 is a different problem [SG13]: the two input masks (v, w) are fixed, and one chooses an output mask*

$$u^*(v, w) \in \arg \max_{u \in \mathbb{F}_2^N} |\phi_2(u, v, w)|.$$

This is the mathematical role of

$$\text{find_optimal_output_u_ccz.}$$

It is not a Q_1 evaluator, because Q_1 does not choose u . The column theorem is a completion theorem

$$(v, w) \mapsto u^*(v, w),$$

followed, if a signed value or a weight is needed, by the single-entry oracle

$$Q_1^{\text{LC},+}(u^*(v, w), v, w).$$

The derivation starts from the support part of the point formula. Put

$$s = u \oplus v \oplus w.$$

For a nonzero point value, Theorem 4 requires

$$s \oplus w = u \oplus v \preceq M^t(s), \quad s \oplus v = u \oplus w \preceq M^t(s).$$

The map $I \oplus L$ is invertible over $\mathbb{F}_2^N$, because $L^N = 0$ and

$$(I \oplus L)^{-1} = I \oplus L \oplus L^2 \oplus \dots \oplus L^{N-1}.$$

Thus every s can be written uniquely as

$$s = a \oplus L(a),$$

and this parameterization is exactly what the column proof uses. With the code's bit order, $L(a)_i = a_{i+1}$ and $a_N = 0$. Therefore

$$\begin{aligned} (M^t(a \oplus L(a)))_i &= \bigoplus_{j=i+1}^{N-1} (a_j \oplus a_{j+1}) \\ &= a_{i+1} \oplus a_N = a_{i+1} = L(a)_i. \end{aligned}$$

Hence

$$M^t(s) = L(a) \quad \text{when } s = a \oplus L(a).$$

Because the magnitude of a nonzero entry is $2^{-\text{wt}(M^t(s))}$, column maximization is now the problem of minimizing $\text{wt}(L(a))$.

The two support conditions also become purely a -conditions. Substituting $s = a \oplus L(a)$ gives

$$\begin{aligned} u \oplus v &= s \oplus w = a \oplus L(a) \oplus w, \\ u \oplus w &= s \oplus v = a \oplus L(a) \oplus v. \end{aligned}$$

For a fixed coordinate i , the condition

$$a_i \oplus L(a)_i \oplus w_i \leq L(a)_i$$

is equivalent to

$$a_i \oplus w_i \leq L(a)_i.$$

Indeed, if $L(a)_i = 0$, both inequalities require $a_i = w_i$; if $L(a)_i = 1$, both are automatically satisfied. The same argument applies to the v -condition. Therefore the column constraints are

$$a \oplus v \preceq L(a), \quad a \oplus w \preceq L(a).$$

These are precisely the side conditions in the column-max section of Schulte–Geers. Once a is chosen, the output mask is forced by $u = s \oplus v \oplus w$:

$$u = (v \oplus w) \oplus a \oplus L(a).$$

This is the formula implemented by

$$\text{column_best_u_of.}$$

For that candidate,

$$s = u \oplus v \oplus w = a \oplus L(a), \quad M^t(s) = L(a).$$

Theorem 7 gives a constructive minimizer under the column assumption

$$v_{N-1} = w_{N-1} = 1.$$

It defines

$$a(v, w) = a_L(v \wedge w) \vee m(v, w),$$

and then

$$u^*(v, w) = (v \oplus w) \oplus a(v, w) \oplus L(a(v, w)).$$

The theorem states that $a(v, w)$ has minimal $\text{wt}(L(a))$ among the solutions of the column constraints, and therefore

$$|\phi_2(u^*(v, w), v, w)| = \max_{u \in \mathbb{F}_2^N} |\phi_2(u, v, w)|.$$

The sign is not supplied by the minimization theorem. It is read back from the point formula:

$$\text{sgn } \phi_2(u^*, v, w) = (-1)^{((u^* \oplus w) \wedge (u^* \oplus v)) \text{ parity}}.$$

This is why `column_best_correlation_value` first calls `column_best_u_of` and then evaluates `linear_correlation_add_ccz_value` on the resulting triple [SG13].

Global column entry in the C++ code.

Lemma 4 (Global reduction to the active Schulte–Geers column). *Theorem 7 assumes the top coordinate of the current word is shared by v and w . The public code entry handles arbitrary N -bit (v, w) by first cutting to the real active top. Let*

$$t = v \vee w.$$

If $t = 0$, then $(v, w) = (0, 0)$ and the code returns $u^* = 0$. Otherwise let

$$p = \text{msb}(t).$$

If the highest active bit is not shared,

$$(v \wedge w)_p = 0,$$

then the whole column is zero. To see this, suppose $\phi_2(u, v, w) \neq 0$, and set

$$h = \text{msb}(u \vee v \vee w).$$

All coordinates above h of $s = u \oplus v \oplus w$ are zero, hence $(M^t s)_h = 0$. The support tests then force

$$(u \oplus v)_h = (u \oplus w)_h = 0, \quad \text{so} \quad u_h = v_h = w_h.$$

If p is the highest active coordinate of $v \vee w$ and $v_p \neq w_p$, this is impossible. If $v_p = w_p = 1$, then no nonzero entry needs coordinates above p ; any such coordinate would become the h above and force $u_h = v_h = w_h = 0$, a contradiction. The code can therefore reduce to width $p + 1$ before applying Theorem 7.

The exact top-level behavior of `find_optimal_output_u_ccz` is

$$\begin{aligned} v \vee w = 0 &\implies u^* = 0, \\ p = \text{msb}(v \vee w), (v \wedge w)_p = 0 &\implies u^* = 0, \\ (v \wedge w)_p = 1 &\implies \text{apply Theorem 7 in width } p + 1. \end{aligned}$$

The returned word is masked back to the original N -bit width. In a zero-column case, the returned 0 is only a conventional representative of a column whose maximum absolute value is 0. The wrapper `linear_correlation_add_phi2_column_max` still calls the point-weight routine; if the point is infeasible, the optional result is empty and the code stores the fallback weight N . Thus the weight field is exact for nonzero columns and a sentinel for the hard-zero column.

The $m(v, w)$ completion used by the column code.

Construction 3 (Column completion $m(v, w)$). Inside the reduced width, the C++ implementation follows the constructive description of $m(v, w)$ preceding Schulte–Geers’ Theorem 7 [SG13]. The first forced condition is obtained directly from the column constraints:

$$a \oplus v \preceq L(a), \quad a \oplus w \preceq L(a) \implies v \oplus w \preceq L(a).$$

Because R is the opposite shift of L , this implies

$$R(v \oplus w) \preceq a.$$

Accordingly, `m_of` starts with the seed

$$a_0 = R(v \oplus w).$$

Let

$$o = v \vee w, \quad h = v \wedge w, \quad Z = \bar{o} \wedge \mathcal{M}_N.$$

The zero-run closure is the least mask $\text{Close}_Z(a_0)$ containing a_0 and satisfying

$$Z_i = 1, \quad a_i = 1, \quad i < N - 1 \implies a_{i+1} = 1.$$

The function

`propagate_ones_right_through_v_or_w_zero_mask`

computes this closure by doubling the length of zero-runs already known to be covered:

$$\begin{aligned} a &\leftarrow a \vee ((a \wedge r) \ll s), \\ r &\leftarrow r \wedge (r \gg s), \end{aligned} \quad s = 1, 2, 4, \dots$$

Here r is the current mask of zero-runs of length at least s . This is rule (1) in implementation form:

$$v_i = w_i = 0, \quad a_i = 1 \implies a_{i+1} = 1.$$

Rule (2) starts at coordinates where $v \vee w$ has a 1 followed by a zero, but $v \wedge w$ is zero at the starting coordinate:

$$S = (o \wedge \overline{o \gg 1} \wedge \bar{h}) \wedge (\mathcal{M}_N \oplus 2^{N-1}),$$

which is the helper

`m_of_rule2_start_mask`.

The seed used by the main `m_of` routine is therefore

$$R(v \oplus w) \vee (S \ll 1).$$

Closing this union through $Z = \overline{v \vee w}$ gives the same result as closing the rule-(2) seed first and then applying rule (1), because the closure operator is monotone and idempotent. Thus the implementation-level formula is

$$m(v, w) = \text{Close}_{\overline{v \vee w}}(R(v \oplus w) \vee (S \ll 1)).$$

The helper `m_of_rule2_fill_mask_parallel` is the rule-(2) seed ($S \ll 1$) followed by the same closure, without the compulsory $R(v \oplus w)$ part. The main `m_of` routine uses the union and closes once.

The completed column vector is then

$$a(v, w) = a_L(v \wedge w) \vee m(v, w),$$

and the code path is

$$\begin{aligned} (v, w) &\mapsto a(v, w) \text{ via } \text{column_best_a_of}, \\ a(v, w) &\mapsto u^*(v, w) \text{ via } \text{column_best_u_of}, \\ (u^*, v, w) &\mapsto Q_1^{\text{LC},+}(u^*, v, w) \text{ via the point evaluator.} \end{aligned}$$

The column record

`Phi2ColumnMax`

stores only

$(u^*, \text{z_weight})$.

It is produced by

`linear_correlation_add_phi2_column_max,`

which first constructs u^* and then calls the point-weight routine on (u^*, v, w) . This is a column completion followed by Q_1 evaluation, not a new signed point formula.

Algorithm 16 Schulte-Geers column completion, expanded from the C++ implementation

```

1: function RULETWOSTARTMASK( $o, h, N$ )
2:    $o \leftarrow o \wedge \text{MASK}(N)$ ,  $h \leftarrow h \wedge \text{MASK}(N)$ 
3:   if  $N \leq 1$  then
4:     return 0
5:   end if
6:    $S \leftarrow o \wedge \neg(o \gg 1) \wedge \neg h$ 
7:    $S \leftarrow S \wedge \neg(1 \ll (N - 1))$ 
8:   return  $S \wedge \text{MASK}(N)$ 
9: end function

10: function ZERORUNCLOSURE( $a, Z, N$ )
11:    $a \leftarrow a \wedge \text{MASK}(N)$ 
12:    $r \leftarrow Z \wedge \text{MASK}(N)$ 
13:    $d \leftarrow 1$ 
14:   while  $d < N$  do
15:      $a \leftarrow a \vee ((a \wedge r) \ll d) \wedge \text{MASK}(N)$ 
16:      $r \leftarrow r \wedge (r \gg d)$ 
17:      $d \leftarrow 2d$ 
18:   end while
19:   return  $a \wedge \text{MASK}(N)$ 
20: end function

21: function MCOMPLETION( $v, w, N$ )
22:    $v \leftarrow v \wedge \text{MASK}(N)$ ,  $w \leftarrow w \wedge \text{MASK}(N)$ 
23:    $o \leftarrow v \vee w$ ,  $h \leftarrow v \wedge w$ ,  $Z \leftarrow \neg o \wedge \text{MASK}(N)$ 
24:    $S \leftarrow \text{RULETWOSTARTMASK}(o, h, N)$ 
25:    $a \leftarrow R(v \oplus w, N) \vee ((S \ll 1) \wedge \text{MASK}(N))$ 
26:    $m \leftarrow \text{ZERORUNCLOSURE}(a, Z, N)$ 
27:   return  $m \wedge \text{MASK}(N)$ 
28: end function

29: function COLUMNBESTATTHEOREMSEVEN( $v, w, N$ )
30:    $v \leftarrow v \wedge \text{MASK}(N)$ ,  $w \leftarrow w \wedge \text{MASK}(N)$ 

```

```

31:  if  $N \leq 0$  then
32:    return nullopt
33:  end if
34:  if  $\text{BIT}(v, N - 1) = 0$  or  $\text{BIT}(w, N - 1) = 0$  then
35:    return nullopt
36:  end if
37:   $m \leftarrow \text{MCOMPLETION}(v, w, N)$ 
38:   $a \leftarrow \text{AL}(v \wedge w, N) \vee m$ 
39:  return  $a \wedge \text{MASK}(N)$ 
40: end function

41: function COLUMNBESTULOCAL( $v, w, N$ )
42:   $a \leftarrow \text{COLUMNBESTATHEOREMSEVEN}(v, w, N)$ 
43:  if  $a = \text{nullopt}$  then
44:    return nullopt
45:  end if
46:   $u \leftarrow (v \oplus w) \oplus a \oplus (a \gg 1)$ 
47:  return  $u \wedge \text{MASK}(N)$ 
48: end function

49: function FINDOPTIMALOUTPUTUCCZ( $v, w, N$ )
50:   $v \leftarrow v \wedge \text{MASK}(N)$ ,  $w \leftarrow w \wedge \text{MASK}(N)$ 
51:   $t \leftarrow v \vee w$ 
52:  if  $t = 0$  then
53:    return 0
54:  end if
55:   $p \leftarrow \text{MSB}(t)$ 
56:  if  $\text{BIT}(v, p) = 0$  or  $\text{BIT}(w, p) = 0$  then
57:    return 0
58:  end if
59:   $N' \leftarrow p + 1$ 
60:   $v' \leftarrow v \wedge \text{MASK}(N')$ ,  $w' \leftarrow w \wedge \text{MASK}(N')$ 
61:   $u \leftarrow \text{COLUMNBESTULOCAL}(v', w', N')$ 
62:  if  $u = \text{nullopt}$  then
63:    return 0
64:  end if
65:  return  $u \wedge \text{MASK}(N)$ 
66: end function

67: function PHITWOCOLUMNMAX( $v, w, N$ )
68:   $u \leftarrow \text{FINDOPTIMALOUTPUTUCCZ}(v, w, N)$ 
69:   $W \leftarrow \text{CCZPOINTWEIGHT}(u, v, w, N)$ 
70:  if  $W = \text{nullopt}$  then
71:     $W \leftarrow N$  ▷ sentinel for a hard-zero column
72:  end if
73:  return  $(u, W)$ 
74: end function

```

The correspondence with the file is exact:

RULETWOSTARTMASK(	) m_of_rule2_start_mask
ZERORUNCLOSURE(	) propagate_ones_right_through_v_or_w_zero_mask
MCOMPLETION(	) m_of
COLUMNBESTATHEOREMSEVEN(	) column_best_a_of
COLUMNBESTULOCAL(	) column_best_u_of
FINDOPTIMALOUTPUTUCCZ(	) find_optimal_output_u_ccz.

The loop invariant in `ZERORUNCLOSURE`(i)s the interval version of Schulte–Geers’ rule (1): after the stage $d = 2^r$, every one of a has been propagated through the zero-run mask Z by all distances below 2^{r+1} that stay inside a zero-run. The update

$$r \leftarrow r \wedge (r \gg d)$$

keeps exactly those starting positions from which a zero-run of length at least $2d$ is still available. Hence, when $d \geq N$, the result is the least mask closed under

$$Z_i = 1, a_i = 1 \implies a_{i+1} = 1.$$

This is the word-parallel version of the constructive $m(v, w)$ used before Theorem 7, and it is why the code can inject the rule-(2) starts as $S \ll 1$ and perform a single closure [SG13].

10.9 Two-Variable Linear Row Completion: Our Fixed-Row Direction

Matrix row objective.

Definition 19 (Fixed-row completion objective). For a fixed output mask u , the matrix row is

$$\mathcal{C}_N[u, *] = \{\phi_2(u, v, w) : (v, w) \in \mathbb{F}_2^N \times \mathbb{F}_2^N\}.$$

The row service used by the project constructs one pair (v, w) attaining the maximum absolute value in this row. This is a different direction from the Schulte–Geers column completion of Theorem 7, because the free variables are different:

column: fixed (v, w) and choose u , row: fixed u and choose (v, w) .

Consequently the row code is not a transposed call to

`find_optimal_output_u_ccz.`

It is a fixed-row constructor built from the row maximum theorem and then checked by the signed point evaluator [SG13]. The code entries are:

L -minimal vector	<code>aL_of,</code>
row input-mask constructor	<code>row_best_b_of,</code>
row exponent	<code>row_best_d_of,</code>
fixed-row record	<code>linear_correlation_add_phi2_row_max.</code>

After constructing the row candidate, the code still uses the same signed point evaluator to check the weight.

Row upper bound from the point formula.

Theorem 12 (Fixed-row completion and row maximum). *The row construction begins with the L -admissible set*

$$A_L(u) = \{x : x \oplus u \preceq L(x)\}.$$

The quantity to minimize in the row proof is

$$d(u) = \min_{x \in A_L(u)} \text{wt}(L(x)).$$

For any nonzero entry in row u , set

$$s = u \oplus v \oplus w.$$

The Theorem-4 support tests give

$$u \oplus v \preceq M^t(s), \quad u \oplus w \preceq M^t(s).$$

Taking their XOR cannot create a one outside $M^t(s)$, so

$$s \oplus u = v \oplus w \preceq M^t(s).$$

Write $s = a \oplus L(a)$. As above,

$$M^t(s) = L(a), \quad a = s \oplus M^t(s).$$

The last support condition is then

$$a \oplus L(a) \oplus u \preceq L(a).$$

Coordinatewise, this is equivalent to

$$a \oplus u \preceq L(a),$$

because coordinates with $L(a)_i = 0$ force $a_i = u_i$, while coordinates with $L(a)_i = 1$ impose no restriction. Therefore $a \in A_L(u)$, and every nonzero row entry satisfies the upper bound

$$|\phi_2(u, v, w)| = 2^{-\text{wt}(M^t(s))} = 2^{-\text{wt}(L(a))} \leq 2^{-d(u)}.$$

Theorem 5 gives one L -minimal vector explicitly:

$$a_L(u) = u \oplus (u \wedge L(u)) \oplus (u \wedge L(u) \wedge L^2(u)) \oplus \cdots \oplus (u \wedge L(u) \wedge \cdots \wedge L^{N-1}(u)).$$

The code computes this same vector in `aL_of`. Equivalently, $a_L(u)$ is obtained by looking at each one-block of u and keeping the parity of the one-run beginning at each bit:

$$a_L(u)_i = (\min\{j \geq i : u_j = 0\} - i) \bmod 2,$$

with the minimum taken as N if no zero occurs before the word boundary. The implementation computes this by placing

$$z_i = \begin{cases} i, & u_i = 0, \\ N, & u_i = 1, \end{cases}$$

then applying a parallel suffix minimum over the z_i . This is the `aL_of` function.

This suffix-min implementation is just the bit-parallel version of the displayed a_L formula. If $u_i = 0$, then the nearest zero is i , so $a_L(u)_i = 0$. If $u_i = 1$, then the parity of the distance to the next zero is the parity of the length of the remaining suffix of the one-block beginning at i . This is the same “replace every second one inside each one-block” construction from the paper, expressed in the code’s LSB-first coordinates.

The row-side input mask returned by the helper is

$$b(u) = a_L(u) \oplus L(a_L(u)) = a_L(u) \oplus (a_L(u) \gg 1).$$

The row completion used in the code is therefore

$$v = b(u), \quad w = u.$$

These masks attain the fixed-row bound:

$$\phi_2(u, b(u), u) = 2^{-d(u)}, \quad d(u) = \text{wt}(L(a_L(u))).$$

The a_L formula and the row bound are Schulte–Geers’ Theorems 5 and 6 [SG13]; the code path here is our fixed-row service that turns those statements into the mask pair needed by the search. Let $a = a_L(u)$. Then the ϕ_2 sum uses

$$s = u \oplus v \oplus w = b(u) = a \oplus L(a).$$

By the suffix definition of M^t ,

$$M^t(a \oplus L(a)) = L(a).$$

Indeed, since $L(a)_i = a_{i+1}$, the suffix XOR of $(a_i \oplus a_{i+1})$ telescopes to a_{i+1} , with the virtual boundary $a_N = 0$. Thus the support mask in the point formula is

$$z = L(a_L(u)).$$

For the chosen row pair,

$$q = u \oplus w = 0, \quad p = u \oplus v = u \oplus a_L(u) \oplus L(a_L(u)).$$

The definition of $a_L(u)$ gives the row-side support condition

$$p \preceq L(a_L(u)).$$

Bitwise, if $u_i = 1$, then $a_L(u)_i$ and $a_L(u)_{i+1}$ have opposite parity along the same one-run, so $p_i = 0$. If $u_i = 0$, then $a_L(u)_i = 0$, and hence $p_i = a_L(u)_{i+1} = L(a_L(u))_i$. Therefore

$$p_i \leq L(a_L(u))_i \quad \text{for every } i.$$

Therefore the ϕ_2 feasibility indicators hold, and the sign is positive because

$$p \cdot q = 0.$$

Its row-side exponent is

$$d(u) = \text{wt}(L(a_L(u))) = \text{wt}(a_L(u) \gg 1),$$

which is the helper `row_best_d_of`. The row-max struct stores

$$(v, w, \text{z_weight}),$$

where the weight is obtained by calling the same point oracle as before:

$$\text{z_weight} = Q_1^{\text{LC},+} \text{ weight field on } (u, b(u), u).$$

So the row construction is not a new Q_1 formula. It is a row-side completion built on top of the signed point evaluator. In particular, `linear_correlation_add_phi2_row_max` does not recompute a different linear-correlation law. It constructs $(v, w) = (b(u), u)$, then uses the point-weight routine

$$\text{linear_correlation_add_ccz_weight}$$

to confirm

$$\text{wt}(M^t(u \oplus b(u) \oplus u)) = \text{wt}(L(a_L(u))).$$

The implementation call order is exactly:

a	$aL_of(u, N),$
v	$\text{row_best_b_of}(u, N) = a \oplus (a \gg 1),$
w	$u,$
W	$\text{linear_correlation_add_ccz_weight}(u, v, w, N).$

The last line should be feasible by the row proof above. The code still keeps the optional return type, because the same point-weight function is also used for arbitrary triples where infeasibility is possible.

Algorithm 17 Fixed-row completion used by the project, expanded from the C++ implementation

```

1: function ALOF( $u, N$ )
2:    $u \leftarrow u \wedge \text{MASK}(N)$ 
3:   for  $i \leftarrow 0$  to  $N - 1$  do
4:     if  $\text{BIT}(u, i) = 0$  then
5:        $z_i \leftarrow i$ 
6:     else
7:        $z_i \leftarrow N$ 
8:     end if
9:   end for
10:   $d \leftarrow 1$ 
11:  while  $d < N$  do
12:    for  $i \leftarrow 0$  to  $N - 1 - d$  do
13:       $z_i \leftarrow \min(z_i, z_{i+d})$ 
14:    end for
15:     $d \leftarrow 2d$ 
16:  end while
17:   $a \leftarrow 0$ 
18:  for  $i \leftarrow 0$  to  $N - 1$  do
19:    if  $(z_i - i)$  is odd then
20:       $a \leftarrow a \vee (1 \ll i)$ 
21:    end if
22:  end for
23:  return  $a \wedge \text{MASK}(N)$ 
24: end function

25: function ROWBESTB( $u, N$ )
26:   $u \leftarrow u \wedge \text{MASK}(N)$ 
27:   $a \leftarrow \text{ALOF}(u, N)$ 
28:  return  $(a \oplus (a \gg 1)) \wedge \text{MASK}(N)$ 
29: end function

30: function ROWBESTD( $u, N$ )
31:   $a \leftarrow \text{ALOF}(u, N)$ 
32:  return  $\text{HAMMINGWEIGHT}(a \gg 1)$ 
33: end function

34: function PHITwoRowMAX( $u, N$ )
35:   $u \leftarrow u \wedge \text{MASK}(N)$ 
36:   $v \leftarrow \text{ROWBESTB}(u, N)$ 
37:   $w \leftarrow u$ 
38:   $W \leftarrow \text{CCZPOINTWEIGHT}(u, v, w, N)$ 
39:  if  $W = \text{nullopt}$  then
40:     $W \leftarrow N$  ▷ defensive fallback; the theorem makes this feasible
41:  end if
42:  return  $(v, w, W)$ 
43: end function

```

The exact code names are

```

      ALOF( ) ↔ aL_of,
    ROWBESTB( ) ↔ row_best_b_of,
    ROWBESTD( ) ↔ row_best_d_of,
    PHITWOROWMAX( ) ↔ linear_correlation_add_phi2_row_max.

```

The suffix-min loop has the invariant that, after the stage $d = 2^r$, z_i is the smallest zero position among

$$i, i+1, \dots, \min(i+2^{r+1}-1, N-1),$$

or N if that interval contains no zero. After the final stage,

$$z_i = \min\{j \geq i : u_j = 0\},$$

with $z_i = N$ if the one-block reaches the boundary. Therefore the bit $(z_i - i) \bmod 2$ is exactly the parity of the suffix of the one-block of u starting at i . This is the implemented form of Schulte–Geers’ $a_L(u)$: it leaves zeros fixed and alternates the ones inside each one-block, counted from the high end of that block [SG13]. With $a = a_L(u)$, ROWBESTB(r) returns $(I \oplus L)(a)$, ROWBESTD(r) returns $\text{wt}(L(a))$, and PHITWOROWMAX(t) then verifies the constructed triple through the same Theorem-4 point oracle rather than through a separate row-specific correlation formula [SG13].

10.10 Two-Variable Linear Subtraction: Addition-Oracle Permutation

Corollary 2 (Two-variable subtraction linear permutation). *For subtraction $C = A - B$, the identity $A = C + B$ gives the linear reduction*

$$\text{Cor}_-(u; v, w) = \text{Cor}_+(v; u, w),$$

where u is now the output mask on C , v the input mask on A , and w the input mask on B .

Algorithm 18 Two-variable modular-subtraction linear oracle

```

1: function SUB2LINEARORACLE( $u, v, w, N$ )
                                     ▷  $C = A - B$  is equivalent to  $A = C + B$ .
2:   return ADD2LINEARORACLE( $v, u, w, N$ )
3: end function

```

The matrix-chain file also exposes historical subtraction wrappers:

```

correlation_sub_varvar_reference, correlation_sub_varvar_logdepth,
linear_sub_varvar32,              linear_sub_varvar64.

```

Their parameter order is documented in the code as (α, γ, β) . Internally they form

$$\beta' = (\bar{\beta} + 1) \wedge \mathcal{M}_N$$

and call the corresponding addition matrix-chain routine. This is a code-level subtraction wrapper in `weight_evaluation.hpp`; the local oracle relation used for $C = A - B$ in the MILP-level text above is the explicit variable permutation through $A = C + B$.

In summary, the two-variable differential Q_1 side is the Lipmaa–Moriai impossible-mask test plus the popcount of the non-equal low $N - 1$ positions. The two-variable differential Q_2 side is the Algorithm-4 construction of $\gamma^*(\alpha, \beta)$, followed by a Q_1 call if the optimal weight is required. The two-variable linear Q_1 side is the exact signed CCZ/Walsh point formula, with the matrix chain as an audit-equivalent signed evaluator and Wallén’s CPM route as a feasibility/weight route for the magnitude. The CCZ file’s column completion and the project-side row completion are separate services built on top of this point oracle; neither is the Q_1 point evaluator itself. Subtraction introduces no new local arithmetic law: both differentially and linearly it is the addition oracle with the variables permuted through $A = C + B$.

11 Fixed-Addend Addition and Subtraction Oracles

11.1 Why the Fixed Addend Is a Separate Local Object

Theorem 13 (Addend-dependence of fixed public constants). *The fixed-addend map*

$$X \mapsto X + K \pmod{2^N}$$

is not the two-variable map $X + Y$ with a zero difference silently substituted on the second input. The second operand is no longer sampled. Its bit pattern is public terrain that controls carry or borrow chains. This is the addend-dependency phenomenon studied by Miyano, Machado, and later bit-vector models for constant modular addition [Miy98, Mac01, ARS⁺20, ARS⁺22]. Therefore the fixed-addend differential and linear Q_1 oracles below have their own state variables, their own impossibility conditions, and, in the differential case, noninteger constant-dependent weights.

11.2 Fixed-Addend Differential Q_1 : Exact Probability, Exact Count, and Code Paths

Definition 20 (Fixed-addend differential Q_1 contract). This subsection keeps three objects separate. First, the mathematical $Q_1^{\text{DP},+K}$ oracle is the exact point probability for one fixed triple (K, u, v) . Second, the standalone C++ evaluator has two equivalent exact routes: an exact count route and a zero-chain logarithmic-weight route. Third, the SCIP box later in the paper is a MILP encoding of the same zero-chain rule, not a new probability law. The recurrence is Machado’s fixed-constant differential recurrence, and the bit-vector/logarithmic closed form is the Azimi–Ranea–Salmasizadeh–Mohajeri–Aref–Rijmen model [Mac01, ARS⁺20, ARS⁺22].

Let

$$F_K(X) = X \boxplus K \pmod{2^N}.$$

For input XOR difference u and output XOR difference v , the exact fixed-addend differential probability is

$$\text{DP}_K(u \rightarrow v) = 2^{-N} \# \{X \in \Omega_N : F_K(X) \oplus F_K(X \oplus u) = v\}.$$

The point oracle is

$$Q_1^{\text{DP},+K}(K, u, v) = \begin{cases} (\text{false}, +\infty), & \text{DP}_K(u \rightarrow v) = 0, \\ (\text{true}, -\log_2 \text{DP}_K(u \rightarrow v)), & \text{DP}_K(u \rightarrow v) > 0. \end{cases}$$

There is no completion variable in this contract. The public word K is part of the local rule, and u, v are already fixed.

Carry-difference identity.

Lemma 5 (Fixed-addend carry-difference identity). *Write the two additions as*

$$Y = X \boxplus K, \quad Y' = (X \oplus u) \boxplus K.$$

Let c_i and c'_i be the carries entering bit i , with

$$c_0 = c'_0 = 0.$$

At bit i ,

$$Y_i = X_i \oplus K_i \oplus c_i, \quad Y'_i = X_i \oplus u_i \oplus K_i \oplus c'_i.$$

Hence

$$v_i = Y_i \oplus Y'_i = u_i \oplus c_i \oplus c'_i.$$

Define the carry-difference bit

$$d_i = c_i \oplus c'_i.$$

Then every valid propagation must satisfy

$$d_i = u_i \oplus v_i \quad \text{for all } i, \quad d_0 = 0.$$

Thus $u_0 = v_0$ is a necessary condition. The last carry-out d_N is not observed in arithmetic modulo 2^N , so the exact count implementation processes the edges $i = 0, \dots, N-2$ and leaves the final input bit free.

Machado conditional recurrence.

Theorem 14 (Machado fixed-addend conditional recurrence). *For the probability derivation, compare the equality of the two bit outputs. Set*

$$e_i = u_i \oplus v_i, \quad u_{-1} = v_{-1} = K_{-1} = 0,$$

and define the three-bit state

$$S_i = (u_{i-1}, v_{i-1}, e_i) \in \{0, \dots, 7\}.$$

Following Machado, let

$$\phi_i = \Pr[Y_i = Y'_i \mid Y_0 = Y'_0, \dots, Y_{i-1} = Y'_{i-1}]$$

and let

$$\delta_i = \Pr[c_i = 1 \mid Y_0 = Y'_0, \dots, Y_i = Y'_i], \quad \delta_{-1} = 0.$$

The exact one-bit recurrence is

$$\delta_i = \begin{cases} (K_{i-1} + \delta_{i-1})/2, & S_i = 000, \\ 0, & S_i = 001, \\ K_{i-1}, & S_i \in \{010, 100, 110\}, \\ \delta_{i-1}, & S_i \in \{011, 101\}, \\ 1/2, & S_i = 111, \end{cases}$$

and

$$\phi_i = \begin{cases} 1, & S_i = 000, \\ 0, & S_i = 001, \\ 1/2, & S_i \in \{010, 011, 100, 101\}, \\ 1 - (K_{i-1} + \delta_{i-1} - 2K_{i-1}\delta_{i-1}), & S_i = 110, \\ K_{i-1} + \delta_{i-1} - 2K_{i-1}\delta_{i-1}, & S_i = 111. \end{cases}$$

Therefore

$$\text{DP}_K(u \rightarrow v) = \prod_{i=0}^{N-1} \phi_i.$$

This table is the probability theorem. Everything below is either an unrolling of this table into a closed form or an implementation of the same quantity.

Zero-chain closed form.

Theorem 15 (Zero-chain closed form for fixed-addend differential weight). *The state 001 is impossible because $\phi_i = 0$. The states*

$$010, 011, 100, 101$$

have $\phi_i = 1/2$, and therefore contribute one unit of weight. The base part is

$$W_{\text{base}}(u, v) = \sum_{i=1}^{N-1} [u_{i-1} \oplus v_{i-1} = 1].$$

The states 110 and 111 are the only constant-dependent states. For such a trigger position i , define the exact backward zero-chain length by

$$\lambda_i = 1 + \max\{L \geq 0 : S_{i-1} = S_{i-2} = \dots = S_{i-L} = 000\}.$$

The chain stops either at the first lower nonzero state or at the left boundary. Define

$$q_i = u_i \oplus v_i \oplus K_{i-1}.$$

The constant block read by this chain is

$$B_{i,\lambda} = K_{i-\lambda-1} + \sum_{r=0}^{\lambda-2} 2^{\lambda-2-r} K_{i-2-r},$$

where all negative-index constant bits are zero and the sum is empty when $\lambda = 1$. Equivalently,

$$B_{i,\lambda} = K_{i-\lambda-1} + \sum_{j=i-\lambda}^{i-2} 2^{j-(i-\lambda)} K_j.$$

The numerator of the local conditional probability is

$$p_{i,\lambda,q} = \begin{cases} B_{i,\lambda}, & q = 1, \\ 2^{\lambda-1} - B_{i,\lambda}, & q = 0. \end{cases}$$

For a trigger position,

$$\phi_i = \frac{p_{i,\lambda_i,q_i}}{2^{\lambda_i-1}}.$$

The reason is explicit from the δ -recurrence. Along the lower zero-chain $S_{i-1}, \dots, S_{i-\lambda_i+1} = 000$, the update $\delta_j = (K_{j-1} + \delta_{j-1})/2$ is iterated. Starting from the boundary value at $i - \lambda_i$, this gives

$$\delta_{i-1} = \frac{1}{2^{\lambda_i-1}} \left(K_{i-\lambda_i-1} + \sum_{r=0}^{\lambda_i-2} 2^{\lambda_i-2-r} K_{i-2-r} \right) = \frac{B_{i,\lambda_i}}{2^{\lambda_i-1}}.$$

For $S_i = 111$ the table uses $\phi_i = \delta_{i-1}$ exactly when $q_i = 1$, and for $S_i = 110$ it uses $\phi_i = 1 - \delta_{i-1}$ exactly when $q_i = 0$. Thus the two cases above are not a convention; they are the two possible numerators of the same conditional probability. If any trigger has $p_{i,\lambda_i,q_i} = 0$, then the propagation is impossible. Otherwise the exact fixed-addend differential weight is

$$W_K^{\text{diff}}(u, v) = W_{\text{base}}(u, v) + \sum_{i: S_i \in \{110, 111\}} ((\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}).$$

This is the exact Q_1 weight. It can be nonintegral because the public constant determines the numerator $p_{i,\lambda,q}$.

Exact-count implementation.

Lemma 6 (Exact-count evaluator for fixed-addend differentials). *The ARX library also implements the same Q_1 probability as an exact count. This is the route behind*

diff_addconst_exact_count_n_impl,
diff_addconst_exact_probability_n_impl,
diff_addconst_exact_weight_n_impl.

For $i = 0, \dots, N - 2$, define the two-state carry-edge matrix

$$E_i(c, c^+) = \sum_{x \in \{0,1\}} \left[\begin{array}{l} \text{Maj}(x, K_i, c) = c^+ \\ \wedge \text{Maj}(x \oplus u_i, K_i, c \oplus d_i) = c^+ \oplus d_{i+1} \end{array} \right],$$

where $d_i = u_i \oplus v_i$. With row vector

$$(A_0, B_0) = (1, 0),$$

the uncompressed exact count is

$$(A_{i+1}, B_{i+1}) = (A_i, B_i)E_i, \quad 0 \leq i < N - 1,$$

and

$$\#\{X : F_K(X) \oplus F_K(X \oplus u) = v\} = 2(A_{N-1} + B_{N-1}).$$

The factor 2 is the free most significant input bit: modulo 2^N does not constrain the final carry-out.

The implementation does not multiply every E_i literally. It uses the following exact edge cases for the pair (d_i, d_{i+1}) :

(d_i, d_{i+1})	row-vector update
(1, 1)	$(A, B) \mapsto (A, B),$
(1, 0)	$\begin{cases} (A + B, 0), & K_i = 0, \\ (0, A + B), & K_i = 1, \end{cases}$
(0, 1)	$\begin{cases} \text{impossible}, & u_i = 0, \\ (B, B), & u_i = 1, K_i = 0, \\ (A, A), & u_i = 1, K_i = 1. \end{cases}$

For a run of $(d_i, d_{i+1}) = (0, 0)$, the code groups adjacent positions that have the same constant bit. Suppose the segment length is ℓ , the common constant bit is k , and the input-difference segment is

$$U = \sum_{t=0}^{\ell-1} u_{i+t} 2^t.$$

Let $M = 2^\ell$. If $U = 0$, then

$$(A, B) \mapsto \begin{cases} (MA + (M - 1)B, B), & k = 0, \\ (A, (M - 1)A + MB), & k = 1. \end{cases}$$

If $U \neq 0$, let r be the least index with $u_{i+r} = 1$, and set

$$\rho = 2^\ell - 2^{\ell-r}.$$

Then

$$(A, B) \mapsto \begin{cases} (MA + \rho B, 0), & k = 0, \\ (0, \rho A + MB), & k = 1. \end{cases}$$

These formulas are the closed segment product implemented by the helper

apply_addconst_zero_run
_count_segment_generic.

They are not a heuristic shortcut: they are the exact product of the corresponding local carry-edge matrices.

Proof. For a fixed prefix, the only hidden state needed by the two additions is the entering carry c_i of the first addition; the entering carry of the second addition is then $c_i \oplus d_i$, because the output condition is $d_i = c_i \oplus c'_i$. The entry $E_i(c, c^+)$ counts exactly the choices of the next input bit x_i that move from this entering carry to c^+ while also producing the required next carry difference d_{i+1} . Induction on i therefore shows that (A_i, B_i) counts all valid prefixes ending in carry 0 or 1. After bit $N - 2$, the most significant input bit is free because the final carry-out is discarded modulo 2^N , giving the factor 2.

The displayed special cases are obtained by enumerating the two choices of x_i in E_i . On a maximal $(d_i, d_{i+1}) = (0, 0)$ run with constant $K_i = k$, multiplying the same two-state matrices gives the stated closed forms; the case $U = 0$ is a triangular matrix power, and the case $U \neq 0$ splits at the least active bit of U . Hence the grouped implementation is the same integer matrix product as the uncompressed exact count. $\square$

Algorithm 19 Fixed-addend differential Q_1 : zero-chain evaluator

```

1: function STATEINDEX( $u, v, i$ )
2:    $a \leftarrow \begin{cases} 0, & i = 0 \\ \text{BIT}(u, i - 1), & i > 0 \end{cases}$ 
3:    $b \leftarrow \begin{cases} 0, & i = 0 \\ \text{BIT}(v, i - 1), & i > 0 \end{cases}$ 
4:    $e \leftarrow \text{BIT}(u, i) \oplus \text{BIT}(v, i)$ 
5:   return  $4a + 2b + e$ 
6: end function

7: function ZEROCHAINLENGTH( $u, v, i$ )
8:    $\lambda \leftarrow 1$ 
9:    $j \leftarrow i - 1$ 
10:  while  $j \geq 0$  and STATEINDEX( $u, v, j$ ) = 0 do
11:     $\lambda \leftarrow \lambda + 1$ 
12:     $j \leftarrow j - 1$ 
13:  end while
14:  return  $\lambda$ 
15: end function

16: function CONSTANTBLOCKVALUE( $K, i, \lambda$ )
17:    $B \leftarrow \text{BIT}(K, i - \lambda - 1)$ 
18:   for  $r \leftarrow 0$  to  $\lambda - 2$  do
19:      $B \leftarrow B + 2^{\lambda - 2 - r} \text{BIT}(K, i - 2 - r)$ 
20:   end for
21:   return  $B$ 
22: end function

23: function FIXEDADDENDDIFFORACLE( $K, u, v, N$ )
24:    $W \leftarrow 0$ 
25:   for  $i \leftarrow 0$  to  $N - 1$  do
26:      $S \leftarrow \text{STATEINDEX}(u, v, i)$ 
27:     if  $S = 1$  then
```

```

28:     return (false, +∞)
29: end if
30: if  $i \geq 1$  and  $\lfloor S/2 \rfloor \in \{1, 2\}$  then
31:      $W \leftarrow W + 1$ 
32: end if
33: if  $i \geq 1$  and  $\lfloor S/2 \rfloor = 3$  then
34:      $\lambda \leftarrow \text{ZEROCHAINLENGTH}(u, v, i)$ 
35:      $q \leftarrow \text{BIT}(u, i) \oplus \text{BIT}(v, i) \oplus \text{BIT}(K, i - 1)$ 
36:      $B \leftarrow \text{CONSTANTBLOCKVALUE}(K, i, \lambda)$ 
37:     if  $q = 1$  then
38:          $p \leftarrow B$ 
39:     else
40:          $p \leftarrow 2^{\lambda-1} - B$ 
41:     end if
42:     if  $p = 0$  then
43:         return (false, +∞)
44:     end if
45:      $W \leftarrow W + (\lambda - 1) - \log_2 p$ 
46: end if
47: end for
48: return (true,  $W$ )
49: end function

```

Algorithm 20 Fixed-addend differential Q_1 : exact-count evaluator

```

1: function ZERORUNCOUNTSEGMENT( $A, B, k, U, \ell$ )
2:      $M \leftarrow 2^\ell$ 
3:     if  $U = 0$  then
4:         if  $k = 0$  then
5:             return ( $MA + (M - 1)B$ ,  $B$ )
6:         else
7:             return ( $A$ ,  $(M - 1)A + MB$ )
8:         end if
9:     end if
10:     $r \leftarrow \min\{t : 0 \leq t < \ell, \text{BIT}(U, t) = 1\}$ 
11:     $\rho \leftarrow 2^\ell - 2^{\ell-r}$ 
12:    if  $k = 0$  then
13:        return ( $MA + \rho B$ ,  $0$ )
14:    else
15:        return ( $0$ ,  $\rho A + MB$ )
16:    end if
17: end function

18: function FIXEDADDENDEXACTCOUNT( $K, u, v, N$ )
19:      $d \leftarrow u \oplus v$ 
20:     if  $\text{BIT}(d, 0) = 1$  then
21:         return 0
22:     end if
23:      $(A, B) \leftarrow (1, 0)$ 
24:      $i \leftarrow 0$ 
25:     while  $i + 1 < N$  do
26:          $\pi \leftarrow (\text{BIT}(d, i), \text{BIT}(d, i + 1))$ 
27:         if  $\pi = (1, 1)$  then

```

```

28:       $\ell \leftarrow 1$ 
29:      while  $i + \ell < N$  and  $\text{BIT}(d, i + \ell) = 1$  do
30:           $\ell \leftarrow \ell + 1$ 
31:      end while
32:       $i \leftarrow i + \ell - 1$ 
33:  else if  $\pi = (0, 0)$  then
34:       $r \leftarrow i$ 
35:      while  $r < N$  and  $\text{BIT}(d, r) = 0$  do
36:           $r \leftarrow r + 1$ 
37:      end while
38:       $z_{\text{end}} \leftarrow r - 1$ 
39:      while  $i < z_{\text{end}}$  do
40:           $k \leftarrow \text{BIT}(K, i)$ 
41:           $\ell \leftarrow 1$ 
42:          while  $i + \ell < z_{\text{end}}$  and  $\text{BIT}(K, i + \ell) = k$  do
43:               $\ell \leftarrow \ell + 1$ 
44:          end while
45:           $U \leftarrow \sum_{t=0}^{\ell-1} \text{BIT}(u, i + t) 2^t$ 
46:           $(A, B) \leftarrow \text{ZERORUNCOUNTSEGMENT}(A, B, k, U, \ell)$ 
47:          if  $A = B = 0$  then
48:              return 0
49:          end if
50:           $i \leftarrow i + \ell$ 
51:      end while
52:  else if  $\pi = (1, 0)$  then
53:      if  $\text{BIT}(K, i) = 0$  then
54:           $(A, B) \leftarrow (A + B, 0)$ 
55:      else
56:           $(A, B) \leftarrow (0, A + B)$ 
57:      end if
58:       $i \leftarrow i + 1$ 
59:  else
60:      if  $\text{BIT}(u, i) = 0$  then
61:          return 0
62:      end if
63:      if  $\text{BIT}(K, i) = 0$  then
64:           $(A, B) \leftarrow (B, B)$ 
65:      else
66:           $(A, B) \leftarrow (A, A)$ 
67:      end if
68:       $i \leftarrow i + 1$ 
69:  end if
70: end while
71: return  $2(A + B)$ 
72: end function

```

The probability and weight returned by the exact-count route are

$$\text{DP}_K(u \rightarrow v) = 2^{-N} \text{FIXEDADDENDEXACTCOUNT}(K, u, v, N),$$

and

$$W = N - \log_2 \text{FIXEDADDENDEXACTCOUNT}(K, u, v, N)$$

when the count is nonzero.

Azimi $\log\text{-}\pi$ and Q_κ fixed-point route.

Theorem 16 (Azimi $\log\text{-}\pi$ and Q_κ fixed-point route). *The library also keeps the Azimi bit-vector structure visible. In the paper's parallel notation, define*

$$s_{000} = \neg(u \ll 1) \wedge \neg(v \ll 1),$$

truncated to N bits, and

$$s'_{000} = s_{000} \wedge \neg \text{LZ}(\neg s_{000}).$$

The vector s'_{000} marks exactly the 000 states that belong to zero-chains below trigger states; trailing 000 states at the top of the word are removed by LZ. Therefore

$$\sum_{i \in I} (\lambda_i - 1) = \text{wt}(s'_{000}),$$

where I is the set of constant-dependent trigger positions whose zero-chain contributes a nontrivial numerator [ARS⁺20, ARS⁺22]. The code reconstructs the same numerator

$$p_{i,\lambda,q}$$

chain by chain, accumulates either $\log_2 p_{i,\lambda,q}$ for the exact real weight or the integer and fractional pieces of an approximate logarithm for Q_κ .

The original bit-vector expression constructs additional masks

$$\begin{aligned} t &= \neg s'_{000} \wedge (s'_{000} \ll 1), \\ t' &= s'_{000} \wedge \neg(s'_{000} \ll 1), \\ s &= ((K \ll 1) \wedge t) \vee (K \wedge (s'_{000} \ll 1)), \\ q &= (\neg((K \ll 1) \oplus u \oplus v) \ll 1) \wedge t', \end{aligned}$$

then propagates q across each marked chain by

$$d = \text{RevCarry}(s'_{000}, q) \vee q,$$

and forms

$$w = (q \wedge (s \wedge d)) \vee (s \wedge \neg d).$$

For every trigger $i \in I$, the subvector $w[i-1, i-\lambda_i]$ is exactly the binary representation of p_{i,λ_i,q_i} . The paper then computes

$$\begin{aligned} r &= \text{ParallelLog}((w \wedge s'_{000}) \ll 1, s'_{000} \ll 1), \\ f &= \text{ParallelTrunc}(w \ll 1, \text{RevCarry}((w \wedge s'_{000}) \ll 1, s'_{000} \ll 1)), \end{aligned}$$

so that r is the sum of $\lfloor \log_2 p_i \rfloor$ and f is the sum of the four fraction-bit truncations. The C++ path does not need to build the global w, r, f words: it scans the same runs in s'_{000} and reads each p_i directly. The following pseudocode is that direct C++ path.

Algorithm 21 Fixed-addend differential Q_1 : direct $\log\text{-}\pi$ and Q_κ paths

```

1: function PIATRUN( $K, u, v, L, H, N$ )
2:    $i \leftarrow H + 1$ 
3:    $\lambda \leftarrow H - L + 2$ 
4:    $B \leftarrow 0$ 
5:   for  $t \leftarrow 0$  to  $\lambda - 2$  do
6:      $B \leftarrow B + 2^t \text{BIT}(K, i - \lambda + t)$ 
7:   end for
```

```

8:    $B \leftarrow B + \text{BIT}(K, i - \lambda - 1)$ 
9:    $q \leftarrow \text{BIT}(u, i) \oplus \text{BIT}(v, i) \oplus \text{BIT}(K, i - 1)$ 
10:  if  $q = 1$  then
11:    return  $B$ 
12:  else
13:    return  $2^{\lambda-1} - B$ 
14:  end if
15: end function

16: function  $\text{FIXEDADDENDLOGPIWEIGHT}(K, u, v, N)$ 
17:  if  $\text{FIXEDADDENDEXACTCOUNT}(K, u, v, N) = 0$  then
18:    return  $+\infty$ 
19:  end if
20:   $s \leftarrow (\neg(u \ll 1) \wedge \neg(v \ll 1)) \wedge \text{MASK}(N)$ 
21:   $s' \leftarrow s \wedge \neg\text{LEADINGZEROS}(\neg s, N)$ 
22:   $W \leftarrow \text{wt}(((u \oplus v) \ll 1) \wedge \text{MASK}(N)) + \text{wt}(s')$ 
23:  for all one-runs  $[L, H]$  of  $s'$  do
24:     $p \leftarrow \text{PIATRUN}(K, u, v, L, H, N)$ 
25:    if  $p = 0$  then
26:      return  $+\infty$ 
27:    end if
28:     $W \leftarrow W - \log_2 p$ 
29:  end for
30:  return  $W$ 
31: end function

32: function  $\text{FIXEDADDENDBVWEIGHTKAPPA}(K, u, v, N, \kappa)$ 
33:  if  $\text{FIXEDADDENDEXACTCOUNT}(K, u, v, N) = 0$  then
34:    return invalid
35:  end if
36:  construct  $s'$  as in  $\text{FIXEDADDENDLOGPIWEIGHT}$ 
37:   $I \leftarrow \text{wt}(((u \oplus v) \ll 1) \wedge \text{MASK}(N)) + \text{wt}(s')$ 
38:   $R \leftarrow 0$ 
39:  for all one-runs  $[L, H]$  of  $s'$  do
40:     $p \leftarrow \text{PIATRUN}(K, u, v, L, H, N)$ 
41:    if  $p = 0$  then
42:      return invalid
43:    end if
44:     $m \leftarrow \lfloor \log_2 p \rfloor$ 
45:     $I \leftarrow I - m$ 
46:     $R \leftarrow R + \text{KAPPABITSAFTERMSB}(p, m, \kappa)$ 
47:  end for
48:  return  $(I \ll \kappa) - R$ 
49: end function

```

For $\kappa = 4$, this is the same quantity as Azimi's BvWeight in Algorithm 1, represented as a Q_4 fixed-point integer [ARS⁺20, ARS⁺22]. The paper writes the parallel version with RevCarry , LZ , ParallelLog , and ParallelTrunc . The C++ path

diff_addconst_bvweight_fixed_point_n_impl

computes the same fixed-point value by explicitly enumerating the zero-chain runs and extracting the κ bits after the most significant bit of each p_i . That direct form is easier to audit in a C++ search implementation; it is not a different mathematical model.

Code-name map.

Code object	Mathematical role
differential_oracle:: fixed_addend_state	$S_i = (u_{i-1}, v_{i-1}, u_i \oplus v_i)$.
differential_oracle:: fixed_addend_block_value	$B_{i,\lambda}$ with negative-index constant bits equal to zero.
differential_oracle:: fixed_addend_chain	$p_{i,\lambda,q}$.
_probability_numerator	
differential_oracle:: oracle_add_const	zero-chain exact $Q_1^{\text{DP},+K}$ evaluator.
differential_oracle:: oracle_sub_const	same evaluator with $K \leftarrow -K \bmod 2^N$.
diff_addconst_exact _count_n_impl	exact carry-difference count $2(A + B)$.
diff_addconst_weight _log2pi_n_impl	exact Azimi $\log\pi$ weight.
diff_addconst_bvweight _fixed_point_n_impl	Q_κ fixed-point BvWeight $^\kappa$.
arithmetic_model:: add_fixed_public	SCIP one-hot MILP encoding of the same zero-chain Q_1 rule.
_constant_exact	
arithmetic_model:: add_fixed_public	SCIP wrapper with $K \leftarrow -K \bmod 2^N$.
_constant_sub_exact	

11.3 Fixed-Addend Linear Q_1 : Signed-Numerator Oracle

Definition 21 (Fixed-addend linear signed-correlation contract). This subsection concerns only the fixed public addend map

$$F_K : X \mapsto Y = X \boxplus K \pmod{2^N},$$

where X is uniformly random and K is fixed. It is not the two-variable Wallen/Schulte-Geers oracle, and it is not a linear-hull aggregation. The local Q_1 value is the exact signed Walsh correlation

$$C_K(\alpha, \beta) = 2^{-N} \sum_{X \in \Omega_N} (-1)^{\alpha \cdot X \oplus \beta \cdot F_K(X)}.$$

Miyano formulates the addend-dependence of this quantity as a function of the fixed addend K [Miy98]. The implementation uses the equivalent two-state carry-transfer form described below.

11.3.1 Carry-State Derivation

Theorem 17 (Fixed-addend linear signed-numerator recurrence). *Let c_i be the carry entering bit i . Then*

$$c_0 = 0, \quad Y_i = X_i \oplus K_i \oplus c_i, \quad c_{i+1} = \text{Maj}(X_i, K_i, c_i).$$

The phase separates into one random part, one public constant part, and one carry-state part:

$$\alpha \cdot X \oplus \beta \cdot Y = \bigoplus_{i=0}^{N-1} (\alpha_i \oplus \beta_i) X_i \oplus \bigoplus_{i=0}^{N-1} \beta_i K_i \oplus \bigoplus_{i=0}^{N-1} \beta_i c_i.$$

Set

$$t_i = \alpha_i \oplus \beta_i, \quad b_i = \beta_i.$$

For $c, c' \in \{0, 1\}$, define the normalized one-bit transfer

$$M_i(c, c') = \frac{1}{2} \sum_{\substack{x \in \{0, 1\}: \\ \text{Maj}(x, K_i, c) = c'}} (-1)^{t_i x \oplus b_i K_i \oplus b_i c}.$$

If $R_i = (R_i^{(0)}, R_i^{(1)})$ is the row vector of signed prefix averages conditioned on the carry entering bit i , then

$$R_0 = (1, 0), \quad R_{i+1} = R_i M_i, \quad C_K(\alpha, \beta) = R_N^{(0)} + R_N^{(1)}.$$

The exact integer numerator version multiplies every step by 2. Let

$$S_i = 2^i R_i = (P_i, Q_i).$$

Then

$$S_0 = (1, 0), \quad S_{i+1} = S_i A_i, \quad A_i = 2M_i,$$

and the final numerator is

$$\mathcal{N} = P_N + Q_N, \quad C_K(\alpha, \beta) = 2^{-N} \mathcal{N}.$$

Thus the exact signed Q_1 value is $2^{-N} \mathcal{N}$. If $\mathcal{N} = 0$, the mask pair is impossible for a nonzero single-trail linear contribution; otherwise

$$W_K^{\text{lin}}(\alpha, \beta) = -\log_2 |C_K(\alpha, \beta)| = N - \log_2 |\mathcal{N}|.$$

There is also a simple triangular-map zero test. Since bit Y_j depends only on $X_0, \dots, X_j$ and contains X_j with coefficient 1, the highest active input and output mask bits must coincide. With

$$h(m) = \max\{i : m_i = 1\}, \quad h(0) = -1,$$

one has

$$C_K(\alpha, \beta) \neq 0 \implies h(\alpha) = h(\beta).$$

The sequential C++ oracle uses this as an early exact rejection before running the carry recurrence.

11.3.2 Integer Transfer Table

Lemma 7 (Integer transfer table for fixed-addend linear correlation). *The integer transfer A_i is the matrix used by the exact numerator and by the MILP state recurrence. Written in the row-vector convention $(P_{i+1}, Q_{i+1}) = (P_i, Q_i) A_i$, the expanded updates are as follows. For $K_i = 0$,*

t_i	b_i	P_{i+1}	Q_{i+1}
0	0	$2P_i + Q_i$	Q_i
0	1	$2P_i - Q_i$	$-Q_i$
1	0	Q_i	$-Q_i$
1	1	$-Q_i$	Q_i

For $K_i = 1$,

t_i	b_i	P_{i+1}	Q_{i+1}
0	0	P_i	$P_i + 2Q_i$
0	1	$-P_i$	$-P_i + 2Q_i$
1	0	P_i	$-P_i$
1	1	$-P_i$	P_i

These two tables are obtained by enumerating the two choices $x_i \in \{0, 1\}$ under the full-adder constraint $c_{i+1} = \text{Maj}(x_i, K_i, c_i)$. No approximation or independence assumption is used.

Proof. The two-state numerator recurrence keeps the signed contribution of all prefixes ending with carry 0 and carry 1. For fixed (K_i, t_i, b_i) , appending bit $x_i \in \{0, 1\}$ contributes the corresponding linear phase sign and moves the carry according to $c_{i+1} = \text{Maj}(x_i, K_i, c_i)$. Enumerating the two choices of x_i for each entering carry $c_i \in \{0, 1\}$ gives four integer contributions, which are exactly the four rows displayed for $K_i = 0$ and for $K_i = 1$. Thus the tables are the expanded form of the exact transfer matrix A_i . $\square$

Algorithm 22 Fixed-addend linear Q_1 exact numerator

```

1: function FIXEDADDENDLINEARNUMERATOR( $K, \alpha, \beta, N$ )
2:   if HIGHESTACTIVEBIT( $\alpha, N$ )  $\neq$  HIGHESTACTIVEBIT( $\beta, N$ ) then
3:     return (0, 0,  $+\infty$ )
4:   end if
5:   ( $P, Q$ )  $\leftarrow$  (1, 0)
6:   for  $i \leftarrow 0$  to  $N - 1$  do
7:      $k \leftarrow \text{BIT}(K, i)$ 
8:      $t \leftarrow \text{BIT}(\alpha, i) \oplus \text{BIT}(\beta, i)$ 
9:      $b \leftarrow \text{BIT}(\beta, i)$ 
10:    if  $k = 0$  then
11:      if ( $t, b$ ) = (0, 0) then
12:        ( $P, Q$ )  $\leftarrow$  ( $2P + Q, Q$ )
13:      else if ( $t, b$ ) = (0, 1) then
14:        ( $P, Q$ )  $\leftarrow$  ( $2P - Q, -Q$ )
15:      else if ( $t, b$ ) = (1, 0) then
16:        ( $P, Q$ )  $\leftarrow$  ( $Q, -Q$ )
17:      else
18:        ( $P, Q$ )  $\leftarrow$  ( $-Q, Q$ )
19:      end if
20:    else
21:      if ( $t, b$ ) = (0, 0) then
22:        ( $P, Q$ )  $\leftarrow$  ( $P, P + 2Q$ )
23:      else if ( $t, b$ ) = (0, 1) then
24:        ( $P, Q$ )  $\leftarrow$  ( $-P, -P + 2Q$ )
25:      else if ( $t, b$ ) = (1, 0) then
26:        ( $P, Q$ )  $\leftarrow$  ( $P, -P$ )
27:      else
28:        ( $P, Q$ )  $\leftarrow$  ( $-P, P$ )
29:      end if
30:    end if
31:  end for
32:   $\mathcal{N} \leftarrow P + Q$ 
33:  if  $\mathcal{N} = 0$  then
34:    return (0, 0,  $+\infty$ )
35:  end if
36:  return ( $\mathcal{N}, 2^{-N}\mathcal{N}, N - \log_2 |\mathcal{N}|$ )
37: end function

```

11.3.3 Run-Event Exact Numerator

Theorem 18 (Run-event exact numerator equivalence). *The public evaluator in*

*linear_correlation/
constant_weight_evaluation.hpp*

contains both the sequential recurrence and an exact run-event numerator routine. The run-event routine does not change the mathematical object: it computes the same $\mathcal{N} = 2^N C_K(\alpha, \beta)$. Its purpose is to factor the constant sign and to group commuting kernels inside maximal runs of equal K -bits [Twi26a].

First factor out the public sign

$$(-1)^{\beta \cdot K}$$

and keep

$$t = \alpha \oplus \beta.$$

The remaining $\beta_i c_i$ part is a carry-state sign. Let

$$D = \text{diag}(1, -1)$$

act on the current carry-state row whenever $\beta_i = 1$. Let $B_{0,0}, B_{0,1}, B_{1,0}, B_{1,1}$ be the integer kernels obtained from the above table with $b_i = 0$:

$$B_{0,0} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}, \quad B_{0,1} = \begin{bmatrix} 0 & 0 \\ 1 & -1 \end{bmatrix},$$

$$B_{1,0} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \quad B_{1,1} = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}.$$

Here the first subscript is the constant run bit K_i , and the second is t_i . Inside a maximal constant run, the $\beta = 0$ steps use only $B_{r,0}$ and $B_{r,1}$; these commute in the closed forms used by the C++ code. The positions with $\beta_i = 1$ are synchronization events: the code first applies D , and then applies the corresponding B_{r,t_i} .

For a row $U = (u_0, u_1)$, the closed powers used by the implementation are:

run bit	kernel power	$U \cdot B^k$
0	$B_{0,0}^k$	$((u_0 + u_1)2^k - u_1, u_1)$
0	$B_{0,1}^k$	$\begin{cases} (u_1, -u_1), & k \text{ odd}, \\ (-u_1, u_1), & k \text{ even} \end{cases}$
1	$B_{1,0}^k$	$(u_0, (u_0 + u_1)2^k - u_0)$
1	$B_{1,1}^k$	$(u_0, -u_0).$

The variable σ below records the accumulated denominator exponent; for the exact all-bit evaluator it ends at $\sigma = N$.

Algorithm 23 Run-event exact numerator for $X \boxplus K$

```

1: function FIXEDADDENDRUNEVENTNUMERATOR( $K, \alpha, \beta, N$ )
2:    $K, \alpha, \beta \leftarrow$  the low  $N$  bits of  $K, \alpha, \beta$ 
3:    $\pi \leftarrow \text{PARITY}(\beta \wedge K)$ 
4:    $t \leftarrow \alpha \oplus \beta$ 
5:    $(u_0, u_1, \sigma) \leftarrow (1, 0, 0)$ 
6:   for all maximal runs  $I = [s, s + \ell - 1]$  of equal  $K$ -bit  $r$  do
7:      $\beta_I \leftarrow (\beta \gg s) \wedge (2^\ell - 1)$ 
8:      $t_I \leftarrow (t \gg s) \wedge (2^\ell - 1)$ 
9:      $j \leftarrow 0$ 
10:    while  $j < \ell$  do
11:       $p \leftarrow$  the least index  $p \geq j$  with  $(\beta_I)_p = 1$ , or  $p = \ell$  if none exists
12:       $L \leftarrow p - j$ 

```

```

13:      if  $L > 0$  then
14:           $k_1 \leftarrow \text{wt}((t_I \gg j) \wedge (2^L - 1))$ 
15:           $k_0 \leftarrow L - k_1$ 
16:          apply the closed power for  $B_{r,0}^{k_0}$ 
17:          apply the closed power for  $B_{r,1}^{k_1}$ 
18:           $\sigma \leftarrow \sigma + L$ 
19:      end if
20:      if  $p = \ell$  then
21:          break
22:      end if
23:       $(u_0, u_1) \leftarrow (u_0, -u_1)$   $\triangleright$  apply  $D$ 
24:      apply one step of  $B_{r,(t_I)_p}$ 
25:       $\sigma \leftarrow \sigma + 1$ 
26:       $j \leftarrow p + 1$ 
27:  end while
28: end for
29:  $\mathcal{N} \leftarrow u_0 + u_1$ 
30: if  $\pi = 1$  then
31:      $\mathcal{N} \leftarrow -\mathcal{N}$ 
32: end if
33: return  $(\mathcal{N}, \sigma)$ 
34: end function

```

The exact functions

```

correlation_add_const
_exact_numerator_logdepth,
correlation_add_const
_weight_ceil_int_logdepth,
correlation_add_const
_has_nonzero_correlation_logdepth

```

are wrappers around this integer numerator. The historical word `logdepth` is an API name: the present implementation is the exact run-event semigroup evaluator, not a heuristic approximation.

11.3.4 Flat and Windowed Engineering Evaluators

Proposition 1 (Exact flat and windowed fixed-addend evaluators). *The companion file*

```

linear_correlation/
constant_weight_evaluation_flat.hpp

```

keeps the same algebra but packages the result as a dyadic rational

$$\widehat{C} = \frac{\widehat{\mathcal{N}}}{2^d}.$$

For the exact flat evaluator, $d = N$ and $\widehat{\mathcal{N}} = \mathcal{N}$. It uses the same constant-run table, the same pulled sign $(-1)^{\beta \cdot K}$, and the same $t = \alpha \oplus \beta$ kernels. Its exact online routine is therefore:

Algorithm 24 *Exact BvCorr-FLAT fixed-addend correlation*

1: **function** BVCORRFLATEXACT(K, α, β, N)

```

2:  build or reuse the maximal-run table of the low  $N$  bits of  $K$ 
3:   $\pi \leftarrow \text{PARITY}(\beta \wedge K), \quad t \leftarrow \alpha \oplus \beta$ 
4:  initialize the scaled row  $(u_0, u_1, \sigma) = (1, 0, 0)$ 
5:  for all constant runs  $I$  in low-to-high order do
6:      call the exact run-span processor on  $I, t|_I$ , and  $\beta|_I$ 
7:  end for
8:   $\hat{N} \leftarrow u_0 + u_1$ 
9:  if  $\pi = 1$  then
10:      $\hat{N} \leftarrow -\hat{N}$ 
11:  end if
12:  return  $(\hat{N}, \sigma, \hat{N}/2^\sigma)$ 
13: end function

```

The windowed estimator is deliberately a different engineering object. It restricts the same event algebra to the working set

$$S = \text{supp}(\alpha \oplus \beta) \cup \text{LeftBand}(\beta, L),$$

where

$$\text{LeftBand}(\beta, L) = \bigcup_{\beta_i=1} \{j : \max(0, i - L) \leq j \leq i\}.$$

Bits outside S are skipped in the event product, and the numerator is then renormalized to denominator 2^N . The certified absolute error bound used by the code is

$$|C_K(\alpha, \beta) - \hat{C}_L| \leq 2 \text{wt}(\beta) 2^{-L}.$$

The conservative weight reported by the windowed routine is therefore

$$W_L = -\log_2 \max \left\{ |\hat{C}_L| - 2 \text{wt}(\beta) 2^{-L}, 2^{-N} \right\}.$$

This windowed path is useful for engineering experiments and audits; the strict NeoAlzette linear MILP below does not use it to price the fixed-public subtractions.

Algorithm 25 Windowed BvCorr-FLAT estimator

```

1: function BvCORRFLATWINDOW( $K, \alpha, \beta, N, L$ )
2:    $t \leftarrow \alpha \oplus \beta$ 
3:    $S \leftarrow \text{supp}(t) \cup \text{LeftBand}(\beta, L)$ 
4:   process only active constant runs intersecting  $S$ 
5:   within each active run, process either the exact span, a count-only span, sparse
      $\beta$ -events, or a segmented span
6:   normalize the resulting dyadic numerator to denominator  $2^N$ 
7:    $\delta \leftarrow 2 \text{wt}(\beta) 2^{-L}$ 
8:   return  $(\hat{C}_L, \delta, -\log_2 \max\{|\hat{C}_L| - \delta, 2^{-N}\})$ 
9: end function

```

The relevant implementation names are:

<code>correlation_add_const</code>	sequential normalized two-state chain,
<code>_exact</code>	
<code>correlation_add_const</code>	exact run-event numerator,
<code>_exact_numerator</code>	
<code>_logdepth</code>	
<code>linear_correlation</code>	exact dyadic BvCorr-FLAT evaluator,
<code>_add_const_exact</code>	
<code>_flat_dyadic</code>	
<code>linear_correlation</code>	windowed estimator with certified error bound,
<code>_add_const_flat</code>	
<code>_bin_report</code>	
<code>linear_oracle::</code>	SCIP trace oracle, same two-state recurrence.
<code>oracle_add_const</code>	

11.4 Fixed-Addend Subtraction Wrappers

Corollary 3 (Fixed-addend subtraction by constant negation). *Fixed public subtraction introduces no separate recurrence. For both the differential and linear fixed-addend oracles,*

$$X \boxminus C = X \boxplus (-C \bmod 2^N).$$

Thus

$$Q_1^{\text{DP}, -K}(C, u, v) = Q_1^{\text{DP}, +K}(-C \bmod 2^N, u, v),$$

and

$$Q_1^{\text{LC}, -K}(C, \alpha, \beta) = Q_1^{\text{LC}, +K}(-C \bmod 2^N, \alpha, \beta).$$

The C++ wrappers are

```
oracle_sub_const,
diff_subconst_exact_*,
add_fixed_public_constant_sub_exact.
```

The round builder uses this through

```
add_fixed_constant_subtraction_operation.
```

In the exact model it calls the fixed-public-constant subtraction box, while the optional zero-difference-average branch is only a comparison model and does not use the public constant terrain.

12 MILP Constraints for Two-Variable Addition and Subtraction

12.1 From the Differential Oracle to a One-Hot MILP Box

Proposition 2 (One-hot encoding of the two-variable differential oracle). *The two-variable differential oracle says that*

$$\alpha_{i-1} = \beta_{i-1} = \gamma_{i-1} \implies \alpha_i \oplus \beta_i \oplus \gamma_i = \alpha_{i-1}.$$

MILP solvers do not consume implications directly, so each bit is first expanded into a one-hot state

$$s_{abc,i} \in \{0, 1\}, \quad (a, b, c) \in \{0, 1\}^3,$$

with

$$\sum_{a,b,c} s_{abc,i} = 1.$$

The state variables bind to the difference bits by

$$\begin{aligned}\alpha_i &= s_{100,i} + s_{101,i} + s_{110,i} + s_{111,i}, \\ \beta_i &= s_{010,i} + s_{011,i} + s_{110,i} + s_{111,i}, \\ \gamma_i &= s_{001,i} + s_{011,i} + s_{101,i} + s_{111,i}.\end{aligned}$$

The parity

$$t_i = \alpha_i \oplus \beta_i \oplus \gamma_i$$

is obtained without a separate XOR gadget:

$$t_i = s_{001,i} + s_{010,i} + s_{100,i} + s_{111,i}.$$

Since the virtual previous triple is 000, the boundary is $t_0 = 0$. For $i = 1, \dots, N-1$, the local implication becomes

$$t_i + s_{000,i-1} \leq 1, \quad t_i \geq s_{111,i-1}.$$

The weight bit is

$$w_i = 1 - s_{000,i} - s_{111,i}, \quad 0 \leq i \leq N-2,$$

and the objective contribution is

$$W_+^{\text{diff}} = \sum_{i=0}^{N-2} w_i.$$

For subtraction $C = A - B$, use the identity $A = C + B$:

$$\alpha_{\text{add}} = \gamma, \quad \beta_{\text{add}} = \beta, \quad \gamma_{\text{add}} = \alpha.$$

12.2 Code-Level Derivation of the Two-Variable MILP Boxes

Proposition 3 (Code-to-model map for two-variable MILP boxes). *The C++ helpers make the modeling boundary explicit before the compact local polytope is written down. On the differential side, `oracle_add2(alpha,beta,gamma,bits)` performs the Lipmaa–Moriai feasibility test bit by bit: if the previous triple is equal and the next parity bit disagrees, the trail is impossible. When the trail is feasible, the oracle charges one unit of weight exactly at positions where the current triple is not in $\{000, 111\}$. This is why the MILP layer introduces a binary weight bit p_i with*

$$p_i = 1 - \text{eq}(\alpha_i, \beta_i, \gamma_i).$$

The helper `add_two_input_add_diff(...)` then adds the low-bit boundary $\alpha_0 \oplus \beta_0 = \gamma_0$, the five local weight inequalities for p_i , and the eight parity-cut inequalities for the next triple. Subtraction is not modeled separately: `add_two_input_sub_diff(...)` simply calls the addition box with (α, γ) swapped, i.e.

$$x - y = z \iff z + y = x.$$

On the linear side, `oracle_add2(u,v,w,bits)` computes

$$p = u \oplus v, \quad q = u \oplus w, \quad m = \text{cpm}(u, \neg(p \oplus q), \text{bits}),$$

and returns zero correlation unless

$$(p \& \neg m) = 0 \quad \text{and} \quad (q \& \neg m) = 0.$$

When the correlation is nonzero, its absolute value is $2^{-\text{popcount}(m)}$, and the sign is determined by the parity of $p \& q$. The Fu–Wang–Guo transition helper in the MILP layer rewrites the same local relation as a two-state automaton with state bits s_i , fixes the top boundary to $s_N = 0$, leaves the lower boundary free, and encodes the ten legal local transitions by the eight facet inequalities. Subtraction again reuses addition through the algebraic rewrite

$$x - y = z \iff z + y = x.$$

This code-to-model map is the engineering derivation; the next paragraph extracts the compact local inequalities explicitly and shows the exact convex-hull form.

Deriving the compact local inequalities explicitly.

Theorem 19 (Compact local MILP boxes for two-variable addition). *This paragraph derives the compact local inequalities used by the two standard MILP boxes for modular addition. The differential side follows the Lipmaa–Moriai criterion for XOR differences through addition [LM02]; the compact inequality system is the one used in the MILP-based Speck trail-search model of Fu et al. [FWG⁺16]. The linear side follows Wallen’s matrix representation for linear approximations of addition modulo 2^N [Wal03a], written in the same local polytope form used by Fu et al. [FWG⁺16].*

Differential box. Consider

$$z = x \boxplus y$$

and the XOR difference triple $(\alpha, \beta \rightarrow \gamma)$, meaning

$$(x \boxplus y) \oplus ((x \oplus \alpha) \boxplus (y \oplus \beta)) = \gamma.$$

The Lipmaa–Moriai condition says that whenever

$$\alpha_i = \beta_i = \gamma_i,$$

the next triple must satisfy

$$\alpha_{i+1} \oplus \beta_{i+1} \oplus \gamma_{i+1} = \alpha_i, \quad 0 \leq i \leq N-2.$$

If the current triple is not equal, this local implication does not constrain the next triple. Therefore

$$\text{DP}^+(\alpha, \beta \rightarrow \gamma) = 2^{-\sum_{i=0}^{N-2} (1 - \text{eq}(\alpha_i, \beta_i, \gamma_i))},$$

where

$$\text{eq}(a, b, c) = \begin{cases} 1, & a = b = c, \\ 0, & \text{otherwise.} \end{cases}$$

The local weight bit is

$$p_i = 1 - \text{eq}(\alpha_i, \beta_i, \gamma_i).$$

For one local window, set

$$a = \alpha_i, \quad b = \beta_i, \quad c = \gamma_i, \quad A = \alpha_{i+1}, \quad B = \beta_{i+1}, \quad C = \gamma_{i+1}, \quad p = p_i.$$

The local feasible set is

$$\mathcal{V} = \left\{ (a, b, c, A, B, C, p) \in \{0, 1\}^7 : \begin{array}{l} (a, b, c) \notin \{000, 111\}, \ p = 1, \ A, B, C \text{ arbitrary;} \\ \text{or } (a, b, c) = 000, \ p = 0, \ A \oplus B \oplus C = 0; \\ \text{or } (a, b, c) = 111, \ p = 0, \ A \oplus B \oplus C = 1 \end{array} \right\}.$$

There are $6 \cdot 8 = 48$ points with a non-equal current triple, four points for 000 followed by even parity, and four points for 111 followed by odd parity. Hence

$$|\mathcal{V}| = 56.$$

The first five inequalities characterize

$$p = 1 - \text{eq}(a, b, c) :$$

$$\begin{aligned} b - c + p &\geq 0, \\ a - b + p &\geq 0, \\ -a + c + p &\geq 0, \\ a + b + c - p &\geq 0, \\ -a - b - c - p &\geq -3. \end{aligned}$$

If $p = 0$, the first three inequalities imply

$$b \geq c, \quad a \geq b, \quad c \geq a,$$

so $a = b = c$. Conversely, 000 forces $p = 0$ through the fourth inequality and 111 forces $p = 0$ through the fifth. If the current triple is not equal, setting $p = 0$ contradicts the first three inequalities, so $p = 1$. The third inequality must be written as

$$-a + c + p \geq 0,$$

that is,

$$-\alpha_i + \gamma_i + p_i \geq 0.$$

The remaining eight inequalities exclude exactly the forbidden next-parity patterns when $p = 0$:

$$\begin{aligned} A - b + B + C + p &\geq 0, \\ b + A - B + C + p &\geq 0, \\ b - A + B + C + p &\geq 0, \\ a + A + B - C + p &\geq 0, \\ c - A - B - C + p &\geq -2, \\ -b + A - B - C + p &\geq -2, \\ -b - A + B - C + p &\geq -2, \\ -b - A - B + C + p &\geq -2. \end{aligned}$$

They exclude, respectively,

$$\begin{aligned} 111 \rightarrow 000, \quad 000 \rightarrow 010, \quad 000 \rightarrow 100, \quad 000 \rightarrow 001, \\ 000 \rightarrow 111, \quad 111 \rightarrow 011, \quad 111 \rightarrow 101, \quad 111 \rightarrow 110. \end{aligned}$$

When $p = 1$, inequalities with right side 0 have minimum $-1 + p = 0$, and inequalities with right side -2 have minimum $-3 + p = -2$. Therefore they do not remove any of the 48 non-equal-current-triple patterns.

Substituting the original variables back gives the thirteen differential local inequalities:

$$\begin{cases} \beta_i - \gamma_i + p_i \geq 0, \\ \alpha_i - \beta_i + p_i \geq 0, \\ -\alpha_i + \gamma_i + p_i \geq 0, \\ \alpha_i + \beta_i + \gamma_i - p_i \geq 0, \\ -\alpha_i - \beta_i - \gamma_i - p_i \geq -3, \\ \alpha_{i+1} - \beta_i + \beta_{i+1} + \gamma_{i+1} + p_i \geq 0, \\ \beta_i + \alpha_{i+1} - \beta_{i+1} + \gamma_{i+1} + p_i \geq 0, \\ \beta_i - \alpha_{i+1} + \beta_{i+1} + \gamma_{i+1} + p_i \geq 0, \\ \alpha_i + \alpha_{i+1} + \beta_{i+1} - \gamma_{i+1} + p_i \geq 0, \\ \gamma_i - \alpha_{i+1} - \beta_{i+1} - \gamma_{i+1} + p_i \geq -2, \\ -\beta_i + \alpha_{i+1} - \beta_{i+1} - \gamma_{i+1} + p_i \geq -2, \\ -\beta_i - \alpha_{i+1} + \beta_{i+1} - \gamma_{i+1} + p_i \geq -2, \\ -\beta_i - \alpha_{i+1} - \beta_{i+1} + \gamma_{i+1} + p_i \geq -2. \end{cases}$$

Together with the binary restrictions

$$\alpha_i, \beta_i, \gamma_i, \alpha_{i+1}, \beta_{i+1}, \gamma_{i+1}, p_i \in \{0, 1\},$$

this system has exactly the 56 feasible local points.

Linear box. Now consider the compact local model for linear approximations of

$$z = x \boxplus y.$$

Let Γ be the output mask and let A, B be the two input masks. The correlation coefficient is

$$\text{cor}^+(\Gamma, A, B) = 2^{-2N} \sum_{x, y \in \mathbb{F}_2^N} (-1)^{\Gamma \cdot (x \boxplus y) \oplus A \cdot x \oplus B \cdot y}.$$

Wallen's representation writes this as a product of 2×2 matrices,

$$\text{cor}^+(\Gamma, A, B) = L M_{u_{N-1}} M_{u_{N-2}} \cdots M_{u_0} C, \quad L = (1, 0), \quad C = (1, 1)^T,$$

with

$$u_i = 4\Gamma_i + 2A_i + B_i.$$

The matrices are

$$\begin{aligned} M_0 &= \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}, & M_1 &= M_2 = -M_4 = \frac{1}{2} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \\ M_7 &= \frac{1}{2} \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}, & -M_3 &= M_5 = M_6 = \frac{1}{2} \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}. \end{aligned}$$

Let

$$e_0 = (1, 0), \quad e_1 = (0, 1).$$

After left multiplication by these matrices, the running row vector can only be $0, e_0$, or e_1 . A zero row vector gives zero correlation. A nonzero path can therefore be compressed to the two-state automaton

$$\varepsilon_N = e_0 \xrightarrow{u_{N-1}} \varepsilon_{N-1} \xrightarrow{u_{N-2}} \cdots \xrightarrow{u_0} \varepsilon_0, \quad \varepsilon_i \in \{e_0, e_1\}.$$

Direct multiplication from e_0 gives

$$e_0 M_0 = e_0, \quad e_0 M_7 = e_1, \quad e_0 M_u = 0 \quad (u \in \{1, 2, 3, 4, 5, 6\}),$$

so from source state e_0 there are only two legal transitions:

$$e_0 \xrightarrow{0} e_0, \quad e_0 \xrightarrow{7} e_1.$$

Direct multiplication from e_1 gives

u_i	0	1	2	3	4	5	6	7
$e_1 M_{u_i}$	$\frac{1}{2}e_1$	$\frac{1}{2}e_0$	$\frac{1}{2}e_0$	$-\frac{1}{2}e_1$	$-\frac{1}{2}e_0$	$\frac{1}{2}e_1$	$\frac{1}{2}e_1$	$\frac{1}{2}e_0$

Thus from source state e_1 all eight input values are legal, and each transition has absolute coefficient 2^{-1} .

Define

$$s_i = \begin{cases} 0, & \varepsilon_i = e_0, \\ 1, & \varepsilon_i = e_1. \end{cases}$$

Then the local transition at bit i is represented by the five-tuple $(s_{i+1}, \Gamma_i, A_i, B_i, s_i)$. The ten legal local patterns are

$$\begin{aligned} & (0, 0, 0, 0, 0), \quad (0, 1, 1, 1, 1), \\ & (1, 0, 0, 0, 1), \quad (1, 0, 0, 1, 0), \quad (1, 0, 1, 0, 0), \quad (1, 0, 1, 1, 1), \\ & (1, 1, 0, 0, 0), \quad (1, 1, 0, 1, 1), \quad (1, 1, 1, 0, 1), \quad (1, 1, 1, 1, 0). \end{aligned}$$

There are 10 legal linear patterns. The number 13 belongs to the differential inequality system above, not to the linear automaton. Equivalently, with

$$x = s_{i+1}, \quad g = \Gamma_i, \quad a = A_i, \quad b = B_i, \quad y = s_i,$$

the automaton relation is

$$(x = 0 \wedge g = a = b = y) \quad \vee \quad (x = 1 \wedge y = 1 \oplus g \oplus a \oplus b).$$

Taking the convex hull of these ten binary points and removing redundant facets gives the following eight linear inequalities:

$$\begin{aligned} x - g - a + b + y &\geq 0, \\ x + g - a - b + y &\geq 0, \\ x + g - a + b - y &\geq 0, \\ -x + g + a + b + y &\geq 0, \\ x + g + a - b - y &\geq 0, \\ x - g + a - b + y &\geq 0, \\ x - g + a + b - y &\geq 0, \\ x + g + a + b + y &\leq 4. \end{aligned}$$

In the original variables this is

$$\begin{aligned} s_{i+1} - \Gamma_i - A_i + B_i + s_i &\geq 0, \\ s_{i+1} + \Gamma_i - A_i - B_i + s_i &\geq 0, \\ s_{i+1} + \Gamma_i - A_i + B_i - s_i &\geq 0, \\ -s_{i+1} + \Gamma_i + A_i + B_i + s_i &\geq 0, \\ s_{i+1} + \Gamma_i + A_i - B_i - s_i &\geq 0, \\ s_{i+1} - \Gamma_i + A_i - B_i + s_i &\geq 0, \\ s_{i+1} - \Gamma_i + A_i + B_i - s_i &\geq 0, \\ s_{i+1} + \Gamma_i + A_i + B_i + s_i &\leq 4. \end{aligned}$$

If $s_{i+1} = 0$, these inequalities leave only

$$(\Gamma_i, A_i, B_i, s_i) = (0, 0, 0, 0) \quad \text{or} \quad (1, 1, 1, 1).$$

If $s_{i+1} = 1$, they leave all eight mask triples and force

$$s_i = 1 \oplus \Gamma_i \oplus A_i \oplus B_i.$$

Hence the eight inequalities are exactly the local H -representation of the ten-state automaton transitions. Each transition out of e_1 contributes a factor 2^{-1} in absolute value, while transitions out of e_0 contribute a factor 1. Since the high boundary is $s_N = 0$, every nonzero single-characteristic approximation represented by this automaton has

$$|\text{cor}^+(\Gamma, A, B)| = 2^{-\sum_{i=0}^{N-1} s_{i+1}} = 2^{-\sum_{i=1}^{N-1} s_i}.$$

Therefore maximizing the absolute correlation is equivalent to minimizing

$$W_+^{\text{lin, char}} = \sum_{i=1}^{N-1} s_i.$$

For an R -round trail search this contribution is summed over rounds:

$$\min \sum_{r=1}^R \sum_{i=1}^{N-1} s_i^{(r)}.$$

The boundary constraint

$$s_N = 0$$

is part of the model because the automaton starts from $\varepsilon_N = e_0$. A separate nonzero-mask constraint is still needed if the global search must exclude the trivial all-zero approximation. To exclude only the zero mask, one uses

$$\sum_{i=0}^{N-1} x_i \geq 1,$$

not

$$\sum_{i=0}^{N-1} x_i = 1,$$

which would force exactly one active bit rather than merely excluding the zero mask.

13 MILP Constraints for Fixed-Addend Addition and Subtraction

13.1 Code-Level Derivation of the Fixed-Addend MILP Boxes

Proposition 4 (Code-to-model map for fixed-addend MILP boxes). *The fixed-addend code is derived directly from the two exact Q_1 oracles. On the differential side, `add_fixed_public_constant_exact` linearizes the oracle’s zero-chain scan and attaches the exact negative-log probability coefficients to selector variables. In the round builder, fixed public subtraction is not modeled separately. The subtraction helper replaces the public constant by its negation modulo 2^n and then calls the same addition box. The optional “zero-difference-average” branch is only a comparison model.*

On the linear side, the round model uses two helpers in sequence. First, the exact-numerator threshold helper builds the exact two-state signed-numerator recurrence and, with threshold $T = 1$ in the best-trail backend, forces the final numerator to be nonzero. Second, the exact-log objective helper binds an integer variable A to the absolute numerator and charges the exact log-weight

$$W = n - \log_2 A$$

through a SCIP epigraph constraint handler. Thus the implemented linear fixed-addend box is not merely a threshold feasibility filter. The threshold part creates the exact numerator and the sign branch; the objective part turns that numerator into the exact logarithmic local cost.

13.2 Fixed-Addend Differential MILP: The Exact Code-Level Constraints

Definition 22 (Fixed-public-constant differential MILP object). This subsection derives the exact SCIP model used for fixed public constants in the differential search. The mathematical object is

$$F_K : X \mapsto X \boxplus K \pmod{2^n}$$

with a public fixed addend K . For input and output XOR differences $u, v \in \{0, 1\}^n$,

$$\text{DP}_K(u \rightarrow v) = 2^{-n} \# \{X : (X \boxplus K) \oplus ((X \oplus u) \boxplus K) = v\}.$$

Miyano identifies this as an addend-dependent probability, Machado gives an exact fixed-addend computation, and Azimi et al. give the bit-vector model and the λ_i, p_i weight decomposition used here [Miy98, Mac01, ARS⁺20, ARS⁺22]. The implemented SCIP model does not use Azimi's Q_κ truncated fixed-point approximation. It precomputes the integer p_i exactly and charges the exact real coefficient

$$(\lambda_i - 1) - \log_2 p_i.$$

Thus the model is an exact real-objective MILP at the mathematical level for the fixed public constant, not a Q_κ -integer truncated surrogate. The implementation stores these real coefficients in SCIP floating-point objective coefficients, so numerical acceptance is still subject to the usual SCIP tolerances.

There are two different MILP models in the implementation for constant-like modular-addition steps, and they solve different modeling problems. The first one is the optional average comparison model

```
arithmetic_model::add_zero_diff_operand_average.
```

It follows the compact zero-difference-operand / constant-input style of MILP model used in the automatic differential search literature [BA20]. This model receives only two difference masks, written here as β, γ , and does not use the concrete public constant bits. It is therefore not a fixed-public-constant exact model.

The second one is the exact fixed-public-constant model

```
arithmetic_model::add_fixed_public_constant_exact.
```

This is the model derived in the rest of this subsection. It receives the actual public constant K , constructs the addend-dependent state sequence, selects the exact zero-run length λ_i , binds the required class

$$q_i = (u_i \oplus v_i) \oplus K_{i-1},$$

and charges the exact coefficient

$$(\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}$$

computed from the concrete constant terrain. In short, the average comparison model abstracts away the constant bits, while the fixed-public exact model uses them.

13.2.1 Average Constant-Input Comparison Model

Proposition 5 (Average zero-difference-operand comparison model). *The optional comparison model*

$$\text{add_zero_diff_operand_average}$$

takes two n -bit difference masks β, γ . It first enforces the least significant bit condition

$$\beta_0 = \gamma_0.$$

For $0 \leq i < n - 1$, it introduces an activity variable

$$e_i \in \{0, 1\}$$

and encodes

$$e_i = \beta_i \vee \gamma_i$$

with the three linear inequalities

$$e_i \geq \beta_i, \quad e_i \geq \gamma_i, \quad e_i \leq \beta_i + \gamma_i.$$

The next-bit consistency constraints are

$$\beta_{i+1} - \gamma_{i+1} + e_i \geq 0,$$

$$-\beta_{i+1} + \gamma_{i+1} + e_i \geq 0.$$

Equivalently,

$$|\beta_{i+1} - \gamma_{i+1}| \leq e_i.$$

Thus, if $e_i = 0$, the model forces

$$\beta_{i+1} = \gamma_{i+1},$$

while if $e_i = 1$, the next-bit pair is left unconstrained. The objective coefficient of every e_i is 1, so the model charges

$$W_{\text{avg}}(\beta, \gamma) = \sum_{i=0}^{n-2} e_i = \sum_{i=0}^{n-2} (\beta_i \vee \gamma_i).$$

This model is useful as a compact literature-style comparison constraint, but it does not read K . Therefore it cannot distinguish two public constants with different bit patterns and cannot represent the exact fixed-addend probability

$$\text{DP}_K(u \rightarrow v)$$

for a concrete K . The exact addend-dependent model is the

$$\text{FIXED_PUBLIC_EXACT}$$

path derived below.

13.2.2 Carry-Pair Oracle

Lemma 8 (Carry-pair exact-count oracle). *Let c_i, c'_i be the carry entering bit i in the two additions*

$$Y = X \boxplus K, \quad Y' = (X \oplus u) \boxplus K.$$

Then

$$\begin{aligned} c_0 &= c'_0 = 0, \\ Y_i &= X_i \oplus K_i \oplus c_i, \quad Y'_i = X_i \oplus u_i \oplus K_i \oplus c'_i, \end{aligned}$$

and therefore

$$v_i = Y_i \oplus Y'_i = u_i \oplus c_i \oplus c'_i.$$

Set

$$d_i = u_i \oplus v_i.$$

The output condition at bit i is exactly

$$d_i = c_i \oplus c'_i.$$

Since $c_0 = c'_0 = 0$, the lowest-bit condition is $d_0 = 0$, equivalently

$$u_0 = v_0.$$

A direct exact-count oracle can be written as a four-state dynamic program over the carry pair

$$\sigma_i = (c_i, c'_i) \in \{00, 01, 10, 11\}.$$

For a local pattern (u_i, K_i, v_i) , define the integer transfer matrix

$$T_i(\sigma, \tau) = \# \left\{ x \in \{0, 1\} : \begin{array}{l} \tau = (\text{Maj}(x, K_i, c), \text{Maj}(x \oplus u_i, K_i, c')), \\ v_i = u_i \oplus c \oplus c' \end{array} \right\},$$

where $\sigma = (c, c')$. With row vector $R_0 = (1, 0, 0, 0)$,

$$R_{i+1} = R_i T_i, \quad N_K(u, v) = \sum_{\tau} R_n(\tau),$$

and

$$\text{DP}_K(u \rightarrow v) = 2^{-n} N_K(u, v), \quad W_K(u, v) = n - \log_2 N_K(u, v).$$

The final carry pair is marginalized because the modulo- 2^n addition discards the carry leaving the most significant bit. This carry-pair dynamic program is the direct counting interpretation. The implemented reference oracle

`differential_oracle::oracle_add_const`

uses the equivalent Azimi λ_i, p_i decomposition directly, because that decomposition also becomes sparse linear selectors in the SCIP model.

Proof. The bit equations for Y_i and Y'_i are the ordinary full-adder recurrences applied to the two words X and $X \oplus u$. XORing the two output-bit equations cancels K_i and X_i , leaving $v_i = u_i \oplus c_i \oplus c'_i$; this is equivalent to $d_i = c_i \oplus c'_i$. The initial carries are both zero, hence $u_0 = v_0$ is necessary and sufficient at the least significant bit.

For each state $\sigma = (c, c')$, the transfer entry $T_i(\sigma, \tau)$ counts exactly the input bits x_i that satisfy the bit- i output condition and produce the next carry pair τ . By induction, $R_i(\sigma)$ is the number of length- i prefixes that realize the required output difference and end in carry pair σ . Summing over τ after the last bit marginalizes the discarded final carry pair, and division by 2^n converts the count into the uniform probability. $\square$

13.2.3 Azimi Weight Decomposition

Theorem 20 (Azimi weight decomposition for exact fixed-public differentials). *The fixed-public exact model uses the three-bit state*

$$S_i = (u_{i-1}, v_{i-1}, e_i), \quad e_i = u_i \oplus v_i,$$

with the boundary convention

$$u_{-1} = v_{-1} = K_{-1} = 0.$$

The state 001 is impossible. In carry-pair language, it is the forbidden transition $d_{i-1} = 0$ to $d_i = 1$ with $u_{i-1} = 0$. In particular, at $i = 0$ it enforces $e_0 = 0$, i.e.

$$u_0 = v_0.$$

The C++ oracle implements this validity test by returning impossible whenever

$$\text{fixed_addend_state}(u, v, i) = 001.$$

For a valid differential, the non-chain part contributes one unit of weight whenever

$$u_{i-1} \oplus v_{i-1} = 1, \quad 1 \leq i \leq n-1.$$

Equivalently, these are the states

$$S_i \in \{010, 011, 100, 101\}.$$

Let

$$W_{\text{base}} = \sum_{i=1}^{n-1} [u_{i-1} \oplus v_{i-1} = 1].$$

The remaining exact factors occur at trigger states

$$S_i \in \{110, 111\}, \quad 1 \leq i \leq n-1.$$

For such an i , define λ_i by walking to lower bits through consecutive 000 states:

$$\lambda_i = 1 + \max\{\ell \geq 0 : S_{i-1} = S_{i-2} = \dots = S_{i-\ell} = 000\}.$$

If the walk reaches below bit 0, it stops at the word boundary. This is exactly the while-loop in the C++ oracle:

$$j = i - 1; \quad \text{while } j \geq 0 \text{ and } S_j = 000, \text{ increment } \lambda \text{ and decrement } j.$$

Define

$$q_i = u_i \oplus v_i \oplus K_{i-1} = e_i \oplus K_{i-1}.$$

The exact integer terrain value used by the code is

$$B_{i,\lambda} = K_{i-\lambda-1} + \sum_{j=i-\lambda}^{i-2} 2^{j-(i-\lambda)} K_j,$$

where all negative-index constant bits are zero and the sum is empty when $i-2 < i-\lambda$. Equivalently,

$$B_{i,\lambda} = K_{i-\lambda-1} + \sum_{r=0}^{\lambda-2} 2^r K_{i-\lambda+r}.$$

This is exactly the value computed by

$$\begin{aligned} & \text{fixed_addend_block_value}(K, i, \lambda) \\ &= \text{constant_slice_value_with_origin}(K, i - 2, i - \lambda) + K_{i-\lambda-1}. \end{aligned}$$

This value is not a plain contiguous binary slice. The boundary bit $K_{i-\lambda-1}$ contributes as an additional unit term, exactly as required by the fixed-addend terrain formula.

The Azimi integer numerator for the trigger is

$$p_{i,\lambda,q} = \begin{cases} B_{i,\lambda}, & q = 1, \\ 2^{\lambda-1} - B_{i,\lambda}, & q = 0. \end{cases}$$

This is the value returned by

$$\text{fixed_addend_chain_probability_numerator}(K, i, \lambda, q).$$

If $p_{i,\lambda,q_i} = 0$, the differential is impossible. Otherwise the trigger contributes the exact factor

$$\phi_i = \frac{p_{i,\lambda_i,q_i}}{2^{\lambda_i-1}},$$

so its weight contribution is

$$-\log_2 \phi_i = (\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}.$$

Therefore the exact fixed-addend weight compiled by SCIP is

$$W_K(u, v) = W_{\text{base}} + \sum_{\substack{1 \leq i \leq n-1: \\ S_i \in \{110, 111\}}} ((\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}).$$

Equivalently, the exact count can be read as the integer identity

$$N_K(u, v) = 2^{n-W_{\text{base}} - \sum_i (\lambda_i - 1)} \prod_i p_{i,\lambda_i,q_i},$$

where the product ranges over the trigger states. This identity is the bridge between the direct Machado-style count oracle and the Azimi weight decomposition:

$$\text{DP}_K(u \rightarrow v) = 2^{-W_{\text{base}}} \prod_i \frac{p_{i,\lambda_i,q_i}}{2^{\lambda_i-1}},$$

and hence

$$-\log_2 \text{DP}_K(u \rightarrow v) = W_{\text{base}} + \sum_i ((\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}).$$

13.2.4 Exact Fixed-Public MILP Variables

Definition 23 (Exact fixed-public differential MILP variables). The exact helper is

$$\text{arithmetic_model}::\text{add_fixed_public_constant_exact}.$$

It receives binary vectors u, v and a public constant K . It first creates binary variables e_i and enforces

$$e_i = u_i \oplus v_i$$

by the four linear inequalities

$$e_i \leq u_i + v_i, \quad e_i \geq u_i - v_i, \quad e_i \geq v_i - u_i, \quad e_i \leq 2 - u_i - v_i.$$

Then it creates one-hot state variables

$$s_{i,r} \in \{0, 1\}, \quad r \in \{0, \dots, 7\},$$

with

$$\sum_{r=0}^7 s_{i,r} = 1.$$

Write $r = (r_2 r_1 r_0)_2$. The binding constraints are

$$\sum_{r:r_2=1} s_{i,r} = \bar{u}_{i-1}, \quad \sum_{r:r_1=1} s_{i,r} = \bar{v}_{i-1}, \quad \sum_{r:r_0=1} s_{i,r} = e_i,$$

where

$$\bar{u}_{i-1} = \begin{cases} 0, & i = 0, \\ u_{i-1}, & i > 0, \end{cases} \quad \bar{v}_{i-1} = \begin{cases} 0, & i = 0, \\ v_{i-1}, & i > 0. \end{cases}$$

The impossible state is not modeled by a soft penalty. The variable

$$s_{i,1} \quad (1 = (001)_2)$$

is created with lower and upper bound 0:

$$s_{i,1} = 0.$$

Thus the lowest-bit condition and all later 001 impossibilities are hard constraints.

The base objective is attached directly to the state variables:

$$\text{obj}(s_{i,2}) = \text{obj}(s_{i,3}) = \text{obj}(s_{i,4}) = \text{obj}(s_{i,5}) = 1, \quad 1 \leq i \leq n-1.$$

No base coefficient is added at $i = 0$. Hence the model contributes exactly

$$\sum_{i=1}^{n-1} (s_{i,2} + s_{i,3} + s_{i,4} + s_{i,5}) = W_{\text{base}}.$$

13.2.5 Trigger Selectors

Definition 24 (Trigger selectors for exact fixed-public differentials). For every $i = 1, \dots, n-1$, every

$$1 \leq \lambda \leq i+1,$$

and every $q \in \{0, 1\}$, the code computes the coefficient through

$$\text{chain_exact_log_weight_coefficient}(K, i, \lambda, q).$$

Internally, this coefficient is finite exactly when

$$p_{i,\lambda,q} > 0.$$

If

$$p_{i,\lambda,q} = 0,$$

the coefficient is infinite and no selector is created. This is not a relaxation: the trigger and q -binding equations are still added over the remaining positive- p selectors. Therefore, if the required q -class and exact chain length have no positive- p selector, the corresponding trigger state is forced infeasible.

If $p_{i,\lambda,q} > 0$, the helper creates

$$z_{i,\lambda,q} \in \{0, 1\}$$

with objective coefficient

$$c_{i,\lambda,q} = (\lambda - 1) - \log_2 p_{i,\lambda,q}.$$

Only trigger states may choose a selector:

$$\sum_{\lambda=1}^{i+1} \sum_{\substack{q \in \{0,1\}: \\ p_{i,\lambda,q} > 0}} z_{i,\lambda,q} = s_{i,6} + s_{i,7}.$$

Thus non-trigger states choose none, and a trigger state chooses exactly one finite (λ, q) branch.

The q -class is fixed by the public bit K_{i-1} , because

$$q = e_i \oplus K_{i-1}.$$

State 110 has $e_i = 0$, and state 111 has $e_i = 1$. Therefore, if $K_{i-1} = 0$,

$$\sum_{\lambda: p_{i,\lambda,0} > 0} z_{i,\lambda,0} = s_{i,6}, \quad \sum_{\lambda: p_{i,\lambda,1} > 0} z_{i,\lambda,1} = s_{i,7}.$$

If $K_{i-1} = 1$, the two right-hand sides are swapped:

$$\sum_{\lambda: p_{i,\lambda,0} > 0} z_{i,\lambda,0} = s_{i,7}, \quad \sum_{\lambda: p_{i,\lambda,1} > 0} z_{i,\lambda,1} = s_{i,6}.$$

This is the exact fixed-constant terrain. If every selector in the required q -class has $p = 0$, the corresponding equality forces the trigger state itself to zero.

13.2.6 Exact Chain-Length Constraints

Lemma 9 (Exactness of fixed-public zero-chain length selectors). *The selector $z_{i,\lambda,q}$ must mean that λ is the exact chain length, not merely a lower bound. The helper enforces the lower part by*

$$z_{i,\lambda,q} \leq s_{i-k,0}, \quad 1 \leq k \leq \lambda - 1,$$

so all $\lambda - 1$ immediately preceding states are 000. If the chain does not reach the word boundary, it also enforces the stopping state

$$z_{i,\lambda,q} \leq 1 - s_{i-\lambda,0}, \quad i - \lambda \geq 0.$$

If $i - \lambda < 0$, the chain stops at the boundary and no additional stopping constraint is needed. For a fixed state sequence, at most one value of λ satisfies both the preceding-zero constraints and the stopping constraint. Thus the selected λ is the exact zero-run length seen by the oracle, not a freely chosen lower bound. These two families are the MILP translation of the C++ while-loop

$$j = i - 1; \quad \text{while } j \geq 0 \text{ and } S_j = 000, \text{ increment } \lambda.$$

13.2.7 Static MILP Pseudocode

Algorithm 26 Average zero-difference-operand comparison MILP

```

1: function ZERO_DIFF_OPERAND_AVERAGE_MILP( $\beta, \gamma, n$ )
2:   add  $\beta_0 = \gamma_0$ 
3:   for  $i \leftarrow 0$  to  $n - 2$  do
4:     create  $e_i \in \{0, 1\}$ 
5:     add  $e_i \geq \beta_i$ 
6:     add  $e_i \geq \gamma_i$ 
7:     add  $e_i \leq \beta_i + \gamma_i$ 
8:     add  $\beta_{i+1} - \gamma_{i+1} + e_i \geq 0$ 
9:     add  $-\beta_{i+1} + \gamma_{i+1} + e_i \geq 0$ 
10:    set objective coefficient of  $e_i$  to 1
11:   end for
12: end function

```

Algorithm 27 Exact fixed-public-constant differential MILP

```

1: function FIXED_PUBLIC_CONSTANT_DIFF_EXACT_MILP( $K, u, v, n$ )
2:   mask  $K$  to  $n$  bits
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:     create  $e_i \in \{0, 1\}$ 
5:     add the four inequalities for  $e_i = u_i \oplus v_i$ 
6:     create  $s_{i,r} \in \{0, 1\}, r = 0, \dots, 7$ 
7:     fix  $s_{i,1} = 0$ 
8:     add  $\sum_r s_{i,r} = 1$ 
9:     bind the state bits to  $\bar{u}_{i-1}, \bar{v}_{i-1}$ , and  $e_i$ 
10:    if  $i \geq 1$  then
11:      add objective coefficient 1 on  $s_{i,2}, s_{i,3}, s_{i,4}, s_{i,5}$ 
12:    end if
13:  end for
14:  for  $i \leftarrow 1$  to  $n - 1$  do
15:    initialize trigger equation with  $-s_{i,6} - s_{i,7}$ 
16:    initialize  $q = 0$  and  $q = 1$  binding equations
17:    for  $\lambda \leftarrow 1$  to  $i + 1$  do
18:      for  $q \leftarrow 0$  to 1 do
19:         $B \leftarrow \text{CONSTANT\_SLICE\_WITH\_ORIGIN}(K, i - 2, i - \lambda) + K_{i-\lambda-1}$ 
20:         $p \leftarrow B$  if  $q = 1$ , otherwise  $p \leftarrow 2^{\lambda-1} - B$ 
21:        if  $p = 0$  then
22:          do not create this selector
23:          continue
24:        end if
25:        create  $z_{i,\lambda,q} \in \{0, 1\}$ 
26:        set its objective coefficient to  $(\lambda - 1) - \log_2 p$ 
27:        add  $+z_{i,\lambda,q}$  to the trigger equation
28:        add  $+z_{i,\lambda,q}$  to the  $q$ -binding equation
29:        for  $k \leftarrow 1$  to  $\lambda - 1$  do
30:          add  $z_{i,\lambda,q} \leq s_{i-k,0}$ 
31:        end for
32:        if  $i - \lambda \geq 0$  then
33:          add  $z_{i,\lambda,q} \leq 1 - s_{i-\lambda,0}$ 
34:        end if

```

```

35:         end for
36:     end for
37:     add the trigger equality  $\sum_{\lambda,q} z_{i,\lambda,q} = s_{i,6} + s_{i,7}$ 
38:     bind  $q = e_i \oplus K_{i-1}$  by equating the  $q$ -selector sums to  $s_{i,6}$  and  $s_{i,7}$ , swapped
    when  $K_{i-1} = 1$ 
39: end for
40: end function

```

13.2.8 Equivalence Proof

Theorem 21 (Exactness of the fixed-public-constant differential MILP). *For every public n -bit constant K and every binary mask pair (u, v) , the exact fixed-public differential MILP is feasible if and only if $\text{DP}_K(u \rightarrow v) > 0$. In every feasible integral assignment, the selected objective terms equal*

$$-\log_2 \text{DP}_K(u \rightarrow v),$$

and zero-probability transitions are infeasible rather than finite-weight.

Proof. Fix binary vectors u, v . The XOR constraints force

$$e_i = u_i \oplus v_i.$$

The one-hot and binding constraints force $s_{i,r} = 1$ exactly for

$$r = (u_{i-1}, v_{i-1}, e_i),$$

with $u_{-1} = v_{-1} = 0$. Since $s_{i,1} = 0$, the MILP is feasible only if the fixed-addend state 001 never appears. This matches the exact oracle's validity test.

For every non-trigger state, the trigger equality forces all z -selectors at that bit to zero. For every trigger state, it forces exactly one selector. The q -binding equalities force the chosen selector to have

$$q = e_i \oplus K_{i-1}.$$

The chain inequalities force precisely the same λ that the oracle obtains by walking downward through consecutive 000 states. Therefore the selected objective coefficient is exactly

$$(\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}.$$

Adding the base state coefficients gives

$$W_{\text{base}} + \sum_i ((\lambda_i - 1) - \log_2 p_{i,\lambda_i,q_i}) = -\log_2 \text{DP}_K(u \rightarrow v).$$

Equivalently, for every feasible binary mask pair, the selected objective terms reproduce the exact count

$$N_K(u, v) = 2^{n - W_{\text{base}} - \sum_i (\lambda_i - 1)} \prod_i p_{i,\lambda_i,q_i}.$$

If any required integer numerator p is zero, no selector exists in that required q -class, and the binding equality forbids the trigger state. Hence zero-probability differentials are infeasible, not merely expensive. $\square$

Remark 2 (Scope of the average comparison model). The average comparison model has a different equivalence statement. Its activity variables satisfy

$$e_i = \beta_i \vee \gamma_i$$

and its next-bit constraints enforce

$$e_i = 0 \implies \beta_{i+1} = \gamma_{i+1}.$$

Therefore its objective is exactly

$$\sum_{i=0}^{n-2} (\beta_i \vee \gamma_i),$$

but this objective is not the exact addend-dependent weight of a concrete public constant. It is a separate comparison abstraction and should not be used as a proof of $\text{DP}_K(u \rightarrow v)$ for fixed K .

13.2.9 Round-Level Addition and Subtraction Wrappers

Corollary 4 (Round-level fixed-public differential wrappers). *The round builder calls a fixed-constant step through two wrapper functions:*

$$\begin{array}{ll} \text{add_fixed_constant} & \text{add_fixed_constant} \\ \text{_addition_operation,} & \text{_subtraction_operation.} \end{array}$$

These wrappers dispatch between the two MILP models described above.

In the exact public-constant mode,

$$\text{ConstantModel}::\text{FIXED_PUBLIC_EXACT},$$

addition calls

$$\text{arithmetic_model}::\text{add_fixed_public_constant_exact},$$

and subtraction calls

$$\text{arithmetic_model}::\text{add_fixed_public_constant_sub_exact}.$$

The subtraction helper is just the exact addition helper with the two's complement constant:

$$X \boxminus C = X \boxplus (-C \bmod 2^n).$$

Thus

$$\text{DP}^{\boxminus C}(u \rightarrow v) = \text{DP}^{\boxplus K}(u \rightarrow v), \quad K = -C \bmod 2^n.$$

In the average comparison mode,

$$\text{ConstantModel}::\text{ZERO_DIFFERENCE_AVERAGE},$$

the wrapper calls

$$\text{arithmetic_model}::\text{add_zero_diff_operand_average}.$$

This path ignores the concrete public constant bits and is not the fixed-public exact model.

Algorithm 28 Round fixed-constant differential wrapper

```

1: function ADDFIXEDCONSTANTOPERATION(op, C, u, v, prefix)
2:   if constant_model = FIXED_PUBLIC_EXACT then
3:     if op =  $\boxminus$  then
4:       call FIXEDPUBLICCONSTANTDIFFEXACTMILP(C, u, v, n)
5:     else
6:        $K \leftarrow -C \bmod 2^n$ 

```

```

7:      call FIXEDPUBLICCONSTANTDIFFEXACTMILP( $K, u, v, n$ )
8:    end if
9:  else
10:    call ZERODIFFOPERANDAVERAGEMILP( $u, v, n$ )
11:  end if
12: end function

```

In the implemented NeoAlzette differential round, the exact fixed-public subtraction model is used at the two public subtraction stages:

stage	changed branch	C	$K = -C \bmod 2^{32}$
CONST_SUB_RC1	B	RC[1]	$-\text{RC}[1] \bmod 2^{32}$
CONST_SUB_RC6	A	RC[6]	$-\text{RC}[6] \bmod 2^{32}$

The differential weight trace records the objective window belonging to each round step. In the current differential backend, the fixed-public-constant step weight is read from the exact state and selector objective terms generated by this construction. A separate post-solve count oracle, if enabled, should therefore agree with the same value.

13.3 Fixed-Addend Linear MILP: Exact Numerator and Exact-Log Weight

Definition 25 (Fixed-addend linear exact-log MILP object). This subsection is the code-level derivation of the strict fixed-addend linear MILP used by the linear round search. The mathematical object is the Miyano fixed-addend linear correlation for

$$F_K : X \mapsto Y = X \boxplus K \pmod{2^n},$$

where X is random and K is public and fixed [Miy98]. The local oracle value is

$$C_K(a, b) = 2^{-n} \sum_{X \in \{0,1\}^n} (-1)^{a \cdot X \oplus b \cdot F_K(X)}.$$

This is the signed linear correlation. The corresponding linear approximation probability is

$$\Pr[a \cdot X = b \cdot F_K(X)] = \frac{1 + C_K(a, b)}{2}.$$

The integer quantity compiled into the MILP is the signed numerator

$$\mathcal{N}_K(a, b) = 2^n C_K(a, b).$$

The exact local correlation weight is therefore

$$W_K(a, b) = \begin{cases} n - \log_2 |\mathcal{N}_K(a, b)|, & \mathcal{N}_K(a, b) \neq 0, \\ +\infty, & \mathcal{N}_K(a, b) = 0. \end{cases}$$

The implementation does not enumerate $\mathcal{N}$ outside SCIP and then repair the solution. It builds $\mathcal{N}_K(a, b)$, binds $|\mathcal{N}_K(a, b)|$, and prices $n - \log_2 |\mathcal{N}_K(a, b)|$ inside the model.

13.3.1 Linear Oracle Formula

Theorem 22 (Fixed-addend linear transfer formula). *The sequential oracle*

```

linear_oracle::
oracle_add_const

```

and the public evaluator

`linear_correlation/
constant_weight_evaluation.hpp`

compute the same two-state carry recurrence. Let s be the carry entering bit i , and r the carry leaving bit i . For fixed bits

$$k = k_i, \quad a = a_i, \quad b = b_i,$$

the normalized one-bit transfer is

$$T_{k,a,b}(s, r) = \frac{1}{2} \sum_{\substack{x \in \{0,1\}: \\ \text{Maj}(x,k,s)=r}} (-1)^{ax \oplus b(x \oplus k \oplus s)}.$$

This formula is the oracle, not a heuristic. It is just the one-bit summation over the only random bit x_i . Enumerating the two possible values of x gives:

$$T_{k,a,b}(s, r) = \begin{cases} 1, & s = k, r = k, a = b, \\ 0, & s = k \text{ and not } (r = k, a = b), \\ \frac{1}{2}(-1)^b, & s \neq k, r = 0, \\ \frac{1}{2}(-1)^a, & s \neq k, r = 1. \end{cases}$$

Indeed, when $s = k$, both values $x = 0, 1$ keep the outgoing carry equal to k , and the two signs cancel unless $a = b$. When $s \neq k$, the value $x = 0$ gives $r = 0$ and sign $(-1)^b$, while $x = 1$ gives $r = 1$ and sign $(-1)^a$.

With row-vector convention, let

$$R_i = (R_i^{(0)}, R_i^{(1)})$$

be the signed prefix average conditioned on the carry entering bit i . Then

$$R_0 = (1, 0), \quad R_{i+1} = R_i T_{k_i, a_i, b_i}.$$

The final carry is not an output bit of the modulo-2ⁿ addition. It is therefore marginalized, giving

$$C_K(a, b) = R_n^{(0)} + R_n^{(1)}.$$

The C++ MILP helper stores the scaled signed prefix state

$$S_i = 2^i R_i = (S_i^{(0)}, S_i^{(1)}).$$

Equivalently, $S_i^{(d)}$ is the signed sum of all processed i -bit prefixes that enter carry state d . Hence its mathematical value is integral and lies in $[-2^i, 2^i]$. Thus $S_0 = (1, 0)$, and each local matrix is multiplied by 2:

$$M_{k,a,b} = 2T_{k,a,b}.$$

Equivalently,

$$M_{k,a,b}(s, r) = \begin{cases} 2, & s = k, r = k, a = b, \\ 0, & s = k \text{ and not } (r = k, a = b), \\ (-1)^b, & s \neq k, r = 0, \\ (-1)^a, & s \neq k, r = 1. \end{cases}$$

This is exactly the coefficient rule implemented by

fixed_addend_scaled
_transition_coefficients.

For clarity, the eight direct-mask matrices are

$$\begin{aligned} M_{0,00} &= \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}, & M_{0,01} &= \begin{bmatrix} 0 & 0 \\ -1 & 1 \end{bmatrix}, & M_{0,10} &= \begin{bmatrix} 0 & 0 \\ 1 & -1 \end{bmatrix}, & M_{0,11} &= \begin{bmatrix} 2 & 0 \\ -1 & -1 \end{bmatrix}, \\ M_{1,00} &= \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, & M_{1,01} &= \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}, & M_{1,10} &= \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}, & M_{1,11} &= \begin{bmatrix} -1 & -1 \\ 0 & 2 \end{bmatrix}. \end{aligned}$$

After the last bit,

$$\mathcal{N}_K(a, b) = S_n^{(0)} + S_n^{(1)}, \quad C_K(a, b) = 2^{-n} \mathcal{N}_K(a, b).$$

The oracle also has an exact triangular-map rejection. Since Y_j depends only on $X_0, \dots, X_j$ and contains X_j with coefficient 1, a nonzero correlation requires the same highest active mask bit:

$$h(a) = h(b), \quad h(m) = \max\{i : m_i = 1\}, \quad h(0) = -1.$$

This condition is an exact rejection test in the necessary direction: if the highest active bits differ, the correlation is forced to be zero. It is not used as a complete nonzero oracle; mask pairs with $h(a) = h(b)$ may still have zero numerator, and the exact numerator threshold below decides those cases. The function `oracle_add_const` checks this rejection condition before it runs the recurrence, and the MILP encodes the same condition as linear constraints.

13.3.2 Oracle-to-MILP Variables

Definition 26 (Fixed-addend linear numerator MILP variables). The static model builder

```
add_fixed_public
_constant_linear_exact
_numerator_threshold
_constraints
```

takes binary mask vectors a, b and a public n -bit constant K . It first creates one-hot selectors for the local pair (a_i, b_i) :

$$z_{i,00}, z_{i,01}, z_{i,10}, z_{i,11} \in \{0, 1\}, \quad z_{i,00} + z_{i,01} + z_{i,10} + z_{i,11} = 1,$$

with direct bindings

$$a_i = z_{i,10} + z_{i,11}, \quad b_i = z_{i,01} + z_{i,11}.$$

Thus exactly one matrix M_{k_i, a_i, b_i} is active at bit i .

The highest-active-bit test is represented by suffix-nonzero variables. For a mask m , define

$$\rho_i(m) = \bigvee_{j=i}^{n-1} m_j.$$

The code linearizes this OR as

$$\rho_{n-1}(m) = m_{n-1},$$

and for $0 \leq i < n-1$,

$$\rho_i(m) \geq m_i, \quad \rho_i(m) \geq \rho_{i+1}(m), \quad \rho_i(m) \leq m_i + \rho_{i+1}(m).$$

It then enforces

$$\rho_i(a) = \rho_i(b), \quad 0 \leq i < n.$$

These equalities are exactly $h(a) = h(b)$, including the all-zero/all-zero case. Since every nonzero fixed-addend correlation satisfies this condition, the suffix constraints do not remove any feasible nonzero numerator case; they only remove mask pairs that are already known to have zero correlation.

The scaled carry states are model variables

$$S_i^{(0)}, S_i^{(1)} \in [-2^i, 2^i], \quad 0 \leq i \leq n,$$

with

$$S_0^{(0)} = 1, \quad S_0^{(1)} = 0.$$

The implementation declares these state variables as continuous variables with finite bounds. For every binary selector assignment, the transition equalities force them to the same signed prefix sums as the recurrence above, so the mathematical values are integral even though no separate integrality declaration is required for the state variables.

The selected matrix product is made linear by split-state variables. For each case $t \in \{00, 01, 10, 11\}$ and carry state $d \in \{0, 1\}$, the intended identity is

$$Y_{i,t}^{(d)} = z_{i,t} S_i^{(d)}.$$

With $L_i = -2^i$ and $U_i = 2^i$, the exact binary-continuous product hull is

$$\begin{aligned} Y &\geq L_i z, & Y &\leq U_i z, \\ Y &\geq S_i^{(d)} - U_i(1 - z), & Y &\leq S_i^{(d)} - L_i(1 - z). \end{aligned}$$

The formulation is exact because z is binary and $S_i^{(d)}$ has known finite bounds. If $z = 0$, these four inequalities force $Y = 0$. If $z = 1$, they force $Y = S_i^{(d)}$. Since exactly one selector is one, the disaggregation constraints

$$S_i^{(d)} = \sum_{t \in \{00, 01, 10, 11\}} Y_{i,t}^{(d)} \quad (d = 0, 1)$$

are also exact.

Let $M_{k_i,t}(d, r)$ denote the coefficient from source carry d to target carry r in the selected direct-mask matrix. The transition constraints are the linear equalities

$$S_{i+1}^{(r)} = \sum_{t \in \{00, 01, 10, 11\}} \sum_{d \in \{0, 1\}} M_{k_i,t}(d, r) Y_{i,t}^{(d)}, \quad r \in \{0, 1\}.$$

Finally,

$$\mathcal{N} = S_n^{(0)} + S_n^{(1)}.$$

Thus the MILP variable $\mathcal{N}$ is the same signed numerator returned by the linear oracle for the selected mask pair.

13.3.3 Numerator Threshold and Sign Branch

Lemma 10 (Numerator threshold and sign selector). *The numerator-threshold builder receives a requested threshold T_{req} . The sanitizer used in code is*

$$T = \begin{cases} 1, & T_{\text{req}} = 0, \\ T_{\text{req}}, & 1 \leq T_{\text{req}} \leq 2^n, \\ 2^n + 1, & T_{\text{req}} > 2^n. \end{cases}$$

Here $T_{\text{req}} = 0$ is interpreted as the default request for a nonzero correlation numerator, not as permission to keep zero numerators. The weight-threshold wrapper uses the equivalent conversion

$$T = \lceil 2^{n-\tau} \rceil$$

before sanitization. If $T > 2^n$, the helper adds an immediate contradiction. Otherwise it creates a binary sign selector

$$\eta \in \{0, 1\},$$

where $\eta = 0$ is the positive numerator branch and $\eta = 1$ is the negative numerator branch. With

$$M_T = 2^n + T,$$

the two constraints are

$$\mathcal{N} + M_T \eta \geq T, \quad \mathcal{N} + M_T \eta \leq M_T - T.$$

If $\eta = 0$, these reduce to

$$T \leq \mathcal{N} \leq 2^n.$$

If $\eta = 1$, they reduce to

$$-2^n \leq \mathcal{N} \leq -T.$$

Therefore the constraint pair is exactly

$$|\mathcal{N}| \geq T$$

and simultaneously records the sign branch that will be used to bind the absolute numerator. In the NeoAlzette best-trail backend, the round builder passes $T_{\text{req}} = 1$, so the local condition is precisely $\mathcal{N} \neq 0$.

13.3.4 Absolute Numerator and Exact-Log Objective

Lemma 11 (Absolute numerator binding and exact-log epigraph). *The helper*

```
add_fixed_addend
_exact_log_weight
_milp_objective
```

adds an integer absolute numerator

$$A \in \mathbb{Z}, \quad 1 \leq A \leq 2^n$$

and a continuous weight variable

$$W \in [0, n]$$

with objective coefficient 1. Let

$$M_A = 2^{n+1} + 2.$$

Using the sign selector η produced by the threshold model, it adds

$$A - \mathcal{N} - M_A \eta \leq 0, \quad A - \mathcal{N} + M_A \eta \geq 0,$$

$$A + \mathcal{N} + M_A \eta \leq M_A, \quad A + \mathcal{N} - M_A \eta \geq -M_A.$$

For $\eta = 0$, the first two inequalities give $A = \mathcal{N}$, while the last two are inactive under $|\mathcal{N}| \leq 2^n$. For $\eta = 1$, the last two inequalities give $A = -\mathcal{N}$, while the first two are inactive. Since the threshold constraints already force $|\mathcal{N}| \geq 1$, this is an exact equality

$$A = |\mathcal{N}|.$$

The nonlinear part is isolated in the SCIP constraint handler

fixed_addend_exact_log_weight.

It enforces the epigraph

$$W \geq f(A), \quad f(A) = n - \log_2 A.$$

The function f is not an additional search space over possible numerator magnitudes. The numerator magnitude A is already produced by the solver-internal carry recurrence and the absolute-value branch above; f only prices that exact magnitude as a correlation weight.

Since

$$f(A) = n - \log_2 A = n - \frac{\ln A}{\ln 2},$$

we have

$$f'(A) = -\frac{1}{A \ln 2}, \quad f''(A) = \frac{1}{A^2 \ln 2} > 0.$$

Hence f is convex on $A > 0$, and every tangent line gives a valid lower supporting inequality for the epigraph constraint $W \geq f(A)$. Therefore every tangent at an integer point $a \in [1, 2^n]$ is valid:

$$W \geq f(a) + f'(a)(A - a), \quad f'(a) = -\frac{1}{a \ln 2}.$$

Equivalently,

$$W \geq -\frac{A}{a \ln 2} + n - \log_2 a + \frac{1}{\ln 2}.$$

The handler initially adds the tangents at $a = 1$ and $a = 2^n$. During LP separation, pseudo-solution enforcement, and incumbent checking, it evaluates the current pair (A^, W^*) . If*

$$W^* < n - \log_2 \max\{1, A^*\} - 10^{-8},$$

it adds tangents at the rounded and clamped point

$$a_0 = \text{round_clamp}(A^*, 1, 2^n)$$

and, when available, also at $a_0 - 1$ and $a_0 + 1$.

The finite tangent pool used inside the node relaxations is only a relaxation of the nonlinear epigraph. Exactness for accepted MIP solutions is provided by the SCIP constraint handler itself: the checker rejects nonintegral A and rejects any candidate solution whose W is below the true value $n - \log_2 A$. Since W has positive objective coefficient and only lower-bounds the local weight, every accepted optimum tightens it to

$$W = n - \log_2 A = n - \log_2 |\mathcal{N}|.$$

Proof. The four big- M inequalities have two cases. If $\eta = 0$, the first pair forces $A = \mathcal{N}$, while the second pair is inactive for $|\mathcal{N}| \leq 2^n$. If $\eta = 1$, the second pair forces $A = -\mathcal{N}$, while the first pair is inactive. Together with the threshold condition $|\mathcal{N}| \geq 1$, this gives $A = |\mathcal{N}|$.

The derivative calculation shows that $f(A) = n - \log_2 A$ is convex on $A > 0$, so every tangent line is a globally valid lower bound for the epigraph $W \geq f(A)$. The finite tangent pool is therefore a relaxation during node processing. At accepted integer solutions, the SCIP constraint handler checks the true nonlinear inequality $W \geq n - \log_2 A$; since W has positive objective coefficient, any optimum tightens this lower bound to equality. $\square$

13.3.5 Static MILP Pseudocode

Algorithm 29 Static fixed-addend linear numerator-threshold MILP

```

1: function FIXEDADDENDLINEARNUMERATORMILP( $K, a, b, n, T_{\text{req}}$ )
2:    $K \leftarrow K \bmod 2^n$ 
3:    $T \leftarrow \text{SANITIZE\_NUMERATOR\_THRESHOLD}(n, T_{\text{req}})$ 
4:   if  $T > 2^n$  then
5:     add two contradictory constraints on one binary guard
6:     return empty handle
7:   end if
8:   create suffix OR variables  $\rho_i(a)$  and  $\rho_i(b)$ 
9:   add  $\rho_i(a) = \rho_i(b)$  for all  $i$ 
10:  create  $S_i^{(0)}, S_i^{(1)} \in [-2^i, 2^i]$  for  $0 \leq i \leq n$ 
11:  add  $S_0^{(0)} = 1$  and  $S_0^{(1)} = 0$ 
12:  for  $i \leftarrow 0$  to  $n - 1$  do
13:    create  $z_{i,00}, z_{i,01}, z_{i,10}, z_{i,11} \in \{0, 1\}$ 
14:    add  $z_{i,00} + z_{i,01} + z_{i,10} + z_{i,11} = 1$ 
15:    add  $a_i = z_{i,10} + z_{i,11}$ 
16:    add  $b_i = z_{i,01} + z_{i,11}$ 
17:    for all  $t \in \{00, 01, 10, 11\}$  and  $d \in \{0, 1\}$  do
18:      create  $Y_{i,t}^{(d)} \in [-2^i, 2^i]$ 
19:      add the four hull inequalities for  $Y_{i,t}^{(d)} = z_{i,t} S_i^{(d)}$ 
20:    end for
21:    for  $d \in \{0, 1\}$  do
22:      add  $S_i^{(d)} = \sum_t Y_{i,t}^{(d)}$ 
23:    end for
24:     $k \leftarrow K_i$ 
25:    for  $r \in \{0, 1\}$  do
26:      add

$$S_{i+1}^{(r)} = \sum_t \sum_d M_{k,t}(d, r) Y_{i,t}^{(d)}$$

27:    end for
28:  end for
29:  create  $\mathcal{N} \in [-2^n, 2^n]$ 
30:  add  $\mathcal{N} = S_n^{(0)} + S_n^{(1)}$ 
31:  create  $\eta \in \{0, 1\}$ 
32:   $M_T \leftarrow 2^n + T$ 
33:  add  $\mathcal{N} + M_T \eta \geq T$ 
34:  add  $\mathcal{N} + M_T \eta \leq M_T - T$ 
35:  return handle  $(\mathcal{N}, \eta, T)$ 
36: end function

```

Algorithm 30 Attach exact log-weight objective

```

1: function ATTACHFIXEDADDENDEXACTLOGWEIGHT( $\mathcal{N}, \eta, n$ )
2:   create integer  $A \in [1, 2^n]$ 
3:   create continuous  $W \in [0, n]$  with objective coefficient 1
4:    $M_A \leftarrow 2^{n+1} + 2$ 
5:   add  $A - \mathcal{N} - M_A \eta \leq 0$ 
6:   add  $A - \mathcal{N} + M_A \eta \geq 0$ 
7:   add  $A + \mathcal{N} + M_A \eta \leq M_A$ 

```

```

8:   add  $A + \mathcal{N} - M_A \eta \geq -M_A$ 
9:   include fixed_addend_exact_log_weight
10:  add tangents at  $A = 1$  and  $A = 2^n$ 
11:  return  $W$ 
12: end function

```

Algorithm 31 Separate fixed-addend exact-log epigraph

```

1: function SEPARATEFIXEDADDENDLOGWEIGHT( $A^*, W^*, n$ )
2:    $\hat{A} \leftarrow \max\{1, A^*\}$ 
3:    $W_{\text{req}} \leftarrow n - \log_2 \hat{A}$ 
4:   if  $W^* \geq W_{\text{req}} - 10^{-8}$  then
5:     return no cut
6:   end if
7:    $a_0 \leftarrow \text{ROUNDANDCLAMP}(A^*, 1, 2^n)$ 
8:   for all  $a \in \{a_0 - 1, a_0, a_0 + 1\} \cap [1, 2^n]$  do
9:     add

```

$$W \geq -\frac{A}{a \ln 2} + n - \log_2 a + \frac{1}{\ln 2}$$

```

10:  end for
11: end function

```

13.3.6 Equivalence Proof

Theorem 23 (Exactness of the fixed-addend linear exact-log MILP). *For every public n -bit constant K and every binary mask pair (a, b) , the fixed-addend linear exact-log MILP with threshold $T = 1$ is feasible if and only if*

$$C_K(a, b) \neq 0.$$

For every accepted optimal assignment, the local weight variable is tight and equals

$$W = -\log_2 |C_K(a, b)|.$$

Proof. Fix a binary assignment of a, b and a public constant K . The one-hot constraints select exactly the pair (a_i, b_i) at each bit. The hull constraints then make each split variable either zero or the current scaled state. Hence the transition equality at bit i is exactly

$$S_{i+1} = S_i M_{k_i, a_i, b_i}.$$

By induction on i ,

$$S_i = 2^i R_i$$

for the same row vector R_i computed by the sequential linear oracle. Therefore the final MILP equality gives

$$\mathcal{N} = S_n^{(0)} + S_n^{(1)} = 2^n (R_n^{(0)} + R_n^{(1)}) = \mathcal{N}_K(a, b).$$

The suffix constraints encode the triangular necessary rejection condition $h(a) = h(b)$. This condition excludes only mask pairs whose fixed-addend correlation is forced to be zero. The sign-selector constraints are exactly $|\mathcal{N}| \geq T$. The absolute-value constraints are exactly $A = |\mathcal{N}|$. The exact-log constraint handler accepts only solutions satisfying

$$W \geq n - \log_2 A.$$

Since W is minimized with positive objective coefficient, every accepted optimum satisfies equality. Consequently, with $T = 1$, the submodel is feasible exactly for nonzero fixed-addend correlations, and an optimal accepted solution prices that box by

$$W = n - \log_2 |\mathcal{N}_K(a, b)| = -\log_2 |C_K(a, b)|.$$

□

13.3.7 Public Subtraction Call Chain

Corollary 5 (Fixed-public subtraction linear call chain). *The round-level helper*

add_fixed_public
_constant_subtraction_step

uses the fixed-addend addition model through the identity

$$X \boxminus C = X \boxplus K, \quad K = -C \bmod 2^n.$$

No mask transformation is needed: subtraction by a public constant and addition by $K = -C \bmod 2^n$ define the same word-to-word Boolean mapping. The wrapper

add_fixed_public
_constant_subtraction
_linear_exact_numerator
_threshold_constraints

computes

$$K = (0 - (C \bmod 2^n)) \bmod 2^n$$

and then calls the addition numerator-threshold builder above. The exact round helper follows this order:

Algorithm 32 Round-level fixed public subtraction step

```

1: function FIXEDPUBLICCONSTANTSUBTRACTIONSTEP( $C, \lambda_{\text{in}}, \lambda_{\text{other}}, \text{branch}, \text{prefix}$ )
2:   create output mask  $\lambda_{\text{out}}$  named prefix_after
3:    $K \leftarrow -C \bmod 2^n$ 
4:    $q \leftarrow$  current number of objective terms
5:    $H \leftarrow \text{FIXEDADDENDLINEARNUMERATORMILP}(K, \lambda_{\text{in}}, \lambda_{\text{out}}, n, 1)$ 
6:    $W \leftarrow \text{ATTACHFIXEDADDENDEXACTLOGWEIGHT}(H.\mathcal{N}, H.\eta, n)$ 
7:   record a WeightStepSpec with operation = fixed_public_constant
                                         _subtraction_linear
                                         _exact_log_weight_milp

8:   record weight_model = miyano_fixed_addend
                        _exact_log_weight_milp
9:   store  $C$  as public_constant and  $K$  as effective_constant
10:  store the objective window from  $q$  to the new objective length
11:  set has_fixed_exact_oracle to true
12:  return  $\lambda_{\text{out}}$ 
13: end function

```

*The mask λ_{other} and the flag *branch* do not enter the local arithmetic. They only tell the trace recorder whether the changed branch is A or B, so that the full-round checkpoint contains both branch masks. The local fixed-addend box is always the two-mask relation*

$$(\lambda_{\text{in}}, \lambda_{\text{out}}) \quad \text{through} \quad X \mapsto X \boxplus K.$$

Inside `build_one_round`, this helper is used twice:

stage	changed mask	C	$K = -C \bmod 2^{32}$
<code>CONST_SUB_B_RC1</code>	B	$\text{RC}[1]$	$-\text{RC}[1] \bmod 2^{32}$
<code>CONST_SUB_A_RC6</code>	A	$\text{RC}[6]$	$-\text{RC}[6] \bmod 2^{32}$

Thus the implemented chain for a fixed-public subtraction is

$$\begin{aligned}
 (\lambda_{\text{in}}, \lambda_{\text{out}}, C) &\mapsto K = -C \bmod 2^n \\
 &\mapsto \mathcal{N}_K(\lambda_{\text{in}}, \lambda_{\text{out}}) \\
 &\mapsto A = |\mathcal{N}_K(\lambda_{\text{in}}, \lambda_{\text{out}})| \\
 &\mapsto W = n - \log_2 A.
 \end{aligned}$$

13.3.8 Trace Oracle and Consistency Check

Proposition 6 (Trace oracle consistency for fixed-addend linear steps). *The exact oracle is still used after solving, but only as an audit of the MILP solution. In*

`collect_weight_trace`

the fixed-exact branch executes

$$\text{oracle_add_const}(K, \lambda_{\text{in}}, \lambda_{\text{out}}, n),$$

where K is the recorded `effective_constant`. It stores

$$\text{possible}, \quad \text{sign}(\mathcal{N}_K), \quad n - \log_2 |\mathcal{N}_K|, \quad C_K, \quad |C_K|.$$

It then sets the trace local weight to this exact oracle weight and the local sign to the oracle sign. The later consistency tests require

$$\text{trace local weight} = \text{fixed constant exact oracle weight},$$

and also require the accumulated trace weight to match the SCIP objective. Therefore a reported characteristic is paper-usable only when the solver objective, the static MILP numerator model, and the independent linear oracle all agree.

14 Mathematical Analysis of the Joint Injection Layer

This section analyzes the implemented injection layer from the viewpoint of first-order differential and first-order linear trail search. Here “first-order” means the usual Boolean difference operator in the differential setting, and the usual Walsh-mask propagation in the linear setting. It does not mean calculus over the reals. The purpose is to make explicit why the injection layer is not a local XOR gadget plus an unrelated addend, but a coupled source oracle whose Boolean front half and modular arithmetic back half must be modeled together.

14.1 Design Rationale Under the Four-Addition Budget

Remark 3 (Quadratic front-half rationale under the arithmetic budget). The main design question is why the injection layer is built in this form at all. The answer is constrained by the round budget. The implemented box is allowed to spend only four modular add/sub operations in the full round: two fixed public subtractions that anchor the bit coordinates, and two source-dependent modular additions that consume injection-derived addends. Therefore the front half of the injection layer cannot use additional modular arithmetic

without changing the cost model and the round semantics. Its available tools are binary Boolean word operations over $\mathbb{F}_2$, public constants, rotations, and zero-filling shifts.

The front half is consequently designed as a quadratic Boolean generator. The linear diffusion masks, companion rotations, shifted views, and bridge-state views are all linear or affine over $\mathbb{F}_2^{32}$. Nonlinearity is introduced only through coordinatewise AND and OR products, with

$$a \wedge b = ab, \quad a \vee b = a + b + ab, \quad \neg a = 1 + a \quad (a, b \in \mathbb{F}_2).$$

Thus the front half remains a degree-two Boolean map. This is deliberate: degree two is still exactly analyzable by first-order differential affine-rank methods and by first-order linear Walsh-rank methods, while being nonlinear enough to prevent the layer from collapsing into a mere rotation/XOR permutation.

The front half is not meant to be a standalone PRF claim. The round should be read as a main ARX path assisted by an injection side path. The main products of the design are still the ARX updates: fixed modular subtraction, source-dependent modular addition, and bit rotations. The injection layer is useful only if this side path contributes enough diffusion and nonlinearity, or confusion, to be worth feeding back into the main path. In this sense it acts as a turbo-style auxiliary generator: it derives secondary values from the current source branch and pushes them into the main ARX schedule so that the diffusion and confusion process is accelerated rather than merely decorated.

Concretely, the side path produces two coupled words from the same source branch:

$$\Phi_B(x) = (X_B(x), Y_B(x)), \quad \Phi_A(x) = (X_A(x), Y_A(x)).$$

The word X is consumed immediately as the cross-branch XOR-injection material, while the word Y is consumed as the nonconstant operand of the following two-variable modular addition. Thus the Boolean front half creates a high-rank coupled source, and the arithmetic back half turns one part of that source into carry/borrow constraints on the main ARX path. A trail that tries to neutralize the XOR side must still remain compatible with the addend side and with the subsequent modular addition.

The relevant mathematical signal is the first-order rank of the front-half map. In the differential view, a fixed input difference Δ gives an affine derivative

$$D_\Delta \Phi(x) = M_\Delta x \oplus c_\Delta.$$

The rank $\text{rank}(M_\Delta)$ determines both the affine image dimension and the probability $2^{-\text{rank}(M_\Delta)}$ of each reachable output difference. In the linear view, a packed output mask β gives a scalar quadratic Boolean function

$$g_\beta(x) = \langle \beta, \Phi(x) \rangle,$$

whose Walsh spectrum is controlled by the bilinear matrix S_β , with nonzero correlation magnitude $2^{-\text{rank}(S_\beta)/2}$. Therefore a high differential rank and a high linear bilinear rank are the two first-order indicators that the Boolean front half is doing real work before the modular addition sees its derived operand.

The implementation was checked with these rank audits. In the low-Hamming weight audit class, where the tested first-order input values have $\text{wt}(\cdot) \leq 16$, the observed front-half ranks did not fall below 26. In the random full-width audit class, the observed front-half ranks did not fall below 28. These numbers are used here as design-audit evidence rather than as a closed-form theorem over the entire 2^{32} -point space. They explain why the front half is structured as a quadratic Boolean layer: even under the four-addition budget, it supplies a high-rank source for both the XOR-injection path and the subsequent two-variable modular-addition path.

14.2 Construction Validity of the Dynamic Diffusion Masks

Find Dynamic Diffusion Masks. The two dynamic diffusion masks were selected with the public `linear-box-search` C++ tool [Tw26c]. The relevant source files are the search driver `find_linear_box.cpp`, the GF(2) row-reduction backend `gf2_m4ri.cpp`, the deterministic verifier `heuristic_decomposer.cpp`, and the GF(2) inversion self-test `test_m4ri.cpp`. The search follows the same design philosophy as rotation-XOR linear-layer search by information-set decoding (ISD): candidates are compact circulant maps, and the Prange/ISD component is used to look for low-weight witnesses against their differential and linear branch-number targets [LP24].

For an n -bit generator word

$$g(z) = \sum_{i=0}^{n-1} g_i z^i \in \mathbb{F}_2[z]/(z^n - 1),$$

let $\mathcal{C}(g)$ denote the binary circulant operator

$$\mathcal{C}(g) = \sum_{i=0}^{n-1} g_i \lll^i.$$

Composition of two such maps is multiplication in $\mathbb{F}_2[z]/(z^n - 1)$. Thus g is invertible precisely when there exists h such that

$$g(z)h(z) \equiv 1 \pmod{z^n - 1}.$$

For $n = 32 = 2^5$, $z^{32} - 1 = (z + 1)^{32}$ over $\mathbb{F}_2$, so the fast invertibility screen reduces to $g(1) = 1$, i.e. odd coefficient parity. The implementation still prints and self-checks the inverse word h , so the accepted object has a directly reproducible inverse certificate.

Branch code and exact target.

Definition 27 (Graph code and low-weight decision predicate). For a generator g , define the graph code of the circulant map by

$$\mathcal{G}_g = \{(x, \mathcal{C}(g)x) : x \in \mathbb{F}_2^n\} \subset \mathbb{F}_2^{2n}.$$

It is the binary linear code generated by the $n \times 2n$ matrix

$$G_g = [I_n \mid \mathcal{C}(g)].$$

The threshold decision problem used by the search is the nonexistence of a nonzero low-weight codeword:

$$\neg \text{LowWt}(G_g, \tau - 1), \quad \text{LowWt}(G, w) \iff \exists c \in \langle G \rangle \setminus \{0\} : \text{wt}(c) \leq w.$$

Lemma 12 (Graph-code branch-number identity). *The differential branch number of $\mathcal{C}(g)$ is the minimum distance of the graph code $\mathcal{G}_g$:*

$$B_{\text{diff}}(g) = \min_{x \in \mathbb{F}_2^n \setminus \{0\}} (\text{wt}(x) + \text{wt}(\mathcal{C}(g)x)) = d(G_g).$$

Proof. The nonzero codewords of G_g are exactly $(x, \mathcal{C}(g)x)$ with $x \neq 0$. Their Hamming weights are $\text{wt}(x) + \text{wt}(\mathcal{C}(g)x)$. Taking the minimum over all nonzero x therefore gives both quantities. $\square$

Lemma 13 (Transpose identity for linear branch number). *The linear branch number is the same minimum-distance problem for the transpose map:*

$$B_{\text{lin}}(g) = \min_{\lambda \neq 0} (\text{wt}(\lambda) + \text{wt}(\mathcal{C}(g)^T \lambda)) = B_{\text{diff}}(g^*), \quad \mathcal{C}(g^*) = \mathcal{C}(g)^T.$$

Here g^* is the cyclic reflection of g . Thus the combined target is

$$B_{\text{comb}}(g) = \min(B_{\text{diff}}(g), B_{\text{lin}}(g)).$$

Proof. For a linear map $y = \mathcal{C}(g)x$, output masks pull back by the transpose: $\lambda \cdot y = (\mathcal{C}(g)^T \lambda) \cdot x$. Therefore a nonzero linear-mask transition has activity $\text{wt}(\lambda) + \text{wt}(\mathcal{C}(g)^T \lambda)$, which is the differential branch expression of the transpose circulant. $\square$

The search adds an implementation constraint to this mathematical target: g must be generated by a short straight-line program over XOR and word rotations. The right-rotation companion is obtained by evaluating the same straight-line program with all rotation directions exchanged.

ISD counterexample search.

Definition 28 (Information-set decoding counterexample oracle). Here ISD is the acronym for *information-set decoding*, specialized to the minimum-distance problem for a binary linear code. Let $G \in \mathbb{F}_2^{k \times m}$ generate $\mathcal{C}_G = \{uG : u \in \mathbb{F}_2^k\}$. An information set is a subset $I \subset \{1, \dots, m\}$, $|I| = k$, such that the column submatrix G_I is nonsingular. After a coordinate permutation π that moves I to the first k positions and row reduction over $\mathbb{F}_2$, one obtains a systematic generator

$$G^\pi \sim [I_k \mid A].$$

Every message u then gives

$$c^\pi(u) = u[I_k \mid A] = (u, uA), \quad \text{wt}(c^\pi(u)) = \text{wt}(u) + \text{wt}(uA).$$

Prange's ISD samples many information sets. If a low-weight codeword exists, then under a favorable information set its projection u on the systematic coordinates has small weight; enumerating low-weight row combinations can then recover the codeword. In this paper ISD is used only as the randomized counterexample oracle

$$\text{ISD}(G, w) = \begin{cases} c \in \mathcal{C}_G \setminus \{0\}, & \text{if it finds } \text{wt}(c) \leq w, \\ \perp, & \text{otherwise.} \end{cases}$$

The answer $c \neq \perp$ is a verifiable mathematical witness against the threshold. The answer $\perp$ is not a proof; it only means that this randomized pass did not find a violating codeword.

Algorithm 33 Prange/ISD low-weight witness oracle for a branch code

Require: binary generator matrix $G = [I_n \mid \mathcal{C}(g)]$, target $w = \tau - 1$

Require: number of trials T , row-combination radius ρ

Ensure: either a nonzero codeword $c \in \langle G \rangle$ with $\text{wt}(c) \leq w$, or $\perp$

- 1: **for** $t = 1, \dots, T$ **do**
- 2: sample a random permutation π of the $2n$ code coordinates.
- 3: let G^π be G with columns permuted by π .
- 4: **if** the first n columns of G^π are singular **then**
- 5: continue.
- 6: **end if**

```

7:   row-reduce  $G^\pi$  over  $\mathbb{F}_2$  to systematic form  $[I_n \mid A]$ .
8:   for each nonzero  $u \in \mathbb{F}_2^n$  with  $\text{wt}(u) \leq \rho$  do
9:      $c^\pi \leftarrow (u, uA)$ .
10:    if  $\text{wt}(c^\pi) \leq w$  then
11:      undo the coordinate permutation and return  $c = \pi^{-1}(c^\pi)$ .
12:    end if
13:  end for
14: end for
15: return  $\perp$ .

```

For the branch code $G_g = [I_n \mid \mathcal{C}(g)]$, any returned word

$$c = (x, y)$$

is checked by the identity $y = \mathcal{C}(g)x$. Hence an ISD hit with $\text{wt}(c) \leq \tau - 1$ proves $B_{\text{diff}}(g) < \tau$. The same oracle is applied to $G_{g^*} = [I_n \mid \mathcal{C}(g)^T]$ for the linear branch number. This gives a fast rejection mechanism for bad candidates.

Deterministic acceptance certificate. The final acceptance test is deterministic. It does not rely on ISD failure. For $N = \tau - 1$, define

$$E_N(g) = \min_{\substack{x \neq 0 \\ \text{wt}(x) \leq N}} (\text{wt}(x) + \text{wt}(\mathcal{C}(g)x)).$$

The verifier enumerates all inputs of weight at most N . Inputs outside the enumerated set already have $\text{wt}(x) \geq \tau$, so they cannot produce total activity below τ . Exactness is then obtained by also supplying a weight- τ witness.

Algorithm 34 Deterministic acceptance test for one circulant generator

Require: generator $g \in \mathbb{F}_2[z]/(z^{32} - 1)$, threshold $\tau = 12$
Ensure: exact pair $(B_{\text{diff}}(g), B_{\text{lin}}(g)) = (12, 12)$, or reject

```

1: compute  $h$  with  $gh \equiv 1 \pmod{z^{32} - 1}$ ; reject if no such  $h$  exists.
2: verify  $\mathcal{C}(h)\mathcal{C}(g) = I_{32}$ ; reject if the identity fails.
3: for  $q \in \{g, g^*\}$  do
4:   set  $m \leftarrow +\infty$ .
5:   for each  $x \in \mathbb{F}_2^{32} \setminus \{0\}$  with  $\text{wt}(x) \leq \tau - 1$  do
6:      $a \leftarrow \text{wt}(x) + \text{wt}(\mathcal{C}(q)x)$ .
7:      $m \leftarrow \min(m, a)$ .
8:     if  $a \leq \tau - 1$  then
9:       reject.
10:    end if
11:  end for
12:  find or verify an explicit witness  $x_0 \neq 0$  with  $\text{wt}(x_0) + \text{wt}(\mathcal{C}(q)x_0) = \tau$ ; reject if none is supplied.
13: end for
14: accept  $g$  with exact differential and linear branch number  $\tau$ .

```

Theorem 24 (Deterministic branch-number acceptance). *If Algorithm 34 accepts g , then*

$$B_{\text{diff}}(g) = B_{\text{lin}}(g) = \tau.$$

Proof. For $q = g$, the enumeration proves that no nonzero x with $\text{wt}(x) \leq \tau - 1$ has total activity at most $\tau - 1$. If $\text{wt}(x) \geq \tau$, then $\text{wt}(x) + \text{wt}(\mathcal{C}(g)x) \geq \tau$ automatically. Hence

every nonzero input has total activity at least τ . The supplied witness has total activity exactly τ , so $B_{\text{diff}}(g) = \tau$. Applying the same argument to $q = g^*$ and Lemma 13 gives $B_{\text{lin}}(g) = \tau$. $\square$

The recorded command was

```
find_linear_box.exe -bits 32 -efficient-implementation
-quality-threshold-branch-number 12 -max-xor 6 -seed 4
-need-found-result 2 -no-progress.
```

In this run, $n = 32$, the target threshold was $\tau = 12$, and the search was restricted to at most six XOR-combine steps. The high-level procedure is shown in Algorithm 35.

Algorithm 35 Search and acceptance test for the dynamic diffusion masks

Require: word size $n = 32$, threshold $\tau = 12$, XOR-step cap $K = 6$, seed s
Require: deterministic certification cap $N = \tau - 1 = 11$
Ensure: certified left/right generator words g_L, g_R , inverses h_L, h_R

- 1: Initialize the pseudorandom generator from s .
- 2: **repeat**
- 3: Sample an XOR-rotation straight-line program P with at most K XOR-combine steps.
- 4: $g_L \leftarrow P_{\ll}(e_0)$, where every rotation in P is evaluated as a left rotation.
- 5: $g_R \leftarrow P_{\gg}(e_0)$, where the same program is evaluated with right rotations.
- 6: **if** $g_L(1) = 0$ or $g_R(1) = 0$ **then**
- 7: reject the candidate and continue.
- 8: **end if**
- 9: Compute h_L, h_R such that $g_L h_L \equiv g_R h_R \equiv 1 \pmod{z^{32} - 1}$.
- 10: **if** $\mathcal{C}(h_L)\mathcal{C}(g_L) \neq I$ or $\mathcal{C}(h_R)\mathcal{C}(g_R) \neq I$ **then**
- 11: reject the candidate and continue.
- 12: **end if**
- 13: **for** $g \in \{g_L, g_R, g_L^*, g_R^*\}$ **do**
- 14: Run deterministic unit-input scans and exhaustive enumeration for $\text{wt}(x) \leq 2$.
- 15: **if** a word x with $\text{wt}(x) + \text{wt}(\mathcal{C}(g)x) \leq \tau - 1$ is found **then**
- 16: reject the candidate and continue the outer loop.
- 17: **end if**
- 18: $c \leftarrow \text{ISD}([I \mid \mathcal{C}(g)], \tau - 1)$, using Algorithm 33.
- 19: **if** $c \neq \perp$ **then**
- 20: reject the candidate and continue the outer loop.
- 21: **end if**
- 22: **end for**
- 23: **for** $g \in \{g_L, g_R, g_L^*, g_R^*\}$ **do**
- 24: Deterministically enumerate all $x \neq 0$ with $\text{wt}(x) \leq N$.
- 25: **if** some enumerated x has $\text{wt}(x) + \text{wt}(\mathcal{C}(g)x) \leq \tau - 1$ **then**
- 26: reject the candidate and continue the outer loop.
- 27: **end if**
- 28: **end for**
- 29: Verify that a total-weight- τ witness exists for each of g_L, g_R, g_L^*, g_R^* .
- 30: **until** all four branch-number checks are exact with value τ
- 31: **return** g_L, g_R, h_L, h_R .

For the two selected words, the deterministic verifier was run with $\tau = 12$ and $N = 11$:

```
linear_box_heuristic_decomposer.exe -matrix-bits 32
-word-size 32 -word-count 1 -verify
-exhaustive-input-weight 11 d05a0889,

linear_box_heuristic_decomposer.exe -matrix-bits 32
-word-size 32 -word-count 1 -verify
-exhaustive-input-weight 11 2220b417.
```

It reports exact metrics

g	$B_{\text{diff}}(g)$	$B_{\text{lin}}(g)$	$B_{\text{comb}}(g)$
0xd05a0889	12	12	12
0x2220b417	12	12	12

with the transpose pairing

$$B_{\text{lin}}(0xd05a0889) = B_{\text{diff}}(0x2220b417), \quad B_{\text{lin}}(0x2220b417) = B_{\text{diff}}(0xd05a0889).$$

Thus the numbers used below are not merely the best weights found by ISD: the accepted masks have explicit inverse generators and exact deterministic branch-number certificates for the threshold 12.

Equivalently, the certificate consumed by the specification is the finite object

$$\text{Cert} = (g_L, g_R, h_L, h_R, \mathcal{W}_L, \mathcal{W}_R, \mathcal{W}_L^*, \mathcal{W}_R^*),$$

where

$$g_L = 0xd05a0889, \quad g_R = 0x2220b417, \quad h_L = 0x5fc08ef4, \quad h_R = 0x5ee207f4.$$

It satisfies the inverse equations

$$g_L h_L \equiv 1 \pmod{z^{32} - 1}, \quad g_R h_R \equiv 1 \pmod{z^{32} - 1},$$

the four exhaustive nonexistence claims

$$\forall g \in \{g_L, g_R, g_L^*, g_R^*\}, \quad \forall x \neq 0, \text{wt}(x) \leq 11 : \quad \text{wt}(x) + \text{wt}(\mathcal{C}(g)x) \geq 12,$$

and the four weight-12 witnesses

g	differential witness	linear witness
0xd05a0889	$0x00000001 \mapsto 0xd05a0889$	$0x00000001 \mapsto 0x2220b417$
0x2220b417	$0x00000001 \mapsto 0x2220b417$	$0x00000001 \mapsto 0xd05a0889$

where each arrow denotes $x \mapsto \mathcal{C}(g)x$ for the differential column and $x \mapsto \mathcal{C}(g)^T x$ for the linear column. The lower-bound and witness parts together imply exact branch number 12, not only absence of an ISD-found counterexample.

Dynamic Diffusion Masks Maths.

Construction 4 (Dynamic diffusion-mask maps). Before analyzing the nonlinear branch-injection layer, we first justify the mathematical object used to generate the dynamic masks. The purpose of this subsection is not yet to re-prove the branch number of the selected linear maps. Instead, it explains how the masks are constructed and why the resulting objects are valid fixed linear maps over $\mathbb{F}_2^{32}$. Only after this construction is well-defined does it make sense to analyze its differential and linear diffusion properties.

Let $\lll$ denote the 32-bit left rotation operator. All additions in this subsection are XOR additions over $\mathbb{F}_2$. A word-level expression built only from XOR and rotations is a linear map

$$L : \mathbb{F}_2^{32} \longrightarrow \mathbb{F}_2^{32}.$$

Moreover, because rotations commute with powers of the same cyclic shift, such a map is a binary circulant linear operator. Equivalently, it can be written as a polynomial in the rotation operator:

$$L(X) = \text{Polynomial}(\lll)X, \quad \text{Polynomial}(\lll) \in \mathbb{F}_2[\lll]/(\lll^{32} - 1).$$

Therefore, a mask generator constructed from XOR and rotations is not a random procedure or a heuristic table. It is a fixed linear transformation of the input word.

The implementation uses two directionally paired generators. The first one is the left-rotation generator

$$\begin{aligned} v_0 &= X, \\ v_1 &= v_0 \oplus \lll^2 v_0, \\ v_2 &= v_0 \oplus \lll^{17} v_1, \\ v_3 &= v_0 \oplus \lll^4 v_2, \\ v_4 &= v_3 \oplus \lll^{24} v_3, \\ M_0(X) &= v_2 \oplus \lll^7 v_4. \end{aligned}$$

This is the mathematical form of

`generate_dynamic_diffusion_mask0.`

Expanding the recurrence gives

$$M_0(X) = (I + \lll^3 + \lll^7 + \lll^{11} + \lll^{17} + \lll^{19} + \lll^{20} + \lll^{22} + \lll^{28} + \lll^{30} + \lll^{31})X.$$

Hence the image of the basis word 1 is

$$M_0(1) = \text{0xd05a0889}.$$

The search tool also reports the inverse generator

$$M_0^{-1}(1) = \text{0x5fc08ef4},$$

which means the selected linear map is invertible after verification:

$$M_0^{-1} \circ M_0 = I.$$

The second generator is the right-rotation counterpart. Let $\ggg$ be the right rotation operator. The implementation computes

$$\begin{aligned} w_0 &= X, \\ w_1 &= w_0 \oplus \ggg^2 w_0, \\ w_2 &= w_0 \oplus \ggg^{17} w_1, \\ w_3 &= w_0 \oplus \ggg^4 w_2, \\ w_4 &= w_3 \oplus \ggg^{24} w_3, \\ M_1(X) &= w_2 \oplus \ggg^7 w_4. \end{aligned}$$

This is the mathematical form of

`generate_dynamic_diffusion_mask1.`

Equivalently, written again in powers of the left-rotation operator $\lll$,

$$M_1(X) = (I + \lll^1 + \lll^2 + \lll^4 + \lll^{10} + \lll^{12} + \lll^{13} + \lll^{15} + \lll^{21} + \lll^{25} + \lll^{29})X.$$

Thus

$$M_1(1) = \text{0x2220b417}.$$

The corresponding inverse reported by the search tool is

$$M_1^{-1}(1) = \text{0x5ee207f4}.$$

After inverse verification, this gives

$$M_1^{-1} \circ M_1 = I.$$

The compact words above are the first-column, or equivalently basis-response, descriptions of the two 32×32 circulant matrices. With the little-endian unit basis $e_i = \lll^i e_0$,

$$M_0(e_i) = \lll^i M_0(e_0), \quad M_1(e_i) = \lll^i M_1(e_0).$$

Thus the two hexadecimal words determine all columns of the corresponding binary matrices; no additional table is hidden in the implementation.

The two masks are therefore not independent constants and not externally sampled randomness. They are deterministic source-dependent linear views:

$$\text{mask}_0 = M_0(B)$$

in the $B \rightarrow A$ injection and

$$\text{mask}_1 = M_1(A)$$

in the $A \rightarrow B$ injection. The word called a “dynamic mask” is dynamic only because it depends on the current branch value. The map that produces it is fixed, public, and linear.

Algorithm 35 records the bounded rotation-XOR search used to obtain the construction. A candidate was represented by the image of the basis word 1, because a binary circulant map is fully determined by that generator word. The current implementation uses the printed pair

$$M_0(1) = \text{0xd05a0889}, \quad M_1(1) = \text{0x2220b417}.$$

The search also produced inverse generator words, so the selected candidates are not many-to-one projections. They are invertible linear permutations of the 32-bit word space.

This point is important for the later injection-layer analysis. If the mask generator collapsed information, then a good-looking nonlinear expression using that mask could still hide a structural degeneracy. Here the mask generation step is a bijective linear preprocessing of the source branch. Therefore, before the Boolean nonlinear terms are considered, the branch value is first exposed through an invertible rotation-XOR view. This makes the subsequent differential and linear analysis meaningful: the nonlinear injection is analyzed on top of fixed, public, invertible linear maps rather than on top of an ill-defined or lossy mask source.

In the branch-injection code, these maps are used as follows. For the $B \rightarrow A$ direction,

$$\text{mask} = M_0(B),$$

and the nonlinear terms combine B , rotated companions of B , the mask, rotated views of the mask, and public round constants through XOR, AND, OR, NOT, shifts, and rotations. For the $A \rightarrow B$ direction,

$$\text{mask} = M_1(A),$$

with the analogous opposite-direction construction. The injection functions therefore have a Feistel-like form: the opposite branch is not overwritten directly by a table lookup, but by a deterministic nonlinear function whose internal mask source is a verified linear permutation of the active branch.

Consequently, the construction qualifies for analysis for three basic reasons.

First, the mask maps are linear maps over $\mathbb{F}_2^{32}$, because they are built only from XOR and rotations. Second, they are invertible maps, as witnessed by the reported inverse generator words and verified by composition with the inverse. Third, they are efficiently implementable straight-line rotation-XOR programs, so the mathematical maps and the C++ implementation are the same object, not two different descriptions.

Thus the later diffusion, differential, and linear analyses are applied to a well-defined fixed construction:

$$B \mapsto M_0(B) \quad \text{and} \quad A \mapsto M_1(A),$$

followed by explicit Boolean nonlinear mixing. The role of this subsection is only to establish that the mask-generation layer is a valid mathematical construction. The quality of its branch number and its contribution to the full injection layer are separate analysis questions.

14.3 Quadratic Boolean Front Half

Definition 29 (Local front-half oracle domain). Let

$$V = \mathbb{F}_2^{32}, \quad W = \mathbb{F}_2^{64},$$

and let $\text{Aff}(V, V)$ denote the affine maps $x \mapsto Lx \oplus a$ with $L \in \text{Hom}(V, V)$ and $a \in V$. The local front-half oracles are

$$J_B, J_A : V \rightarrow W, \quad J_s(x) = (C_s(x), D_s(x)), \quad s \in \{A, B\}.$$

They are evaluated only after the fixed-public subtractions

$$B_0 = B \boxminus \text{RC}[1], \quad A_0 = A \boxminus \text{RC}[6].$$

Those subtractions belong to the fixed-addend arithmetic model. In this subsection the variables $B_0, A_0 \in V$ are the already reduced source words.

Lemma 14 (Quadratic affine-view normal form of the local front half). *For each source side $s \in \{A, B\}$, the local front-half oracle has the form*

$$J_s(x) = b_s \oplus L_s x \oplus \bigoplus_{t \in \mathcal{T}_s} P_{s,t} \left((L_{s,t} x \oplus a_{s,t}) \wedge (M_{s,t} x \oplus b_{s,t}) \right),$$

where

$$b_s \in W, \quad L_s : V \rightarrow W, \quad L_{s,t}, M_{s,t} : V \rightarrow V, \quad a_{s,t}, b_{s,t} \in V, \quad P_{s,t} : V \rightarrow W$$

are public maps or constants. Equivalently,

$$J_s \in W \otimes (\mathbb{F}_2 \oplus V^* \oplus \text{Sym}^2(V^*)).$$

This is the algebraic type used below: a degree-at-most-two vectorial Boolean map built from products between affine views of one source branch.

Proof. All operations before a product gate are affine over $\mathbb{F}_2$. Rotations, zero-filling shifts, and `mask0`, `mask1` are linear maps on V ; XOR by a public constant and bitwise complement are affine:

$$\neg z = z \oplus \mathbf{1}.$$

For $U, Z \in \text{Aff}(V, V)$, the coordinatewise product

$$(U \wedge Z)_i = U_i Z_i$$

has degree at most 2. Bitwise OR adds no new algebraic class:

$$U \vee Z = U \oplus Z \oplus (U \wedge Z).$$

Collecting all affine terms into $b_s \oplus L_s x$, and collecting each coordinatewise product into a public projection $P_{s,t}$, gives the stated normal form. $\square$

Construction 5 (Concrete B -side and A -side front-half formulas). For the B -side oracle, set

$$\begin{aligned} m_B &= \text{mask0}(B_0), \\ p_B &= B_0 \ggg_{24}, \\ \bar{m}_B &= m_B \ggg_{24} \oplus \text{MASK0_RC7}, \\ r_B &= m_B \ggg_5, \\ x_{0,B} &= p_B \oplus m_B, \\ x_{1,B} &= B_0 \oplus m_B, \\ v_B &= p_B \oplus \bar{m}_B, \\ s_B &= B_0 \ggg_{19} \oplus (B_0 \lll 9). \end{aligned}$$

The nonlinear carrier is

$$\begin{aligned} q_{\text{state}}^B &= \text{RC7_R24} \oplus \neg(B_0 \wedge m_B), \\ q_{\text{comp}}^B &= p_B \oplus B_0 \oplus \text{RC8_R24} \oplus \neg(p_B \vee r_B), \\ q_{\text{bridge}}^B &= s_B \oplus B_0 \oplus \text{RC13_R24} \oplus \neg(s_B \wedge \bar{m}_B), \\ q_{\text{shared}}^B &= q_{\text{state}}^B \oplus q_{\text{comp}}^B, \\ \text{cross}_B &= (B_0 \oplus r_B) \wedge m_B \oplus \bar{m}_B \ggg_7, \\ \text{anti}_B &= ((x_{1,B} \ggg 3) \oplus (v_B \ggg 5) \oplus r_B) \wedge (B_0 \oplus x_{0,B} \ggg_{11}). \end{aligned}$$

The output pair is

$$\begin{aligned} C_B &= q_{\text{shared}}^B \oplus q_{\text{comp}}^B \ggg_5 \oplus q_{\text{comp}}^B \ggg_{11} \oplus \text{anti}_B, \\ D_B &= q_{\text{shared}}^B \oplus q_{\text{state}}^B \ggg_5 \oplus q_{\text{bridge}}^B \ggg_{13} \oplus \text{cross}_B \oplus \text{anti}_B. \end{aligned}$$

For the A -side oracle, set

$$\begin{aligned} m_A &= \text{mask1}(A_0), \\ p_A &= A_0 \lll^8, \\ \bar{m}_A &= m_A \lll^8 \oplus \text{MASK1_RC2}, \\ r_A &= m_A \ggg_5, \\ x_{0,A} &= p_A \oplus m_A, \\ x_{1,A} &= A_0 \oplus m_A, \\ v_A &= p_A \oplus \bar{m}_A, \\ s_A &= A_0 \lll^{19} \oplus (A_0 \ggg 9). \end{aligned}$$

The nonlinear carrier is

$$\begin{aligned}
q_{\text{state}}^A &= \text{RC2_L8} \oplus \neg(A_0 \vee m_A), \\
q_{\text{comp}}^A &= p_A \oplus A_0 \oplus \text{RC3_L8} \oplus \neg(p_A \wedge r_A), \\
q_{\text{bridge}}^A &= s_A \oplus A_0 \oplus \text{RC12_L8} \oplus \neg(s_A \vee \bar{m}_A), \\
q_{\text{shared}}^A &= q_{\text{state}}^A \oplus q_{\text{comp}}^A, \\
\text{cross}_A &= (A_0 \oplus r_A) \wedge m_A \oplus \bar{m}_A \lll^{13}, \\
\text{anti}_A &= ((x_{1,A} \ll 3) \oplus (v_A \ll 5) \oplus r_A) \vee (A_0 \oplus x_{0,A} \lll^{11}).
\end{aligned}$$

The output pair is

$$\begin{aligned}
C_A &= q_{\text{shared}}^A \oplus q_{\text{comp}}^A \lll^5 \oplus q_{\text{comp}}^A \lll^{11} \oplus \text{anti}_A, \\
D_A &= q_{\text{shared}}^A \oplus q_{\text{state}}^A \lll^5 \oplus q_{\text{bridge}}^A \lll^{13} \oplus \text{cross}_A \oplus \text{anti}_A.
\end{aligned}$$

The terms $m_s, p_s, r_s, x_{0,s}, x_{1,s}, s_s$ are linear source views; the terms $\bar{m}_s, v_s$ are affine views because of the public constants MASK0_RC7 and MASK1_RC2. The products appearing in $q_{\text{state}}, q_{\text{comp}}, q_{\text{bridge}}, \text{cross}$, and anti are the set $\mathcal{T}_s$ in the normal form above.

Lemma 15 (Affine derivative of a quadratic vector map). *For any quadratic vector map $F : V \rightarrow W$ and any source input difference $\Delta_{\text{in}} \in V$, the derivative*

$$D_{\Delta_{\text{in}}} F(x) = F(x \oplus \Delta_{\text{in}}) \oplus F(x)$$

is affine. Indeed, for affine scalar views u, v ,

$$D_{\Delta_{\text{in}}}(uv) = (D_{\Delta_{\text{in}}} u)v \oplus u(D_{\Delta_{\text{in}}} v) \oplus (D_{\Delta_{\text{in}}} u)(D_{\Delta_{\text{in}}} v),$$

and $D_{\Delta_{\text{in}}} u, D_{\Delta_{\text{in}}} v$ are constants. Hence

$$D_{\Delta_{\text{in}}} J_s(x) = M_{s, \Delta_{\text{in}}}^J x \oplus c_{s, \Delta_{\text{in}}}^J$$

for a public matrix $M_{s, \Delta_{\text{in}}}^J$ and offset $c_{s, \Delta_{\text{in}}}^J$. For an output difference $\Delta_{\text{out}} \in W$,

$$\text{DP}_{J_s}(\Delta_{\text{in}} \rightarrow \Delta_{\text{out}}) = \begin{cases} 2^{-\text{rank}(M_{s, \Delta_{\text{in}}}^J)}, & \Delta_{\text{out}} \oplus c_{s, \Delta_{\text{in}}}^J \in \text{im}(M_{s, \Delta_{\text{in}}}^J), \\ 0, & \text{otherwise.} \end{cases}$$

Theorem 25 (Walsh support of the contracted quadratic phase). *For any output mask $\beta \in W$, the contracted phase*

$$g_{s, \beta}(x) = \langle \beta, J_s(x) \rangle$$

is a scalar quadratic Boolean function. Write it as

$$g_{s, \beta}(x) = g_{s, \beta}(0) \oplus \langle \ell_{s, \beta}, x \rangle \oplus q_{s, \beta}(x),$$

where $q_{s, \beta}$ is homogeneous quadratic. Its polar form

$$B_{s, \beta}(u, v) = g_{s, \beta}(0) \oplus g_{s, \beta}(u) \oplus g_{s, \beta}(v) \oplus g_{s, \beta}(u \oplus v)$$

is represented by an alternating matrix $S_{s, \beta}^J$. The normalized correlation

$$C_{J_s}(\alpha, \beta) = 2^{-32} \sum_{x \in V} (-1)^{\langle \alpha, x \rangle \oplus \langle \beta, J_s(x) \rangle}$$

is governed by the radical

$$\mathcal{R}_{s, \beta} = \ker(S_{s, \beta}^J).$$

The support condition is

$$C_{J_s}(\alpha, \beta) \neq 0 \iff g_{s,\beta}(z) \oplus g_{s,\beta}(0) = \langle \alpha, z \rangle \text{ for all } z \in \mathcal{R}_{s,\beta}.$$

Equivalently, the valid input masks form one affine coset

$$\alpha \in \alpha_{s,\beta}^0 \oplus \text{im}(S_{s,\beta}^J),$$

where $\alpha_{s,\beta}^0$ is any representative satisfying the radical equations. Every nonzero value has magnitude

$$|C_{J_s}(\alpha, \beta)| = 2^{-\text{rank}(S_{s,\beta}^J)/2}.$$

Thus the local linear weight is $\text{rank}(S_{s,\beta}^J)/2$, while masks outside the coset have zero correlation.

Proof. Let

$$S(\alpha) = \sum_{x \in V} (-1)^{g_{s,\beta}(x) \oplus \langle \alpha, x \rangle}.$$

For any $z \in \mathcal{R}_{s,\beta}$, translating the summation variable by z gives

$$S(\alpha) = (-1)^{g_{s,\beta}(z) \oplus g_{s,\beta}(0) \oplus \langle \alpha, z \rangle} S(\alpha),$$

because the polar form vanishes on z . Hence $S(\alpha) = 0$ unless the displayed radical equations hold for every $z \in \mathcal{R}_{s,\beta}$. These equations define an affine coset of $\mathcal{R}_{s,\beta}^\perp$, and for the representing alternating matrix $\mathcal{R}_{s,\beta}^\perp = \text{im}(S_{s,\beta}^J)$.

When the radical equations hold, the phase is constant on cosets of $\mathcal{R}_{s,\beta}$ and descends to a nondegenerate quadratic form on $V/\mathcal{R}_{s,\beta}$, whose dimension is $\text{rank}(S_{s,\beta}^J)$. The standard quadratic Gauss sum over $\mathbb{F}_2$ has absolute value $2^{\text{rank}(S_{s,\beta}^J)/2}$ on the quotient. Multiplying by the radical size and normalizing by 2^{32} gives $|C_{J_s}(\alpha, \beta)| = 2^{-\text{rank}(S_{s,\beta}^J)/2}$. $\square$

Corollary 6 (Rank-analyzable class of the front-half maps). *The design role of the front half is contained in these definitions. With $x_B = B_0$ and $x_A = A_0$, the view system*

$$\{x_s, m_s, p_s, \bar{m}_s, r_s, x_{0,s}, x_{1,s}, v_s, s_s\}$$

*feeds quadratic products between different coordinate systems of the same source word. The shared carrier q_{shared} appears in both output words; the **cross** term multiplies a source-mask view by a rotated mask-difference view; the **anti** term mixes shifted views with a source-versus-companion expression. The B-side and A-side keep the same degree bound but swap the AND/OR polarity in the state, companion, bridge, and anti channels. Thus both directions remain in the same rank-analyzable quadratic class while producing different derivative matrices and different masked bilinear forms.*

The packed maps Φ_B and Φ_A used next are obtained from J_B, J_A by linear post-processing and direct source terms. They therefore remain vectorial Boolean maps of degree at most 2, with the same affine-derivative and quadratic-Walsh consequences.

14.4 Quadratic Boolean Injection Function Differential Behavior

Theorem 26 (Differential coupling of the packed injection oracle). *The previous subsection identifies the Boolean front half as a packed quadratic source oracle*

$$\Phi_s(x) = (X_s(x), Y_s(x)), \quad s \in \{A, B\},$$

where $X_s(x)$ is the XOR-injection word and $Y_s(x)$ is the injection-derived arithmetic operand. The important point is that these two words are not independent outputs. They

are produced by the same source word x , the same dynamic diffusion mask, and the same quadratic carrier system.

Thus, when the trail reaches the following modular addition or subtraction, it does not enter an ordinary unconstrained two-variable add/sub box. It enters a two-variable modular arithmetic box whose second input difference is constrained by the previous quadratic oracle.

For a fixed source difference

$$\Delta s \in \mathbb{F}_2^{32},$$

the first-order derivative of the packed oracle is affine:

$$D_{\Delta s} \Phi_s(x) = M_{s, \Delta s} x \oplus c_{s, \Delta s}.$$

The reachable packed output differences are therefore exactly the affine subspace

$$\mathcal{A}_{s, \Delta s} = c_{s, \Delta s} \oplus \text{im}(M_{s, \Delta s}) \subseteq \mathbb{F}_2^{64}.$$

Every reachable pair

$$(\delta_X, \delta_Y) \in \mathcal{A}_{s, \Delta s}$$

occurs with probability

$$2^{-\text{rank}(M_{s, \Delta s})},$$

and every pair outside this affine image has probability zero.

Now consider the local source-controlled update in target/source order. Let t be the target branch being modified and s the preserved source branch. The Boolean front half first applies the XOR-injection word:

$$t \mapsto t \oplus X_s(s).$$

Then the arithmetic back half applies a modular addition with the injection-derived operand:

$$t' = (t \oplus X_s(s)) \boxplus H_s(s),$$

where

$$H_s(s) = Y_s(s) \oplus K_s$$

and K_s is the corresponding public round constant. In the differential setting, this public XOR constant has no effect on the addend difference, so the addend difference entering the two-variable addition is still

$$\delta_Y.$$

Therefore, for a selected reachable oracle output difference

$$(\delta_X, \delta_Y) \in \mathcal{A}_{s, \Delta s},$$

the input difference of the first addition operand is not simply Δt . It is

$$\alpha = \Delta t \oplus \delta_X,$$

because the target word has already been XOR-routed by X_s . The second addition operand has difference

$$\beta = \delta_Y.$$

The target output difference is

$$\gamma = \Delta t'.$$

Hence the following arithmetic condition is the ordinary two-variable modular addition transition

$$\text{DP}^+(\alpha, \beta \rightarrow \gamma) = \text{DP}^+(\Delta t \oplus \delta_X, \delta_Y \rightarrow \Delta t'),$$

but with the crucial restriction that

$$(\delta_X, \delta_Y) \in c_{s, \Delta s} \oplus \text{im}(M_{s, \Delta s}).$$

This is the key coupling. The modular addition model itself is the standard two-variable add-difference model, but its two input differences are not freely chosen. One input is the target difference after XOR injection,

$$\Delta t \oplus \delta_X,$$

and the other is the addend difference generated by the same source oracle,

$$\delta_Y.$$

Both δ_X and δ_Y come from the same affine derivative image of Φ_s . Separating them would destroy the implemented structure.

At the single-characteristic level, a local differential transition is feasible only if there exists a packed oracle witness

$$(\delta_X, \delta_Y) \in \mathcal{A}_{s, \Delta s}$$

such that the two-variable modular addition transition

$$\text{DP}^+(\Delta t \oplus \delta_X, \delta_Y \rightarrow \Delta t')$$

is feasible. Its local characteristic weight is

$$w_{\mathcal{I}_s}^{\text{diff}} = \text{rank}(M_{s, \Delta s}) + W_+^{\text{diff}}(\Delta t \oplus \delta_X, \delta_Y, \Delta t'),$$

where

$$W_+^{\text{diff}} = -\log_2 \text{DP}^+.$$

Equivalently, if one writes the full local differential probability by summing over the hidden oracle witness pairs, the factorized form is

$$\text{DP}_{\mathcal{I}_s}^{\text{fact}}((\Delta t, \Delta s) \rightarrow (\Delta t', \Delta s)) = \sum_{(\delta_X, \delta_Y) \in \mathcal{A}_{s, \Delta s}} 2^{-\text{rank}(M_{s, \Delta s})} \text{DP}^+(\Delta t \oplus \delta_X, \delta_Y \rightarrow \Delta t').$$

The implemented characteristic MILP uses a selected witness rather than aggregating the full hull sum. Therefore the model should be read as a joint-oracle characteristic model:

$$\text{quadratic oracle witness} \quad + \quad \text{two-variable modular-addition transition}.$$

For subtraction, the same principle applies through the usual reduction

$$x \boxminus y = z \quad \Longleftrightarrow \quad z \boxplus y = x.$$

Thus the two-variable subtraction model is not a separate kind of oracle here. It is the same source-controlled arithmetic transition written in the inverse addition orientation. The important point is unchanged: the borrow or carry chain is fed by an operand difference δ_Y that is constrained by the same packed Boolean oracle that also produced the XOR-routing difference δ_X .

This is why the first-order differential analysis must not stop at the rank of the Boolean front half. The rank says which packed pairs

$$(\delta_X, \delta_Y)$$

can leave the quadratic oracle and at what cost. The following two-variable modular addition then decides which of those pairs can actually pass through the carry chain together with the target difference. The layer is therefore a coupled object:

$$\Delta s \longmapsto (\delta_X, \delta_Y) \longmapsto (\Delta t \oplus \delta_X, \delta_Y) \xrightarrow{\boxplus} \Delta t'.$$

That is the first-order differential meaning of the injection layer.

14.5 Quadratic Boolean Injection Function Linear Behavior

Theorem 27 (Linear coupling of the packed injection oracle). *The linear side has the same coupling, but in the dual mask direction. The packed oracle*

$$\Phi_s(x) = (X_s(x), Y_s(x))$$

does not provide an arbitrary mask for X_s and an unrelated arbitrary mask for Y_s . A packed output mask

$$\beta = (\beta_X, \beta_Y) \in \mathbb{F}_2^{64}$$

contracts both oracle outputs at once:

$$g_{s,\beta}(x) = \langle \beta_X, X_s(x) \rangle \oplus \langle \beta_Y, Y_s(x) \rangle.$$

Because Φ_s is quadratic, this is a scalar quadratic Boolean function. Its Walsh support and its weight are governed by the bilinear matrix $S_{s,\beta}$. If

$$r_{s,\beta} = \text{rank}(S_{s,\beta}),$$

then every nonzero oracle correlation has magnitude

$$2^{-r_{s,\beta}/2},$$

and the local oracle weight is

$$r_{s,\beta}/2.$$

The input mask must also satisfy the exact support-coset condition

$$\alpha_s \in \text{support_offset}_{s,\beta} \oplus \text{im}(S_{s,\beta}).$$

Now connect this oracle to the following modular addition. In local target/source order, the value update is

$$t' = (t \oplus X_s(s)) \boxplus (Y_s(s) \oplus K_s).$$

Let

$$\lambda$$

be the input mask on t , let

$$\gamma$$

be the output mask on t' , and let

$$\nu, \mu$$

be the source-branch masks before and after the local source-preserving update. Since the source branch is preserved as a state word, the source-side mask difference that must be supplied by the oracle is

$$\alpha_s = \nu \oplus \mu.$$

The modular addition has two input operands:

$$u = t \oplus X_s(s), \quad h = Y_s(s) \oplus K_s.$$

In the linear approximation of

$$t' = u \boxplus h,$$

let

$$\eta$$

be the internal mask assigned to the addend operand h . Then the two-variable modular addition contributes the ordinary two-input addition correlation

$$\text{Corr}^+(\gamma, \lambda, \eta),$$

where γ is the output mask, λ is the mask on the first input operand u , and η is the mask on the second input operand h .

But $u = t \oplus X_s(s)$. Therefore the mask λ appears twice: once on the original target word t , and once as the output mask on the XOR-injection word $X_s(s)$. Similarly, the addend mask η becomes the output mask on $Y_s(s)$. Thus the packed output mask entering the quadratic oracle is not arbitrary; it is

$$\beta = (\lambda, \eta).$$

The oracle term is therefore

$$\text{Corr}_{\Phi_s}(\nu \oplus \mu, (\lambda, \eta)).$$

The public constant K_s in

$$h = Y_s(s) \oplus K_s$$

does not change the mask propagation, but it contributes a known sign:

$$(-1)^{\langle \eta, K_s \rangle}.$$

Consequently, the factorized local linear hull is

$$\text{Corr}_{\mathcal{I}_s}^{\text{fact}}((\lambda, \nu) \rightarrow (\gamma, \mu)) = \sum_{\eta \in \mathbb{F}_2^{32}} (-1)^{\langle \eta, K_s \rangle} \text{Corr}^+(\gamma, \lambda, \eta) \text{Corr}_{\Phi_s}(\nu \oplus \mu, (\lambda, \eta)).$$

At the single-characteristic level, the implemented linear MILP chooses one internal addend mask η , rather than aggregating the full hull sum. The selected transition is feasible only if the two conditions below both hold.

First, the two-variable modular addition correlation must be nonzero:

$$\text{Corr}^+(\gamma, \lambda, \eta) \neq 0.$$

Second, the source mask required by the oracle must lie in the packed oracle support:

$$\nu \oplus \mu \in \text{support_offset}_{s,(\lambda,\eta)} \oplus \text{im}(S_{s,(\lambda,\eta)}).$$

When both conditions hold, the local characteristic weight is

$$W_{\mathcal{I}_s}^{\text{lin}} = W_+^{\text{lin}}(\gamma, \lambda, \eta) + \frac{1}{2} \text{rank}(S_{s,(\lambda,\eta)}).$$

The sign contributed by the public constant is tracked separately as

$$(-1)^{\langle \eta, K_s \rangle}.$$

This is the linear counterpart of the differential coupling. The two-variable modular addition is still the ordinary two-variable addition model, but the addend mask η is not floating in isolation. It is simultaneously the Y -component of the packed oracle output mask

$$(\lambda, \eta),$$

while λ is simultaneously the target-input mask of the modular addition and the X -component of the packed oracle output mask. The source mask entering the oracle is forced by the preserved source branch:

$$\alpha_s = \nu \oplus \mu.$$

Thus the local linear transition has the structure

$$\begin{aligned} (\lambda, \nu) &\leftarrow \text{target/source input masks,} \\ \eta &\leftarrow \text{internal addend mask,} \\ (\lambda, \eta) &\leftarrow \text{packed oracle output mask,} \\ \nu \oplus \mu &\leftarrow \text{oracle input mask.} \end{aligned}$$

The model is therefore not merely a two-variable modular addition followed by a detached Boolean approximation. It is a coupled Walsh problem in which the same internal addend mask η simultaneously drives the addition correlation and the quadratic-oracle support condition.

For subtraction, the same interpretation is applied after rewriting the subtraction as an equivalent addition relation. The algebraic orientation of the addition box changes, but the packed-oracle coupling does not: the mask on the source-derived arithmetic operand must still be the Y -component of the same packed oracle mask that also contains the XOR-injection mask.

14.6 Oracle-to-MILP Bridge for the Joint Injection Layer

Definition 30 (Packed joint-injection oracle and forward transition). The MILP object called `joint_injection` is not an informal connection between a Boolean gadget and an addition box. It is the finite-field realization of one packed source oracle together with the witness variables needed to connect that oracle to the following modular addition.

All notation in this subsection is over

$$V = \mathbb{F}_2^{32}, \quad W = V \times V \simeq \mathbb{F}_2^{64}.$$

There are two code-level kinds:

$$\kappa = B \rightarrow A, \quad \kappa = A \rightarrow B.$$

For each kind, the implemented joint oracle is

$$\Phi_\kappa : V \rightarrow W, \quad \Phi_\kappa(x) = (X_\kappa(x), Y_\kappa(x)).$$

The first component is the word XORed into the opposite branch. The second component is the source-derived modular-addition operand before the public XOR constant. The packing convention used by the differential and linear oracles is

$$\text{pack}(u, v) = u + 2^{32}v, \quad u, v \in V,$$

so bits $0, \dots, 31$ are the XOR-routing component and bits $32, \dots, 63$ are the addend component.

The exact functions packed by `joint_injection` are the same functions used by the witness-bit builder and by the linear Walsh oracle. For $B \rightarrow A$,

$$\begin{aligned} X_{B \rightarrow A}(x) &= x \lll^{24} \oplus C_B(x) \lll^{16} \oplus x \lll^8, \\ Y_{B \rightarrow A}(x) &= (C_B(x) \ll 2) \oplus (D_B(x) \gg 2) \lll^{31} \\ &\quad \oplus (C_B(x) \gg 5) \oplus (D_B(x) \ll 5) \lll^{17}. \end{aligned}$$

For $A \rightarrow B$,

$$\begin{aligned} X_{A \rightarrow B}(x) &= x \ggg^{24} \oplus D_A(x) \ggg^{16} \oplus x \ggg^8, \\ Y_{A \rightarrow B}(x) &= (C_A(x) \gg 3) \oplus (D_A(x) \ll 3) \oplus (C_A(x) \ll 1) \oplus (D_A(x) \gg 1). \end{aligned}$$

The public constants

$$K_{B \rightarrow A} = \text{RC}[0], \quad K_{A \rightarrow B} = \text{RC}[5]$$

are not packed into Φ_κ . They appear only in the arithmetic operand

$$H_\kappa(x) = Y_\kappa(x) \oplus K_\kappa.$$

Consequently, they have no effect on XOR differences, and in a linear trail they contribute only the phase factor associated with the addend mask.

For a local target/source pair $(t, x) \in V \times V$, the forward source-preserving transition represented by a joint-injection site is

$$\mathcal{I}_\kappa(t, x) = ((t \oplus X_\kappa(x)) \boxplus (Y_\kappa(x) \oplus K_\kappa), x).$$

The MILP does not encode $\mathcal{I}_\kappa$ as an unstructured 64-bit black box. It factors it through the packed oracle:

$$x \xrightarrow{\Phi_\kappa} (X_\kappa(x), Y_\kappa(x)) \xrightarrow{t \mapsto t \oplus X_\kappa(x)} u \xrightarrow{u \boxplus (Y_\kappa(x) \oplus K_\kappa)} t'.$$

The mathematical point of the implementation is that the two words $X_\kappa(x)$ and $Y_\kappa(x)$ are never split into independent MILP objects. They are always the two components of one packed oracle value.

14.6.1 Boundary Arithmetic and Inverse Orientation

Remark 4 (Boundary arithmetic and inverse orientation). The fixed-public subtractions that prepare the source words are outside the joint oracle:

$$B_0 = B \boxminus \text{RC}[1], \quad A_0 = A \boxminus \text{RC}[6].$$

They are one-variable fixed-addend arithmetic layers, not components of Φ_κ . Their differential and linear costs are handled by the fixed-public add/sub models described earlier. The joint-injection MILP begins after those reductions, with source input B_0 for $B \rightarrow A$ and source input A_0 for $A \rightarrow B$.

The inverse implementation uses the same source-controlled object in reverse. For $B \rightarrow A$,

$$\mathcal{I}_{B \rightarrow A}^{-1}(A', B_0) = ((A' \boxminus (Y_{B \rightarrow A}(B_0) \oplus \text{RC}[0])) \oplus X_{B \rightarrow A}(B_0), B_0),$$

and for $A \rightarrow B$,

$$\mathcal{I}_{A \rightarrow B}^{-1}(A_0, B') = (A_0, (B' \boxminus (Y_{A \rightarrow B}(A_0) \oplus \text{RC}[5])) \oplus X_{A \rightarrow B}(A_0)).$$

Thus subtraction does not introduce a separate joint-injection object. It is the inverse orientation of the same packed source oracle followed by the corresponding modular arithmetic relation.

14.6.2 Differential Bridge

Theorem 28 (Differential oracle-to-MILP bridge for joint injection). *For each kind κ , the differential model contains the following variables:*

$\Delta_x \in V$	source/control difference, called <i>controlling_difference</i> ,
$\Delta_X \in V$	XOR-routing output difference, called <i>xor_delta</i> ,
$\Delta_Y \in V$	addend-output difference, called <i>add_operand_delta</i> ,
$\Delta_\Phi \in W$	packed output difference, called <i>joint_delta</i> ,
$w_\Phi \in [0, 32]$	rank weight, called <i>rank_weight</i> .

They are tied by the packing equation

$$\Delta_\Phi = \text{pack}(\Delta_X, \Delta_Y).$$

The implementation also introduces an explicit support witness

$$z \in V, \quad z' = z \oplus \Delta_x.$$

It evaluates the same straight-line Boolean oracle on both z and z' and binds the output differences by

$$\begin{aligned} \Delta_X &= X_\kappa(z) \oplus X_\kappa(z'), \\ \Delta_Y &= Y_\kappa(z) \oplus Y_\kappa(z'). \end{aligned}$$

Thus the static MILP witness equations impose the graph condition

$$(\Delta_x, \Delta_X, \Delta_Y) \in \mathcal{G}_\kappa^{\text{diff}},$$

where

$$\mathcal{G}_\kappa^{\text{diff}} = \left\{ (\Delta_x, \Delta_X, \Delta_Y) : \exists z \in V, \begin{array}{l} \Delta_X = X_\kappa(z) \oplus X_\kappa(z \oplus \Delta_x), \\ \Delta_Y = Y_\kappa(z) \oplus Y_\kappa(z \oplus \Delta_x) \end{array} \right\}.$$

This witness z is not a state variable of the cipher trail. It is an existential certificate that the packed output difference belongs to the derivative image of the quadratic oracle.

Because Φ_κ is quadratic, for each fixed Δ_x the derivative is affine:

$$D_{\Delta_x} \Phi_\kappa(z) = \Phi_\kappa(z \oplus \Delta_x) \oplus \Phi_\kappa(z) = M_{\kappa, \Delta_x} z \oplus c_{\kappa, \Delta_x}.$$

The reachable packed difference set is the affine subspace

$$\mathcal{A}_{\kappa, \Delta_x} = c_{\kappa, \Delta_x} \oplus \text{im}(M_{\kappa, \Delta_x}) \subseteq W.$$

If

$$r_{\kappa, \Delta_x} = \text{rank}(M_{\kappa, \Delta_x}),$$

then every reachable packed difference has probability

$$2^{-r_{\kappa, \Delta_x}},$$

and every packed difference outside $\mathcal{A}_{\kappa, \Delta_x}$ has probability 0.

The custom differential constraint handler enforces the same mathematical relation in rank form. For a fixed Δ_x , let

$$\mathcal{P}_{\kappa, \Delta_x} = \ker(M_{\kappa, \Delta_x}^T) \subseteq W^*$$

be the parity-check space of the image. The image condition

$$\Delta_\Phi \in c_{\kappa, \Delta_x} \oplus \text{im}(M_{\kappa, \Delta_x})$$

is equivalent to the family of parity equations

$$\langle h, \Delta_\Phi \rangle = \langle h, c_{\kappa, \Delta_x} \rangle \quad (h \in \mathcal{P}_{\kappa, \Delta_x}).$$

When the bits of Δ_x are locally fixed, the handler materializes these equations as local XOR constraints on the 64 bits of Δ_Φ . It also imposes the epigraph condition

$$w_\Phi \geq r_{\kappa, \Delta_x}.$$

For an integral assignment (Δ_x, Δ_Φ) that violates the affine image, the handler rejects that assignment, equivalently adding a local no-good cut for the invalid pair. Therefore the feasible integral differential joint-injection assignments are exactly

$$\{(\Delta_x, \Delta_X, \Delta_Y, w_\Phi) : \text{pack}(\Delta_X, \Delta_Y) \in \mathcal{A}_{\kappa, \Delta_x}, \quad w_\Phi \geq r_{\kappa, \Delta_x}\},$$

with the objective choosing $w_\Phi = r_{\kappa, \Delta_x}$ in a minimum weight trail.

The connection to the following modular addition is then purely algebraic. Let Δ_t be the target-branch difference before the XOR-routing step and let $\Delta_{t'}$ be the target-branch difference after the modular addition. The XOR-routing word gives the first addition input difference

$$\Delta_u = \Delta_t \oplus \Delta_X.$$

The public constant K_κ in $Y_\kappa(x) \oplus K_\kappa$ has zero XOR difference, so the second addition input difference is

$$\Delta_h = \Delta_Y.$$

The following two-variable modular-addition model is therefore applied to

$$\text{DP}^+(\Delta_u, \Delta_h \rightarrow \Delta_{t'}) = \text{DP}^+(\Delta_t \oplus \Delta_X, \Delta_Y \rightarrow \Delta_{t'}).$$

At the selected-characteristic level, the joint-injection arithmetic step is feasible exactly when

$$\text{pack}(\Delta_X, \Delta_Y) \in \mathcal{A}_{\kappa, \Delta_x}$$

and

$$\text{DP}^+(\Delta_t \oplus \Delta_X, \Delta_Y \rightarrow \Delta_{t'}) \neq 0.$$

Its local characteristic weight is

$$W_{\mathcal{I}_\kappa}^{\text{diff}} = r_{\kappa, \Delta_x} + W_+^{\text{diff}}(\Delta_t \oplus \Delta_X, \Delta_Y, \Delta_{t'}).$$

This is the weight represented in the model by the objective coefficient on `rank_weight` plus the objective variables created by `add_two_input_add_diff`.

For $B \rightarrow A$, the code instantiates this formula as

$$\begin{aligned} \Delta_x &= \Delta B_0, \\ \Delta_t &= \Delta A, \\ \Delta_u &= \Delta A \oplus \Delta X_{B \rightarrow A}, \\ \Delta_{t'} &= \Delta A^+, \end{aligned}$$

and the addition constraint is

$$\text{DP}^+(\Delta A \oplus \Delta X_{B \rightarrow A}, \Delta Y_{B \rightarrow A} \rightarrow \Delta A^+).$$

For $A \rightarrow B$, it instantiates the same equations as

$$\begin{aligned} \Delta_x &= \Delta A_0, \\ \Delta_t &= \Delta B, \\ \Delta_u &= \Delta B \oplus \Delta X_{A \rightarrow B}, \\ \Delta_{t'} &= \Delta B^+, \end{aligned}$$

with addition condition

$$\text{DP}^+(\Delta B \oplus \Delta X_{A \rightarrow B}, \Delta Y_{A \rightarrow B} \rightarrow \Delta B^+).$$

The exact factorized local differential transition probability, before selecting one characteristic witness, is

$$\begin{aligned} \text{DP}_{\mathcal{I}_\kappa}^{\text{fact}}((\Delta_t, \Delta_x) \rightarrow (\Delta_{t'}, \Delta_x)) = & \sum_{\substack{(\Delta_X, \Delta_Y): \\ \text{pack}(\Delta_X, \Delta_Y) \in \mathcal{A}_{\kappa, \Delta_x}}} 2^{-r_{\kappa, \Delta_x}} \\ & \cdot \text{DP}^+(\Delta_t \oplus \Delta_X, \Delta_Y \rightarrow \Delta_{t'}). \end{aligned}$$

The MILP search is not this full local hull sum. It chooses one packed oracle-output difference (Δ_X, Δ_Y) , proves its membership in the affine image, charges r_{κ, Δ_x} , and then charges the selected two-variable addition transition. This is the characteristic model implemented by the *joint_injection* variables.

14.6.3 Linear Bridge

Theorem 29 (Linear oracle-to-MILP bridge for joint injection). *The linear *joint_injection* object is the Walsh-dual analogue of the differential object, but the variables are different. For a local transition*

$$(t, x) \mapsto (t', x),$$

write the input masks as

$$(\lambda, \nu) \in V \times V$$

and the output masks as

$$(\gamma, \mu) \in V \times V.$$

Here λ is the target mask before the joint-injection transition, γ is the target mask after the modular addition, ν is the source mask before the source-preserving transition, and μ is the source mask after it.

The XOR-routing substep

$$u = t \oplus X_\kappa(x)$$

forces the target mask on t to equal the mask on u . Denote this common mask by

$$\lambda.$$

It is also the mask applied to the X -component of the packed oracle. The two-variable addition substep introduces an internal addend mask

$$\eta \in V.$$

This η is simultaneously the mask on the second input of the modular addition and the mask applied to the Y -component of the packed oracle. Thus the packed oracle output mask used by the implementation is

$$\beta = \text{pack}(\lambda, \eta) = (\lambda, \eta) \in W.$$

This is exactly the bit order constructed by *pack_joint_beta_bits*: the lower 32 bits are *beta_xor_mask*, and the upper 32 bits are *beta_addend_mask*.

The source branch is preserved as a state word, but it participates in the phase through the oracle. The phase terms depending only on x are

$$\langle \nu, x \rangle \oplus \langle \mu, x \rangle \oplus \langle \lambda, X_\kappa(x) \rangle \oplus \langle \eta, Y_\kappa(x) \rangle.$$

Therefore the input mask supplied to the quadratic Walsh oracle is

$$\alpha = \nu \oplus \mu.$$

This is the equality encoded in the model by

$$\nu = \mu \oplus \alpha.$$

For $B \rightarrow A$, the code implements it as

$$\text{mask_B_after_sub_RC1} = \text{mask_B_after_B_to_A_injection} \oplus \text{alpha_B_to_A},$$

while the target-mask equality is

$$\text{mask_A_at_round_start} = \text{mask_A_after_B_to_A_injection}.$$

For $A \rightarrow B$, the corresponding source equation is

$$\text{mask_A_after_sub_RC6} = \text{mask_A_after_A_to_B_injection} \oplus \text{alpha_A_to_B},$$

and the target-mask equality is

$$\text{mask_B_after_xor_A_rotB13} = \text{mask_B_after_A_to_B_injection}.$$

For a packed mask $\beta = (\lambda, \eta)$, define the scalar quadratic function

$$g_{\kappa, \beta}(x) = \langle \beta, \Phi_{\kappa}(x) \rangle = \langle \lambda, X_{\kappa}(x) \rangle \oplus \langle \eta, Y_{\kappa}(x) \rangle.$$

Let

$$S_{\kappa, \beta}$$

be the alternating matrix of the polar form of $g_{\kappa, \beta}$, and let

$$\mathcal{R}_{\kappa, \beta} = \ker(S_{\kappa, \beta})$$

be its radical. The normalized oracle correlation is

$$\text{Corr}_{\Phi_{\kappa}}(\alpha, \beta) = 2^{-32} \sum_{x \in V} (-1)^{\langle \alpha, x \rangle \oplus \langle \beta, \Phi_{\kappa}(x) \rangle}.$$

It is nonzero if and only if

$$g_{\kappa, \beta}(z) \oplus g_{\kappa, \beta}(0) = \langle \alpha, z \rangle \quad \text{for every } z \in \mathcal{R}_{\kappa, \beta}.$$

Equivalently, the admissible α -masks form one affine coset

$$\mathcal{S}_{\kappa, \beta} = \alpha_{\kappa, \beta}^0 \oplus \text{im}(S_{\kappa, \beta}) \subseteq V.$$

When $\alpha \in \mathcal{S}_{\kappa, \beta}$, the magnitude is

$$|\text{Corr}_{\Phi_{\kappa}}(\alpha, \beta)| = 2^{-\text{rank}(S_{\kappa, \beta})/2}.$$

When $\alpha \notin \mathcal{S}_{\kappa, \beta}$, the correlation is zero.

The linear custom constraint handler therefore represents the mathematical set

$$\mathcal{G}_{\kappa}^{\text{lin}} = \left\{ (\alpha, \beta, w_{\Phi}) : \alpha \in \mathcal{S}_{\kappa, \beta}, \quad w_{\Phi} \geq \frac{1}{2} \text{rank}(S_{\kappa, \beta}) \right\}.$$

The stored **support_offset** is $\alpha_{\kappa, \beta}^0$. The stored **support_paritys** are the radical equations

$$\langle z, \alpha \rangle = g_{\kappa, \beta}(z) \oplus g_{\kappa, \beta}(0), \quad z \in \mathcal{R}_{\kappa, \beta}.$$

For a locally fixed β , the handler materializes these equations as local XOR constraints on the 32 α -bits, and it tightens

$$w_\Phi \geq \frac{1}{2} \text{rank}(S_{\kappa,\beta}).$$

If the support descriptor is impossible, the handler excludes that β . If an integral pair (α, β) violates the support equations, the handler excludes that pair. Thus the feasible integral assignments of the linear **joint_injection** constraint are precisely the nonzero Walsh support pairs with the correct rank weight.

The two-variable modular-addition correlation used by the following arithmetic box is

$$\text{Corr}_+(\gamma, \lambda, \eta) = 2^{-64} \sum_{u,h \in V} (-1)^{\langle \lambda, u \rangle \oplus \langle \eta, h \rangle \oplus \langle \gamma, u \oplus h \rangle}.$$

The public XOR constant in $h = Y_\kappa(x) \oplus K_\kappa$ changes the phase by

$$(-1)^{\langle \eta, K_\kappa \rangle}$$

and does not change the support or the absolute weight of the addition box.

Substituting the addition Walsh expansion into the full local map gives the exact factorized linear hull

$$\begin{aligned} \text{Corr}_{\mathcal{I}_\kappa}^{\text{fact}}((\lambda, \nu) \rightarrow (\gamma, \mu)) \\ = \sum_{\eta \in V} (-1)^{\langle \eta, K_\kappa \rangle} \text{Corr}_+(\gamma, \lambda, \eta) \text{Corr}_{\Phi_\kappa}(\nu \oplus \mu, \text{pack}(\lambda, \eta)). \end{aligned}$$

This formula is the linear bridge. The same η appears in the addition correlation and in the Y -component of the packed oracle mask. The same λ appears in the first input of the addition and in the X -component of the packed oracle mask. The same $\nu \oplus \mu$ is the input mask of the quadratic oracle.

The implemented linear MILP is the selected-characteristic version of this sum. It creates one η , forms

$$\beta = \text{pack}(\lambda, \eta),$$

enforces

$$\nu \oplus \mu \in \mathcal{S}_{\kappa,\beta},$$

charges

$$\frac{1}{2} \text{rank}(S_{\kappa,\beta}),$$

and applies the ordinary two-variable modular-addition linear characteristic constraint

$$\text{Corr}_+(\gamma, \lambda, \eta) \neq 0$$

with addition weight

$$W_+^{\text{lin}}(\gamma, \lambda, \eta) = -\log_2 |\text{Corr}_+(\gamma, \lambda, \eta)|.$$

The selected local linear weight is therefore

$$W_{\mathcal{I}_\kappa}^{\text{lin}} = W_+^{\text{lin}}(\gamma, \lambda, \eta) + \frac{1}{2} \text{rank}(S_{\kappa,\text{pack}(\lambda,\eta)}).$$

The sign

$$(-1)^{\langle \eta, K_\kappa \rangle}$$

belongs to the signed correlation audit; it is not part of the absolute-weight objective.

14.6.4 Joint-Injection MILP Identity

Corollary 7 (Joint-injection MILP identity). *The resulting MILP identity is the following. A differential `joint_injection` instance is the graph-plus-rank object*

$$(\Delta_x, \Delta_X, \Delta_Y, w_\Phi) \in \mathcal{G}_\kappa^{\text{diff}} \quad \text{and} \quad w_\Phi \geq r_{\kappa, \Delta_x},$$

followed by the modular-addition relation

$$\Delta_t \oplus \Delta_X, \Delta_Y \mapsto \Delta_{t'}.$$

A linear `joint_injection` instance is the Walsh-support object

$$(\alpha, \text{pack}(\lambda, \eta), w_\Phi) \in \mathcal{G}_\kappa^{\text{lin}}, \quad \alpha = \nu \oplus \mu,$$

followed by the modular-addition correlation

$$\text{Corr}_+(\gamma, \lambda, \eta).$$

Thus the shared variables are not auxiliary notation. They are the mathematical content of the implementation:

<i>model</i>	<i>shared packed-oracle variable</i>	<i>arithmetic use</i>
<i>differential</i>	(Δ_X, Δ_Y)	$(\Delta_t \oplus \Delta_X, \Delta_Y \rightarrow \Delta_{t'})$
<i>linear</i>	(λ, η)	$\text{Corr}_+(\gamma, \lambda, \eta).$

This is why the implementation is a joint injection model: the modular addition consumes a word whose difference or mask is constrained as one component of the same packed quadratic oracle that supplies the XOR-routing word.

14.6.5 Differential MILP Pseudocode

The differential implementation follows the mathematical graph condition above with two simultaneous mechanisms. The explicit witness circuit proves existence of a source value realizing the selected packed output difference. The rank oracle supplies the affine-image parity checks and the exact rank weight.

Algorithm 36 Differential joint-injection MILP instance

```

1: function CREATEJOINTINJECTIONDIFFERENCES( $\kappa, \Delta_x, \text{PREFIX}$ )
2:   create  $\Delta_X \in V$  ▷ prefix_xor_delta
3:   create  $\Delta_Y \in V$  ▷ prefix_add_operand_delta
4:   create  $w_\Phi \in [0, 32]$  ▷ prefix_rank_weight
5:   create witness  $z \in V$  ▷ prefix_support_witness_x
6:   set  $z' = z \oplus \Delta_x$  ▷ prefix_support_witness_x_prime
7:   compute

```

$$(X, Y) = \Phi_\kappa(z)$$

by the corresponding straight-line Boolean injection circuit

```

8:   compute

```

$$(X', Y') = \Phi_\kappa(z')$$

by a second copy of the same circuit

```

9:   add equality constraints

```

$$\Delta_X = X \oplus X', \quad \Delta_Y = Y \oplus Y'$$

```

10:   pack
      
$$\Delta_\Phi = \text{pack}(\Delta_X, \Delta_Y)$$

      
$$\triangleright \text{prefix\_joint\_delta}$$

11:   add custom rank constraint
      
$$\Delta_\Phi \in c_{\kappa, \Delta_x} \oplus \text{im}(M_{\kappa, \Delta_x}), \quad w_\Phi \geq \text{rank}(M_{\kappa, \Delta_x})$$

      
$$\triangleright \text{add\_injection\_rank\_constraint}$$

12:   return  $(\Delta_X, \Delta_Y, w_\Phi)$ 
13: end function

```

The returned pair is then consumed by the arithmetic half of the same joint-injection site. For a target input difference Δ_t and target output difference $\Delta_{t'}$, the code creates

$$\Delta_u = \Delta_t \oplus \Delta_X$$

and calls the two-variable modular-addition differential MILP on

$$(\Delta_u, \Delta_Y, \Delta_{t'}).$$

Algorithm 37 Differential use of the joint-injection pair in the round MILP

```

1: function ADDDIFFERENTIALJOINTINJECTIONSITE( $\kappa, \Delta_t, \Delta_x, \Delta_{t'}, \text{PREFIX}$ )
2:    $(\Delta_X, \Delta_Y, w_\Phi) \leftarrow \text{CREATEJOINTINJECTIONDIFFERENCES}(\kappa, \Delta_x, \text{PREFIX})$ 
3:   create XOR-routed target difference

```

$$\Delta_u = \Delta_t \oplus \Delta_X$$

```

4:   call the two-variable modular-addition differential model

```

$$\text{ADDTWOINPUTADDDIFF}(\Delta_u, \Delta_Y, \Delta_{t'})$$

```

5:   charge objective

```

$$w_\Phi + W_+^{\text{diff}}(\Delta_u, \Delta_Y, \Delta_{t'})$$

```

6: end function

```

For the first injection site,

$$\kappa = B \rightarrow A, \quad \Delta_x = \Delta B_0, \quad \Delta_t = \Delta A.$$

For the second injection site,

$$\kappa = A \rightarrow B, \quad \Delta_x = \Delta A_0, \quad \Delta_t = \Delta B.$$

14.6.6 Linear MILP Pseudocode

The linear implementation does not build a witness copy of the Boolean network. It creates the packed oracle mask $\beta = (\lambda, \eta)$, creates the oracle input mask α , and delegates the support and rank test to the custom Walsh constraint handler.

Algorithm 38 Linear joint-injection Walsh MILP instance

```

1: function CREATEJOINTINJECTIONALPHAWITHWALSHCONSTRAINT( $\kappa, \lambda, \eta, \text{PREFIX}$ )
2:   create  $\alpha \in V$ 
      
$$\triangleright \text{prefix\_alpha}$$


```

```

3:   pack
                                      $\beta = \text{pack}(\lambda, \eta)$ 
                                      $\triangleright \text{pack\_joint\_beta\_bits}$ 
4:   create  $w_\Phi \in [0, 32]$ 
                                      $\triangleright \text{prefix\_linear\_weight}$ 
5:   add custom Walsh constraint
                                      $\alpha \in \mathcal{S}_{\kappa, \beta} = \alpha_{\kappa, \beta}^0 \oplus \text{im}(S_{\kappa, \beta}), \quad w_\Phi \geq \frac{1}{2} \text{rank}(S_{\kappa, \beta})$ 
                                      $\triangleright \text{add\_linear\_injection\_walsh\_constraint}$ 
6:   return  $(\alpha, w_\Phi)$ 
7: end function

```

The returned α is not a free external mask. It is tied to the source mask before and after the source-preserving local transition:

$$\alpha = \nu \oplus \mu.$$

The addend mask η is then reused as the second-input mask of the two-variable modular-addition linear model.

Algorithm 39 Linear use of the joint-injection mask pair in the round MILP

```

1: function ADDLINEARJOINTINJECTIONSITE( $\kappa, \lambda, \nu, \gamma, \mu, \text{PREFIX}$ )
2:   create internal addend mask  $\eta \in V$ 
3:    $(\alpha, w_\Phi) \leftarrow \text{CREATEJOINTINJECTIONALPHAWITHWALSHCONSTRAINT}(\kappa, \lambda, \eta, \text{PREFIX})$ 
4:   add source-mask relation
                                      $\nu = \mu \oplus \alpha$ 
5:   call the two-variable modular-addition linear characteristic model
                                      $\text{ADDTWOINPUTMODULARADDITIONLINEAR}(\gamma, \lambda, \eta)$ 
6:   record public-constant phase
                                      $(-1)^{\langle \eta, K_\kappa \rangle}$ 
7:   charge objective
                                      $w_\Phi + W_+^{\text{lin}}(\gamma, \lambda, \eta)$ 
8: end function

```

For $B \rightarrow A$, the implementation passes

$$\lambda = \text{mask_A_after_B_to_A_injection}, \quad \eta = \text{beta_B_to_A_addend},$$

and enforces

$$\text{mask_B_after_sub_RC1} = \text{mask_B_after_B_to_A_injection} \oplus \alpha.$$

For $A \rightarrow B$, it passes

$$\lambda = \text{mask_B_after_A_to_B_injection}, \quad \eta = \text{beta_A_to_B_addend},$$

and enforces

$$\text{mask_A_after_sub_RC6} = \text{mask_A_after_A_to_B_injection} \oplus \alpha.$$

15 Top-Level MILP Question Model Assembly

15.1 Differential

The previous sections define the local differential objects used by the implementation: the two-variable modular-addition differential box, the fixed-public-addend and fixed-public-subtractend differential boxes, and the joint-injection graph-plus-rank operator. This subsection gives the top-level assembly procedure used by the NeoAlzette XOR-differential MILP backend.

This pseudocode is not a standalone derivation of the model. It assumes the reader has already reconstructed the oracle-to-MILP components defined earlier. In particular, the routines below are not informal placeholders. Each call refers to the exact differential MILP component defined in the corresponding previous section. Replacing one of these calls by an averaged, table-based, or uncoupled surrogate produces a different MILP model from the one analyzed in this paper.

The model built here is an XOR-differential single-characteristic search model. It is not a linear model, not a division-property model, not a SAT/SMT bit-vector input file, and not a full differential-hull enumeration unless an additional bounded enumeration layer is explicitly enabled. Public XOR constants and rotations are zero-weight linear transport steps in the XOR-difference domain, while modular additions, fixed-public subtractions, and joint-injection derivative ranks contribute to the objective.

Let the input difference of round r be $(\Delta A_r, \Delta B_r) \in V \times V$, where $V = \mathbb{F}_2^{32}$. The round builder creates checkpoint variables after every nonlinear or structural step. The objective is the sum of all local negative-log differential weights created by the called components:

$$W_{\text{diff}} = \sum W_{\text{fixed}}^{\text{diff}} + \sum W_{+}^{\text{diff}} + \sum w_{\Phi}.$$

Here $W_{\text{fixed}}^{\text{diff}}$ is supplied by the exact fixed-public addend/subtractend differential box, W_{+}^{diff} is supplied by the two-variable modular-addition differential box, and w_{Φ} is the affine-derivative rank cost of the corresponding joint-injection site.

Algorithm 40 Build the NeoAlzette XOR-differential MILP instance

- 1: **function** BUILDDIFFERENTIALNEOALZETTEMILP($R, \text{options}$)
- 2: create MILP/CIP model $\mathcal{M}$
- 3: create input difference words

$$\Delta A_0, \Delta B_0 \in V$$

- 4: set

$$(\Delta A_{\text{cur}}, \Delta B_{\text{cur}}) \leftarrow (\Delta A_0, \Delta B_0)$$

- 5: **for** $r = 0$ to $R - 1$ **do**
- 6:

$$(\Delta A_{\text{cur}}, \Delta B_{\text{cur}}) \leftarrow \text{BUILDDIFFERENTIALNEOALZETTEROUND}(r, \Delta A_{\text{cur}}, \Delta B_{\text{cur}})$$

- 7: **end for**
- 8: define output words

$$\Delta A_R \leftarrow \Delta A_{\text{cur}}, \quad \Delta B_R \leftarrow \Delta B_{\text{cur}}.$$

- 9: add nonzero external input constraint

$$\sum_{i=0}^{31} \Delta A_0[i] + \sum_{i=0}^{31} \Delta B_0[i] \geq 1.$$

```

10:   if options.fix_input_da is set then
11:       constrain  $\Delta A_0$  to the requested 32-bit value
12:   end if
13:   if options.fix_input_db is set then
14:       constrain  $\Delta B_0$  to the requested 32-bit value
15:   end if
16:   if options.fix_output_da is set then
17:       constrain  $\Delta A_R$  to the requested 32-bit value
18:   end if
19:   if options.fix_output_db is set then
20:       constrain  $\Delta B_R$  to the requested 32-bit value
21:   end if
22:   apply all stored no-good cuts if bounded characteristic enumeration is enabled
23:   if an objective-weight window is enabled then
24:       add

$$W_{\min} \leq W_{\text{diff}} \leq W_{\max}.$$

25:   end if
26:   set the model objective to minimize  $W_{\text{diff}}$ 
27:   return  $\mathcal{M}$ 
28: end function

```

The round-level construction follows the implemented forward order of the Core. The two public subtractions are modeled as fixed-public subtractend differential operators, not as XOR constants and not as zero-weight whitening. The command-line comparison mode may replace them by a zero-difference averaged model, but the V6.5 exact model uses the fixed-public-exact branch.

Algorithm 41 Build one NeoAlzette XOR-differential round

```

1: function BUILDDIFFERENTIALNEOALZETTEROUND( $r, \Delta A, \Delta B$ )
 $\triangleright$  First fixed-public subtraction on the  $B$  branch
2:   create  $\Delta B_1 \in V$ 
3:    $\text{ADDFIXEDPUBLICCONSTANTSUBDIFF}(RC[1], \Delta B, \Delta B_1).$ 
 $\triangleright$  First joint injection:  $B$  controls an injection into  $A$ 
4:    $(\Delta X_0, \Delta Y_0, w_{\Phi,0}) \leftarrow \text{CREATEJOINTINJECTIONDIFFERENCES}(B \rightarrow A, \Delta B_1, \text{prefix}_{r,0}).$ 
5:   create XOR-injected  $A$  difference

$$\Delta A_1 = \Delta A \oplus \Delta X_0.$$

 $\triangleright$  First modular addition using the injection-derived operand
6:   create  $\Delta A_2 \in V$ 
7:    $\text{ADDTWOINPUTADDDIFF}(\Delta A_1, \Delta Y_0, \Delta A_2).$ 
 $\triangleright$  First cross-branch bridge: rotation pair (22, 13)
8:   create

$$\Delta B_2 = \Delta B_1 \oplus \text{rotl}(\Delta A_2, 22).$$

9:   record public XOR by  $RC[4]$  as a zero-weight differential no-op
10:  create

$$\Delta A_3 = \Delta A_2 \oplus \text{rotl}(\Delta B_2, 13).$$


```

▷ Second fixed-public subtraction on the A branch

11: create $\Delta A_4 \in V$

12: $\text{ADDFIXEDPUBLICCONSTANTSUBDIFF}(RC[6], \Delta A_3, \Delta A_4).$

▷ Second joint injection: A controls an injection into B

13: $(\Delta X_1, \Delta Y_1, w_{\Phi,1}) \leftarrow \text{CREATEJOINTINJECTIONDIFFERENCES}(A \rightarrow B, \Delta A_4, \text{prefix}_{r,1}).$

14: create XOR-injected B difference

$\Delta B_3 = \Delta B_2 \oplus \Delta X_1.$

▷ Second modular addition using the injection-derived operand

15: create $\Delta B_4 \in V$

16: $\text{ADDTWOINPUTADDDIFF}(\Delta B_3, \Delta Y_1, \Delta B_4).$

▷ Second cross-branch bridge: rotation pair (5, 25)

17: create

$\Delta A_5 = \Delta A_4 \oplus \text{rotl}(\Delta B_4, 5).$

18: record public XOR by $RC[9]$ as a zero-weight differential no-op

19: create

$\Delta B_5 = \Delta B_4 \oplus \text{rotl}(\Delta A_5, 25).$

▷ Final public whitening

20: record public XOR by $RC[10]$ on A as a zero-weight differential no-op

21: record public XOR by $RC[11]$ on B as a zero-weight differential no-op

22: **return** $(\Delta A_5, \Delta B_5)$

23: **end function**

The joint-injection calls above are deliberately shared with the arithmetic calls that follow them. For the first site, the same packed derivative output $(\Delta X_0, \Delta Y_0)$ supplies both the XOR-routed branch difference ΔX_0 and the modular-addition operand difference ΔY_0 . For the second site, $(\Delta X_1, \Delta Y_1)$ plays the same role with the branches exchanged. Thus the following two relations are not independent:

$$\Delta A_1 = \Delta A \oplus \Delta X_0, \quad (\Delta A_1, \Delta Y_0) \rightarrow \Delta A_2,$$

and

$$\Delta B_3 = \Delta B_2 \oplus \Delta X_1, \quad (\Delta B_3, \Delta Y_1) \rightarrow \Delta B_4.$$

The variable ΔY_j consumed by the modular addition is constrained as one component of the same quadratic-derivative output that supplies ΔX_j . Splitting these two words into unrelated MILP variables would destroy the joint-injection model.

Algorithm 42 Differential joint-injection subroutine used by the round assembly

1: **function** $\text{CREATEJOINTINJECTIONDIFFERENCES}(\kappa, \Delta x, \text{prefix})$

2: create output-difference words

$\Delta X, \Delta Y \in V$

3: create rank-weight variable

$w_{\Phi} \in [0, 32].$

4: create witness word $z \in V$

5: set

$$z' = z \oplus \Delta x.$$

6: bit-blast the corresponding injection circuit twice:

$$(X, Y) = \Phi_\kappa(z), \quad (X', Y') = \Phi_\kappa(z').$$

7: impose witness graph equalities

$$\Delta X = X \oplus X', \quad \Delta Y = Y \oplus Y'.$$

8: pack the joint output difference

$$\Delta_\Phi = \text{pack}(\Delta X, \Delta Y).$$

9: impose the affine-image support and rank constraint

$$\Delta_\Phi \in c_{\kappa, \Delta x} \oplus \text{im}(M_{\kappa, \Delta x}), \quad w_\Phi \geq \text{rank}(M_{\kappa, \Delta x}).$$

10: add w_Φ to the objective

11: **return** $(\Delta X, \Delta Y, w_\Phi)$

12: **end function**

The differential objective assembled by one round can be read schematically as

$$\begin{aligned} W_r = & W_{\text{sub}, RC[1]}^{\text{diff}}(\Delta B \rightarrow \Delta B_1) + w_{\Phi, 0} + W_+^{\text{diff}}(\Delta A_1, \Delta Y_0 \rightarrow \Delta A_2) \\ & + W_{\text{sub}, RC[6]}^{\text{diff}}(\Delta A_3 \rightarrow \Delta A_4) + w_{\Phi, 1} + W_+^{\text{diff}}(\Delta B_3, \Delta Y_1 \rightarrow \Delta B_4). \end{aligned}$$

All remaining bridge, rotation, and public-XOR steps are zero-weight linear constraints in the XOR-difference model. For R rounds, the global objective is

$$W_{\text{diff}} = \sum_{r=0}^{R-1} W_r.$$

Finally, the implementation records checkpoints and local weight metadata after each stage. These trace objects are not part of the cryptographic primitive; they are an engineering audit layer used to verify that the exported solution, the local weight decomposition, and the SCIP objective refer to the same characteristic.

15.2 Linear

The previous sections define the local linear objects used by the implementation: the two-variable modular-addition linear characteristic box, the fixed-public-addend and fixed-public-subtractend signed-numerator oracle, and the joint-injection Walsh support-rank operator. This subsection gives the top-level assembly procedure used by the NeoAlzette linear MILP backend.

This pseudocode is not a standalone introduction to linear cryptanalysis of ARX operations. It assumes the reader has already reconstructed the two-variable addition linear oracle, the fixed-addend signed transfer oracle, the exact-log fixed-constant MILP objective, and the joint-injection Walsh support-rank constraint. The routines called below are not informal placeholders. Each call refers to the corresponding oracle-to-MILP construction defined earlier in the paper. Replacing one of these routines by a local shortcut, an averaged constant model, a detached Boolean approximation, or a table-only surrogate produces a different linear MILP model from the one analyzed here.

The model built here is a single-characteristic Walsh-correlation search model. It is not a differential model, not a division-property model, not an endpoint-hull enumeration,

and not a full linear-hull computation unless an additional enumeration layer is explicitly enabled. The endpoint-hull workflow, if used, belongs to a separate driver. The present backend constructs one linear trail, minimizes its additive negative-log correlation weight, and then uses the trace layer to validate the local oracle interpretation.

For a forward operation $Y = F(X)$, a linear trail relates an output mask to the corresponding input mask by the transpose action of F . Therefore the round builder is written in implementation order, but each linear transport constraint must be read as a transpose-mask relation. Public XOR constants do not change the support of a linear mask; they only contribute a possible sign phase. Rotations propagate masks by the inverse rotation:

$$\text{rotr}(\cdot, r)^T = \text{rotr}(\cdot, r).$$

Let the boundary masks of round r be

$$(mA_r, mB_r) \in V \times V, \quad V = \mathbb{F}_2^{32}.$$

The linear objective is the sum of all local negative-log absolute-correlation weights:

$$W_{\text{lin}} = \sum W_{\text{fixed}}^{\text{lin}} + \sum W_+^{\text{lin}} + \sum w_{\Phi}.$$

Here $W_{\text{fixed}}^{\text{lin}}$ is supplied by the exact fixed-public addend/subtractend signed-numerator model, W_+^{lin} is supplied by the two-variable modular-addition linear box, and w_{Φ} is supplied by the joint Walsh support-rank operator.

Algorithm 43 Build the NeoAlzette linear MILP instance

```

1: function BUILDLINEARNEOALZETTEMILP( $R, \text{options}$ )
2:   create MILP/CIP model  $\mathcal{M}$ 
3:   create boundary mask words
      
$$mA_0, mB_0 \in V$$

4:   set
      
$$(mA_{\text{cur}}, mB_{\text{cur}}) \leftarrow (mA_0, mB_0).$$

5:   for  $r = 0$  to  $R - 1$  do
6:      $(mA_{\text{cur}}, mB_{\text{cur}}) \leftarrow \text{BUILDLINEARNEOALZETTEROUND}(r, mA_{\text{cur}}, mB_{\text{cur}}).$ 
7:   end for
8:   define final boundary masks
      
$$mA_R \leftarrow mA_{\text{cur}}, \quad mB_R \leftarrow mB_{\text{cur}}.$$

9:   if  $\text{options.require\_nonzero\_input\_mask}$  and the input mask is not fully fixed then
10:    add
      
$$\sum_{i=0}^{31} mA_0[i] + \sum_{i=0}^{31} mB_0[i] \geq 1.$$

11:  end if
12:  if  $\text{options.require\_nonzero\_output\_mask}$  and the output mask is not fully fixed then
13:    add
      
$$\sum_{i=0}^{31} mA_R[i] + \sum_{i=0}^{31} mB_R[i] \geq 1.$$

14:  end if

```

```

15:   if fixed input masks are supplied then
16:       constrain  $mA_0$  and/or  $mB_0$  to the requested values
17:   end if
18:   if fixed output masks are supplied then
19:       constrain  $mA_R$  and/or  $mB_R$  to the requested values
20:   end if
21:   if an objective-weight window is enabled then
22:       add
           
$$W_{\min} \leq W_{\text{lin}} \leq W_{\max}.$$

23:   end if
24:   apply stored no-good cuts if characteristic enumeration is enabled
25:   set the model objective to minimize  $W_{\text{lin}}$ 
26:   return  $\mathcal{M}$ 
27: end function

```

The round-level builder follows the implemented NeoAlzette forward schedule, but the constraints are transpose-mask constraints. Thus every XOR-rotation bridge is modeled by moving the mask through the transpose of the corresponding linear map. Every fixed-public subtraction is modeled as addition by the two's-complement public constant and charged by the exact fixed-addend signed-numerator weight.

Algorithm 44 Build one NeoAlzette linear round

```

1: function BUILDLINEARNEOALZETTEROUND( $r, mA, mB$ )
    $\triangleright$  Fixed-public subtraction on the  $B$  branch
2:   create  $mB_1 \in V$ 
3:   ADDFIXEDPUBLICCONSTANTSUBLINEAREXACTLOG( $RC[1], mB, mB_1$ ).
4:   keep the other branch mask unchanged:
       
$$mA_1 = mA.$$

        $\triangleright$  First joint injection: source  $B$  controls the  $B \rightarrow A$  site
5:   create target XOR-output mask  $\lambda_0 \in V$ 
6:   create internal addend mask  $\eta_0 \in V$ 
7:   set
       
$$\lambda_0 = mA_1.$$

8:    $(\alpha_0, w_{\Phi,0}) \leftarrow \text{CREATEJOINTINJECTIONALPHAWITHWALSHCONSTRAINT}$ 
        $(B \rightarrow A, \lambda_0, \eta_0, \text{prefix}_{r,0}).$ 
9:   impose the source-mask transpose relation
       
$$mB_1 = mB_2 \oplus \alpha_0.$$

10:  set
       
$$mA_2 = mA_1.$$

        $\triangleright$  First modular addition using the same internal addend mask
11:  create  $mA_3, mB_3 \in V$ 
12:  ADDTWOINPUTMODULARADDITIONLINEAR( $mA_3, mA_2, \eta_0$ ).
13:  preserve the source branch:
       
$$mB_3 = mB_2.$$


```

-
- 14: record the public addend phase from $RC[0]$ and η_0
 $\triangleright$ First bridge: $B \leftarrow B \oplus \text{rotr}(A, 22) \oplus RC[4]$
- 15: create $mA_4, mB_4 \in V$
- 16: impose the transpose bridge equations

$$mA_3 = mA_4 \oplus \text{rotr}(mB_4, 22), \quad mB_3 = mB_4.$$
- 17: record the public XOR phase from $RC[4]$
 $\triangleright$ Second bridge half: $A \leftarrow A \oplus \text{rotr}(B, 13)$
- 18: create $mA_5, mB_5 \in V$
- 19: impose

$$mA_4 = mA_5, \quad mB_4 = mB_5 \oplus \text{rotr}(mA_5, 13).$$

 $\triangleright$ Fixed-public subtraction on the A branch
- 20: create $mA_6 \in V$
- 21: $\text{ADDFIXEDPUBLICCONSTANTSUBLINEAREXACTLOG}(RC[6], mA_5, mA_6).$
- 22: keep

$$mB_6 = mB_5.$$

 $\triangleright$ Second joint injection: source A controls the $A \rightarrow B$ site
- 23: create target XOR-output mask $\lambda_1 \in V$
- 24: create internal addend mask $\eta_1 \in V$
- 25: set

$$\lambda_1 = mB_6.$$
- 26: $(\alpha_1, w_{\Phi,1}) \leftarrow \text{CREATEJOINTINJECTIONALPHAWITHWALSHCONSTRAINT}$
 $(A \rightarrow B, \lambda_1, \eta_1, \text{prefix}_{r,1}).$
- 27: impose the source-mask transpose relation

$$mA_6 = mA_7 \oplus \alpha_1.$$
- 28: set

$$mB_7 = mB_6.$$

 $\triangleright$ Second modular addition using the same internal addend mask
- 29: create $mA_8, mB_8 \in V$
- 30: $\text{ADDTWOINPUTMODULARADDITIONLINEAR}(mB_8, mB_7, \eta_1).$
- 31: preserve the source branch:

$$mA_8 = mA_7.$$
- 32: record the public addend phase from $RC[5]$ and η_1
 $\triangleright$ Second bridge: $A \leftarrow A \oplus \text{rotr}(B, 5) \oplus RC[9]$
- 33: create $mA_9, mB_9 \in V$
- 34: impose

$$mA_8 = mA_9, \quad mB_8 = mB_9 \oplus \text{rotr}(mA_9, 5).$$
- 35: record the public XOR phase from $RC[9]$
 $\triangleright$ Final bridge half: $B \leftarrow B \oplus \text{rotr}(A, 25)$
- 36: create $mA_{10}, mB_{10} \in V$
- 37: impose

$$mA_9 = mA_{10} \oplus \text{rotr}(mB_{10}, 25), \quad mB_9 = mB_{10}.$$

 $\triangleright$ Final public whitening

```

38:    record the public XOR phase from  $RC[10]$  and  $RC[11]$ 
39:    return  $(mA_{10}, mB_{10})$ 
40: end function

```

The fixed-public subtraction subroutine is strict. A subtraction $X \mapsto X - C$ is treated as fixed-addend addition by $K = -C \bmod 2^{32}$ at the correlation level. For input and output masks (a, b) , the oracle computes the signed numerator

$$N_K(a, b) = 2^{32} C_K(a, b),$$

where

$$C_K(a, b) = 2^{-32} \sum_{X \in V} (-1)^{a \cdot X \oplus b \cdot (X \boxplus K)}.$$

The model binds $A_K(a, b) = |N_K(a, b)|$ and charges

$$W_K(a, b) = \begin{cases} 32 - \log_2 A_K(a, b), & A_K(a, b) \neq 0, \\ +\infty, & A_K(a, b) = 0. \end{cases}$$

This is the exact-log fixed-addend model. There is no outer numerator sweep, no post-solve correction, and no logarithmic ladder approximation in the V6.5 strict linear backend.

Algorithm 45 Fixed-public subtraction exact-log linear subroutine

```

1: function ADDFIXEDPUBLICCONSTANTSUBLINEAREXACTLOG( $C, a, b$ )
2:    set

```

$$K = -C \bmod 2^{32}.$$

```

3:    build the Miyano-style two-state signed carry-transfer recurrence for

```

$$X \mapsto X \boxplus K.$$

```

4:    construct the signed numerator variable

```

$$N = 2^{32} C_K(a, b).$$

```

5:    bind the absolute numerator

```

$$A = |N|.$$

```

6:    add the exact-log epigraph objective

```

$$W \geq 32 - \log_2 A.$$

```

7:    add  $W$  to the global linear objective
8:    return  $W$ 
9: end function

```

The joint-injection subroutine is the linear counterpart of the differential joint-injection builder, but it is not implemented by duplicating the Boolean circuit on two witnesses. Instead, it uses the Walsh support of the packed vectorial oracle

$$\Phi_\kappa : V \rightarrow V \times V, \quad \Phi_\kappa(x) = (X_\kappa(x), Y_\kappa(x)).$$

The first component is the XOR-routed component, and the second component is the source-derived addend component. The packed output mask is

$$\beta = \text{pack}(\lambda, \eta),$$

where λ is the mask on the XOR-routed component and η is the mask on the addend component. The oracle creates the required source mask α and charges the Walsh-rank weight.

Algorithm 46 Linear joint-injection Walsh subroutine

```

1: function CREATEJOINTINJECTIONALPHAWITHWALSHCONSTRAINT( $\kappa, \lambda, \eta$ , prefix)
2:   create source mask
                                      $\alpha \in V$ .

3:   pack the output mask
                                      $\beta = \text{pack}(\lambda, \eta) \in V \times V$ .

4:   create weight variable
                                      $w_\Phi \in [0, 32]$ .

5:   call the custom Walsh support-rank constraint handler:
                                      $\alpha \in \alpha_{\kappa, \beta}^0 \oplus \text{im}(S_{\kappa, \beta}), \quad w_\Phi \geq \frac{1}{2} \text{rank}(S_{\kappa, \beta})$ .

6:   add  $w_\Phi$  to the global linear objective
7:   return ( $\alpha, w_\Phi$ )
8: end function

```

The crucial coupling is the reuse of η . At a $B \rightarrow A$ site, the same internal mask η_0 is simultaneously:

1. the Y -component of the packed Walsh output mask $\text{pack}(\lambda_0, \eta_0)$ for the joint-injection oracle; and
2. the second-input mask of the following two-variable modular-addition linear characteristic.

Likewise, at an $A \rightarrow B$ site, η_1 is simultaneously the addend component of the packed joint-injection Walsh mask and the second-input mask of the subsequent addition on the B branch. Therefore the two constraints

$$\alpha_j \in \alpha_{\kappa, \text{pack}(\lambda_j, \eta_j)}^0 \oplus \text{im}(S_{\kappa, \text{pack}(\lambda_j, \eta_j)})$$

and

$$\text{Corr}_+(\gamma_j, \lambda_j, \eta_j) \neq 0$$

are not independent local choices. They share the same η_j . Detaching the Boolean approximation from the modular-addition addend mask would destroy the joint Walsh model and produce a different linear characteristic search problem.

For one round, the local linear objective can be summarized as

$$W_r = W_{\text{sub}, RC[1]}^{\text{lin}} + w_{\Phi, 0} + W_+^{\text{lin}}(mA_3, mA_2, \eta_0) \\ + W_{\text{sub}, RC[6]}^{\text{lin}} + w_{\Phi, 1} + W_+^{\text{lin}}(mB_8, mB_7, \eta_1).$$

All XOR, rotation, and public-whitening layers are zero-weight transpose-linear constraints, with only sign phases recorded for public constants. For R rounds, the global objective is

$$W_{\text{lin}} = \sum_{r=0}^{R-1} W_r.$$

Finally, the backend records a signed trace for every local stage. The trace layer stores the local masks, selected objective terms, injection support-rank validation, fixed-addend exact numerator data, local sign, local weight, and cumulative weight. This trace is not part of the primitive. It is the audit mechanism used to verify that the SCIP objective, the local oracle weights, and the exported characteristic all describe the same linear trail.

16 Executable Model Correspondence, Characteristic Certification, and Forest/HULL Analysis

This section closes the gap between the local derivations and the current C++ analysis workflow. It introduces no new numerical result. Its purpose is to state exactly what is certified when the program prints a trail, enumerates a HULL window, or grows a multi-layer Forest path.

16.1 Round Graph and Four-Case Arithmetic Dispatch

Let $G = (V_G, E_G)$ be the acyclic stage graph obtained from the checkpointed round. A differential characteristic assigns a two-word difference to every state vertex and local input/output differences to every nonlinear edge. A linear characteristic assigns masks in the reverse/transpose direction and a local sign to every nonzero Walsh transition.

Theorem 30 (Completeness of the arithmetic dispatch). *Every modular arithmetic edge in the implemented round is dispatched to exactly one of the following local models:*

1. *two-variable differential addition/subtraction;*
2. *fixed-public differential addition/subtraction;*
3. *two-variable linear addition/subtraction;*
4. *fixed-public linear addition/subtraction.*

No two cases have the same probability space, and therefore no case may be replaced by another merely by fixing a difference or mask to zero.

Proof. The two source-dependent additions receive one branch word and one injection-derived word; both operands vary over the characteristic and are modeled by the two-variable relations. The operations $B \leftarrow B - \text{RC}[1]$ and $A \leftarrow A - \text{RC}[6]$ contain one variable word and one public constant; their probability or correlation depends on the actual constant bit pattern and is modeled by the fixed-addend recurrences. Differential and linear analyses use different observables (XOR differences versus Walsh masks), giving the remaining separation. These four categories cover every modular arithmetic statement in the forward schedule and are pairwise disjoint by operand status and analysis type. $\square$

The implementation dispatch is summarized in Table 4.

Table 4: Four-case arithmetic dispatch in the C++ implementation.

Case	Point oracle	MILP constructor
Two-variable differential	<code>differential_oracle::oracle_add2/sub2</code>	<code>add_two_input_add_diff/sub_diff</code>
Fixed-public differential	<code>differential_oracle::oracle_add_const/sub_const</code>	<code>add_fixed_public_constant_exact/sub_exact</code>
Two-variable linear	<code>linear_oracle::oracle_add2/sub2</code>	modular-addition characteristic constructor and its subtraction wrapper
Fixed-public linear	<code>linear_oracle::oracle_add_const/sub_const</code>	fixed-public exact-numerator constructor plus the exact-log epigraph handler

16.2 Global Refinement of the Round MILP

Refinement Claim 1 (Differential round refinement). An integral feasible solution of the differential round MILP projects to a stage-consistent differential characteristic of the implemented checkpoint graph. Its objective is the sum of the local differential weights selected by the four arithmetic boxes and the two joint-injection rank boxes.

Proof. Proceed in forward checkpoint order. Equality, XOR, and rotation constraints identify the same state differences as the corresponding affine value-domain updates. The fixed-public subtraction boxes connect the pre- and post-subtraction branch differences. At each joint injection, one source difference is connected to the packed pair consisting of the XOR-routed output difference and the addend-routed output difference. The addend-routed variable is reused as the second input of the subsequent two-variable addition box; therefore the local witnesses cannot be selected independently. Induction over all stages gives a consistent assignment to the full graph. Every nonzero objective coefficient belongs to exactly one local nonlinear edge, so the global objective is their sum. $\square$

Refinement Claim 2 (Linear round refinement). An integral feasible solution of the linear round MILP projects to a stage-consistent linear characteristic of the implemented round in transpose order. Its objective equals the sum of local absolute-correlation weights, and its sign equals the product of local oracle signs and public-constant phases.

Proof. The proof follows the reverse data-flow of masks. For every XOR-rotation update, the MILP applies the transpose map, which is the inverse rotation of the output mask added to the unchanged-branch mask. Each arithmetic box enforces a nonzero local Walsh coefficient and contributes its exact local weight. Each joint injection uses one packed output mask containing the XOR-routed mask and the addend-routed mask; the latter is reused by the following addition box. Public XOR constants contribute only a sign phase. Multiplication of local Walsh coefficients adds their logarithmic absolute weights and multiplies their signs, proving the claim. $\square$

16.3 Semantic Characteristic Certificates

Definition 31 (Semantic characteristic signature). A semantic signature $\sigma(\tau)$ contains the external input and output pair, every local nonlinear input/output tuple, and both output components of each joint injection. It excludes auxiliary solver witnesses that do not change the projected characteristic. In the linear case, the signature contains the corresponding masks; the sign is recomputed from the trace rather than used to distinguish two otherwise identical characteristics.

The signature is the unit of HULL enumeration. In particular, recording only round checkpoints is insufficient because two joint-injection transitions may share the same branch checkpoints while producing different addend-routed outputs and different subsequent arithmetic costs.

Algorithm 47 Independent audit of one differential characteristic

```

1: function AUDITDIFFERENTIALCHARACTERISTIC( $\tau$ )
2:   verify all affine checkpoint equations of the round graph
3:   for each two-variable arithmetic stage  $e$  do
4:     evaluate oracle_add2 or oracle_sub2 on the exported triple
5:     reject if impossible or if the oracle weight differs from the trace weight
6:   end for
7:   for each fixed-public arithmetic stage  $e$  do
8:     evaluate the constant-dependent differential oracle on the exported pair
9:     reject if impossible or if the exact local weight differs from the trace weight
10:  end for
11:  for each joint-injection stage  $j$  do
12:    evaluate the affine-image/rank oracle on the source and packed two-output
    difference
13:    reject if the packed output is outside the image or the rank weight differs

```

```

14:   end for
15:   require the sum of recomputed local weights to equal the SCIP objective within
      tolerance
16:   return ACCEPT
17: end function

```

Algorithm 48 Independent audit of one linear characteristic

```

1: function AUDITLINEARCHARACTERISTIC( $\tau$ )
2:   verify all transpose checkpoint equations of the round graph
3:    $s \leftarrow +1$ ,  $W \leftarrow 0$ 
4:   for each two-variable arithmetic stage  $e$  do
5:     evaluate linear_oracle::oracle_add2/sub2
6:     reject if the coefficient is zero or the oracle weight differs from the trace
7:      $s \leftarrow s \cdot s_e$ ,  $W \leftarrow W + W_e$ 
8:   end for
9:   for each fixed-public arithmetic stage  $e$  do
10:    evaluate the exact signed-numerator carry-transfer oracle
11:    reject if the numerator is zero or the exact weight differs from the trace
12:     $s \leftarrow s \cdot s_e$ ,  $W \leftarrow W + W_e$ 
13:  end for
14:  for each joint-injection stage  $j$  do
15:    evaluate the quadratic Walsh support/rank oracle on the packed output mask
16:    reject if unsupported or if the rank weight/sign differs from the trace
17:  end for
18:  multiply  $s$  by all recorded public-constant phases
19:  require  $W$  to equal the SCIP objective within tolerance
20:  return (ACCEPT,  $s$ ,  $W$ )
21: end function

```

Theorem 31 (Soundness of an accepted certificate). *If Algorithm 47 or Algorithm 48 accepts, then the exported artifact is a stage-consistent characteristic under the local models specified in this paper, and its reported local-weight sum agrees with the solver objective. In the linear case, the returned sign is the sign used for HULL summation.*

Proof. Every nonlinear edge is independently checked against its exact point oracle, and every affine edge is checked by direct equality. Since adjacent stages share the same exported intermediate word, these local checks compose into a path through the full round graph. The final equality check establishes the objective identity. The linear sign follows by the piling-up product of the independently evaluated local signs and constant phases. $\square$

16.4 Semantic No-Good Cuts and Reoptimization

Let $z = (z_1, \dots, z_m)$ be the binary variables in a semantic signature and let z^* be an already counted assignment. The canonical no-good cut is

$$\sum_{i: z_i^* = 0} z_i + \sum_{i: z_i^* = 1} (1 - z_i) \geq 1.$$

Theorem 32 (Exact exclusion by a semantic no-good). *The canonical no-good inequality excludes every solver witness with semantic signature z^* and preserves every witness whose semantic signature is different in at least one position.*

Proof. At $z = z^*$, every summand is zero, so the inequality is violated. If $z \neq z^*$, choose a coordinate at which they differ. The corresponding summand is one, while all summands are nonnegative, so the inequality holds. Auxiliary solver witnesses are deliberately absent from the signature; hence different encodings of the same projected characteristic are excluded together, whereas a different characteristic is preserved. $\square$

Invariant 1 (HULL reoptimization invariant). Within one HULL enumeration session, the endpoint, objective, local arithmetic models, joint-injection models, and weight window are fixed. Between successive calls to `solve_next`, the only mathematical change is the addition of one semantic no-good cut for an accepted characteristic.

Theorem 33 (Reoptimization preserves the enumeration problem). *The differential and linear HULL reoptimization-session classes enumerate the same sequence of residual feasible sets as rebuilding the identical model after each accumulated semantic no-good cut.*

Proof. SCIP reoptimization reuses internal search information but does not change the algebraic constraints. After the k -th accepted characteristic, both the reused model and a rebuilt model contain the original endpoint/window model and exactly the first k semantic no-good inequalities. Therefore their feasible integral points and objective values are identical. Induction on k proves the claim. $\square$

Algorithm 49 Endpoint-local HULL enumeration with exact semantic blocking

```

1: function ENUMERATEHULLWINDOW( $E, [W_{\min}, W_{\max}], T$ )
2:   construct one reoptimization session with fixed endpoint  $E$  and weight window
3:   initialize accumulator  $H \leftarrow 0$  and counted-signature set  $S \leftarrow \emptyset$ 
4:   while wall-clock budget  $T$  remains do
5:      $r \leftarrow \text{SOLVENEXT}(T_{\text{remaining}})$ 
6:     if the residual model is infeasible then
7:       return ( $H, S, \text{COMPLETE}$ )
8:     else if no incumbent is available then
9:       return ( $H, S, \text{PARTIAL}$ )
10:    end if
11:    audit the exported characteristic; reject the session on audit failure
12:    compute  $\sigma(\tau)$  and require  $\sigma(\tau) \notin S$ 
13:    add its differential contribution, or signed linear contribution, to  $H$ 
14:    insert  $\sigma(\tau)$  into  $S$ 
15:    add the canonical semantic no-good cut to the reoptimization session
16:  end while
17:  return ( $H, S, \text{PARTIAL}$ )
18: end function

```

A HULL result is complete for the declared endpoint and window only when the residual model is proved infeasible after all accepted characteristics have been blocked. A time, memory, solution, or external enumeration limit yields a partial HULL result. This distinction is independent of whether reoptimization or full model rebuilding is used.

16.5 Forest Layer as Iterated Weakness Growth

The Forest Layer is not an endpoint-only search. It composes analyzed boxes: the output difference or mask of one accepted trunk becomes the input of the next layer. For a path $P = (s_0, \tau_0, s_1, \dots, \tau_{L-1}, s_L)$,

$$s_{i+1} = \text{out}(\tau_i), \quad \text{in}(\tau_i) = s_i.$$

The cumulative trail weight is $W(P) = \sum_{i=0}^{L-1} W(\tau_i)$. The outer wall-clock limit, rather than a mathematical endpoint, terminates a resource-limited Forest campaign.

Algorithm 50 Multi-layer Forest growth used by the HULL executables

```

1: function GROWFOREST( $s_0, T$ )
2:   initialize priority queue  $Q$  with root state  $s_0$ 
3:   initialize per-path state history and best-known cumulative weights
4:   while  $Q \neq \emptyset$  and wall-clock budget  $T$  remains do
5:     extract the next candidate  $(s, \ell, W_{\text{prefix}})$ 
6:     solve the one-box MILP with input fixed to  $s$ 
7:     if no audited incumbent is available then
8:       terminate this branch and continue
9:     end if
10:    let  $\tau$  be the audited incumbent and  $s' = \text{out}(\tau)$ 
11:    record local and cumulative weights, operator profile, and endpoint statistics
12:    optionally enumerate the declared endpoint HULL window using Algorithm 49
13:    if  $s'$  occurs earlier on the same path then
14:      record a cycle and do not create an infinite duplicate branch
15:    else if the new cumulative weight is not dominated for  $s'$  then
16:      insert  $(s', \ell + 1, W_{\text{prefix}} + W(\tau))$  into  $Q$ 
17:    end if
18:  end while
19:  return all audited trunks and resource-limited Forest statistics
20: end function

```

Proposition 7 (Validity of a Forest path). *Every path emitted by Algorithm 50 is a composable sequence of audited one-box characteristics: the output pair of each trunk is identical to the fixed input pair of its successor.*

Proof. The successor candidate is created directly from the exported output pair of the accepted trunk, and the next MILP fixes its input variables to that pair. The claim follows by induction on the layer index. Priority ordering, dominance pruning, and cycle detection change which valid paths are explored within the time budget; they do not change the validity of an emitted path. $\square$

16.6 Multi-Process Campaigns Without Model Mutation

The differential and linear directories each contain a `hull_multiple_thread.runner.cpp` program implementing process-level parallelism. Each worker launches an unmodified HULL executable in a separate process and output directory. Scheme A assigns distinct Forest seeds for broad exploration. Scheme B fixes externally chosen input pairs to obtain an interpretable fan/sector partition, while each child process retains the normal output-to-input Forest feedback internally.

Algorithm 51 Scheme A/B multi-process campaign runner

```

1: function RUNCAMPAIGN( $\text{mode}, J, P, T$ )
2:   create  $J$  deterministic job specifications
3:   if  $\text{mode} = A$  then
4:     assign a distinct Forest seed to every job
5:   else
6:     assign a distinct externally fixed input pair/sector to every job

```

```

7:   end if
8:   maintain at most  $P$  concurrent child processes
9:   for each completed child do
10:    retain its command, standard output, standard error, and JSON artifact
11:    parse coverage and candidate statistics
12:  end for
13:  merge counts of unique inputs, outputs, sectors, layers, cycles, and best weights
14:  do not add HULL probabilities/correlations across workers without semantic-
    signature deduplication
15:  return campaign summary
16: end function

```

Process-level parallelism does not strengthen a local proof and does not turn a partial HULL into a complete one. Its role is throughput: independent seeds or externally partitioned inputs cover more of the search space under a fixed calendar-time budget. Exact endpoint-local HULL claims remain attached to the termination status of the child process that performed the semantic enumeration.

16.7 Research-Artifact Acceptance Checklist

A result is eligible for inclusion in a scientific table only if all applicable checks below succeed:

1. the executable and paper use the same core constants, rotations, and stage order;
2. all local oracle checks pass and the trace/objective delta is within tolerance;
3. the joint-injection addend variable is shared with the following addition box;
4. the printed input/output pair agrees with the first/last trace checkpoint;
5. the SCIP status and every resource limit are reported;
6. “optimal” is used only for a solver-certified optimum;
7. “complete HULL” is used only after residual infeasibility is proved;
8. multi-process summaries do not double-count a characteristic contribution.

17 Resource-Limited MILP Experiment

17.1 Differential Result

We finally report the resource-limited XOR-differential single-characteristic search performed on one full NeoAlzette V6.5 box. The backend is the C++/SCIP MILP model described above. The constant model is the fixed-public-exact model, and the joint injection model is the explicit MILP witness-support model with the joint affine-rank weight. The run was capped at 43200 seconds.

The solver stopped by time limit. Therefore the characteristic reported in this subsection is the best incumbent found within the resource budget, not a solver-certified global optimum. The final incumbent has objective weight

$$W_{\text{diff}} = 80.431292345473963,$$

corresponding to the estimated single-characteristic probability

$$\Pr(\Delta_{\text{in}} \mapsto \Delta_{\text{out}}) = 2^{-80.431292345} \approx 6.134348059789 \cdot 10^{-25}.$$

The trace-recomputed weight is equal to the SCIP objective, with zero objective delta.

Table 5: SCIP resource-limited differential search summary for one NeoAlzette V6.5 box.

Item	Value
Analysis type	XOR-differential single characteristic
Cipher object	NeoAlzette V6.5, one full 64-bit box
Backend	C++ / SCIP C API
Rounds	1
Time limit	43200 seconds
Solver status	time limit reached
Optimality proved	no
Memory limit reached	no
Solution limit reached	no
Best incumbent weight	80.431292345473963
Best incumbent probability	$2^{-80.431292345}$
Decimal probability	$6.134348059789 \cdot 10^{-25}$
Trace weight	80.431292345473963
Trace-objective delta	0
Trace matches objective	yes

The endpoint difference of the incumbent characteristic is

$$(\Delta A_{\text{in}}, \Delta B_{\text{in}}) = (0x3492db6a, 0x00000000),$$

and

$$(\Delta A_{\text{out}}, \Delta B_{\text{out}}) = (0x5f727fe6, 0xbac57b00).$$

Table 6: Endpoint of the best incumbent XOR-differential characteristic.

Boundary	ΔA	ΔB
Input	0x3492db6a	0x00000000
Output	0x5f727fe6	0xbac57b00

The corresponding internal propagation is shown in Table 7. Only modular addition, fixed-public subtraction, and the joint injection rank contribute positive local weight. Rotations, XOR bridges, and public XOR constants are zero-weight linear transport in the XOR-difference model.

Table 7: Best incumbent differential propagation through one NeoAlzette V6.5 box.

Step	Stage	ΔA	ΔB	Local weight
0	input	3492db6a	00000000	–
1	$B \leftarrow B - RC[1]$	3492db6a	00000000	0
2	joint injection $B \rightarrow A$	3492db6a	00000000	0
3	$A \leftarrow A \boxplus D_0$	54925b6a	00000000	17
4	bridge $A \rightarrow B$, rotation 22	54925b6a	da952496	0
5	public XOR $RC[4]$ on B	54925b6a	da952496	0
6	bridge $B \rightarrow A$, rotation 13	f0008038	da952496	0
7	$A \leftarrow A - RC[6]$	90018008	da952496	7.431292345474
8	joint injection $A \rightarrow B$	90018008	735dbc01	31
9	$B \leftarrow B \boxplus D_1$	90018008	767b9fff	25
10	bridge $B \rightarrow A$, rotation 5	5f727fe6	767b9fff	0
11	public XOR $RC[9]$ on A	5f727fe6	767b9fff	0
12	final bridge $A \rightarrow B$, rotation 25	5f727fe6	bac57b00	0
13	final public XOR $RC[10], RC[11]$	5f727fe6	bac57b00	0
Total weight				80.431292345474

The local positive-weight decomposition is

$$\begin{aligned}
W_{\text{diff}} &= W_{\text{ADD0}} + W_{\text{SUB}, RC[6]} + W_{\Phi_1} + W_{\text{ADD1}} \\
&= 17 + 7.4312923454739579 + 31 + 25 \\
&= 80.431292345473963.
\end{aligned}$$

The first fixed-public subtraction by $RC[1]$ and the first joint injection have zero local weight on this characteristic because the active difference is carried only by the A branch at that point. The second joint injection is active and contributes rank weight 31.

The two joint-injection local transitions are:

Table 8: Joint-injection derivative transitions in the incumbent characteristic.

Site	Input difference	XOR output	Addend output	Joint output	Rank weight
$B \rightarrow A$	00000000	00000000	00000000	0000000000000000	0
$A \rightarrow B$	90018008	a9c89897	09dc5ce4	09dc5ce4a9c89897	31

The resource cost of the run is summarized in Table 9. After presolving, the model contains 11681 variables and 27024 constraints. The reported final memory footprint is about 2892 MB. The search processed 14024 solving nodes in the final run, with 15073 total nodes over two SCIP runs. The run ended at the time limit with primal bound 80.431292345474 and dual bound 0.30007633376790; consequently, the solver did not close the optimality gap.

Table 9: Time and space cost of the 12-hour differential MILP run.

Item	Value
Presolving time	23.00 seconds
Solving time	43200.00 seconds
SCIP status	time limit reached
Presolved variables	11681
Binary variables	11679
Continuous variables	2
Presolved constraints	27024
Injection-rank constraints	2
Set partitioning/packing constraints	21029
AND constraints	640
XOR constraints	4240
Linear constraints	58
Logic-or constraints	1055
Solving nodes	14024
Total nodes over two runs	15073
LP iterations near final log line	about $49.059 \cdot 10^6$
Final reported memory	about 2892 MB
Primal bound	80.431292345474
Dual bound	0.30007633376790
Optimality gap	not closed

Therefore, under the exact fixed-public constant model and the joint affine-rank injection model, the best one-box XOR-differential characteristic found within the 12-hour resource budget has weight

$$80.431292345473963.$$

This value should be read as a resource-limited incumbent bound for the implemented NeoAlzette V6.5 model, not as a mathematically proven maximum differential characteristic bound.

17.2 Linear Result

We also report the resource-limited linear single-characteristic search performed on one full NeoAlzette V6.5 box. The backend is the C++/SCIP MILP model described above. The constant model is the fixed-addend exact-log-weight MILP model, and the joint-injection model is the joint linear Walsh MILP constraint model. The run was capped at 43200 seconds.

The solver stopped by time limit. Therefore the characteristic reported in this subsection is the best incumbent found within the resource budget, not a solver-certified global optimum. The final incumbent has objective weight

$$W_{\text{lin}} = 16.615385952228642,$$

corresponding to the absolute single-characteristic correlation

$$|C| = 2^{-16.6153859522} \approx 9.960254577518 \cdot 10^{-6}.$$

The exact trace evaluator recomputes the total weight as

$$W_{\text{trace}} = 16.615385959600772,$$

with objective delta

$$|W_{\text{trace}} - W_{\text{obj}}| = 7.3721295734685555 \cdot 10^{-9}.$$

The trace oracles are valid, and the trace weights are consistent with the local oracle evaluations.

Table 10: SCIP resource-limited linear search summary for one NeoAlzette V6.5 box.

Item	Value
Analysis type	Linear single characteristic
Cipher object	NeoAlzette V6.5, one full 64-bit box
Backend	C++ / SCIP C API
Rounds	1
Time limit	43200 seconds
Solver status	time limit reached
Feasible incumbent found	yes
Optimality proved	no
Memory limit reached	no
Constant model	fixed-addend exact-log-weight MILP
Injection model	joint linear Walsh MILP constraints
Best incumbent objective weight	16.615385952228642
Exact trace-evaluated weight	16.615385959600772
Absolute correlation	$2^{-16.6153859522}$
Decimal absolute correlation	$9.960254577518 \cdot 10^{-6}$
Characteristic sign	+1
Signed correlation contribution	$+9.960254577518 \cdot 10^{-6}$
Trace-objective delta	$7.3721295734685555 \cdot 10^{-9}$
Trace oracles valid	yes
Paper-usable characteristic	yes

The boundary masks of the incumbent characteristic are

$$(mA_{\text{in}}, mB_{\text{in}}) = (0x00000000, 0xbffee75d),$$

and

$$(mA_{\text{out}}, mB_{\text{out}}) = (0x10bca2ab, 0xed7ecbae).$$

Table 11: Endpoint of the best incumbent linear characteristic.

Boundary	mA	mB
Input	0x00000000	0xbffee75d
Output	0x10bca2ab	0xed7ecbae

The corresponding linear-mask propagation is shown in Table 12. The table is written in the same style as a standard linear characteristic table: each row records the mask pair after one local operation, together with the local sign and the local negative-log correlation weight. Only the fixed-public subtraction layers contribute positive weight in this incumbent. The two joint-injection Walsh constraints and the two two-variable modular-addition linear boxes are traversed with zero local weight for the selected masks. XOR bridges, rotations, and public XOR constants are zero-weight transpose-linear layers; public constants may still contribute sign phases.

Table 12: Best incumbent linear-mask propagation through one NeoAlzette V6.5 box.

Step	Stage	mA	mB	Local sign	Local weight
0	input	00000000	bffee75d	–	–
1	$B \leftarrow B - RC[1]$	00000000	ffffffff	+1	7.691438210390
2	joint injection $B \rightarrow A$	00000000	ffffffff	+1	0
3	$A \leftarrow A \boxplus D_0$	00000000	ffffffff	+1	0
4	bridge $B \leftarrow B \oplus \text{rotl}(A, 22) \oplus RC[4]$	ffffffff	ffffffff	+1	0
5	bridge $A \leftarrow A \oplus \text{rotl}(B, 13)$	ffffffff	00000000	+1	0
6	$A \leftarrow A - RC[6]$	afd975dd	00000000	–1	8.923947749211
7	joint injection $A \rightarrow B$	afd975dd	00000000	+1	0
8	$B \leftarrow B \boxplus D_1$	afd975dd	00000000	+1	0
9	bridge $A \leftarrow A \oplus \text{rotl}(B, 5) \oplus RC[9]$	afd975dd	ed7ecbae	+1	0
10	bridge $B \leftarrow B \oplus \text{rotl}(A, 25)$	10bca2ab	ed7ecbae	+1	0
11	final public XOR $RC[10], RC[11]$	10bca2ab	ed7ecbae	–1	0
				Total sign	+1
				Total exact trace weight	16.615385959601

The positive-weight decomposition is

$$\begin{aligned}
W_{\text{lin}} &= W_{\text{sub}, RC[1]}^{\text{lin}} + W_{\text{sub}, RC[6]}^{\text{lin}} \\
&= 7.6914382103902188 + 8.923947749210555 \\
&= 16.615385959600772.
\end{aligned}$$

The product of the local signs is

$$(+1) \cdot (-1) \cdot (-1) = +1,$$

where the second negative sign comes from the fixed-public subtraction by $RC[6]$, and the final negative sign comes from the final public-constant phase. Hence the reported signed contribution is positive:

$$C_{\text{char}} = +2^{-16.6153859522} \approx +9.960254577518 \cdot 10^{-6}.$$

The two fixed-public subtraction transitions are shown separately because they are the only positive-weight local linear events in the incumbent characteristic.

Table 13: Fixed-public exact-log linear events in the incumbent characteristic.

Stage	Constant	Effective constant	Input mask	Output mask	Sign	Weight
$B - RC[1]$	c117176a	3ee8e896	bffee75d	ffffffff	+1	7.691438210390
$A - RC[6]$	13198a2e	ece675d2	ffffffff	afd975dd	–1	8.923947749211

The two joint-injection Walsh constraints are inactive in the sense that their packed output masks are zero and their rank weights are zero. They are still present in the MILP model and are validated by the oracle trace.

Table 14: Joint-injection Walsh events in the incumbent characteristic.

Site	α	β_{xor}	β_{add}	Packed β	Rank weight
$B \rightarrow A$	00000000	00000000	00000000	0000000000000000	0
$A \rightarrow B$	00000000	00000000	00000000	0000000000000000	0

The resource cost of the run is summarized in Table 15. After presolving, the model contains 1692 variables and 5294 constraints. The SCIP progress table reports the final memory field near the time limit as approximately 60 GB. The run processed 517403 solving nodes and ended with primal bound 16.6153859522286 and dual bound 5.82243782154161. The remaining gap is 185.37%, so SCIP did not prove global optimality.

Table 15: Time and space cost of the 12-hour linear MILP run.

Item	Value
Presolving time	1.00 second
Solving time	43200.00 seconds
SCIP status	time limit reached
Presolved variables	1692
Binary variables	1078
Integer variables	2
Continuous variables	612
Presolved constraints	5294
Fixed-addend exact-log constraints	2
Linear-injection Walsh constraints	2
Variable-bound constraints	956
Knapsack constraints	372
Set-partitioning/packing constraints	248
AND constraints	62
XOR constraints	192
Linear constraints	3272
Logic-or constraints	188
Solving nodes	517403
Solutions found	26
Final reported memory field	about 60 GB
Primal bound	16.6153859522286
Dual bound	5.82243782154161
Final gap	185.37%

Therefore, under the fixed-addend exact-log-weight model and the joint linear Walsh injection model, the best one-box linear characteristic found within the 12-hour resource budget has absolute correlation

$$|C| = 2^{-16.6153859522}.$$

This value should be read as a resource-limited incumbent for the implemented NeoAlzette V6.5 linear model, not as a mathematically proven maximum linear characteristic bound and not as a bound on the full linear hull.

A Detailed Code-to-Mathematics Correspondence

This appendix records the stable interface names used by the refinement arguments. The purpose is not to freeze line numbers, but to make semantic drift detectable during future maintenance.

Table 16: Core and analysis interface correspondence.

C++ interface	Mathematical object
NeoAlzetteCore::generate_dynamic_diffusion_mask0/1	The two 32-bit linear diffusion maps <code>mask0</code> and <code>mask1</code> .
NeoAlzetteCore::cd_injection_from_B/A	The value-domain maps $J_B, J_A : \mathbb{W}_{32} \rightarrow \mathbb{W}_{32}^2$.
NeoAlzetteCore::forward/backward	The permutation F and inverse F^{-1} of Theorem 2.
differential_oracle::oracle_add2/sub2	Lipmaa–Moriai point probability and subtraction permutation.
differential_oracle::oracle_add_const/sub_const	Exact fixed-public carry-pair probability.
DifferentialInjectionOracle::transition	Affine derivative image membership and rank weight of the packed joint output.
linear_oracle::oracle_add2/sub2	Signed two-variable modular-addition/subtraction Walsh coefficient.
linear_oracle::oracle_add_const/sub_const	Exact signed fixed-addend numerator and logarithmic weight.
LinearInjectionOracle::transition	Quadratic Walsh support, offset, rank, and sign data of the packed joint output mask.
NeoAlzetteScip	Differential composition of the checkpoint graph.
EngineeringSearch::build_one_round	
NeoAlzetteScip	
LinearSearch::build_one_round	Transpose-mask composition of the checkpoint graph.
capture_semantic_no_good	Projection of a solver solution to $\sigma(\tau)$.
DifferentialHull	Residual differential endpoint/window model under accumulated semantic no-goods.
ReoptimizationSession	
LinearHull	Residual linear endpoint/window model under accumulated semantic no-goods.
ReoptimizationSession	
run_forest_hull_search	Algorithm 50.
hull_multiple_thread.runner.cpp	Algorithm 51; process scheduling only.

B Verification Protocol for Future Revisions

A future change to the core or analysis should be accepted only after the following protocol has been rerun. This protocol is stated as pseudocode so that a separate verifier can be implemented without sharing the MILP code.

Algorithm 52 Regression protocol for a future NeoAlzette revision

- 1: **function** VERIFYREVISION(Core, DiffModel, LinModel)
- 2: compare every public constant and rotation used by the three components
- 3: run Algorithm 8 on deterministic and random states
- 4: **for** small word sizes for which exhaustive enumeration is feasible **do**
- 5: compare two-variable differential oracle values with brute force
- 6: compare fixed-public differential oracle values with brute force for every constant
- 7: compare two-variable signed linear oracle values with Walsh enumeration

```

8:      compare fixed-public signed linear oracle values with Walsh enumeration for
      every constant
9:    end for
10:   compare every MILP local box with its independent oracle on exhaustive small-
      width assignments
11:   generate one feasible differential and one feasible linear characteristic
12:   audit them with Algorithms 47 and 48
13:   add their semantic no-good cuts and verify that the same signatures cannot reappear
14:   verify that forward/backward, single-round MILP, HULL, Forest, and multi-process
      runners use the same model identifiers and stage order
15:   return ACCEPT only if every check succeeds
16: end function

```

Result immutability for this revision. The preceding methodological additions do not modify the numerical tables in the experimental section. In particular, the 43200-second differential and linear incumbent weights, solver statuses, node counts, memory figures, and reported gaps remain exactly those of the designated V6.5 artifacts.

C Citations and Source Lineage

The implementation comments and local derivation notes identify the following source lineage for the arithmetic models. The paper does not need to reproduce the original proofs in full, but it should cite the relevant works when the models are described:

References

- [ARS⁺20] S. A. Azimi, A. Ranea, M. Salmasizadeh, J. Mohajeri, M. R. Aref, and V. Rijmen. A bit-vector differential model for the modular addition by a constant. In *ASIACRYPT 2020*, volume 12491 of *Lecture Notes in Computer Science*. Springer, 2020.
- [ARS⁺22] S. A. Azimi, A. Ranea, M. Salmasizadeh, J. Mohajeri, M. R. Aref, and V. Rijmen. A bit-vector differential model for the modular addition by a constant and its applications to differential and impossible-differential cryptanalysis. *Designs, Codes and Cryptography*, 90:1797–1855, 2022.
- [BA20] E. Bagherzadeh and Z. Ahmadian. Milp-based automatic differential search for lea and hight block ciphers. IET Information Security, 2020.
- [BBdS⁺20] Christof Beierle, Alex Biryukov, Luan Cardoso dos Santos, Johann Großschädl, Léo Perrin, Aleksei Udovenko, Vesselin Velichkov, and Qingju Wang. Alzette: A 64-bit arx-box (feat. crax and trax). In *CRYPTO 2020*, volume 12171 of *Lecture Notes in Computer Science*, pages 419–448. Springer, 2020.
- [BBdS⁺21] Christof Beierle, Alex Biryukov, Luan Cardoso dos Santos, Johann Großschädl, Amir Moradi, Léo Perrin, Alireza Rezaei Shahmirzadi, Aleksei Udovenko, Vesselin Velichkov, and Qingju Wang. Schwaemm and esch: Lightweight authenticated encryption and hashing using the sparkle permutation family. Specification version 1.2, 2021.
- [BK98] Alex Biryukov and Eyal Kushilevitz. Improved cryptanalysis of rc5. In *EUROCRYPT 1998*, volume 1403 of *Lecture Notes in Computer Science*, pages 85–99. Springer, 1998.

- [BPV99] Johan Borst, Bart Preneel, and Joos Vandewalle. Linear cryptanalysis of rc5 and rc6. In *FSE 1999*, volume 1636 of *Lecture Notes in Computer Science*, pages 16–30. Springer, 1999.
- [BV] Alex Biryukov and Vesselin Velichkov. Automatic search for differential trails in arx ciphers. *Cryptology preprint*.
- [BVC16] Alex Biryukov, Vesselin Velichkov, and Y. Le Corre. Automatic search for the best trails in arx: Application to block cipher speck. In *FSE 2016*, 2016.
- [CRRY99] Scott Contini, Ronald L. Rivest, M. J. B. Robshaw, and Yiqun Lisa Yin. Improved analysis of some simplified variants of rc6. In *Fast Software Encryption 1999*, volume 1636 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 1999.
- [Dat] Data Security Technology. Zuc stream cipher algorithm, part 1: Specification of the zuc stream cipher algorithm. GM/T 0001.1 and related public specification material.
- [DPU⁺16] R. Dinu, L. Perrin, A. Udovenko, V. Velichkov, J. Großschädl, and A. Biryukov. Design strategies for arx with provable bounds: Sparx and lax. In *ASIACRYPT 2016*, volume 10031 of *Lecture Notes in Computer Science*. Springer, 2016.
- [DRS15] S. M. Dehnavi, A. Mahmoodi Rishakani, and M. R. Mirzaee Shamsabad. A more explicit formula for linear probabilities of modular addition modulo a power of two. *Cryptology preprint*, 2015.
- [DRS⁺16] S. M. Dehnavi, A. Mahmoodi Rishakani, M. R. Mirzaee Shamsabad, Hamidreza Maimani, and Einollah Pasha. Cryptographic properties of addition modulo 2^n . *Preprint*, 2016.
- [EAY] M. ElSheikh, A. Abdelkhalek, and A. M. Youssef. On milp-based automatic search for differential trails through modular additions with application to bel-t. *Cryptology preprint*.
- [ElS21] M. ElSheikh. *MILP-aided Cryptanalysis of Some Block Ciphers*. PhD thesis, Concordia University, 2021.
- [FFW23] Ting Fan, Wei Feng, and Yongzhuang Wei. Design and analysis of a large-state lightweight cryptographic s-box. *Journal of Xidian University*, 50(4):170–179, 2023.
- [FWG⁺16] Kai Fu, Meiqin Wang, Yinghua Guo, Siwei Sun, and Lei Hu. Milp-based automatic search algorithms for differential and linear trails for speck. In *FSE 2016*, 2016.
- [GD10] Jie Guan and Lin Ding. Probability analysis and application of xor difference between $(x + k) \bmod 2^n$ and $x \oplus k$. In *IEEE Conference Proceedings*. IEEE, 2010.
- [HW19] M. Huang and L. Wang. Automatic search for the linear (hull) characteristics of arx ciphers: Applied to speck, sparx, chaskey and cham-64. *Cryptology ePrint Archive*, Report 2019/1319, 2019.
- [HXW22] M. Huang, Z. Xu, and L. Wang. On the probability and automatic search of rotational-xor cryptanalysis on arx ciphers. *The Computer Journal*, 65(12):3062–3080, 2022.

- [JR94] Burton S. Kaliski Jr. and M. J. B. Robshaw. Linear cryptanalysis using multiple approximations. In *CRYPTO 1994*, volume 839 of *Lecture Notes in Computer Science*, pages 26–39. Springer, 1994.
- [JY95] Burton S. Kaliski Jr. and Yiqun Lisa Yin. On differential and linear cryptanalysis of the rc5 encryption algorithm. In *CRYPTO 1995*, volume 963 of *Lecture Notes in Computer Science*, pages 171–184. Springer, 1995.
- [KM96] Lars R. Knudsen and Willi Meier. Improved differential attacks on rc5. In *CRYPTO 1996*, volume 1109 of *Lecture Notes in Computer Science*, pages 216–228. Springer, 1996.
- [Leu15] Gaëtan Leurent. Differential and linear cryptanalysis of arx with partitioning: Application to feal and chaskey. HAL preprint, hal-01243166, 2015.
- [LHS⁺11] Zhenhua Li, Xiaoying Huang, Fei Shen, Jihong Teng, and Kai Li. Analysis of addition modulo 2^n on boolean function. In *2011 Third International Conference on Multimedia Information Networking and Security*, pages 386–389. IEEE, 2011.
- [LJC23] Guangqiu Lv, Chenhui Jin, and Ting Cui. A miqcp-based automatic search algorithm for differential-linear trails of arx ciphers. Manuscript, 2023.
- [LLJW] Z. Liu, Y. Li, L. Jiao, and M. Wang. A new method for searching optimal differential and linear trails in arx ciphers. Cryptology preprint.
- [LM02] Helger Lipmaa and Shiho Moriai. Efficient algorithms for computing differential properties of addition. In *Fast Software Encryption 2001*, volume 2355 of *Lecture Notes in Computer Science*, pages 336–350. Springer, 2002.
- [LNS⁺23] Y. Liu, Z. Niu, S. Sun, et al. Rotational differential-linear cryptanalysis revisited. *Journal of Cryptology*, 36(3), 2023.
- [LP24] Gaëtan Leurent and Cédric Pernot. Design of a linear layer optimised for bitsliced 32-bit implementation. *IACR Transactions on Symmetric Cryptology*, 2024.
- [LSL21] Y. Liu, S. Sun, and C. Li. Rotational cryptanalysis from a differential-linear perspective. In *EUROCRYPT 2021*, 2021.
- [LWR16] Y. Liu, Q. Wang, and Vincent Rijmen. Automatic search of linear trails in arx with applications to speck and chaskey. In *ACNS 2016*, volume 9696 of *Lecture Notes in Computer Science*, pages 485–499. Springer, 2016.
- [Mac01] A. W. Machado. Differential probability of modular addition with a constant operand. Cryptology ePrint Archive, Report 2001/052, 2001.
- [Miy98] Hiroshi Miyano. Addend dependency of differential/linear probability of addition. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E81-A(1):106–110, 1998.
- [MKA⁺21] Nicky Mouha, Nikolay Kolomeec, Danil Akhtiamov, Ivan Sutormin, Matvey Panferov, Kseniya Titova, Tatiana Bonich, Evgeniya Ishchukova, Natalia Tokareva, and Bulat Zhantulikov. Maximums of the additive differential probability of exclusive-or. *IACR Transactions on Symmetric Cryptology*, 2021(2):292–313, 2021.

- [MN02] Atsuko Miyaji and Masao Nonaka. Cryptanalysis of the reduced-round rc6. In *Information and Communications Security – ICICS 2002*, volume 2513 of *Lecture Notes in Computer Science*, pages 480–494. Springer, 2002.
- [NSL⁺22] Z. Niu, S. Sun, Y. Liu, et al. Rotational differential-linear distinguishers of arx ciphers with arbitrary output linear masks. In *CRYPTO 2022*, 2022.
- [Sel98] Ali Aydın Selçuk. New results in linear cryptanalysis of rc5. In *Fast Software Encryption 1998*, volume 1372 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 1998.
- [SG13] Ernst Schulte-Geers. On ccz-equivalence of addition mod 2^n . *Designs, Codes and Cryptography*, 66:111–127, 2013.
- [SHW⁺14] Siwei Sun, Lei Hu, P. Wang, K. Qiao, X. Ma, and L. Song. Towards finding the best characteristics of some bit-oriented block ciphers and automatic enumeration of differential and linear characteristics with predefined properties. In *ASIACRYPT 2014*, 2014.
- [STK02] T. Shimoyama, M. Takenaka, and T. Koshihara. Multiple linear cryptanalysis of a reduced round rc6. In *FSE 2002*, 2002.
- [TW18] R. Tse and W. Wong. The sm4 blockcipher algorithm and its modes of operations. Internet-Draft draft-ribose-cfrg-sm4-02, IETF, 2018.
- [Twi26a] Twilight-Dream-Of-Magic. A bit-vector linear correlation model for modular addition by a constant (bvcrr-flat). Technical manuscript, 2026.
- [Twi26b] Twilight-Dream-Of-Magic. Exact fixed-addend differential and linear theory of modular addition with linear-time algorithms. Technical manuscript, 2026.
- [Twi26c] Twilight-Dream-Of-Magic. linear-box-search: Search tools for binary circulant linear layers. GitHub repository, <https://github.com/Twilight-Dream-Of-Magic/linear-box-search>, 2026. C++ sources include `cpp/src/find_linear_box.cpp`, `cpp/src/gf2_m4ri.cpp`, `cpp/src/heuristic_decomposer.cpp`, and `cpp/src/test_m4ri.cpp`.
- [Wal03a] Johan Wallén. Linear approximations of addition modulo 2^n . In *Fast Software Encryption 2003*, volume 2887 of *Lecture Notes in Computer Science*, pages 261–273. Springer, 2003.
- [Wal03b] Johan Wallén. On the differential and linear properties of addition. Technical Report HUT-TCS-A84, Helsinki University of Technology, Laboratory for Theoretical Computer Science, 2003.
- [XLW22] Zheng Xu, Yongqiang Li, and Mingsheng Wang. Security analysis of alzette. *Journal of Cryptologic Research*, 9(4):698–708, 2022.